

GA10-220501097-AA10-EV01 elabora documentos técnicos y de usuario del
software

APRENDIZ

OCTAVIO ANDRES ROJAS AGUILERA

INSTRUCTOR

JAIRON ANTONIO MUÑOZ

FICHA: 3118568

CENTRO AGROPEACUARIO LA GRANJA – SERVICIO NACIONAL DE
APRENDIZAJE SENA

ANALISIS Y DESARROLLO DE SOFTWARE

2026

MANUAL TÉCNICO Y DE OPERACIÓN DE LOS SISTEMAS DE INFORMACIÓN

Control de Versiones

Versión	Fecha	Descripción	Autor
1.0	28/05/2026	Versión inicial del documento	[Tu Nombre]
2.0	01/06/2026	Ajustes al documento (Guía DNP)	[Tu Nombre]
3.0	02/06/2026	Resolución técnica integral de puntos	[Tu Nombre]

Derechos de Autor

La elaboración de este documento y sus diferentes componentes fueron desarrollados por aprendices del programa de formación Tecnología en Análisis y Desarrollo de Software del SENA. Por esta razón, los derechos de autor y, en particular, los derechos patrimoniales de este documento y su contenido pertenecen exclusivamente al SENA y al proyecto formativo PSG-SHOP.

Por lo tanto, su uso y reproducción por terceros está sujeto a la autorización expresa del aprendiz responsable, en cumplimiento de la Ley 23 de 1982 sobre derechos de autor y demás normas que la modifiquen o complementen.

Este documento se encuentra protegido por derechos de autor y no puede ser copiado, reproducido o distribuido por personas o entidades diferentes al SENA y al proyecto formativo PSG-SHOP.

TABLA DE CONTENIDO

1. Objetivo
2. Alcance
3. Definiciones
4. ¿Qué es el Manual Técnico y de Operación del Sistema?
 - 4.1 Propósito del Manual
 - 4.2 Contenido del Manual
 - 4.3 Audiencia Objetivo
 - 4.4 Objetivo Final
5. Introducción
6. Taxonomía y contenido del manual técnico y de operación del sistema

- 6.1 Prerrequisitos
 - 6.2 Frameworks y estándares
 - 6.3 Diagrama de casos de uso
 - 6.4 Diagrama ER (Modelo Entidad Relación)
 - 6.5 Diagrama de componentes
 - 6.6 Diagrama de servicios
 - 6.7 Diagrama de despliegue
 - 6.8 Diagrama de clases
7. Diseño técnico del sistema de información
- 7.1 Gestión de sesiones y seguridad de pago
 - 7.2 Inventario y sincronización en tiempo real
 - 7.3 Reglas de aplicación de cupones
8. Software base del sistema y prerrequisitos
- 8.1 Requisitos de hardware
 - 8.2 Requisitos de software
9. Componentes y estándares
10. Despliegue y configuración de los componentes
11. Instalación
12. Configuración
13. Despliegue
14. Resolución de problemas
15. Software base del sistema y prerrequisitos (instalación servidor)
16. Componentes y estándares
17. Documentación del código fuente
18. Descripción de los acuerdos de niveles de servicios

1. OBJETIVO

El objetivo fundamental de este manual es documentar de forma exhaustiva la arquitectura, operación y mantenimiento del sistema PSG-SHOP.

Se busca proporcionar al personal técnico e instructores del SENA una guía clara para comprender la integración de tecnologías modernas como Vite, React, Firebase y Stripe.

El sistema tiene como finalidad resolver la necesidad de comercialización digital de accesorios y moños artesanales, automatizando el proceso desde la visualización del catálogo hasta el cierre de la transacción bancaria.

Este manual garantiza que cualquier desarrollador pueda instalar, configurar y escalar el proyecto sin pérdida de conocimiento técnico.

2. ALCANCE

Este documento abarca desde la instalación de dependencias en un entorno local hasta el despliegue final en la plataforma Vercel.

Incluye:

- Infraestructura NoSQL en Firebase.
- Flujo de pagos seguro mediante Stripe.
- Organización de componentes UI bajo Atomic Design simplificado.
- Plan de mantenimiento operativo.
- Resolución de incidentes técnicos.

3. DEFINICIONES

Vite.js: Herramienta de compilación ultrarrápida basada en módulos ES nativos.

React 18: Biblioteca JavaScript para construir interfaces de usuario basadas en componentes.

Firebase Firestore: Base de datos flexible y escalable perteneciente a Firebase y Google Cloud.

Stripe SDK: Conjunto de herramientas para integrar pagos seguros y suscripciones.

Tailwind CSS: Framework CSS basado en utilidades para construir interfaces modernas.

Context API: Sistema nativo de React para compartir estados globales entre componentes.

Vercel: Plataforma cloud especializada en despliegue de aplicaciones frontend.

Serverless: Modelo de computación donde el proveedor administra automáticamente la infraestructura.

Hook: Función especial de React como useState o useEffect que permite acceder a características internas del framework.

4. ¿QUÉ ES EL MANUAL TÉCNICO Y DE OPERACIÓN DEL SISTEMA?

El Manual Técnico de PSG-SHOP es el documento maestro que describe la estructura tecnológica del software. A diferencia de un manual de usuario, este documento explica el funcionamiento interno de la aplicación. Su propósito es garantizar la sostenibilidad, mantenibilidad y transferencia de conocimiento del proyecto. Se encuentra alineado con los lineamientos de documentación técnica establecidos por MinTIC.

4.1 Propósito del Manual

Eje de Instalación: Detallar dependencias, configuraciones y comandos necesarios para ejecutar el sistema.

Eje de Arquitectura: Explicar la comunicación entre Frontend, Firebase y Stripe.

Eje de Seguridad: Documentar el manejo adecuado de variables de entorno y credenciales.

4.2 Contenido del Manual

El manual describe técnicamente:

- Catálogo dinámico.
- Carrito de compras.
- Módulo administrativo.
- Pasarela de pagos.

4.3 Audiencia Objetivo

- Instructores ADSO.
- Desarrolladores Frontend.
- Administradores de bases de datos.

- Personal de soporte técnico.

4.4 Objetivo Final

Garantizar una disponibilidad del 99.9% de la plataforma PSG-SHOP mediante el conocimiento detallado de todos sus componentes.

5. INTRODUCCIÓN

PSG-SHOP surge como una solución tecnológica para digitalizar la comercialización de accesorios y moños artesanales. La aplicación fue desarrollada como una Single Page Application (SPA), permitiendo una navegación rápida sin recargas constantes de página. Toda la lógica de datos reside en la nube, reduciendo la carga sobre el dispositivo del usuario y permitiendo un excelente rendimiento incluso en equipos de bajos recursos.

6. TAXONOMÍA Y CONTENIDO DEL MANUAL TÉCNICO Y DE OPERACIÓN DEL SISTEMA

6.1 PRE-REQUISITOS

Node.js y NPM: Node.js versión 20.x o superior.

Gestión Cloud: Cuenta activa en Firebase.

Infraestructura: Cuenta y acceso a Vercel.

Navegador

- Chrome 90+
- Edge 90+
- Safari 14+

Seguridad

- Puertos habilitados:
 - 5173 (Desarrollo)
 - 443 (Producción)

6.2 FRAMEWORKS Y ESTÁNDARES

Frontend

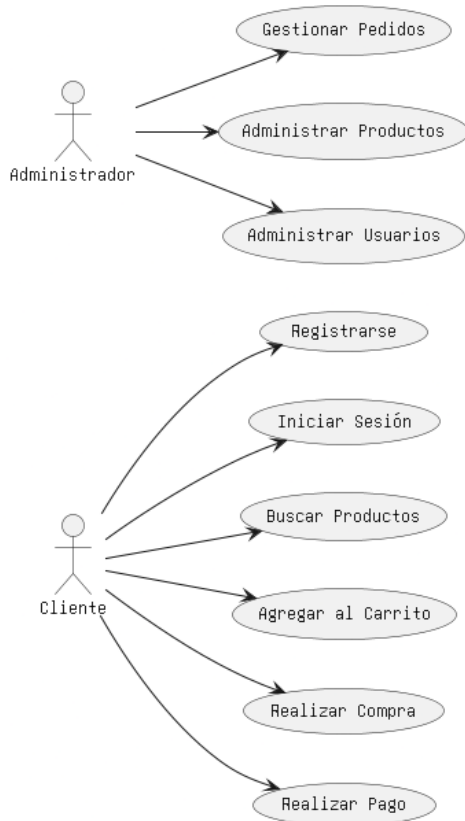
- Vite 5.x
- React 18.2

Datos: Firebase SDK v10

Maquetación: Tailwind CSS 3.x

Pagos: Stripe Payment Intent API

6.3 DIAGRAMA DE CASOS DE USO



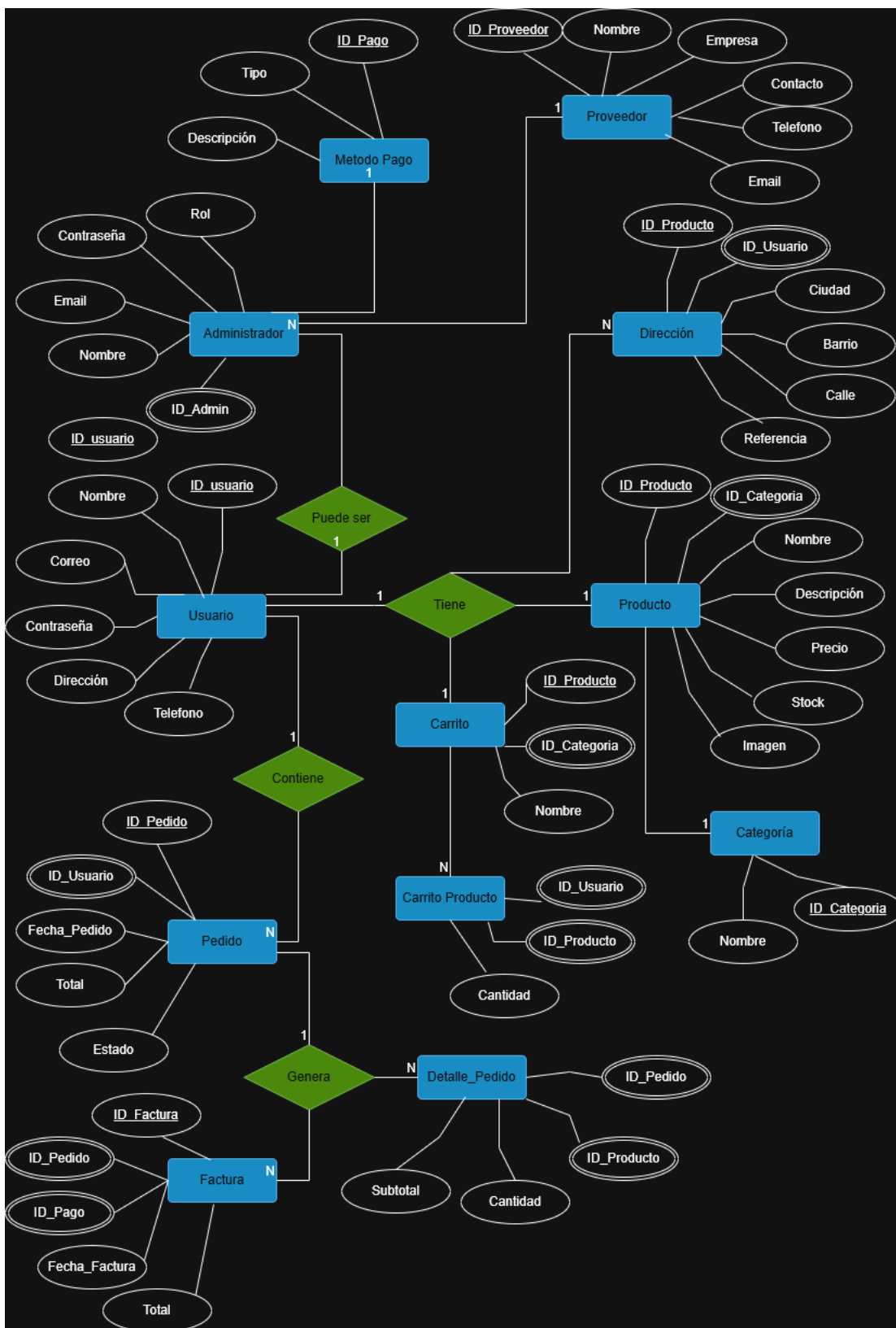
Cliente

- Consultar catálogo.
- Filtrar por categorías.
- Agregar productos al carrito.
- Realizar pagos.

Administrador

- Editar productos.
- Gestionar inventario.
- Consultar ventas.
- Validar pagos.

6.4 DIAGRAMA ER (MODELO ENTIDAD RELACIÓN)



USERS

- UID
- Email
- Nombre
- Rol

PRODUCTS

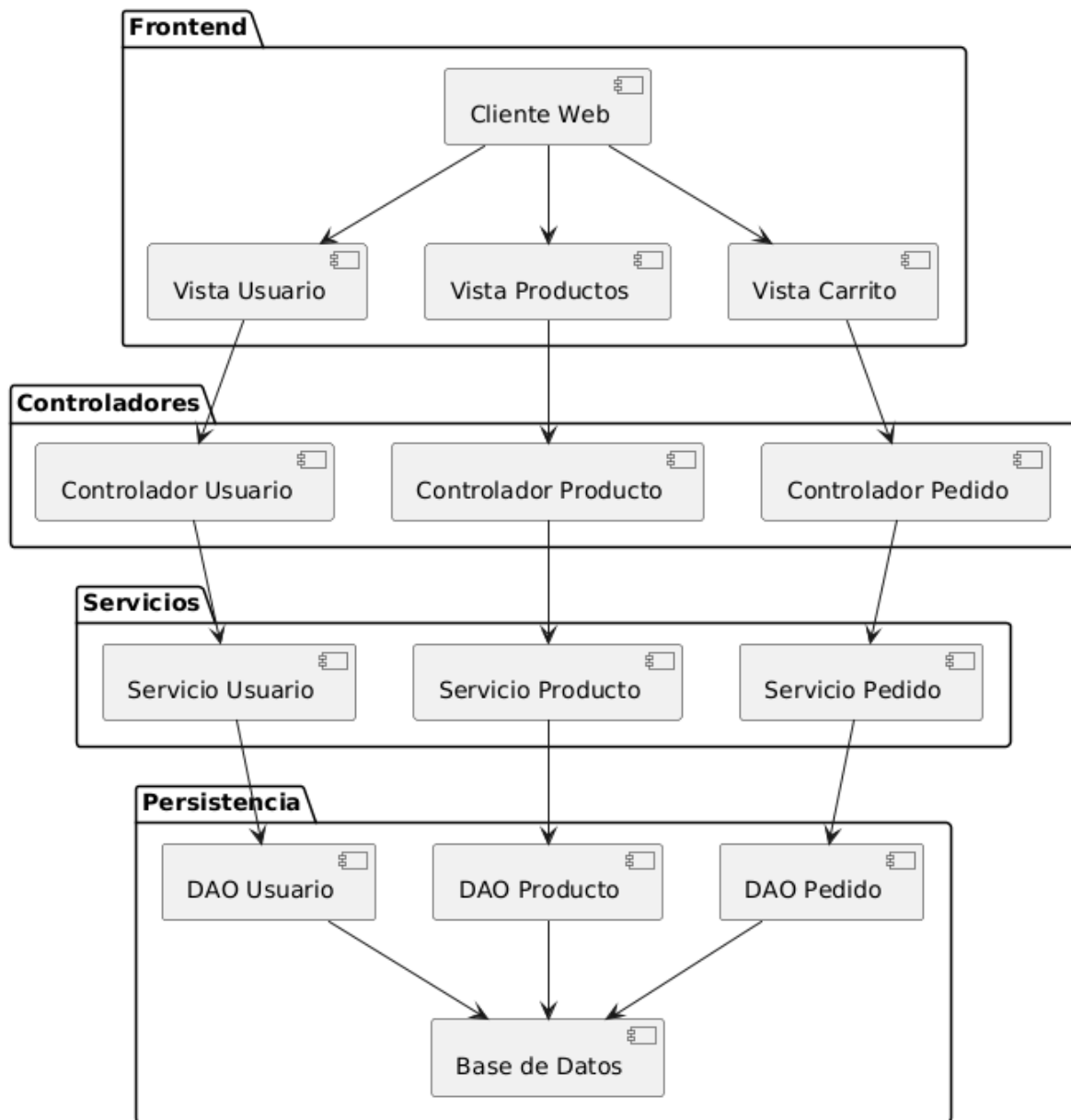
- ID
- Nombre
- Precio
- Stock
- Imagen_URL
- Categoría

ORDERS

- Usuario
- Productos
- Transacción Stripe

6.5 DIAGRAMA DE COMPONENTES

Diagrama de Componentes - Sistema E-commerce

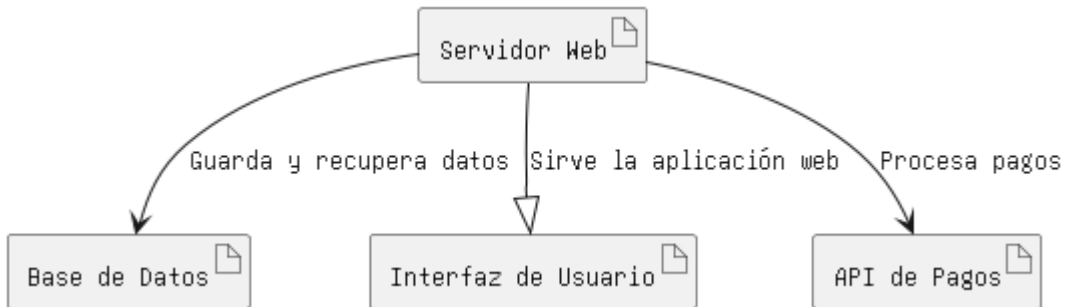


Componentes Principales

- App.jsx
- CartContext.js
- AuthContext.js
- Navbar.jsx

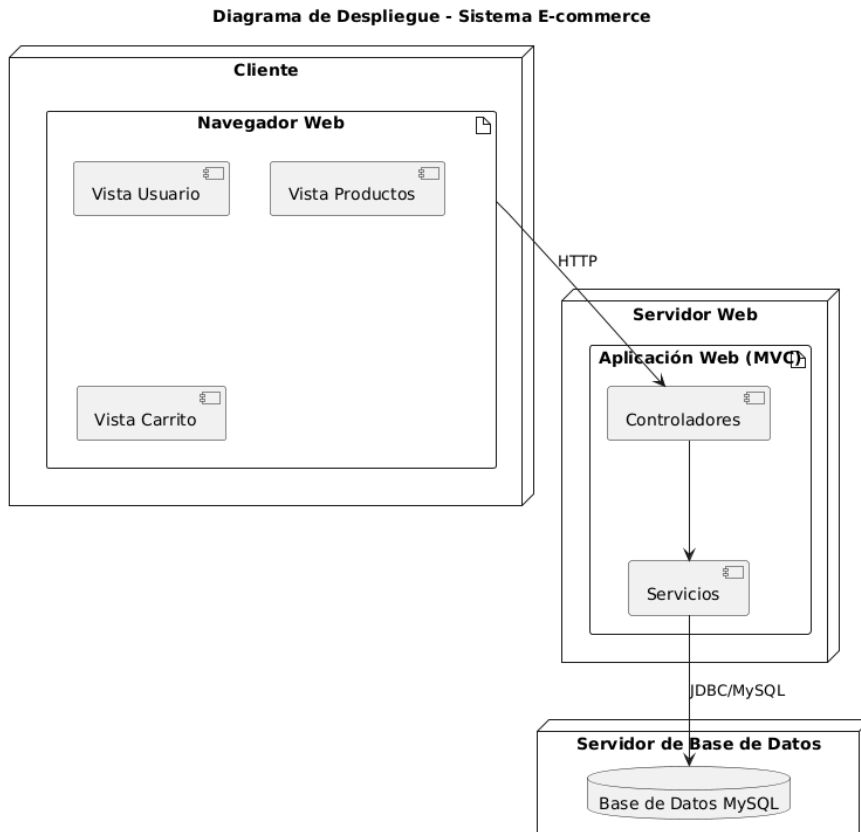
- Footer.jsx
- ItemList.jsx
- StripeButton.jsx

6.6 DIAGRAMA DE SERVICIOS



- App ↔ Firebase Autenticación
- App ↔ Firestore API
- App ↔ Stripe Gateway

6.7 DIAGRAMA DE DESPLIEGUE

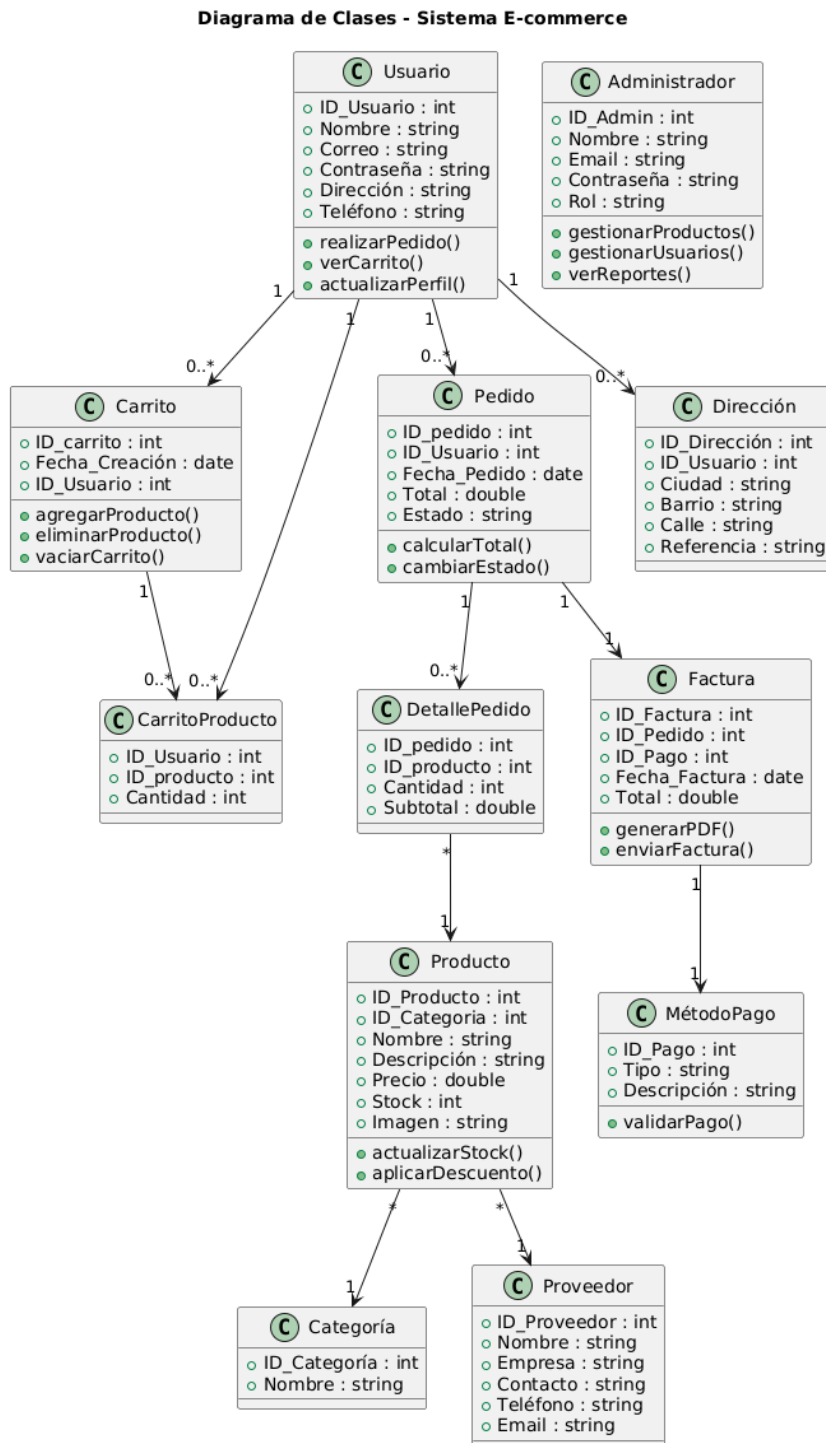


Cliente: Navegador Web.

Hosting: Vercel.

Base de Datos: Firebase Firestore.

6.8 DIAGRAMA DE CLASES



Product

- Cálculo de descuentos.
- Cálculo de IVA.

Cart

- Cálculo de totales.
- Aplicación de cupones.

Order

- Gestión y formateo de pedidos.

7. DISEÑO TÉCNICO DEL SISTEMA

7.1 Gestión de Sesiones y Seguridad de Pago

El sistema utiliza JWT administrados por Firebase Authentication. Stripe procesa los pagos de forma segura sin almacenar información bancaria dentro de PSG-SHOP.

7.2 Inventario y Sincronización en Tiempo Real

Se utiliza el método onSnapshot de Firebase para actualizar automáticamente el inventario en tiempo real.

7.3 Reglas de Aplicación de Cupones

- Validación de existencia.
- Validación de fecha de expiración.
- Máximo un cupón por compra.

8. SOFTWARE BASE DEL SISTEMA Y PRERREQUISITOS

8.1 Requisitos de Hardware

- Intel Core i3 (8ª generación) o equivalente.
- 4 GB RAM mínimo.
- 8 GB RAM recomendados para desarrollo.

8.2 Requisitos de Software

- Windows 10 o superior.
- Linux.

- Chrome, Firefox o Safari actualizados.
- Node.js 20.10 LTS o superior.

9. COMPONENTES Y ESTÁNDARES

9.1 Framework de Desarrollo

React 18: Gestión eficiente de estados e interfaces dinámicas.

Vite.js: Compilación rápida y excelente rendimiento.

9.2 Estándares de Desarrollo

Modularización: Cada módulo funciona de forma independiente.

Convenciones de Nombres

- PascalCase para componentes.
- camelCase para funciones y variables.

9.3 Modelo de Datos

Se utiliza Firestore con una estructura NoSQL optimizada para lecturas rápidas.

9.4 Servicios Ofrecidos

- Catálogo de productos.
- Carrito de compras.
- Autenticación.
- Notificaciones.
- Pasarela de pagos.

10. DESPLIEGUE Y CONFIGURACIÓN DE COMPONENTES

10.1 Organización de Componentes

/src/components

Componentes reutilizables de interfaz.

/src/pages

Páginas principales del sistema.

/src/services

Configuraciones e integraciones externas.

11. INSTALACIÓN

1. Clonar el proyecto

git clone <https://github.com/arktech365/psg-sena.git>

2. Ingresar al proyecto

cd psg-shop-oficial

3. Crear archivo .env

VITE_FIREBASE_API_KEY=
VITE_STRIPE_PUBLIC_KEY=

4. Instalar dependencias

npm install

5. Ejecutar proyecto

npm run dev

12. CONFIGURACIÓN

Firebase

1. Crear proyecto.
2. Activar Firestore.
3. Activar Authentication.
4. Habilitar acceso con Google.

Vercel

1. Importar repositorio.
2. Configurar variables de entorno.
3. Verificar comando de instalación.

13. DESPLIEGUE

1. Realizar commit.
2. Ejecutar push a la rama principal.
3. Esperar despliegue automático de Vercel.
4. Verificar funcionamiento de la aplicación.

14. RESOLUCIÓN DE PROBLEMAS

14.1 API Key de Stripe Inválida

Causa: Llaves incorrectas.

Solución: Verificar variables de entorno en Vercel.

14.2 Pérdida del Carrito al Recargar

Causa: Falta de persistencia.

Solución: Implementar localStorage.

14.3 Error de Login con Google

Causa: Dominio no autorizado.

Solución: Agregar dominio en Firebase Authentication.

14.4 Permisos Denegados en Firestore

Causa: Reglas restrictivas.

Solución:

allow read: if true;

15. SOFTWARE BASE DEL SISTEMA Y PRERREQUISITOS (SERVIDOR)

Plataforma: Vercel

Sistema Operativo: Amazon Linux 2

Runtime: Node.js 20.x

SSL: Let's Encrypt

Base de Datos: Firebase Firestore

16. COMPONENTES Y ESTÁNDARES

16.1 Librerías y Frameworks

React 18: Gestión moderna de interfaces.

Tailwind CSS: Diseño responsivo y eficiente.

Stripe SDK: Procesamiento seguro de pagos.

16.2 Patrones de Diseño

Hooks: Arquitectura basada en componentes funcionales.

Context API: Gestión global del estado de la aplicación.

16.3 Identidad Corporativa y Experiencia de Usuario

Paleta de Colores

Panel Administrativo: Azul Oscuro y Blanco.

Página Principal: Blanco y Negro.

Diseño Responsive: Optimizado para dispositivos móviles.

Experiencia de Usuario: Retroalimentación visual inmediata para las acciones del usuario.

17. Documentación del código fuente:

El código fuente de un programa informático (o software) es un conjunto de líneas de texto con los pasos que debe seguir la computadora para ejecutar un programa. Dicho esto, a continuación, se muestra los distintos códigos que conforman la tienda virtual PSG-SHOP:

Main.jsx

```
• import { StrictMode } from 'react'
• import { createRoot } from 'react-dom/client'
• import './index.css'
• import App from './App.jsx'
• import { HashRouter } from 'react-router-dom'
• import { PayPalScriptProvider } from '@paypal/react-paypal-js'
•
• const paypalOptions = {
•   'client-id': 'AdgxAW2HZC5B-yzBWZKimJnJH8W5r-
PwL5_lppQ0w1w43p_m1pg5FzCD12trocXC23YuWZhhyiJIOf',
•   currency: 'USD',
•   intent: 'capture'
• };
•
• createRoot(document.getElementById('root')).render(
•   <StrictMode>
•     <PayPalScriptProvider options={paypalOptions}>
•       <HashRouter>
•         <App />
•       </HashRouter>
•     </PayPalScriptProvider>
•   </StrictMode>,
• )
```

App.jsx

```
import React, { useState, useEffect } from 'react';
import { Routes, Route, Navigate } from 'react-router-dom';
import { onAuthStateChanged } from 'firebase/auth';
import { auth, db } from './firebase';
import { doc, getDoc } from 'firebase/firestore';
import { CartProvider } from './context/CartContext';
import { WishlistProvider } from './context/WishlistContext';
import { AuthContext, useAuth } from './context/AuthContext';
import Home from './assets/pages/Home';
import Shop from './assets/pages/Shop';
import ProductDetail from './assets/pages/ProductDetail';
import Cart from './assets/pages/Cart';
import Checkout from './assets/pages/Checkout';
import Profile from './assets/pages/Profile';
import Login from './components/login';
import Register from './components/Register';
import ResetPassword from './components/ResetPassword';
import ModernAdminDashboard from './assets/pages/ModernAdminDashboard';
import Orders from './assets/pages/Orders';
import CouponTest from './assets/pages/CouponTest';
import Wishlist from './assets/pages/Wishlist';
import Blog from './assets/pages/Blog';
import BlogPost from './assets/pages/BlogPost';
import Contact from './assets/pages/Contact';
import NotFound from './assets/pages/NotFound';
import Navbar from './components/navbar';
```

```

import Footer from './components/Footer';
import createSampleCoupon from './utils/createSampleCoupon';
import useScrollToTop from './hooks/useScrollToTop';

function App() {
  const [currentUser, setCurrentUser] = useState(null);
  const [userProfile, setUserProfile] = useState(null);
  const [loading, setLoading] = useState(true);
  const [isAdmin, setIsAdmin] = useState(false);

  // Add scroll to top functionality
  useScrollToTop();

  useEffect(() => {
    const unsubscribe = onAuthStateChanged(auth, async (user) => {
      setCurrentUser(user);

      if (user) {
        // Fetch user profile data
        try {
          const userDocRef = doc(db, 'users', user.uid);
          const userDocSnap = await getDoc(userDocRef);

          if (userDocSnap.exists()) {
            const userData = userDocSnap.data();
            setUserProfile(userData);
            setIsAdmin(userData.role === 'admin');
          }
        } catch (error) {
          console.log('Error fetching user profile data', error);
        }
      }
    });
    return unsubscribe;
  }, []);
}

```

```

    } else {
      setUserProfile(null);
      setIsAdmin(false);
    }
  } catch (error) {
    console.error('Error fetching user profile:', error);
    setUserProfile(null);
    setIsAdmin(false);
  }
} else {
  setUserProfile(null);
  setIsAdmin(false);
}

setLoading(false);
});

return unsubscribe;
}, []);

const value = {
  currentUser,
  userProfile,
  isAdmin
};

if (loading) {

```

```

return (
  <div className="flex items-center justify-center min-h-screen">
    <div className="w-12 h-12 border-b-2 border-indigo-600 rounded-full
animate-spin"></div>
  </div>
);
}

```

// Protected Route component

```

const ProtectedRoute = ({ children, adminOnly = false }) => {
  if (!currentUser) {
    return <Navigate to="/login" />;
  }

```

```

  if (adminOnly && !isAdmin) {
    return <Navigate to="/home" />;
  }

```

```

  return children;
};

```

// Redirect authenticated users away from login/register

```

const RedirectIfAuthenticated = ({ children }) => {
  if (currentUser) {
    return <Navigate to="/home" />;
  }
  return children;
}

```

```
};
```

```
const handleCreateSampleCoupon = async () => {  
  try {  
    const couponId = await createSampleCoupon();  
    alert(`Sample coupon created successfully with ID: ${couponId}`);  
  } catch (error) {  
    alert(`Error creating sample coupon: ${error.message}`);  
  }  
};
```

```
return (  
  <AuthContext.Provider value={value}>  
    <CartProvider>  
      <WishlistProvider>  
        <div className="flex flex-col min-h-screen">  
          {/* Only show navbar for non-admin routes */}  
          <Routes>  
            <Route path="/admin/*" element={null} />  
            <Route path="*" element={<Navbar />} />  
          </Routes>  
  
          <main className="flex-grow">  
            <Routes>  
              <Route path="/" element={<Home />} />  
              <Route path="/home" element={<Home />} />  
              <Route path="/shop" element={<Shop />} />  
            </Routes>  
          </main>  
        </div>  
      </WishlistProvider>  
    </CartProvider>  
  </AuthContext.Provider>  
)
```

```

<Route path="/product/:id" element={<ProductDetail />} />
<Route path="/cart" element={<Cart />} />
<Route path="/wishlist" element={<Wishlist />} />
<Route path="/blog" element={<Blog />} />
<Route path="/blog/:id" element={<BlogPost />} />
<Route path="/contact" element={<Contact />} />
  <Route path="/checkout" element={
    <ProtectedRoute>
      <Checkout />
    </ProtectedRoute>
  } />
  <Route path="/profile" element={
    <ProtectedRoute>
      <Profile />
    </ProtectedRoute>
  } />
  <Route path="/orders" element={
    <ProtectedRoute>
      <Orders />
    </ProtectedRoute>
  } />
  <Route path="/login" element={
    <RedirectIfAuthenticated>
      <Login />
    </RedirectIfAuthenticated>
  } />
  <Route path="/register" element={

```

```

    <RedirectIfAuthenticated>
      <Register />
    </RedirectIfAuthenticated>
  } />
  <Route path="/reset-password" element={
    <RedirectIfAuthenticated>
      <ResetPassword />
    </RedirectIfAuthenticated>
  } />
  <Route path="/admin/*" element={
    <ProtectedRoute adminOnly>
      <ModernAdminDashboard />
    </ProtectedRoute>
  } />
  <Route path="/coupon-test" element={
    <ProtectedRoute adminOnly>
      <CouponTest />
    </ProtectedRoute>
  } />
  <Route path="*" element={<NotFound />} />
</Routes>
</main>
{ /* Only show footer for non-admin routes */ }
<Routes>
  <Route path="/admin/*" element={null} />
  <Route path="*" element={<Footer />} />
</Routes>

```



```

    </div>

    </WishlistProvider>

    </CartProvider>

    </AuthContext.Provider>

  );
}

export default App;

```

server.cjs

```

• // Simple local server for Stripe Sessions
• require('dotenv').config();
• const express = require('express');
• const cors = require('cors');
• if (!process.env.STRIPE_SECRET_KEY) {
•   console.error("FATAL: STRIPE_SECRET_KEY no está definido en el
•     archivo .env");
•   process.exit(1);
• }
• const stripe = require('stripe')(process.env.STRIPE_SECRET_KEY);
•
• const app = express();
• app.use(cors());
• app.use(express.json({ limit: '50mb' }));
• app.use(express.urlencoded({ limit: '50mb', extended: true }));
•
• // Sincronización masiva de productos (Admin -> Stripe)
• app.post('/sync-products', async (req, res) => {
•   const { products } = req.body;
•   if (!products || !Array.isArray(products)) {
•     return res.status(400).json({ error: 'Lista de productos no
•       válida' });
•   }
•
•   console.log(`Iniciando sincronización de ${products.length}
•     productos con Stripe...`);
•   const results = { created: 0, updated: 0, errors: [] };
•
•   try {
•     // Obtenemos todos los productos de Stripe para comparar en
•     memoria y evitar latencia del buscador

```

```

• // Expandimos default_price para ver los detalles del monto
• const stripeList = await stripe.products.list({
•   limit: 100,
•   active: true,
•   expand: ['data.default_price']
• });
• const stripeProducts = stripeList.data;
•
• for (const prod of products) {
•   try {
•     // Buscamos si ya existe el producto por nuestro ID de
      Firestore en la metadata
•     const existingProd = stripeProducts.find(s => s.metadata &&
      s.metadata.firestore_id === prod.id);
•
•     const amountInCents = parseInt(parseFloat(prod.price) * 100);
•
•     if (existingProd) {
•       // Existe: Actualizamos info básica
•       const updateData = {
•         name: prod.name,
•         description: prod.description,
•         images: prod.imageUrl ? [prod.imageUrl] : [],
•       };
•
•       // Verificamos si el precio cambió (comparando con el precio
      actual en Stripe si está disponible)
•       // Nota: Traemos los detalles del precio para comparar el
      monto real pasándolo como expansión
•       const currentPrice = existingProd.default_price && typeof
      existingProd.default_price === 'object'
•         ? existingProd.default_price
•         : null;
•
•       if (currentPrice && currentPrice.unit_amount !==
      amountInCents) {
•         // El precio ha cambiado: Guardamos el ID anterior para
      archivarlo DESPUÉS
•         const oldPriceId = currentPrice.id;
•
•         // 1. Creamos un nuevo objeto Price
•         const newPrice = await stripe.prices.create({
•           product: existingProd.id,
•           unit_amount: amountInCents,
•           currency: 'cop',

```

```

•      });
•
•      // 2. Primero actualizamos el producto para que use el
      NUEVO precio como predeterminado
•      // Esto libera el precio antiguo de ser el "default"
•      await stripe.products.update(existingProd.id, {
•        ...updateData,
•        default_price: newPrice.id
•      });
•
•      // 3. Ahora que el antiguo ya no es el predeterminado,
      podemos archivarlo sin errores
•      await stripe.prices.update(oldPriceId, { active: false });
•
•      console.log(`Precio actualizado (Nuevo default:
      ${newPrice.id}) y antiguo archivado (${oldPriceId}) para
      ${prod.name}`);
•
•      // Ya hicimos el update, así que saltamos el update final
      de abajo para este caso
•      results.updated++;
•      continue;
•    } else if (!currentPrice && amountInCents > 0) {
•      // Si no tenía precio predeterminado, se lo ponemos
•      const newPrice = await stripe.prices.create({
•        product: existingProd.id,
•        unit_amount: amountInCents,
•        currency: 'cop',
•      });
•      updateData.default_price = newPrice.id;
•    }
•
•    await stripe.products.update(existingProd.id, updateData);
•    results.updated++;
•  } else {
•    // No existe: Creamos
•    await stripe.products.create({
•      name: prod.name,
•      description: prod.description,
•      images: prod.imageUrl ? [prod.imageUrl] : [],
•      metadata: { firestore_id: prod.id },
•      default_price_data: {
•        currency: 'cop',
•        unit_amount: amountInCents,
•      },
•    },

```

```

•         });
•         results.created++;
•     }
•     } catch (e) {
•         console.error(`Error sincronizando producto ${prod.id}:`,
e.message);
•         results.errors.push({ id: prod.id, error: e.message });
•     }
• }
•
•
•     res.json({ success: true, ...results });
•     console.log(`Sincronización terminada: ${results.created} creados,
${results.updated} actualizados.`);
•     } catch (err) {
•         console.error("Error fatal en sincronización:", err.message);
•         res.status(500).json({ error: err.message });
•     }
• });
• app.post('/create-checkout-session', async (req, res) => {
•     console.log("Nueva petición de pago recibida...");
•     const { cartItems, shippingCost, discount, successUrl, cancelUrl } =
req.body;
•
•     if (!cartItems || cartItems.length === 0) {
•         console.log("Error: Carrito vacío");
•         return res.status(400).json({ error: 'El carrito está vacío' });
•     }
•
•     console.log(`Procesando ${cartItems.length} productos...`);
•
•     let totalAmount = 0;
•     const line_items = cartItems.map(item => {
•         const amountInCents = parseInt(parseFloat(item.price) * 100);
•         totalAmount += amountInCents * item.quantity;
•         return {
•             price_data: {
•                 currency: 'cop',
•                 product_data: { name: item.name },
•                 unit_amount: amountInCents,
•             },
•             quantity: item.quantity,
•         };
•     });
•
•     if (shippingCost && shippingCost > 0) {

```

```

•     const shippingInCents = parseInt(parseFloat(shippingCost) * 100);
•     totalAmount += shippingInCents;
•     line_items.push({
•       price_data: {
•         currency: 'cop',
•         product_data: { name: 'Costo de Envío' },
•         unit_amount: shippingInCents,
•       },
•       quantity: 1,
•     });
•   }
•
•   const sessionConfig = {
•     payment_method_types: ['card'],
•     line_items,
•     mode: 'payment',
•     success_url: successUrl,
•     cancel_url: cancelUrl,
•   };
•
•   // Manejo de descuentos mediante Cupones temporales de Stripe
•   if (discount && discount > 0) {
•     const discountInCents = parseInt(parseFloat(discount) * 100);
•     const coupon = await stripe.coupons.create({
•       amount_off: discountInCents,
•       currency: 'cop',
•       duration: 'once',
•       name: 'Descuento PSG Shop',
•     });
•     sessionConfig.discounts = [{ coupon: coupon.id }];
•   }
•
•   console.log(`Total bruto: ${totalAmount / 100} COP. Descuento:
•   ${discount || 0} COP.`);
•
•   try {
•     const session = await
•     stripe.checkout.sessions.create(sessionConfig);
•     res.json({ id: session.id });
•     console.log("Sesión creada con éxito:", session.id);
•   } catch (e) {
•     console.error("Error completo de Stripe:", JSON.stringify(e, null,
•     2));
•     res.status(500).json({ error: e.message });
•   }

```

```

• });
•
• const PORT = 3001;
• app.listen(PORT, '0.0.0.0', () => console.log(`Server running on all
  interfaces at PORT ${PORT}`));

```

Index.html

```

<!doctype html>

<html lang="en">

  <head>

    <meta charset="UTF-8" />

    <link rel="icon" type="image/svg+xml" href="/vite.svg" />

    <meta name="viewport" content="width=device-width, initial-scale=1.0" />

    <link rel="preconnect" href="https://fonts.googleapis.com">

    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>

    <link
      href="https://fonts.googleapis.com/css2?family=Inter:ital,opsz,wght@0,14..32,100..
      900;1,14..32,100..900&display=swap" rel="stylesheet">

    <link href="/src/index.css" rel="stylesheet">

    <title>PSGSHOP | Welcome</title>

  </head>

  <body>

    <div id="root"> </div>

    <script type="module" src="/src/main.jsx"> </script>

  </body>

</html>

```

📁 ENV (configuraciones variables de entorno)

VITE_API_KEY=AlzaSyDzEL20_gjVbq1gmskOJ2H2mFUocpamJzU

VITE_AUTH_DOMAIN=psgshop-62518.firebaseio.com

VITE_PROJECT_ID=psgshop-62518

VITE_STORAGE_BUCKET=psgshop-62518.firebaseio.com

VITE_MESSAGING_SENDER_ID=951270215395

VITE_APP_ID=1:951270215395:web:d6c1ec0787e43351c17fd6

STRIPE_SECRET_KEY=sk_test_51S8kdbBqZW5OQI7p0PPuTHiroYgBCxKOAETGA7hxgW4c7S7wblzJex82orB90Tjm20H23pXhQJ3bd6vXM3hJmv4800ql1ymqHt

VITE_STRIPE_PUBLISHABLE_KEY=pk_test_51S8kdbBqZW5OQI7pAuakPwvEkUK1Mc1AVG2nwaw4lcWA0hIvgUvW0yWDVewvd4CWDSmbosw2oGs4nWaN0ijtBpa00uy8DnNtl

📁 Firebase.js

```
import { initializeApp } from "firebase/app";
```

```
import { getFirestore } from "firebase/firestore";
```

```
import { getAuth, setPersistence, browserLocalPersistence, GoogleAuthProvider }  
from "firebase/auth";
```

```
import { getStorage } from "firebase/storage";
```

```
const firebaseConfig = {
```

```
  apiKey: import.meta.env.VITE_API_KEY,
```

```
  authDomain: import.meta.env.VITE_AUTH_DOMAIN,
```

```
  projectId: import.meta.env.VITE_PROJECT_ID,
```

```
  storageBucket: import.meta.env.VITE_STORAGE_BUCKET,
```

```
  messagingSenderId: import.meta.env.VITE_MESSAGING_SENDER_ID,
```

```
  appId: import.meta.env.VITE_APP_ID
```

```
};
```

```
console.log("Firebase config values:", {  
  apiKey: importmeta.env.VITE_API_KEY ? "****" : "MISSING",  
  authDomain: importmeta.env.VITE_AUTH_DOMAIN || "MISSING",  
  projectId: importmeta.env.VITE_PROJECT_ID || "MISSING",  
  storageBucket: importmeta.env.VITE_STORAGE_BUCKET || "MISSING",  
  messagingSenderId: importmeta.env.VITE_MESSAGING_SENDER_ID || "MISSING",  
  appId: importmeta.env.VITE_APP_ID || "MISSING"  
});
```

```
const isFirebaseConfigValid = Object.values(firebaseConfig).every(value => value  
&& value !== '');
```

```
console.log("Is Firebase config valid:", isFirebaseConfigValid);
```

```
let app, db, auth, storage;
```

```
if(isFirebaseConfigValid) {  
  try{  
    console.log("Initializing Firebase app...");  
    app = initializeApp(firebaseConfig);  
    console.log("Firebase app initialized successfully");  
  
    console.log("Initializing Firestore...");
```



```

db = getFirestore(app);

console.log("Firestore initialized successfully");


console.log("Initializing Auth...");
auth = getAuth(app);

// Set persistence to local storage to maintain session across browser restarts
setPersistence(auth, browserLocalPersistence)

.then(() => {
    console.log("Auth persistence set to browser local storage");
})

.catch((error) => {
    console.error("Error setting auth persistence:", error);
});

console.log("Auth initialized successfully");


console.log("Initializing Storage...");
storage = getStorage(app);
console.log("Storage initialized successfully");
} catch (error) {
    console.error("Firebase initialization error:", error);
    console.error("Error code:", error.code);
    console.error("Error message:", error.message);
}
} else {

```

```

const missingKeys = Object.entries(firebaseConfig)
  .filter(([key, value]) => !value || value === "")
  .map(([key]) => key);

console.warn("Firebase configuration is missing or incomplete. Missing keys:",
missingKeys);

console.warn("Please check your .env file and ensure all Firebase config variables
are set.");
}

```

```
export { db, auth, app, storage, GoogleAuthProvider };
```

Tailwind config

```

/** @type {import('tailwindcss').Config} */
export default {
  content: [
    "./index.html",
    "./src/**/*.{js,ts,jsx,tsx}",
  ],
  theme: {
    extend: {
      fontFamily: {
        sans: ['Inter', 'sans-serif'],
      },
    },
  },
  plugins: [],
}

```

```
}
```

Vercel.json

```
{
```

```
  "rewrites": [
```

```
    {
```

```
      "source": "/api/(.*)",
```

```
      "destination": "/api/index.js"
```

```
    }
```

```
  ]
```

```
}
```

.gitignore

```
# Logs
```

```
logs
```

```
*.log
```

```
npm-debug.log*
```

```
yarn-debug.log*
```

```
yarn-error.log*
```

```
pnpm-debug.log*
```

```
lerna-debug.log*
```

```
node_modules
```

```
dist
```

```
dist-ssr
```

```
*.local
```

```
.env
```

Editor directories and files

.vscode/*

!.vscode/extensions.json

.idea

.DS_Store

*.suo

.ntvs

*.njsproj

*.sln

*.sw?

 **src/assets/pages/blog.jsx**

import React from 'react';

import { Link } from 'react-router-dom';

const Blog = () => {

const companyInfo = {

name: "PSG SHOP",

tagline: "Elegancia en cada detalle",

description: "Tu destino para moños elegantes y accesorios de moda de alta calidad.",

mission: "Ofrecer productos de la más alta calidad con un servicio excepcional, ayudando a nuestros clientes a expresar su estilo único a través de accesorios elegantes.",

vision: "Ser la marca líder en accesorios de moda en Latinoamérica, reconocida por nuestra calidad, innovación y compromiso con la sostenibilidad.",

```

values: [

  { title: "Calidad Premium", desc: "Cada moño es creado con materiales de la
más alta calidad." },

  { title: "Diseño Único", desc: "Cada pieza es única y cuidadosamente diseñada."
},

  { title: "Sostenibilidad", desc: "Comprometidos con prácticas ecológicas
responsables." },

  { title: "Atención Personalizada", desc: "Servicio al cliente excepcional en cada
interacción." },

  { title: "Innovación", desc: "Siempre a la vanguardia de las tendencias de moda."
},

]

};

```

```

const team = [

{

  id: 1,

  name: "María González",

  role: "Fundadora & Directora Creativa",

  bio: "Con más de 10 años de experiencia en moda y accesorios, María lidera
nuestro equipo con visión artística y pasión por la excelencia.",

  image: "https://ui-
avatars.com/api/?name=María+González&background=111827&color=ffffff&size
=128"

},

{

  id: 2,

```

```

    name: "Carlos Rodríguez",
    role: "Director de Operaciones",
    bio: "Experto en logística y atención al cliente, Carlos asegura que cada pedido
    sea procesado con eficiencia y cuidado.",
    image: "https://ui-
    avatars.com/api/?name=Carlos+Rodríguez&background=374151&color=ffffff&size=128"
  },
  {
    id: 3,
    name: "Ana Martínez",
    role: "Diseñadora Principal",
    bio: "Artista del textil con un ojo meticuloso para el detalle, Ana crea cada
    diseño con inspiración y precisión.",
    image: "https://ui-
    avatars.com/api/?name=Ana+Martínez&background=4b5563&color=ffffff&size=128"
  }
];

const achievements = [
  { number: "500+", label: "Clientes Satisfechos" },
  { number: "100+", label: "Diseños Únicos" },
  { number: "5.0★", label: "Calificación Promedio" },
  { number: "24/7", label: "Soporte al Cliente" },
];

```

```

return (

  <div className="min-h-screen bg-white" style={{ fontFamily: 'Inter, sans-serif'
}}>

    {/ * — Hero — * /}

    <section className="bg-gray-900 relative overflow-hidden">

      <div className="absolute inset-0 opacity-5" style={{ backgroundImage:
'radial-gradient(circle at 2px 2px, white 1px, transparent 0)', backgroundSize: '28px
28px' }} />

      <div className="relative z-10 max-w-4xl mx-auto px-4 py-20 sm:px-6 lg:px-8
text-center">

        <p className="text-xs font-semibold uppercase tracking-widest text-gray-
400 mb-4">Sobre Nosotros</p>

        <h1 className="text-4xl md:text-5xl font-black text-white mb-5 leading-
tight">

          {companyInfo.tagline}

        </h1>

        <p className="text-lg text-gray-400 max-w-xl mx-auto">

          {companyInfo.description}

        </p>

      </div>

    </section>

    {/ * — Stats — * /}

    <section className="border-b border-gray-100">

```

```

<div className="max-w-5xl mx-auto px-4 sm:px-6 lg:px-8">
  <div className="grid grid-cols-2 md:grid-cols-4 divide-x divide-gray-100">
    {achievements.map((item, i) => (
      <div key={i} className="py-10 px-6 text-center">
        <p className="text-3xl font-black text-gray-900 mb-1">{item.number}</p>
        <p className="text-xs font-medium text-gray-500 uppercase tracking-wider">{item.label}</p>
      </div>
    ))}
  </div>
</div>
</section>

```

```

{/* — Main Content — */}

```

```

<section className="max-w-7xl mx-auto px-4 py-14 sm:px-6 lg:px-8">
  <div className="flex flex-col lg:flex-row gap-10">

```

```

{/* Left column */}

```

```

<div className="lg:flex-1 space-y-8">

```

```

{/* Our Story */}

```

```

<div className="bg-white border border-gray-200 rounded-2xl overflow-hidden shadow-sm">

```

```

  <div className="px-8 pt-8 pb-6">

```


`<span className="text-xs font-semibold uppercase tracking-widest text-gray-400">Nuestra Historia</span>`

`<h2 className="text-2xl font-bold text-gray-900 mt-2 mb-4">Cómo comenzamos</h2>`

`<p className="text-gray-600 leading-relaxed mb-4">`

PSG SHOP nació de la pasión por la elegancia y el detalle. Fundada en 2015 por María González,

nuestra tienda comenzó como un pequeño taller artesanal en el corazón de Cajamarca, Tolima.

Con el tiempo, hemos crecido hasta convertirnos en un referente de calidad en accesorios de moda.

`</p>`

`<p className="text-gray-600 leading-relaxed">`

Cada moño que creamos es una obra de arte, elaborada con materiales premium y técnicas tradicionales

que han sido perfeccionadas a lo largo de generaciones. Nuestro compromiso es ayudarte a expresar

tu estilo único con accesorios que no solo complementan tu look, sino que también cuentan tu historia.

`</p>`

`</div>`

`<div className="px-8 pb-8">`

`<img`

`src="https://images.unsplash.com/photo-1591047139829-d91aecb6caea?ixlib=rb-4.0.3&auto=format&fit=crop&w=900&q=80"`

`alt="Nuestro taller"`

```
        className="w-full h-56 object-cover rounded-xl border border-gray-100"
```

```
        onError={(e) => { e.target.onerror = null; e.target.src =  
'https://via.placeholder.com/900x350.png?text=PSG+SHOP'; }}
```

```
    />
```

```
</div>
```

```
</div>
```

```
{/* Mission & Vision */}
```

```
<div className="grid gap-5 md:grid-cols-2">
```

```
    <div className="bg-white border border-gray-200 rounded-2xl p-7  
    shadow-sm">
```

```
        <div className="flex items-center gap-2 mb-3">
```

```
            <span className="w-2 h-2 rounded-full bg-black flex-shrink-0" />
```

```
            <h3 className="text-xs font-semibold uppercase tracking-widest text-  
            gray-400">Misión</h3>
```

```
        </div>
```

```
        <p className="text-gray-700 leading-relaxed text-  
        sm">{companyInfo.mission}</p>
```

```
    </div>
```

```
    <div className="bg-white border border-gray-200 rounded-2xl p-7  
    shadow-sm">
```

```
        <div className="flex items-center gap-2 mb-3">
```

```
            <span className="w-2 h-2 rounded-full bg-gray-400 flex-shrink-0" />
```

```
            <h3 className="text-xs font-semibold uppercase tracking-widest text-  
            gray-400">Visión</h3>
```

```
        </div>
```

```
<p className="text-gray-700 leading-relaxed text-sm">{companyInfo.vision}</p>
```

```
</div>
```

```
</div>
```

```
{/* Values */}
```

```
<div className="bg-white border border-gray-200 rounded-2xl p-8 shadow-sm">
```

```
<span className="text-xs font-semibold uppercase tracking-widest text-gray-400">Nuestros Valores</span>
```

```
<h2 className="text-2xl font-bold text-gray-900 mt-2 mb-6">Lo que nos define</h2>
```

```
<ul className="space-y-4">
```

```
{companyInfo.values.map((value, index) => (
```

```
<li key={index} className="flex items-start gap-4 p-4 rounded-xl hover:bg-gray-50 transition-colors duration-150">
```

```
<span className="flex items-center justify-center flex-shrink-0 w-7 h-7 bg-black text-white text-xs font-bold rounded-full">
```

```
{index + 1}
```

```
</span>
```

```
<div>
```

```
<p className="text-sm font-semibold text-gray-900">{value.title}</p>
```

```
<p className="text-sm text-gray-500 mt-0.5">{value.desc}</p>
```

```
</div>
```

```
</li>
```

```
)))
```

```

    </ul>

  </div>

</div>

{/* Right sidebar */}

<div className="lg:w-80 space-y-6">

  {/* Team */}

  <div className="bg-white border border-gray-200 rounded-2xl p-6 shadow-sm">

    <span className="text-xs font-semibold uppercase tracking-widest text-gray-400">Equipo</span>

    <h3 className="text-lg font-bold text-gray-900 mt-1 mb-5">Las personas detrás de PSG</h3>

    <div className="space-y-6">

      {team.map((member) => (

        <div key={member.id} className="flex gap-3">

          <img

            src={member.image}

            alt={member.name}

            className="w-12 h-12 rounded-full object-cover border border-gray-200 flex-shrink-0"

          />

          <div className="min-w-0">

            <p className="text-sm font-semibold text-gray-900 leading-snug">{member.name}</p>

```

```
<p className="text-[10px] font-semibold uppercase tracking-widest text-gray-400 mb-1">{member.role}</p>
```

```
<p className="text-xs text-gray-500 leading-relaxed">{member.bio}</p>
```

```
</div>
```

```
</div>
```

```
)))
```

```
</div>
```

```
</div>
```

```
{/* Contact CTA */}
```

```
<div className="bg-gray-900 rounded-2xl p-6 relative overflow-hidden">
```

```
<div className="absolute inset-0 opacity-5" style={{ backgroundImage: 'radial-gradient(circle at 2px 2px, white 1px, transparent 0)', backgroundSize: '20px 20px' }} />
```

```
<div className="relative z-10">
```

```
<h3 className="text-base font-bold text-white mb-2">¿Tienes preguntas?</h3>
```

```
<p className="text-sm text-gray-400 mb-5 leading-relaxed">
```

```
Estamos aquí para ayudarte. Nuestro equipo te atenderá personalmente.
```

```
</p>
```

```
<Link
```

```
to="/contact"
```

```
className="inline-flex items-center px-4 py-2 text-sm font-medium text-gray-900 bg-white rounded-lg hover:bg-gray-100 transition-colors duration-150"
```

>

Contáctanos

```
<svg className="w-4 h-4 ml-2" fill="none" viewBox="0 0 24 24"
stroke="currentColor"> <path strokeLinecap="round" strokeLinejoin="round"
strokeWidth="2" d="M9 5l7 7 7-7" /> </svg>
```

</Link>

</div>

</div>

{/* Shop CTA */}

```
<div className="bg-white border border-gray-200 rounded-2xl p-6
shadow-sm">
```

```
<h3 className="text-base font-bold text-gray-900 mb-2">Descubre la
Colección</h3>
```

```
<p className="text-sm text-gray-500 mb-5 leading-relaxed">
```

Explora nuestra exclusiva colección de moños artesanales, diseñados para cada ocasión especial.

</p>

<Link

to="/shop"

```
className="inline-flex items-center px-4 py-2 text-sm font-medium
text-white bg-black rounded-lg hover:bg-gray-800 transition-colors duration-150"
```

>

Ver Productos

```
<svg className="w-4 h-4 ml-2" fill="none" viewBox="0 0 24 24"
stroke="currentColor"> <path strokeLinecap="round" strokeLinejoin="round"
strokeWidth="2" d="M9 5l7 7 7-7" /> </svg>
```

```

        </Link>
      </div>
    </div>
  </div>
</section>
</div>
);
};
export default Blog;

```

 **src/assets/pages/BlogPost.jsx**

```

import React from 'react';
import { Link, useParams } from 'react-router-dom';
// Removed Navbar import since it's already rendered in App.jsx

```

```

const BlogPost = () => {
  // In a real app, this would come from a database or API
  const { id } = useParams();

  // Sample blog post data
  const blogPosts = [
    {
      id: 1,
      title: "Cómo elegir el moño perfecto para tu ocasión especial",
      date: "15 Oct 2025",

```

author: "María González",

category: "Guía de Estilo",

image: "https://images.unsplash.com/photo-1591047139829-d91aecb6caea?ixlib=rb-4.0.3&ixid=M3wxMjA3fDB8MHxwaG90by1wYWdlfHx8fGVufDB8fHx8fA%3D%3D&auto=format&fit=crop&w=800&q=80",

content: `

<p>Seleccionar el moño perfecto para una ocasión especial puede ser un desafío, pero con los consejos adecuados, puedes encontrar el accesorio ideal que complementa tu estilo y la naturaleza del evento.</p>

<h2 className="mt-8 mb-4 text-2xl font-bold">Considera la ocasión</h2>

<p>El tipo de evento es el primer factor a considerar. Un moño para una boda requiere un enfoque diferente que uno para una cena casual. Los eventos formales suelen llamar a diseños más elaborados y materiales de mayor calidad.</p>

<h2 className="mt-8 mb-4 text-2xl font-bold">Combina con tu atuendo</h2>

<p>El color y el estilo de tu ropa deben ser el punto de partida. Un moño debe complementar, no competir con tu outfit. Si llevas un vestido con estampado, opta por un moño en tonos neutros. Para atuendos sólidos, puedes experimentar con colores vibrantes o texturas interesantes.</p>

<h2 className="mt-8 mb-4 text-2xl font-bold">Considera la temporada</h2>

<p>Los materiales y colores apropiados varían según la época del año. En verano, los moños de seda liviana y colores claros son ideales. En invierno, puedes

optar por tejidos más gruesos como el terciopelo o el brocado en tonos más oscuros.</p>

<h2 className="mt-8 mb-4 text-2xl font-bold">Conoce tu rostro</h2>

<p>La forma de tu rostro puede influir en el estilo de moño que mejor te queda. Los moños más anchos tienden a favorecer los rostros más estrechos, mientras que los diseños más delgados pueden equilibrar los rostros más anchos.</p>

<h2 className="mt-8 mb-4 text-2xl font-bold">Calidad sobre cantidad</h2>

<p>Invierte en un moño de calidad que se mantenga bien con el tiempo. Los materiales premium no solo se ven mejor, sino que también duran más. En PSG SHOP, nos especializamos en moños hechos con materiales de primera calidad que mantienen su forma y color después de múltiples usos.</p>

<p className="mt-8">Con estos consejos, estarás lista para elegir el moño perfecto para cualquier ocasión especial. Recuerda que el mejor accesorio es aquel que te hace sentir segura y elegante.</p>

,

},

{

id: 2,

title: "Tendencias de moda en moños para esta temporada",

date: "10 Oct 2025",

author: "Carlos Rodríguez",

category: "Tendencias",

image: "https://images.unsplash.com/photo-1611849785508-1f152a6489d4?ixlib=rb-4.0.3&ixid=M3wxMjA3fDB8MHxwaG90by1wYWdlfHx8fGVufDB8fHx8fA%3D%3D&auto=format&fit=crop&w=800&q=80",

content: `

<p>La moda de los moños continúa evolucionando, y esta temporada trae consigo emocionantes novedades que están revolucionando el mundo de los accesorios. Desde colores vibrantes hasta texturas innovadoras, descubre las tendencias que dominarán tu guardarropa.</p>

<h2 className="mt-8 mb-4 text-2xl font-bold">1. Colores Bold y Saturados</h2>

<p>Esta temporada, los colores intensos están en todas partes. Desde el magenta profundo hasta el turquesa eléctrico, los moños en tonos vibrantes son la forma perfecta de agregar un toque de personalidad a cualquier atuendo. Estos colores no solo llaman la atención, sino que también transmiten confianza y estilo.</p>

<h2 className="mt-8 mb-4 text-2xl font-bold">2. Texturas Mixtas</h2>

<p>La combinación de diferentes texturas en un solo moño es una tendencia que está ganando popularidad. Piensa en moños que combinan seda con encaje, o terciopelo con detalles metálicos. Esta mezcla de texturas agrega profundidad visual y crea piezas verdaderamente únicas.</p>

<h2 className="mt-8 mb-4 text-2xl font-bold">3. Moños Extra Anchos</h2>

<p>El tamaño importa, y esta temporada los moños extra anchos son los protagonistas. Estos moños hacen una declaración audaz y pueden convertirse en el punto focal de cualquier outfit. Son especialmente populares para looks de oficina y eventos formales.</p>

<h2 className="mt-8 mb-4 text-2xl font-bold">4. Estampados Abstractos</h2>

<p>Los patrones abstractos están reemplazando a los estampados florales tradicionales. Diseños geométricos, formas orgánicas y patrones artísticos están apareciendo en moños de alta gama. Estos estampados agregan interés visual sin abrumar el resto del atuendo.</p>

<h2 className="mt-8 mb-4 text-2xl font-bold">5. Detalles Sostenibles</h2>

<p>Con el creciente enfoque en la moda sostenible, los moños hechos de materiales ecológicos y mediante procesos éticos están ganando terreno. Desde sedas orgánicas hasta tintes naturales, los consumidores conscientes buscan opciones que reflejen sus valores.</p>

<p className="mt-8">En PSG SHOP, estamos al tanto de estas tendencias y ofrecemos una cuidadosa selección de moños que combinan estilo contemporáneo con calidad excepcional. Nuestra colección de esta temporada incorpora muchos de estos elementos para que puedas lucir a la moda sin sacrificar la elegancia clásica.</p>

、
}

];

// Find the current post or use the first one as default

const post = blogPosts.find(post => post.id.toString() === id) || blogPosts[0];

// Related posts (excluding the current one)

const relatedPosts = blogPosts.filter(p => p.id.toString() !== id).slice(0, 3);

```

return (
  <div className="min-h-screen bg-gray-50">
    {/ * Breadcrumb */}
    <div className="py-4 bg-white">
      <div className="container px-4 mx-auto">
        <nav className="text-sm">
          <Link to="/home" className="text-black uppercase tracking-widest
hover:text-gray-800">Inicio</Link>
          <span className="mx-2 text-gray-400">/</span>
          <Link to="/blog" className="text-black uppercase tracking-widest
hover:text-gray-800">Sobre Nosotros</Link>
          <span className="mx-2 text-gray-400">/</span>
          <span className="text-gray-500">{post.title}</span>
        </nav>
      </div>
    </div>

    {/ * Blog Post Content */}
    <div className="container px-4 py-12 mx-auto">
      <div className="flex flex-col gap-8 lg:flex-row">
        {/ * Main Content */}
        <div className="lg:w-2/3">
          <article className="overflow-hidden bg-white shadow-md rounded-xl">
            <div className="overflow-hidden h-96">

```

```

<img
  src={post.image}
  alt={post.title}
  className="object-cover w-full h-full"
  onError={(e) => {
    e.target.onerror = null;
    e.target.src =
'https://via.placeholder.com/800x600.png?text=Imagen+Blog';
  }}
/>
</div>
<div className="p-8">
  <div className="flex items-center justify-between mb-4">
    <span className="inline-flex items-center px-3 py-1 text-sm font-
medium text-gray-800 bg-indigo-100 rounded-full">
      {post.category}
    </span>
    <span className="text-gray-500">{post.date}</span>
  </div>

  <h1 className="mb-4 text-3xl font-bold text-gray-900 md:text-
4xl">{post.title}</h1>

  <div className="flex items-center pb-6 mb-8 border-b border-gray-
200">

```

```
<div className="flex items-center justify-center w-10 h-10 mr-3 font-  
bold text-gray-800 bg-indigo-100 rounded-full">
```

```
{post.author.charAt(0)}
```

```
</div>
```

```
<div>
```

```
<p className="font-medium text-gray-900">{post.author}</p>
```

```
<p className="text-sm text-gray-500">Autor en PSG SHOP</p>
```

```
</div>
```

```
</div>
```

```
<div
```

```
  className="prose text-gray-700 max-w-none"
```

```
  dangerouslySetInnerHTML={{ __html: post.content }}
```

```
/>
```

```
<div className="pt-8 mt-12 border-t border-gray-200">
```

```
<div className="flex items-center justify-between">
```

```
<Link
```

```
  to="/blog"
```

```
  className="inline-flex items-center font-medium text-black  
uppercase tracking-widest hover:text-gray-800"
```

```
>
```

```
  ← Volver a Sobre Nosotros
```

```
</Link>
```

```
<div className="flex space-x-4">
```


<svg className="w-6 h-6" fill="currentColor" viewBox="0 0 24 24">

<path d="M24 4.557c-.883.392-1.832.656-2.828.775 1.017-.609 1.798-1.574 2.165-2.724-.951.564-2.005.974-3.127 1.195-.897-.957-2.178-1.555-3.594-1.555-3.179 0-5.515 2.966-4.797 6.045-4.091-.205-7.719-2.165-10.148-5.144-1.29 2.213-.669 5.108 1.523 6.574-.806-.026-1.566-.247-2.229-.616-.054 2.281 1.581 4.415 3.949 4.89-.693.188-1.452.232-2.224.084.626 1.956 2.444 3.379 4.6 3.419-2.07 1.623-4.678 2.348-7.29 2.04 2.179 1.397 4.768 2.212 7.548 2.212 9.142 0 14.307-7.721 13.995-14.646.962-.695 1.797-1.562 2.457-2.549z"/>

</svg>

<svg className="w-6 h-6" fill="currentColor" viewBox="0 0 24 24">

<path d="M9 8h3v4h3v12h5v-12h3.642l.358-4h-4v-1.667c0-.955.192-1.333 1.115-1.333h2.885v-5h-3.808c-3.596 0-5.192 1.583-5.192 4.615v3.385z"/>

</svg>

<svg className="w-6 h-6" fill="currentColor" viewBox="0 0 24 24">

<path d="M12 2.163c3.204 0 3.584.012 4.85.07 3.252.148 4.771 1.691 4.919 4.919.058 1.265.069 1.645.069 4.849 0 3.205-.012 3.584-.069 4.849-.149 3.225-1.664 4.771-4.919 4.919-1.266.057 1.645-.069 4.849-.069zm0-2.163c-3.259 0-3.667.014-4.947.072-4.358.2-6.78 2.618-6.98 6.98-.059 1.281-.073 1.689-.073 4.948

0 3.259.014 3.668.072 4.948.2 4.358 2.618 6.78 6.98 6.98 1.281.058 1.689.072
 4.948.072 3.259 0 3.668-.014 4.948-.072 4.354-.2 6.782-2.618 6.979-6.98.059-
 1.28.073-1.689.073-4.948 0-3.259-.014-3.667-.072-4.947-.196-4.354-2.617-6.78-
 6.979-6.98-1.281-.059-1.69-.073-4.949-.073zm0 5.838c-3.403 0-6.162 2.759-6.162
 6.162s2.759 6.163 6.162 6.163 6.162-2.759 6.162-6.163c0-3.403-2.759-6.162-6.162-
 6.162zm0 10.162c-2.209 0-4-1.79-4-4 0-2.209 1.791-4 4-4s4 1.791 4 4c0 2.21-1.791
 4-4 4zm6.406-11.845c-.796 0-1.441.645-1.441 1.44s.645 1.44 1.441 1.44c.795 0
 1.439-.645 1.439-1.44s-.644-1.44-1.439-1.44z"/>

</svg>

</div>

</div>

</div>

</div>

</article>

</div>

{/ Sidebar */}*

<div className="lg:w-1/3">

{/ Related Posts */}*

<div className="p-6 mb-8 bg-white shadow-md rounded-xl">

<h3 className="mb-4 text-xl font-bold text-gray-900">Artículos
 Relacionados </h3>

<div className="space-y-4">

{relatedPosts.map(post => (

<div key={post.id} className="flex group">


```

lg">
    <div className="flex-shrink-0 w-20 h-20 overflow-hidden rounded-

    <img
      src={post.image}
      alt={post.title}
      className="object-cover w-full h-full"
      onError={(e) => {
        e.target.onerror = null;
        e.target.src = 'https://via.placeholder.com/100x100.png?text= Blog';
      }}
    />
  </div>

  <div className="ml-4">
    <span className="text-xs font-medium text-black uppercase
tracking-widest">{post.category}</span>

    <h4 className="mt-1 font-medium text-gray-900 transition-colors
duration-300 group-hover:text-black uppercase tracking-widest">

      <Link to={`/blog/${post.id}`}>{post.title}</Link>

    </h4>

    <p className="mt-1 text-xs text-gray-500">{post.date}</p>
  </div>
</div>
  )}
</div>
</div>

```

```

{ /* Categories */

<div className="p-6 mb-8 bg-white shadow-md rounded-xl">

  <h3 className="mb-4 text-xl font-bold text-gray-900">Categorías</h3>

  <ul className="space-y-2">

    {[ 'Guía de Estilo', 'Tendencias', 'Cuidado', 'Historia', 'Eventos',
'Personalización'].map((category, index) => (

      <li key={index}>

        <Link

          to="#"

          className="block px-4 py-2 text-gray-600 transition-colors
duration-300 rounded-lg hover:bg-gray-100"

        >

          {category}

        </Link>

      </li>

    )]}

  </ul>

</div>

```

```

{ /* Newsletter */

<div className="p-6 text-gray-900 shadow-md bg-gradient-to-r from-
indigo-600 to-purple-700 rounded-xl">

  <h3 className="mb-2 text-xl font-bold">Boletín Informativo</h3>

  <p className="mb-4 opacity-90">

```

Suscríbete para recibir las últimas novedades y artículos del blog

</p>

<form *className*="space-y-3">

<input

type="email"

placeholder="Tu correo electrónico"

className="w-full px-4 py-2 text-gray-900 rounded-lg focus:outline-none focus:ring-2 focus:ring-indigo-300"

/>

<button

type="submit"

className="w-full py-2 font-medium text-black transition-colors duration-300 bg-white rounded-lg hover:bg-gray-100"

>

Suscribirse

</button>

</form>

</div>

</div>

</div>

</div>

</div>

);

};

```
export default BlogPost;
```

```
 src/assets/pages/Cart.jsx
```

```
import React, { useState } from 'react';
```

```
import { Link, useNavigate } from 'react-router-dom';
```

```
import { useCart } from '../context/CartContext';
```

```
const Cart = () => {
```

```
  const { items, updateQuantity, removeFromCart, getTotalPrice, loading, coupon, applyCoupon, removeCoupon } = useCart();
```

```
  const navigate = useNavigate();
```

```
  const [couponCode, setCouponCode] = useState('');
```

```
  const [couponError, setCouponError] = useState('');
```

```
  const [applyingCoupon, setApplyingCoupon] = useState(false);
```

```
  const handleApplyCoupon = async (e) => {
```

```
    e.preventDefault();
```

```
    if (!couponCode.trim()) return;
```

```
    setApplyingCoupon(true); setCouponError('');
```

```
    try {
```

```
      await applyCoupon(couponCode.trim());
```

```
      setCouponCode('');
```

```
    } catch (error) {
```

```
      setCouponError(error.message || 'Error al aplicar el cupón');
```

```
    } finally {
```

```
      setApplyingCoupon(false);
```

```

    }

};

const handleRemoveCoupon = () => { removeCoupon(); setCouponCode("");
setCouponError(""); };

const subtotal = items.reduce((total, item) => total + (item.price * item.quantity),
0);

if (loading) {

    return (

        <div className="min-h-screen bg-gray-50 flex items-center justify-center"
style={{ fontFamily: 'Inter, sans-serif' }}>

            <div className="w-8 h-8 border-2 border-gray-900 border-t-transparent
rounded-full animate-spin" />

        </div>

    );

}

return (

    <div className="min-h-screen bg-gray-50" style={{ fontFamily: 'Inter, sans-serif'
}}>

        <div className="px-4 py-10 mx-auto max-w-5xl sm:px-6 lg:px-8">

            <h1 className="text-2xl font-bold tracking-tight text-gray-900 mb-8">Tu
Carrito</h1>

            {items.length === 0 ? (

```

```

    <div className="text-center py-16 bg-white rounded-2xl border border-
gray-200 shadow-sm">

    <div className="flex items-center justify-center w-14 h-14 mx-auto
rounded-full bg-gray-100 mb-4">

    <svg className="w-7 h-7 text-gray-400" fill="none" viewBox="0 0 24 24"
stroke="currentColor">

    <path strokeLinecap="round" strokeLinejoin="round" strokeWidth="1.5"
d="M3 3h2l.4 2M7 13h10l4-8H5.4M7 13L5.4 5M7 13l-2.293 2.293c-.63.63-.184
1.707.707 1.707H17m0 0a2 2 0 100 4 2 2 0 000-4zm-8 2a2 2 0 11-4 0 2 2 0 014 0z"
/>

    </svg>

    </div>

    <h3 className="text-base font-semibold text-gray-900">Tu carrito está
vacío</h3>

    <p className="mt-1 text-sm text-gray-500">Empieza a agregar productos
a tu carrito de compras.</p>

    <Link to="/shop" className="inline-flex items-center mt-6 px-5 py-2.5
text-sm font-medium text-white bg-black hover:bg-gray-800 rounded-lg
transition-colors duration-150">

    Continuar Comprando

    </Link>

    </div>

  ): (

    <div className="lg:grid lg:grid-cols-12 lg:gap-8">

    {/* Cart Items */}

    <div className="lg:col-span-7">

    <div className="bg-white border border-gray-200 rounded-2xl shadow-
sm overflow-hidden">

```

```

<ul className="divide-y divide-gray-100">
  {items.map((item) => (
    <li key={item.id} className="p-5 flex gap-4">
      {/* Thumbnail */}
      <div className="flex-shrink-0 w-20 h-20 rounded-xl border border-
gray-200 overflow-hidden bg-gray-50">
        <img
          src={item.imageUrl}
          alt={item.name}
          className="w-full h-full object-contain"
          onError={(e) => { e.target.onerror = null; e.target.src =
'https://via.placeholder.com/80x80.png?text=PSG'; }}
        />
      </div>
      {/* Info */}
      <div className="flex-1 min-w-0">
        <div className="flex items-start justify-between gap-2">
          <div>
            <Link to={`/product/${item.id}`} className="text-sm font-
semibold text-gray-900 hover:text-black line-clamp-2 leading-
snug">{item.name}</Link>
            <p className="text-xs text-gray-400 mt-
0.5">{item.category}</p>
          </div>
          <p className="text-sm font-bold text-gray-900 flex-shrink-
0">${parseFloat(item.price * item.quantity).toLocaleString('es-CO')}</p>

```

```

</div>

<div className="flex items-center justify-between mt-3">

  {/* Quantity */}

  <div className="flex items-center border border-gray-200
rounded-lg overflow-hidden">

    <button

      onClick={() => updateQuantity(item.id, item.quantity - 1)}

      disabled={item.quantity <= 1}

      className="w-8 h-8 flex items-center justify-center text-gray-
500 hover:bg-gray-50 disabled:opacity-30 disabled:cursor-not-allowed transition-
colors"

    >

      <svg className="w-3 h-3" fill="none" viewBox="0 0 24 24"
stroke="currentColor"> <path strokeLinecap="round" strokeLinejoin="round"
strokeWidth="2.5" d="M20 12H4" /> </svg>

    </button>

    <span className="w-8 text-center text-sm font-medium text-
gray-900">{item.quantity}</span>

    <button

      onClick={() => updateQuantity(item.id, item.quantity + 1)}

      disabled={item.stock !== undefined && item.quantity >=
item.stock}

      className="w-8 h-8 flex items-center justify-center text-gray-
500 hover:bg-gray-50 disabled:opacity-30 disabled:cursor-not-allowed transition-
colors"

    >

```



```

        <svg className="w-3 h-3" fill="none" viewBox="0 0 24 24"
stroke="currentColor"> <path strokeLinecap="round" strokeLinejoin="round"
strokeWidth="2.5" d="M12 6v6m0 0v6m0-6h6m-6 0H6" /> </svg>

        </button>

    </div>

    <button

        type="button"

        onClick={() => removeFromCart(item.id)}

        className="text-xs text-gray-400 hover:text-red-500 transition-
colors duration-150 flex items-center gap-1"

        >

        <svg className="w-3.5 h-3.5" fill="none" viewBox="0 0 24 24"
stroke="currentColor"> <path strokeLinecap="round" strokeLinejoin="round"
strokeWidth="2" d="M19 7l-.867 12.142A2 2 0 0116.138 21H7.862a2 2 0 01-1.995-
1.858L5 7m5 4v6m4-6v6m1-10V4a1 1 0 00-1-1h-4a1 1 0 00-1 1v3M4 7h16"
/> </svg>

        Eliminar

    </button>

</div>

</div>

</li>

    )}

</ul>

    <div className="px-5 py-3 border-t border-gray-100">

        <Link to="/shop" className="text-xs font-medium text-gray-500
hover:text-black transition-colors duration-150 flex items-center gap-1">

```

```
<svg className="w-3.5 h-3.5" fill="none" viewBox="0 0 24 24"
stroke="currentColor"><path strokeLinecap="round" strokeLinejoin="round"
strokeWidth="2" d="M15 19l-7-7 7-7" /></svg>
```

Continuar comprando

```
</Link>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
{/* Order Summary */}
```

```
<div className="mt-6 lg:mt-0 lg:col-span-5">
```

```
<div className="bg-white border border-gray-200 rounded-2xl shadow-
sm p-6">
```

```
<h2 className="text-base font-semibold text-gray-900 mb-
5">Resumen del Pedido</h2>
```

```
{/* Coupon */}
```

```
<div className="mb-5">
```

```
<p className="text-xs font-semibold uppercase tracking-widest text-
gray-500 mb-2">Cupón de descuento</p>
```

```
{coupon ? (
```

```
<div className="flex items-center justify-between px-3 py-2.5 bg-
green-50 border border-green-200 rounded-lg">
```

```
<div>
```

```
<span className="text-sm font-semibold text-green-
800">{coupon.code}</span>
```

```
<span className="ml-2 text-xs text-green-600">
```

```

        {coupon.discountType === 'percentage' ? `-${coupon.discountValue}%` : `-${coupon.discountValue.toLocaleString('es-CO')}`}
      </span>
    </div>

    <button onClick={handleRemoveCoupon} className="text-xs text-red-500 hover:text-red-700 transition-colors">Quitar</button>
  </div>
): (
  <form onSubmit={handleApplyCoupon} className="flex gap-2">
    <input
      type="text"
      value={couponCode}
      onChange={(e) => setCouponCode(e.target.value)}
      placeholder="Ingresa tu código"
      className="flex-1 min-w-0 px-3 py-2 text-sm border border-gray-300 rounded-lg focus:outline-none focus:ring-2 focus:ring-black focus:border-transparent placeholder-gray-400 disabled:bg-gray-50"
      disabled={applyingCoupon}
    />
    <button
      type="submit"
      disabled={applyingCoupon || !couponCode.trim()}
      className="px-3 py-2 text-sm font-medium text-white bg-black rounded-lg hover:bg-gray-800 transition-colors disabled:opacity-40"
    >
      {applyingCoupon ? '...' : 'Aplicar'}
    </button>
  </form>
)

```

```

        </button>

    </form>

    )}

    {couponError && <p className="mt-1.5 text-xs text-red-
600">{couponError}</p>}

</div>

{ /* Totals */ }

<div className="space-y-3 py-4 border-t border-gray-100">

    <div className="flex items-center justify-between">

        <dt className="text-sm text-gray-500">Subtotal</dt>

        <dd className="text-sm font-medium text-gray-
900">${parseFloat(subtotal).toLocaleString('es-CO')}</dd>

    </div>

    {coupon && (

        <div className="flex items-center justify-between">

            <dt className="text-sm text-gray-500">Descuento
({coupon.code})</dt>

            <dd className="text-sm font-medium text-green-600">-
${parseFloat(subtotal - getTotalPrice()).toLocaleString('es-CO')}</dd>

        </div>

    )}

    <div className="flex items-center justify-between pt-3 border-t
border-gray-200">

        <dt className="text-base font-semibold text-gray-900">Total</dt>

```

```
      <dd className="text-base font-bold text-gray-900">${parseFloat(getTotalPrice()).toLocaleString('es-CO')}</dd>
```

```
    </div>
```

```
  </div>
```

```
  { /* Stock warning */ }
```

```
  {items.some(item => item.stock !== undefined && item.stock < item.quantity) && (
```

```
    <div className="mt-3 p-3 bg-amber-50 border border-amber-200 rounded-xl">
```

```
      <div className="flex gap-2">
```

```
        <svg className="h-4 w-4 text-amber-500 flex-shrink-0 mt-0.5"
xml:ns="http://www.w3.org/2000/svg" viewBox="0 0 20 20"
fill="currentColor"> <path fillRule="evenodd" d="M8.257 3.099c.765-1.36 2.722-1.36 3.486 0l5.58 9.92c.75 1.334-.213 2.98-1.742 2.98H4.42c-1.53 0-2.493-1.646-1.743-2.98l5.58-9.92zM11 13a1 1 0 11-2 0 1 1 0 0 12 0zm-1-8a1 1 0 00-1 1v3a1 1 0 002 0V6a1 1 0 00-1-1z" clipRule="evenodd" /> </svg>
```

```
        <p className="text-xs text-amber-700">Algunos productos exceden el stock disponible. Ajusta las cantidades.</p>
```

```
      </div>
```

```
    </div>
```

```
  )}
```

```
<button
```

```
  type="button"
```

```
  onClick={() => navigate('/checkout')}
```

```
      className="w-full mt-5 py-3 px-4 bg-black text-white text-sm font-
medium rounded-xl hover:bg-gray-800 active:bg-gray-900 transition-colors
duration-150 focus:outline-none focus:ring-2 focus:ring-offset-2 focus:ring-black
disabled:opacity-40"
```

```
      disabled={items.some(item => item.stock !== undefined && item.stock
< item.quantity)}
```

```
    >
```

```
      Proceder al Pago
```

```
    </button>
```

```
  </div>
```

```
</div>
```

```
</div>
```

```
  )}
```

```
</div>
```

```
</div>
```

```
);
```

```
};
```

```
export default Cart;
```

 **src/assets/pages/Checkout**

```
import React, { useState, useEffect } from 'react';
```

```
import { useNavigate } from 'react-router-dom';
```

```
import { PayPalButtons } from '@paypal/react-paypal-js';
```

```
import { useCart } from '../context/CartContext';
```

```
import { useAuth } from '../context/AuthContext';
```

```
import { db } from '../firebase';
```

```
import { doc, getDoc, updateDoc, collection, addDoc } from 'firebase/firestore';
```



```

    const img = valid[safe];

    if(typeof img === 'string') return img;

    if(img && img.data) return img.data;
  }
}

return product.imageUrl || 'https://via.placeholder.com/100x100.png?text=PSG';
};

```

```

const [addresses, setAddresses] = useState([]);

const [selectedAddress, setSelectedAddress] = useState(null);

const [newAddress, setNewAddress] = useState({ name: '', street: '', city: '', state: '',
zipCode: '', country: 'Colombia' });

const [showNewAddressForm, setShowNewAddressForm] = useState(false);

const [loading, setLoading] = useState(true);

const [processing, setProcessing] = useState(false);

const [shippingCost, setShippingCost] = useState(0);

```

```

const originLocation = { city: 'Cajamarca', state: 'Tolima', country: 'Colombia' };

const subtotal = items.reduce((total, item) => total + (item.price * item.quantity),
0);

const discount = coupon ? subtotal - getTotalPrice() : 0;

const total = getTotalPrice() + shippingCost;

```

```

const calculateShippingCost = (dest) => {

  if(!dest) return 0;

```



```

    if(dest.city?.toLowerCase() === originLocation.city.toLowerCase() &&
dest.state?.toLowerCase() === originLocation.state.toLowerCase()) return 5000;

    if(dest.state?.toLowerCase() === originLocation.state.toLowerCase()) return
15000;

    if(dest.country?.toLowerCase() === 'colombia') return 25000;

    return 50000;

};

```

```

const isLocalDelivery = (addr) => !(addr && addr.city?.toLowerCase() ===
originLocation.city.toLowerCase() && addr.state?.toLowerCase() ===
originLocation.state.toLowerCase());

```

```

useEffect(() => { setShippingCost(calculateShippingCost(selectedAddress)); },
[selectedAddress]);

```

```

useEffect(() => {

const loadAddresses = async () => {

    if(!currentUser) { navigate('/login'); return; }

    try{

        const snap = await getDoc(doc(db, 'users', currentUser.uid));

        if(snap.exists()) {

            const addrs = snap.data().addresses || [];

            setAddresses(addrs);

            const def = addrs.find(a => a.isDefault) || addrs[0] || null;

            setSelectedAddress(def);

        }

    }
}

```

```

    } catch (err) {

        Swal.fire({ title: 'Error', text: 'Error al cargar las direcciones', icon: 'error',
confirmButtonText: 'Aceptar' });

        } finally { setLoading(false); }

    };

    loadAddresses();

}, [currentUser, navigate]);

const handleAddressChange = (e) => {

    const { name, value } = e.target;

    setNewAddress(prev => ({ ...prev, [name]: value }));

};

const handleAddNewAddress = async (e) => {

    e.preventDefault();

    if(!newAddress.name || !newAddress.street || !newAddress.city ||
!newAddress.state || !newAddress.zipCode) {

        Swal.fire({ title: 'Error', text: 'Por favor completa todos los campos', icon: 'error',
confirmButtonText: 'Aceptar' });

        return;

    }

    try{

        const newAddr = { ...newAddress, id: Date.now(), isDefault: addresses.length
=== 0 };

        await updateDoc(doc(db, 'users', currentUser.uid), { addresses: [...addresses,
newAddr] });

```

```

    const updated = [...addresses, newAddr];

    setAddresses(updated); setSelectedAddress(newAddr);
    setShowNewAddressForm(false);

    setNewAddress({ name: '', street: '', city: '', state: '', zipCode: '', country:
'Colombia' });

    Swal.fire({ title: 'Dirección agregada', text: 'Dirección guardada correctamente',
icon: 'success', confirmButtonText: 'Aceptar' });

    } catch (err) {

        Swal.fire({ title: 'Error', text: 'Error al agregar la dirección: ' + err.message, icon:
'error', confirmButtonText: 'Aceptar' });

    }

};

```

```

const createOrder = async (paymentMethod, paymentStatus) => {

    // Limpiamos los items para que no pesen demasiado (evita error de 1MB en
Firestore)

    // Especialmente si los productos tienen imágenes en Base64

    const cleanItems = items.map(it => ({

        id: it.id || '',

        name: it.name,

        price: it.price,

        quantity: it.quantity,

        selectedSize: it.selectedSize || null,

        selectedColor: it.selectedColor || null,

        // Solo guardamos la URL de la imagen si NO es un Base64 pesado

        imageUrl: (it.imageUrl && !it.imageUrl.startsWith('data:')) ? it.imageUrl : null
    }));

```

```
}});
```

```
const orderData = {  
  userId: currentUser.uid, userEmail: currentUser.email,  
  items: cleanItems, subtotal: parseFloat(subtotal), discount: parseFloat(discount),  
  shippingCost: parseFloat(shippingCost), totalAmount: parseFloat(total),  
  totalAmountUSD: parseFloat(convertToUSD(total).toFixed(2)),  
  address: selectedAddress, paymentMethod, paymentStatus,  
  orderStatus: 'pending', coupon: coupon || null,  
  createdAt: new Date(), updatedAt: new Date()  
};  
  
const ref = await addDoc(collection(db, 'orders'), orderData);  
  
return ref.id;  
};
```

```
const handlePayPalSuccess = (data, actions) => {  
  return actions.order.capture().then(async (details) => {  
    setProcessing(true);  
  
    try {  
      await createOrder('PayPal', 'completed');  
  
      clearCart();  
  
      Swal.fire({ title: '¡Pago exitoso!', text: `Gracias por tu compra,  
      ${details.payer.name.given_name}!`, icon: 'success', confirmButtonText: 'Ir a mis  
      pedidos' }).then(() => navigate('/orders'));  
  
    } catch (err) {
```

```
Swal.fire({ title: 'Error', text: 'Error al procesar tu pedido. Contacta al soporte.',  
icon: 'error', confirmButtonText: 'Aceptar' });
```

```
  } finally { setProcessing(false); }  
  
});  
  
};
```

```
const handlePayPalError = (err) => {
```

```
  Swal.fire({ title: 'Error', text: 'Error en el pago con PayPal. Intenta nuevamente.',  
icon: 'error', confirmButtonText: 'Aceptar' });  
  
};
```

```
const handleStripeCheckout = async () => {
```

```
  if(!selectedAddress) {  
  
    Swal.fire({ title: 'Error', text: 'Selecciona una dirección de envío', icon: 'error',  
confirmButtonText: 'Aceptar' });  
  
    return;  
  
  }
```

```
  setProcessing(true);
```

```
  try{  
  
    // 1. Crear el pedido en Firestore primero con estado 'Pendiente de Pago  
(Stripe)'
```

```
    // para que el administrador ya pueda verlo.
```

```
    const orderId = await createOrder('Stripe', 'pending_stripe');
```

```
    // 2. Optimizamos los items para que no pesen tanto
```

```

const cleanItems = items.map(it => ({
  name: it.name,
  price: it.price,
  quantity: it.quantity
}));

const response = await fetch(`${API_BASE_URL}/create-checkout-session`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    cartItems: cleanItems,
    shippingCost: shippingCost,
    discount: discount,
    successUrl:
`${window.location.origin}${window.location.hostname.includes('github.io') ? '/psg-official' : ''}/#/orders?payment=success&orderId=${orderId}`,
    cancelUrl:
`${window.location.origin}${window.location.hostname.includes('github.io') ? '/psg-official' : ''}/#/checkout`,
  }),
});

if (!response.ok) {
  const errorText = await response.text();

  throw new Error(`Error del servidor (${response.status}):
${errorText.substring(0, 50)}...`);
}

```

```

    }

    const session = await response.json();

    if (!session.id) throw new Error(session.error || 'No se pudo crear la sesión');

    // 2. Redirect to Stripe Checkout

    const stripe = await stripePromise;

    if (!stripe) throw new Error('Stripe no se pudo cargar');

    const { error } = await stripe.redirectToCheckout({
      sessionId: session.id,
    });

    if (error) throw error;

  } catch (err) {

    Swal.fire({ title: 'Error', text: 'Stripe Error: ' + err.message, icon: 'error',
confirmButtonText: 'Aceptar' });

  } finally {

    setProcessing(false);

  }

};

const handleCashOnDelivery = async () => {

  if (!selectedAddress) { Swal.fire({ title: 'Error', text: 'Selecciona una dirección de
envío', icon: 'error', confirmButtonText: 'Aceptar' }); return; }

```

```

setProcessing(true);

try {
  await createOrder('Cash on Delivery', 'pending'); clearCart();

  Swal.fire({ title: 'Pedido confirmado', text: 'Pagarás contra entrega.', icon:
'success', confirmButtonText: 'Ver mis pedidos' }).then(() => navigate('/orders'));

  } catch (err) {

    Swal.fire({ title: 'Error', text: 'Error al procesar el pedido: ' + err.message, icon:
'error', confirmButtonText: 'Aceptar' });

    } finally { setProcessing(false); }

};

const inputCls = "block w-full px-3 py-2.5 text-sm text-gray-900 bg-white border
border-gray-300 rounded-lg focus:outline-none focus:ring-2 focus:ring-black
focus:border-transparent transition-all placeholder-gray-400";

if(cartLoading || loading) {

  return (

    <div className="min-h-screen bg-gray-50 flex items-center justify-center"
style={{ fontFamily: 'Inter, sans-serif' }}>

      <div className="w-8 h-8 border-2 border-gray-900 border-t-transparent
rounded-full animate-spin" />

    </div>

  );

}

if(items.length === 0) { navigate('/cart'); return null; }

```



```

return (
  <div className="min-h-screen bg-gray-50" style={{ fontFamily: 'Inter, sans-serif'
}}>
    <div className="px-4 py-10 mx-auto max-w-6xl sm:px-6 lg:px-8">

      {/* Header */}

      <div className="mb-8">
        <h1 className="text-2xl font-bold tracking-tight text-gray-900">Finalizar
Compra</h1>
        <p className="text-sm text-gray-500 mt-1">Revisa tus datos antes de
confirmar el pedido</p>
      </div>

      <div className="lg:grid lg:grid-cols-12 lg:gap-8">

        {/* — Left Column — */}

        <div className="lg:col-span-7 space-y-6">

          {/* Shipping Address */}

          <div className="bg-white border border-gray-200 rounded-2xl shadow-
sm overflow-hidden">
            <div className="px-6 py-5 border-b border-gray-100">
              <div className="flex items-center justify-between">
                <div>

```

```
<h2 className="text-base font-semibold text-gray-900">Dirección  
de Envío</h2>
```

```
<p className="text-xs text-gray-400 mt-0.5">Selecciona o agrega  
una dirección</p>
```

```
</div>
```

```
<button
```

```
  type="button"
```

```
  onClick={() => setShowNewAddressForm(v => !v)}
```

```
  className="inline-flex items-center gap-1.5 px-3 py-1.5 text-xs font-  
medium text-gray-700 bg-white border border-gray-300 rounded-lg hover:bg-  
gray-50 transition-colors"
```

```
>
```

```
<svg className="w-3.5 h-3.5" fill="none" viewBox="0 0 24 24"  
stroke="currentColor"><path strokeLinecap="round" strokeLinejoin="round"  
strokeWidth="2" d="M12 4v16m8-8H4" /></svg>
```

```
{addresses.length === 0 ? 'Agregar dirección' : 'Nueva dirección'}
```

```
</button>
```

```
</div>
```

```
</div>
```

```
<div className="px-6 py-5">
```

```
{addresses.length === 0 && !showNewAddressForm && (
```

```
<div className="text-center py-8 text-gray-400">
```

```
<svg className="w-10 h-10 mx-auto mb-2 text-gray-300" fill="none"  
viewBox="0 0 24 24" stroke="currentColor">
```

```

    <path strokeLinecap="round" strokeLinejoin="round"
strokeWidth="1.5" d="M17.657 16.657L13.414 20.9a2 2 0 01-2.828 0l-4.244-
4.243a8 8 0 1111.314 0z" />

```

```

</svg>

```

```

    <p className="text-sm font-medium text-gray-900">No hay
direcciones guardadas</p>

```

```

    <p className="text-xs text-gray-400 mt-0.5">Agrega una para
continuar</p>

```

```

</div>

```

```

  })

```

```

{addresses.length > 0 && (

```

```

  <div className="space-y-3 mb-5">

```

```

    {addresses.map((addr) => (

```

```

      <div

```

```

        key={addr.id}

```

```

        onClick={() => setSelectedAddress(addr)}

```

```

        className={`flex items-start gap-3 p-4 rounded-xl border cursor-
pointer transition-all duration-150 ${

```

```

          selectedAddress?.id === addr.id ? 'border-black bg-gray-50' :
'border-gray-200 hover:border-gray-400'

```

```

        }}

```

```

      >

```

```

    <div className={`mt-0.5 w-4 h-4 rounded-full border-2 flex-shrink-
0 flex items-center justify-center ${selectedAddress?.id === addr.id ? 'border-black'
: 'border-gray-300'}}>

```

```

        {selectedAddress?.id === addr.id && <div className="w-2 h-2
rounded-full bg-black" />}

      </div>

      <div className="flex-1 min-w-0">

        <div className="flex items-center gap-2">

          <p className="text-sm font-semibold text-gray-
900">{addr.name}</p>

          {addr.isDefault && <span className="inline-flex items-center
px-1.5 py-0.5 rounded text-[10px] font-medium bg-gray-100 text-gray-
500">Predeterminada</span>}

        </div>

        <p className="text-xs text-gray-500 mt-0.5">{addr.street}</p>

        <p className="text-xs text-gray-500">{addr.city}, {addr.state}
{addr.zipCode} · {addr.country}</p>

      </div>

    </div>

  )}

</div>

)}

{showNewAddressForm && (

  <div className={` ${addresses.length > 0 ? 'pt-5 border-t border-gray-
100' : ''}`}>

    <p className="text-xs font-semibold uppercase tracking-widest text-
gray-400 mb-4">Nueva Dirección</p>

    <form onSubmit={handleAddNewAddress}>

      <div className="grid grid-cols-1 gap-4 sm:grid-cols-2">

```

```

    {
      { name: 'name', label: 'Nombre de la Dirección', placeholder: 'Ej:
Casa, Trabajo' },

      { name: 'street', label: 'Calle y Número', placeholder: 'Av. Principal
123' },

      { name: 'city', label: 'Ciudad', placeholder: 'Bogotá' },

      { name: 'state', label: 'Departamento', placeholder: 'Cundinamarca'
},

      { name: 'zipCode', label: 'Código Postal', placeholder: '110111' },
    ].map(({ name, label, placeholder }) => (
      <div key={name}>

        <label htmlFor={name} className="block text-xs font-semibold
text-gray-600 uppercase tracking-wide mb-1.5">{label}</label>

        <input type="text" name={name} id={name}
value={newAddress[name]} onChange={handleAddressChange}
placeholder={placeholder} className={inputCls} />

      </div>
    )))

    <div>

      <label className="block text-xs font-semibold text-gray-600
uppercase tracking-wide mb-1.5">País</label>

      <input type="text" value="Colombia" disabled
className={` ${inputCls} bg-gray-50 text-gray-400`} />

      <p className="mt-1 text-xs text-gray-400">Solo operamos en
Colombia</p>

    </div>

  </div>

```

```

    <div className="flex gap-2 mt-5">

      <button type="submit" className="px-4 py-2 text-sm font-
medium text-white bg-black rounded-lg hover:bg-gray-800 transition-
colors">Guardar Dirección</button>

      <button type="button" onClick={() =>
setShowNewAddressForm(false)} className="px-4 py-2 text-sm font-medium text-
gray-700 bg-white border border-gray-300 rounded-lg hover:bg-gray-50
transition-colors">Cancelar</button>

    </div>

  </form>

</div>

)}

</div>

</div>

```

```

{ /* Order Items */ }

<div className="bg-white border border-gray-200 rounded-2xl shadow-
sm overflow-hidden">

  <div className="px-6 py-5 border-b border-gray-100">

    <h2 className="text-base font-semibold text-gray-900">Productos del
Pedido</h2>

  </div>

  <ul className="divide-y divide-gray-100">

    {items.map((item) => (

      <li key={item.id} className="px-6 py-4 flex gap-4">

        <div className="flex-shrink-0 w-16 h-16 rounded-xl border border-
gray-200 overflow-hidden bg-gray-50">

```

```

    <img
      src={getPrimaryImageUrl(item)} alt={item.name}
      className="w-full h-full object-contain"
      onError={(e) => { e.target.onerror = null; e.target.src =
'https://via.placeholder.com/64x64.png?text=PSG'; }}
    />
  </div>

  <div className="flex-1 min-w-0">
    <div className="flex items-start justify-between gap-2">
      <p className="text-sm font-semibold text-gray-900 line-clamp-2
leading-snug">{item.name}</p>

      <p className="text-sm font-bold text-gray-900 flex-shrink-
0">${parseFloat(item.price * item.quantity).toLocaleString('es-CO')}</p>
    </div>

    <p className="text-xs text-gray-400 mt-0.5">{item.category} · Cant:
{item.quantity}</p>
  </div>
</li>

  )}
</ul>
</div>
</div>

{ /* — Right Column: Summary + Payment — */ }
<div className="mt-6 lg:mt-0 lg:col-span-5">

```

```
<div className="bg-white border border-gray-200 rounded-2xl shadow-sm p-6 sticky top-20">
```

```
<h2 className="text-base font-semibold text-gray-900 mb-5">Resumen del Pedido</h2>
```

```
{/* Totals */}
```

```
<div className="space-y-3 mb-5">
```

```
<div className="flex justify-between text-sm">
```

```
<span className="text-gray-500">Subtotal</span>
```

```
<span className="font-medium text-gray-900">${parseFloat(subtotal).toLocaleString('es-CO')}</span>
```

```
</div>
```

```
{coupon && (
```

```
<div className="flex justify-between text-sm">
```

```
<span className="text-gray-500">Descuento ({coupon.code})</span>
```

```
<span className="font-medium text-green-600">- ${parseFloat(discount).toLocaleString('es-CO')}</span>
```

```
</div>
```

```
)}
```

```
<div className="flex justify-between text-sm">
```

```
<span className="text-gray-500">Envío</span>
```

```
<span className="font-medium text-gray-900">${parseFloat(shippingCost).toLocaleString('es-CO')}</span>
```

```
</div>
```

```
<div className="flex justify-between pt-3 border-t border-gray-200">
```



```
        <span className="text-base font-semibold text-gray-900">Total</span>
```

```
        <span className="text-base font-bold text-gray-900">${parseFloat(total).toLocaleString('es-CO')}</span>
```

```
    </div>
```

```
</div>
```

```
{/* Selected Address Preview */}
```

```
{selectedAddress && (
```

```
    <div className="mb-5 p-3 bg-gray-50 rounded-xl border border-gray-100">
```

```
        <p className="text-[10px] font-semibold uppercase tracking-widest text-gray-400 mb-1.5">Enviar a</p>
```

```
        <p className="text-xs font-semibold text-gray-900">{selectedAddress.name}</p>
```

```
        <p className="text-xs text-gray-500">{selectedAddress.street}</p>
```

```
        <p className="text-xs text-gray-500">{selectedAddress.city},  
{selectedAddress.state}</p>
```

```
    </div>
```

```
)}
```

```
{/* Payment Methods */}
```

```
<div>
```

```
    <p className="text-xs font-semibold uppercase tracking-widest text-gray-400 mb-3">Método de Pago</p>
```

```
{/* Stripe/Card Payment */}
```

```

<div className="mb-3">

  {selectedAddress ? (

    <button

      onClick={() => handleStripeCheckout()}

      disabled={processing}

      className="w-full flex items-center justify-center gap-3 px-4 py-3
bg-indigo-600 rounded-xl text-sm font-black text-white hover:bg-indigo-700
transition-all hover:shadow-xl active:scale-95 disabled:opacity-50"

    >

      {processing ? (

        <div className="w-4 h-4 border-2 border-white border-t-
transparent rounded-full animate-spin" />

        ) : (

          <svg className="w-5 h-5" fill="none" viewBox="0 0 24 24"
stroke="currentColor">

            <path strokeLinecap="round" strokeLinejoin="round"
strokeWidth="2.5" d="M3 10h18M7 15h1m4 0h1m-7 4h12a2 2 0 02-2V7a2 2 0
00-2-2H6a2 2 0 02-2v10a2 2 0 02 2z" />

          </svg>

        )}

        {processing ? 'Procesando...' : 'Pagar con Tarjeta (Stripe)'}

      </button>

    ) : null}

  </div>

  {/* PayPal */}

```

```

<div className="mb-3">
  {selectedAddress ? (
    <PayPalButtons
      style={{ layout: 'vertical', shape: 'rect', label: 'pay' }}
      createOrder={({data, actions}) => actions.order.create({
        purchase_units: [{ amount: { value: convertToUSD(total).toFixed(2),
currency_code: 'USD' } }]
      })}
      onApprove={handlePayPalSuccess}
      onError={handlePayPalError}
    />
  ) : (
    <div className="py-3 px-4 border border-gray-200 rounded-xl text-
center">
      <p className="text-xs text-gray-400">Selecciona una dirección para
activar el pago</p>
    </div>
  )}
</div>

```

```

{/* Cash on Delivery – local only */}
{isLocalDelivery(selectedAddress) && (
  <button
    onClick={handleCashOnDelivery}
    disabled={processing || !selectedAddress}

```

```

        className="w-full flex items-center justify-center gap-2 px-4 py-3
border border-gray-300 rounded-xl text-sm font-medium text-gray-700 bg-white
hover:bg-gray-50 transition-colors disabled:opacity-40"

    >

    {processing ? (

        <div className="w-4 h-4 border-2 border-gray-400 border-t-
transparent rounded-full animate-spin" />

        ) : (

            <svg className="w-4 h-4 text-gray-600" fill="none" viewBox="0 0
24 24" stroke="currentColor">

                <path strokeLinecap="round" strokeLinejoin="round"
strokeWidth="1.5" d="M12 8c-1.657 0-3 .895-3 2s1.343 2 3 2 3 .895 3 2-1.343 2-3
2m0-8c1.11 0 2.08.402 2.599 1M12 8V7m0 1v8m0 0v1m0-1c-1.11 0-2.08-.402-
2.599-1M21 12a9 9 0 11-18 0 9 9 0 0118 0z" />

            </svg>

        )}

    {processing ? 'Procesando...' : 'Pagar Contra Entrega'}

</button>

)}

{/* Info for non-local */}

{!isLocalDelivery(selectedAddress) && selectedAddress && (

    <div className="mt-3 p-3 bg-blue-50 border border-blue-100
rounded-xl">

        <p className="text-xs text-blue-700 text-center">El pago contra
entrega solo está disponible en Cajamarca. Usa PayPal para otras ubicaciones.</p>

    </div>

```

```

    })
  </div>

  <button
    onClick={() => navigate('/cart')}
    className="mt-5 flex items-center justify-center gap-1 w-full text-xs
text-gray-400 hover:text-black transition-colors"
  >
    <svg className="w-3.5 h-3.5" fill="none" viewBox="0 0 24 24"
stroke="currentColor"> <path strokeLinecap="round" strokeLinejoin="round"
strokeWidth="2" d="M15 19l-7-7 7-7" /> </svg>
    Volver al Carrito
  </button>
</div>
</div>
</div>
</div>
</div>
);
};

export default Checkout;

```

src/assets/pages/CheckoutCancel.jsx

```

import React, { useEffect } from 'react';
import { useNavigate } from 'react-router-dom';
import Swal from 'sweetalert2';

```

```

const CheckoutCancel = () => {
  const navigate = useNavigate();

  useEffect(() => {
    Swal.fire({
      title: 'Pago cancelado',
      text: 'El pago ha sido cancelado. Puedes intentar nuevamente.',
      icon: 'info',
      confirmButtonText: 'Reintentar',
      cancelButtonText: 'Ir al carrito',
      showCancelButton: true,
      confirmButtonColor: '#111827',
      cancelButtonColor: '#6b7280',
    }).then((result) => {
      navigate(result.isConfirmed ? '/checkout' : '/cart');
    });
  }, [navigate]);

  return (
    <div className="min-h-screen bg-gray-50 flex flex-col items-center justify-
center px-4" style={{ fontFamily: 'Inter, sans-serif' }}>
      <div className="text-center max-w-sm">
        <div className="flex items-center justify-center w-16 h-16 rounded-full bg-
gray-100 mx-auto mb-5">

```

```
    <svg className="w-8 h-8 text-gray-500" fill="none" viewBox="0 0 24 24"
stroke="currentColor">
```

```
    <path strokeLinecap="round" strokeLinejoin="round" strokeWidth="1.5"
d="M6 18L18 6M6 6l12 12" />
```

```
  </svg>
```

```
</div>
```

```
    <h1 className="text-xl font-bold text-gray-900 mb-2">Pago
cancelado</h1>
```

```
    <p className="text-sm text-gray-500 mb-6">No se realizó ningún cargo.
Serás redirigido automáticamente.</p>
```

```
    <div className="w-8 h-8 border-2 border-gray-400 border-t-transparent
rounded-full animate-spin mx-auto" />
```

```
  </div>
```

```
</div>
```

```
);
```

```
};
```

```
export default CheckoutCancel;
```

```
📁 src/assets/pages/contact.jsx
```

```
import React, { useState } from 'react';
```

```
import { useForm, ValidationError } from '@formspring/react';
```

```
import { Link } from 'react-router-dom';
```

```
const Contact = () => {
```

```
  const [state, handleSubmit] = useForm("mpwybkly");
```

```
  const [formData, setFormData] = useState({
```

```
    name: '',
```

```

email: "",
subject: "",
message: ""
});

```

```

const handleInputChange = (e) => {
  const { name, value } = e.target;
  setFormData(prev => ({ ...prev, [name]: value }));
};

```

```

const inputCls = "block w-full px-4 py-3 text-sm text-gray-900 bg-white border
border-gray-300 rounded-xl focus:outline-none focus:ring-2 focus:ring-black
focus:border-transparent transition-all duration-150 placeholder-gray-400";

```

```

const contactItems = [
  {
    icon: (
      <svg className="w-5 h-5 text-gray-600" fill="none" viewBox="0 0 24 24"
stroke="currentColor">
        <path strokeLinecap="round" strokeLinejoin="round" strokeWidth="1.5"
d="M3 5a2 2 0 012-2h3.28a1 1 0 01.948 1.498 4.493a1 1 0 01-.502 1.211-2.257
1.13a11.042 11.042 0 005.516 5.516 11.13-2.257a1 1 0 011.21-.502 14.493 1.498a1 1 0
01.684 9.49V19a2 2 0 01-2 2h-1C9.716 21 3 14.284 3 6V5z" />
      </svg>
    ),
    label: "Teléfono",

```



```

value: "(123) 456-7890",
sub: "Lun – Vie, 9:00 AM – 6:00 PM",
},
{
  icon: (
    <svg className="w-5 h-5 text-gray-600" fill="none" viewBox="0 0 24 24"
stroke="currentColor">
      <path strokeLinecap="round" strokeLinejoin="round" strokeWidth="1.5"
d="M3 8l7.89 5.26a2 2 0 02.22 0L21 8M5 19h14a2 2 0 02-2V7a2 2 0 02-2-2H5a2
2 0 02-2 2v10a2 2 0 02 2z" />
    </svg>
  ),
  label: "Email",
  value: "psgmoños@gmail.com",
  sub: "Respuesta en 24 horas hábiles",
},
{
  icon: (
    <svg className="w-5 h-5 text-gray-600" fill="none" viewBox="0 0 24 24"
stroke="currentColor">
      <path strokeLinecap="round" strokeLinejoin="round" strokeWidth="1.5"
d="M17.657 16.657L13.414 20.9a1.998 1.998 0 01-2.827 0l-4.244-4.243a8 8 0
1111.314 0z" />
      <path strokeLinecap="round" strokeLinejoin="round" strokeWidth="1.5"
d="M15 11a3 3 0 11-6 0 3 3 0 016 0z" />
    </svg>
  ),

```

```

    label: "Ubicación",
    value: "Cajamarca, Tolima",
    sub: "Colombia",
  },
];

```

```

const schedule = [
  { day: "Lunes – Viernes", hours: "9:00 AM – 6:00 PM", open: true },
  { day: "Sábado", hours: "10:00 AM – 4:00 PM", open: true },
  { day: "Domingo", hours: "Cerrado", open: false },
];

```

```

const faqs = [
  { q: "¿Cuánto tiempo tardan en responder?", a: "Respondemos todos los mensajes dentro de las 24 horas hábiles." },
  { q: "¿Ofrecen envío internacional?", a: "Sí, enviamos a varios países. Los costos y tiempos varían según la ubicación." },
  { q: "¿Puedo cambiar o devolver un producto?", a: "Ofrecemos devoluciones dentro de los 30 días posteriores a la compra." },
  { q: "¿Cómo puedo rastrear mi pedido?", a: "Recibirás un email con el número de seguimiento una vez enviado tu pedido." },
];

```

```

return (
  <div className="min-h-screen bg-white" style={{ fontFamily: 'Inter, sans-serif' }}>

```

```
{/*—— Hero ——*/}
```

```
<section className="bg-gray-900 relative overflow-hidden">
```

```
  <div className="absolute inset-0 opacity-5" style={{ backgroundImage:
'radial-gradient(circle at 2px 2px, white 1px, transparent 0)', backgroundSize: '28px
28px' }} />
```

```
  <div className="relative z-10 max-w-4xl mx-auto px-4 py-20 sm:px-6 lg:px-8
text-center">
```

```
    <p className="text-xs font-semibold uppercase tracking-widest text-gray-
400 mb-4">Contacto</p>
```

```
    <h1 className="text-4xl md:text-5xl font-black text-white mb-5 leading-
tight">
```

```
      Hablemos
```

```
    </h1>
```

```
    <p className="text-lg text-gray-400 max-w-xl mx-auto">
```

```
      ¿Tienes preguntas? Estamos aquí para ayudarte. Envíanos un mensaje y nos
pondremos en contacto contigo pronto.
```

```
    </p>
```

```
  </div>
```

```
</section>
```

```
{/*—— Body ——*/}
```

```
<section className="max-w-7xl mx-auto px-4 py-14 sm:px-6 lg:px-8">
```

```
{/* Success banner */}
```

```
{state.succeeded && (
```

```

    <div className="mb-8 flex gap-3 items-start p-4 bg-green-50 border
border-green-200 rounded-2xl">

      <svg className="w-5 h-5 text-green-500 flex-shrink-0 mt-0.5"
fill="currentColor" viewBox="0 0 20 20">

        <path fillRule="evenodd" d="M10 18a8 8 0 100-16 8 8 0 00 16zm3.707-
9.293a1 1 0 00-1.414-1.414L9 10.586 7.707 9.293a1 1 0 00-1.414 1.414l2 2a1 1 0
001.414 0l4-4z" clipRule="evenodd" />

      </svg>

    <div>

      <p className="text-sm font-semibold text-green-800">¡Mensaje
enviado!</p>

      <p className="text-sm text-green-700 mt-0.5">Hemos recibido tu
mensaje y nos pondremos en contacto pronto.</p>

    </div>

  </div>

)}

```

```

<div className="grid grid-cols-1 gap-10 lg:grid-cols-3">

  { /* — Contact Form — */ }

  <div className="lg:col-span-2">

    <div className="bg-white border border-gray-200 rounded-2xl p-8
shadow-sm">

      <span className="text-xs font-semibold uppercase tracking-widest text-
gray-400">Formulario</span>

      <h2 className="text-2xl font-bold text-gray-900 mt-2 mb-7">Envíanos
un mensaje</h2>

```

```

<form onSubmit={handleSubmit} className="space-y-5">
  <div className="grid grid-cols-1 gap-5 sm:grid-cols-2">
    <div>
      <label htmlFor="name" className="block text-xs font-semibold text-
gray-600 uppercase tracking-wide mb-1.5">
        Nombre
      </label>
      <input
        type="text"
        name="name"
        id="name"
        required
        value={formData.name}
        onChange={handleInputChange}
        className={inputCls}
        placeholder="Tu nombre completo"
      />
    </div>
    <div>
      <label htmlFor="email" className="block text-xs font-semibold text-
gray-600 uppercase tracking-wide mb-1.5">
        Email
      </label>
      <input

```

```

    type="email"
    name="email"
    id="email"
    required
    value={formData.email}
    onChange={handleInputChange}
    className={inputCls}
    placeholder="tu@email.com"
  />

  <ValidationError prefix="Email" field="email" errors={state.errors}
  className="mt-1 text-xs text-red-600" />
</div>
</div>

<div>
  <label htmlFor="subject" className="block text-xs font-semibold text-
  gray-600 uppercase tracking-wide mb-1.5">
    Asunto
  </label>
  <input
    type="text"
    name="subject"
    id="subject"
    required
    value={formData.subject}

```

```

        onChange={handleInputChange}

        className={inputCls}

        placeholder="¿Sobre qué te gustaría hablar?"

    />
</div>

<div>

    <label htmlFor="message" className="block text-xs font-semibold
text-gray-600 uppercase tracking-wide mb-1.5">

        Mensaje

    </label>

    <textarea

        id="message"

        name="message"

        rows={6}

        required

        value={formData.message}

        onChange={handleInputChange}

        className={` ${inputCls} resize-none`}

        placeholder="Escribe tu mensaje aquí..."

    />

    <ValidationError prefix="Message" field="message"
errors={state.errors} className="mt-1 text-xs text-red-600" />

</div>

```

```

<button
  type="submit"
  disabled={state.submitting}

  className="w-full py-3 px-6 bg-black text-white text-sm font-medium
rounded-xl hover:bg-gray-800 active:bg-gray-900 transition-colors duration-150
focus:outline-none focus:ring-2 focus:ring-offset-2 focus:ring-black
disabled:opacity-40"

  >

  {state.submitting ? 'Enviando...' : 'Enviar Mensaje'}

</button>

</form>

</div>

```

```

{ /* — FAQ — */

<div className="mt-8 bg-white border border-gray-200 rounded-2xl p-8
shadow-sm">

  <span className="text-xs font-semibold uppercase tracking-widest text-
gray-400">FAQ</span>

  <h2 className="text-2xl font-bold text-gray-900 mt-2 mb-7">Preguntas
Frecuentes</h2>

  <div className="grid grid-cols-1 gap-4 md:grid-cols-2">

    {faqs.map((faq, i) => (

      <div key={i} className="p-5 bg-gray-50 border border-gray-100
rounded-xl hover:border-gray-300 transition-colors duration-150">

        <h3 className="text-sm font-semibold text-gray-900 mb-
2">{faq.q}</h3>

        <p className="text-sm text-gray-500 leading-relaxed">{faq.a}</p>

```



```

    </div>

  )}

</div>

</div>

</div>

{ /* — Sidebar: Info + Hours + Social — */ }

<div className="space-y-6">

  { /* Contact info */ }

  <div className="bg-white border border-gray-200 rounded-2xl p-6 shadow-sm">

    <span className="text-xs font-semibold uppercase tracking-widest text-gray-400">Información</span>

    <h3 className="text-lg font-bold text-gray-900 mt-1 mb-5">Cómo contactarnos</h3>

    <div className="space-y-5">

      {contactItems.map((item, i) => (

        <div key={i} className="flex gap-3">

          <div className="flex items-center justify-center w-10 h-10 bg-gray-100 rounded-xl flex-shrink-0">

            {item.icon}

          </div>

          <div>

            <p className="text-xs font-medium text-gray-400 mb-0.5">{item.label}</p>

```

```

        <p className="text-sm font-semibold text-gray-
900">{item.value}</p>

        <p className="text-xs text-gray-500">{item.sub}</p>

    </div>

</div>

)}}

</div>

</div>

```

```

{ /* Map */

    <div className="bg-white border border-gray-200 rounded-2xl overflow-
hidden shadow-sm">

        <div className="px-6 pt-6 pb-3">

            <span className="text-xs font-semibold uppercase tracking-widest
text-gray-400">Ubicación</span>

            <h3 className="text-base font-bold text-gray-900 mt-1">Dónde
encontrarnos</h3>

        </div>

        <div className="h-48 overflow-hidden">

            <iframe

                src="https://www.google.com/maps/embed?pb=!1m18!1m12!1m3!1d1
5782.9116518304!2d-
75.31303365!3d4.48527395!2m3!1f0!2f0!3f0!3m2!1i1024!2i768!4f13.1!3m3!1m2!1s0
x8e38f5a5a5a5a5a5%3A0x5a5a5a5a5a5a5a5a!2sCajamarca%2C%20Tolima%2C%20
Colombia!5e0!3m2!1sen!2sco!4v1678901234567!5m2!1sen!2sco"

                width="100%"

                height="100%"

```

```

    style={{ border: 0 }}

    allowFullScreen=""

    loading="lazy"

    referrerPolicy="no-referrer-when-downgrade"

    title="Ubicación PSG SHOP – Cajamarca, Tolima, Colombia"

  />
</div>

<div className="px-6 py-3 border-t border-gray-100">

  <p className="text-xs text-gray-500">Cajamarca, Tolima ·
Colombia</p>

</div>

</div>

{ /* Business hours */ }

<div className="bg-white border border-gray-200 rounded-2xl p-6
shadow-sm">

  <span className="text-xs font-semibold uppercase tracking-widest text-
gray-400">Horario</span>

  <h3 className="text-base font-bold text-gray-900 mt-1 mb-4">Atención
al cliente</h3>

  <ul className="space-y-3">

    {schedule.map((s, i) => (

      <li key={i} className="flex items-center justify-between text-sm pb-3
border-b border-gray-100 last:border-0 last:pb-0">

        <span className="text-gray-600">{s.day}</span>

```

```
<span className={`font-medium ${s.open ? 'text-gray-900' : 'text-gray-400'}`}>{s.hours}</span>
```

```
</li>
```

```
)))
```

```
</ul>
```

```
</div>
```

```
{/* Social media */}
```

```
<div className="bg-gray-900 rounded-2xl p-6 relative overflow-hidden">
```

```
<div className="absolute inset-0 opacity-5" style={{ backgroundImage: 'radial-gradient(circle at 2px 2px, white 1px, transparent 0)', backgroundSize: '20px 20px' }} />
```

```
<div className="relative z-10">
```

```
<h3 className="text-base font-bold text-white mb-1">Síguenos en redes</h3>
```

```
<p className="text-sm text-gray-400 mb-5">Mantente al día con nuestras novedades y promociones</p>
```

```
<div className="flex gap-3">
```

```
{[
```

```
  { label: "Twitter", path: "M24 4.557c-.883.392-1.832.656-2.828.775 1.017-.609 1.798-1.574 2.165-2.724-.951.564-2.005.974-3.127 1.195-.897-.957-2.178-1.555-3.594-1.555-3.179 0-5.515 2.966-4.797 6.045-4.091-.205-7.719-2.165-10.148-5.144-1.29 2.213-.669 5.108 1.523 6.574-.806-.026-1.566-.247-2.229-.616-.054 2.281 1.581 4.415 3.949 4.89-.693.188-1.452.232-2.224.084.626 1.956 2.444 3.379 4.6 3.419-2.07 1.623-4.678 2.348-7.29 2.04 2.179 1.397 4.768 2.212 7.548 2.212 9.142 0 14.307-7.721 13.995-14.646.962-.695 1.797-1.562 2.457-2.549z" },
```

```
{ label: "Facebook", path: "M9 8h-3v4h3v12h5v-12h3.642l.358-4h-4v-1.667c0-.955.192-1.333 1.115-1.333h2.885v-5h-3.808c-3.596 0-5.192 1.583-5.192 4.615v3.385z" },
```

```
{ label: "Instagram", path: "M12 2.163c3.204 0 3.584.012 4.85.07 3.252.148 4.771 1.691 4.919 4.919.058 1.265.069 1.645.069 4.849 0 3.205-.012 3.584-.069 4.849-.149 3.225-1.664 4.771-4.919 4.919-1.266.058-1.644.07-4.85.07-3.204 0-3.584-.012-4.849-.07-3.26-.149-4.771-1.699-4.919-4.92-.058-1.265-.07-1.689-.07-4.849 0-3.204.013-3.583.07-4.849.149-3.227 1.664-4.771 4.919-4.919 1.266-.057 1.689-.072 4.849-.072zm0-2.163c-3.259 0-3.667.014-4.947.072-4.358.2-6.78 2.618-6.98 6.98-.059 1.281-.073 1.689-.073 4.948 0 3.259.014 3.668.072 4.948.2 4.358 2.618 6.78 6.98 6.98 1.281.058 1.689.072 4.948.072 3.259 0 3.668-.014 4.948-.072 4.354-.2 6.782-2.618 6.979-6.98.059-1.28.073-1.689.073-4.948 0-3.259-.014-3.667-.072-4.947-.196-4.354-2.617-6.78-6.979-6.98-1.281-.059-1.69-.073-4.949-.073zm0 5.838c-3.403 0-6.162 2.759-6.162 6.162s2.759 6.163 6.162 6.162-2.759 6.162-6.163c0-3.403-2.759-6.162-6.162-6.162zm0 10.162c-2.209 0-4-1.79-4-4 0-2.209 1.791-4 4-4s4 1.791 4 4c0 2.21-1.791 4-4 4zm6.406-11.845c-.796 0-1.441.645-1.441 1.44s.645 1.44 1.441 1.44c.795 0 1.439-.645 1.439-1.44s-.644-1.44-1.439-1.44z" },
```

```
].map((social) => (
  <a
    key={social.label}
    href="#"
    aria-label={social.label}
    className="w-9 h-9 flex items-center justify-center rounded-full
border border-gray-700 text-gray-400 hover:border-white hover:text-white
transition-all duration-150"
  >
    <svg className="w-4 h-4" fill="currentColor" viewBox="0 0 24 24">
      <path d={social.path} />
    </svg>
```



```

/*----- Product Card-----*/

const ProductCard = ({ product, getPrimaryImageUrl, onAddToCart,
onAddToWishlist }) => {

  const [wishlist, setWishlisted] = useState(false);

  const handleWishlist = (e) => {

    e.preventDefault();

    e.stopPropagation();

    setWishlisted(true);

    onAddToWishlist(product);

    setTimeout(() => setWishlisted(false), 2000);

  };

  const outOfStock = product.stock === 0;

  const lowStock = product.stock !== undefined && product.stock > 0 &&
product.stock <= 5;

  return (

    <div className="group relative bg-white border border-gray-100 rounded-2xl
overflow-hidden shadow-sm hover:shadow-xl hover:-translate-y-1 transition-all
duration-300 flex flex-col">

      {/* Image container */}

      <Link to={`/product/${product.id}`} className="block relative overflow-hidden
aspect-square bg-gray-50 flex-shrink-0">

        <img

```

```

src={getPrimaryImageUrl(product)}

alt={product.name}

className="object-cover w-full h-full group-hover:scale-105 transition-
transform duration-500 ease-out"

onError={(e) => { e.target.onerror = null; e.target.src =
'https://via.placeholder.com/400x400.png?text=PSG'; }}

/>

{/* Gradient overlay on hover */}

<div className="absolute inset-0 bg-gradient-to-t from-black/30 via-
transparent to-transparent opacity-0 group-hover:opacity-100 transition-opacity
duration-300" />

{/* Quick-view label on hover */}

<div className="absolute inset-x-0 bottom-0 flex items-center justify-center
pb-4 opacity-0 group-hover:opacity-100 translate-y-2 group-hover:translate-y-0
transition-all duration-300">

  <span className="px-4 py-1.5 bg-white/90 backdrop-blur-sm text-gray-
900 text-xs font-semibold rounded-full shadow">

    Ver detalles →

  </span>

</div>

{/* Badges */}

<div className="absolute top-3 left-3 flex flex-col gap-1.5">

  {outOfStock && (
```



```
    <span className="inline-flex items-center px-2.5 py-1 rounded-full text-[10px] font-bold uppercase tracking-wide bg-red-600 text-white shadow-sm">
```

```
      Agotado
```

```
    </span>
```

```
  )}
```

```
{lowStock && (
```

```
    <span className="inline-flex items-center px-2.5 py-1 rounded-full text-[10px] font-bold uppercase tracking-wide bg-amber-500 text-white shadow-sm">
```

```
      Solo {product.stock} left
```

```
    </span>
```

```
  )}
```

```
</div>
```

```
{/* Wishlist button */}
```

```
<button
```

```
  className={`absolute top-3 right-3 w-9 h-9 rounded-full flex items-center justify-center shadow-sm transition-all duration-200
```

```
    ${wishlisted
```

```
      ? 'bg-red-500 text-white scale-110'
```

```
      : 'bg-white/80 backdrop-blur-sm text-gray-500 hover:bg-white hover:text-red-500 opacity-0 group-hover:opacity-100 translate-y-1 group-hover:translate-y-0'
```

```
  )}
```

```
  onClick={handleWishlist}
```

```
  aria-label="Agregar a favoritos"
```

```
>
```

```

    <svg className="w-4 h-4" fill={wishlist ? 'currentColor' : 'none'}
viewBox="0 0 24 24" stroke="currentColor" strokeWidth="2">

    <path strokeLinecap="round" strokeLinejoin="round" d="M4.318 6.318a4 4
0 00 6.364L12 20.364l7.682-7.682a4 4 0 00-6.364-6.364L12 7.636l-1.318-1.318a4 4
0 00-6.364 0z" />

    </svg>

</button>

</Link>

```

```

{/* Info */}

<div className="p-4 flex flex-col flex-1">

    {/* Category */}

    <span className="text-[10px] font-bold uppercase tracking-widest text-
indigo-500 mb-1">

        {product.category || 'General'}

    </span>

```

```

    {/* Name */}

    <h3 className="text-sm font-semibold text-gray-900 leading-snug mb-2
line-clamp-2 flex-1">

        {product.name}

    </h3>

```

```

    {/* Rating */}

    {product.rating > 0 && (

        <div className="mb-2">

```

```

    <StarRating rating={product.rating} size="sm" />

  </div>

)}

{/* Price + Add to cart */}

<div className="flex items-center justify-between mt-auto pt-3 border-t
border-gray-50">

  <span className="text-base font-extrabold text-gray-900 tracking-tight">

    ${parseFloat(product.price).toLocaleString('es-CO')}

  </span>

  <button

    className={`flex items-center gap-1.5 px-3.5 py-2 rounded-xl text-xs font-
semibold transition-all duration-150

    ${outOfStock

      ? 'bg-gray-100 text-gray-400 cursor-not-allowed'

      : 'bg-gray-900 text-white hover:bg-gray-700 active:scale-95 shadow-sm
hover:shadow'

    }}

    disabled={outOfStock}

    onClick={(e) => { e.preventDefault(); onAddToCart(product); }}

  >

    {outOfStock ? (

      'Agotado'

    ) : (

      <>

```

```

        <svg className="w-3.5 h-3.5" fill="none" viewBox="0 0 24 24"
stroke="currentColor" strokeWidth="2.2">

            <path strokeLinecap="round" strokeLinejoin="round" d="M3 3h2l.4
2M7 13h10l4-8H5.4M7 13L5.4 5M7 13l-2.293 2.293c-.63.63-.184 1.707.707
1.707H17m0 0a2 2 0 100 4 2 2 0 000-4zm-8 2a2 2 0 11-4 0 2 2 0 014 0z" />

        </svg>

        Agregar

    </>

    )}

</button>

</div>

</div>

</div>

);

};

```

```

const Home = () => {

    const [featuredProducts, setFeaturedProducts] = useState([]);

    const [categoryList, setCategoryList] = useState([]);

    const [categoriesLoading, setCategoriesLoading] = useState(true);

    const [loading, setLoading] = useState(true);

    const [error, setError] = useState(null);

    const { addToCart } = useCart();

    const { addToWishlist } = useWishlist();

    const { currentUser } = useAuth();

```

```
// Detección de host para rutas de imágenes locales

const isGitHubPages = window.location.hostname.includes('github.io');
const assetBase = isGitHubPages ? '/psg-official/' : '/';


// State for testimonials slider

const [currentTestimonial, setCurrentTestimonial] = useState(0);


// Refs for touch events

const sliderRef = useRef(null);
const startXRef = useRef(0);
const endXRef = useRef(0);
const isDraggingRef = useRef(false);


// Formspree form setup

const [state, handleSubmit] = useForm("mpwybkly");


// State for contact form

const [formData, setFormData] = useState({
  name: "",
  email: "",
  subject: "",
  message: ""
});
```

```

useEffect(() => {
  const fetchData = async () => {
    try{
      // Fetch featured products
      setLoading(true);
      const productList = await getProducts();
      // Get top 8 products with highest ratings or randomly select if no ratings
      const sortedProducts = [...productList]
        .sort((a, b) => (b.rating || 0) - (a.rating || 0))
        .slice(0, 8);
      setFeaturedProducts(sortedProducts);

      // Fetch categories
      setCategoriesLoading(true);
      const categoriesList = await getCategories();
      setCategoryList(categoriesList);
    } catch (err) {
      setError('Failed to load data');
      console.error('Error fetching data:', err);
    } finally{
      setLoading(false);
      setCategoriesLoading(false);
    }
  }
}

```

```
};
```

```
fetchData();
```

```
}, []);
```

```
// Testimonials data - Potenciado con más detalles
```

```
const testimonials = [
```

```
{
```

```
  id: 1,
```

```
  name: "María González",
```

```
  role: "Cliente Verificada",
```

```
  title: "¡Calidad insuperable!",
```

```
  text: "Los moños son de excelente calidad y el envío fue increíblemente rápido.  
Cada detalle está cuidado con amor. ¡Definitivamente volveré a comprar!",
```

```
  rating: 5,
```

```
  date: "Hace 2 semanas"
```

```
},
```

```
{
```

```
  id: 2,
```

```
  name: "Carlos Rodríguez",
```

```
  role: "Padre Satisfecho",
```

```
  title: "Excelente para regalos",
```

```
  text: "Compré un set para mi hija y quedó encantada. La atención al cliente es  
personalizada y muy humana. Muy recomendado para ocasiones especiales.",
```

```
  rating: 5,
```

```

    date: "Hace 1 mes"
  },
  {
    id: 3,
    name: "Ana Martínez",
    role: "Estilista Profesional",
    title: "Diseños únicos",
    text: "Como estilista, busco accesorios que destaquen. Estos moños tienen diseños que no se encuentran en tiendas convencionales. La tela es premium.",
    rating: 5,
    date: "Hace 3 días"
  }
];

```

// Auto-rotate testimonials

```

useEffect(() => {
  const interval = setInterval(() => {
    setCurrentTestimonial((prev) => (prev + 1) % testimonials.length);
  }, 5000);
  return () => clearInterval(interval);
}, [testimonials.length]);

```

// Handle form input changes

```

const handleInputChange = (e) => {
  const { name, value } = e.target;

```



```

setFormData(prev => ({
  ...prev,
  [name]: value
}));
};

```

```

// Get the primary image URL for a product
const getPrimaryImageUrl = (product) => {
  if(product.imageUrls && product.imageUrls.length > 0) {
    // Filter out empty images
    const validImages = product.imageUrls.filter(image =>
      image && (typeof image === 'string' || (image.data && typeof image.data
=== 'string'))
    );
    if(validImages.length > 0) {
      const primaryIndex = product.primaryImageIndex || 0;
      // Ensure primaryIndex is within bounds
      const safeIndex = Math.min(primaryIndex, validImages.length - 1);
      const primaryImage = validImages[safeIndex];

      // Handle both string URLs and base64 data objects
      if(typeof primaryImage === 'string') {
        return primaryImage;
      } else if(primaryImage && primaryImage.data) {
        return primaryImage.data;
      }
    }
  }
};

```

```

    }
  }
}

return product.imageUrl ||
'https://via.placeholder.com/300x300.png?text= Moño';
};

// Function to handle adding to cart with authentication check
const handleAddToCart = (product) => {
  // Check if user is logged in
  if(!currentUser) {
    Swal.fire({
      title: 'Inicia sesión',
      text: 'Debes iniciar sesión para agregar productos al carrito',
      icon: 'warning',
      confirmButtonText: 'Iniciar sesión',
      showCancelButton: true,
      cancelButtonText: 'Cancelar'
    }).then((result) => {
      if(result.isConfirmed) {
        // Redirect to login page with hash routing
        window.location.href = '/psg-official/#/login';
      }
    });
  }
  return;
};

```

```
}
```

```
// Check if product has stock
```

```
const stockLimit = product.stock !== undefined ? product.stock : Infinity;
```

```
if(stockLimit === 0) {
```

```
  Swal.fire({
```

```
    title: 'Producto agotado',
```

```
    text: 'Este producto no está disponible actualmente.',
```

```
    icon: 'error',
```

```
    confirmButtonText: 'Aceptar'
```

```
  });
```

```
  return;
```

```
}
```

```
// Get the primary product image as selected in the admin panel
```

```
const mainImage = getPrimaryImageUrl(product);
```

```
addToCart({
```

```
  ...product,
```

```
  imageUrl: mainImage, // Ensure we use the primary image selected in admin panel
```

```
  quantity: 1
```

```
});
```

```
Swal.fire({
```

```

    title: 'Producto agregado',
    text: 'Producto agregado correctamente al carrito',
    icon: 'success',
    confirmButtonText: 'Aceptar'
  });
};

// Function to handle adding to wishlist with authentication check
const handleAddToWishlist = (product) => {
  // Check if user is logged in
  if(!currentUser) {
    Swal.fire({
      title: 'Inicia sesión',
      text: 'Debes iniciar sesión para agregar productos a tu lista de deseos',
      icon: 'warning',
      confirmButtonText: 'Iniciar sesión',
      showCancelButton: true,
      cancelButtonText: 'Cancelar'
    }).then((result) => {
      if(result.isConfirmed) {
        // Redirect to login page with hash routing
        window.location.href = '/psg-official/#/login';
      }
    });
  }
};

```

```

    return,
  }

  // Get the primary product image as selected in the admin panel
  const mainImage = getPrimaryImageUrl(product);

  addToWishlist({
    ...product,
    imageUrl: mainImage // Ensure we use the primary image selected in admin
    panel
  });

  Swal.fire({
    title: 'Producto agregado',
    text: 'Producto agregado correctamente a tu lista de deseos',
    icon: 'success',
    confirmButtonText: 'Aceptar'
  });
};

// Features data
const features = [
  {
    id: 1,
    title: "Envío Gratis",

```

description: "En pedidos superiores a \$100.000 COP",

icon: (

```
<svg className="w-8 h-8 text-black uppercase tracking-widest" fill="none"
stroke="currentColor" viewBox="0 0 24 24" xmlns="http://www.w3.org/2000/svg">
```

```
<path strokeLinecap="round" strokeLinejoin="round" strokeWidth="2"
d="M9 12l2 4-4m5.618-4.016A11.955 11.955 0 0112 2.944a11.955 11.955 0 01-
8.618 3.04A12.02 12.02 0 003 9c0 5.591 3.824 10.29 9 11.622 5.176-1.332 9-6.03 9-
11.622 0-1.042-.133-2.052-.382-3.016z"></path>
```

```
</svg>
```

)

},

{

id: 2,

title: "Devolución Gratuita",

description: "30 días para devoluciones sin complicaciones",

icon: (

```
<svg className="w-8 h-8 text-black uppercase tracking-widest" fill="none"
stroke="currentColor" viewBox="0 0 24 24" xmlns="http://www.w3.org/2000/svg">
```

```
<path strokeLinecap="round" strokeLinejoin="round" strokeWidth="2"
d="M4 4v5h.582m15.356 2A8.001 8.001 0 004.582 9m0 0H9m11 11v-5h-.581m0
0a8.003 8.003 0 01-15.357-2m15.357 2H15"></path>
```

```
</svg>
```

)

},

{

id: 3,

title: "Soporte 24/7",

description: "Asistencia dedicada en cualquier momento",

icon: (

```
<svg className="w-8 h-8 text-black uppercase tracking-widest" fill="none"
stroke="currentColor" viewBox="0 0 24 24" xmlns="http://www.w3.org/2000/svg">
```

```
<path strokeLinecap="round" strokeLinejoin="round" strokeWidth="2"
d="M18 9v3m0 0v3m0-3h3m-3 0h-3m-2-5a4 4 0 11-8 0 4 4 0 018 0zM3 20a6 6 0
0112 0v1H3v-1z"></path>
```

```
</svg>
```

)

},

{

id: 4,

title: "Pago Seguro",

description: "Protegemos tus datos de pago",

icon: (

```
<svg className="w-8 h-8 text-black uppercase tracking-widest" fill="none"
stroke="currentColor" viewBox="0 0 24 24" xmlns="http://www.w3.org/2000/svg">
```

```
<path strokeLinecap="round" strokeLinejoin="round" strokeWidth="2"
d="M12 15v2m-6 4h12a2 2 0 02-2v-6a2 2 0 00-2-2H6a2 2 0 00-2 2v6a2 2 0 002
2zm10-10V7a4 4 0 11-8 0v4h8z"></path>
```

```
</svg>
```

)

}

];

// Handle touch events for mobile swipe

```

const handleTouchStart = (e) => {
  startXRef.current = e.touches[0].clientX;
  isDraggingRef.current = true;
};

const handleTouchMove = (e) => {
  if(!isDraggingRef.current) return;
  endXRef.current = e.touches[0].clientX;
};

const handleTouchEnd = () => {
  if(!isDraggingRef.current) return;
  isDraggingRef.current = false;

  const swipeThreshold = 50;

  if(startXRef.current - endXRef.current > swipeThreshold) {
    // Swipe left - next testimonial
    setCurrentTestimonial((prev) => (prev + 1) % testimonials.length);
  } else if(endXRef.current - startXRef.current > swipeThreshold) {
    // Swipe right - previous testimonial
    setCurrentTestimonial((prev) => (prev === 0 ? testimonials.length - 1 : prev -
1));
  }
}

```



```

// Reset values

startXRef.current = 0;

endXRef.current = 0;

};

// Handle mouse events for desktop drag

const handleMouseDown = (e) => {

  startXRef.current = e.clientX;

  isDraggingRef.current = true;

};

const handleMouseMove = (e) => {

  if(!isDraggingRef.current) return;

  endXRef.current = e.clientX;

};

const handleMouseUp = () => {

  if(!isDraggingRef.current) return;

  isDraggingRef.current = false;

const swipeThreshold = 50;

  if(startXRef.current - endXRef.current > swipeThreshold) {

    // Swipe left - next testimonial

```

```

    setCurrentTestimonial((prev) => (prev + 1) % testimonials.length);
  } else if(endXRef.current - startXRef.current > swipeThreshold) {
    // Swipe right - previous testimonial
    setCurrentTestimonial((prev) => (prev === 0 ? testimonials.length - 1 : prev -
1));
  }

  // Reset values
  startXRef.current = 0;
  endXRef.current = 0;
};

const handleMouseLeave = () => {
  isDraggingRef.current = false;
};

return (
  <div className="min-h-screen bg-white">
    {/ * Hero Section - Restored Original Layout, Maximized Aesthetic */}
    <section className="flex flex-col gap-8 p-6 mx-auto lg:flex-row max-w-7xl
animate-fade-in">
      {/ * Bloque principal - Enhanced Original */}
      <div className="flex flex-col md:flex-row items-center justify-between flex-1
p-10 md:p-16 bg-gray-50 rounded-[2.5rem] border border-gray-100 shadow-2xl
shadow-indigo-50/50 group relative overflow-hidden">
        {/ * Subtle background decoration */}

```

```
<div className="absolute -top-24 -left-24 w-64 h-64 bg-indigo-500/5
rounded-full blur-3xl" />
```

```
<div className="relative z-10 max-w-md">
```

```
{/* Etiqueta superior mejorada */}
```

```
<div className="flex flex-wrap items-center gap-3 mb-6">
```

```
<span className="px-4 py-1.5 text-[10px] font-bold tracking-widest text-
indigo-600 bg-white border border-indigo-100 rounded-full shadow-sm">
```

```
NUEVO
```

```
</span>
```

```
<span className="flex items-center gap-1.5 px-4 py-1.5 text-[10px] font-
bold text-gray-500 rounded-full bg-white/80 border border-gray-100 shadow-
sm">
```

```
<svg className="w-3.5 h-3.5 text-indigo-500" fill="none" viewBox="0 0
24 24" stroke="currentColor" strokeWidth="2.5">
```

```
<path strokeLinecap="round" strokeLinejoin="round" d="M5 13l4 4L19
7" />
```

```
</svg>
```

```
¡Envío gratis hoy!
```

```
</span>
```

```
</div>
```

```
{/* Título Premium */}
```

```
<h1 className="mb-6 text-4xl font-bold leading-tight text-gray-900
md:text-5xl tracking-tighter">
```

```
Elegancia en <br />
```

cada detalle perfecto

</h1>

{/ Descripción */}*

<p *className*="mb-10 text-lg text-gray-500 leading-relaxed font-medium">

Colección exclusiva de moños artesanales. Calidad premium, diseño único y entrega inmediata.

</p>

{/ Botón Mejorado */}*

<Link

to="/shop"

className="inline-flex items-center justify-center px-8 py-4 text-sm font-bold tracking-widest text-white transition-all bg-black rounded-2xl hover:bg-indigo-600 hover:shadow-2xl hover:shadow-indigo-200 active:scale-95 group/btn"

>

VER CATÁLOGO

<svg *className*="w-5 h-5 ml-2 group-hover/btn:translate-x-1 transition-transform" *fill*="none" *viewBox*="0 0 24 24" *stroke*="currentColor" *strokeWidth*="2.5">

<path *strokeLinecap*="round" *strokeLinejoin*="round" *d*="M17 8l4 4m0 0l-4 4m4-4H3" />

</svg>

</Link>

</div>

```

    {/ * Imagen principal mejorada */}

    <div className="mt-12 md:mt-0 relative group-hover:rotate-1 transition-
transform duration-700">

        <div className="absolute inset-0 bg-indigo-500 rounded-[2rem] blur-2xl
opacity-10 group-hover:opacity-20 transition-opacity" />

        <img

            src={` ${assetBase}mejores.jpg`}

            alt="Elegante colección"

            className="relative object-cover w-64 md:w-80 h-auto rounded-[2rem]
shadow-2xl grayscale-[20%] group-hover:grayscale-0 transition-all duration-700"

            onError={(e) => {

                e.target.onerror = null;

                e.target.src =
'https://via.placeholder.com/600x600.png?text=Elegante+Colecci%C3%B3n';

            }}

        />

    </div>

</div>

```

```

    {/ * Panel lateral derecho - Enhanced Original Cards */}

    <div className="flex flex-col gap-6 lg:w-1/3">

        {/ * Tarjeta 1: Mejores Moños */}

        <div className="group flex items-center justify-between flex-1 p-8 bg-
indigo-50/50 border border-indigo-100/50 rounded-[2.5rem] shadow-sm
hover:shadow-xl transition-all duration-500 overflow-hidden relative">

```

```

    <div className="absolute top-0 right-0 w-32 h-32 bg-indigo-500/5
rounded-full blur-2xl" />

    <div className="flex-1 relative z-10">
      <h3 className="mb-2 text-2xl font-bold text-gray-900 leading-tight">
        Mejores <span className="text-indigo-600">moños</span>
      </h3>
      <p className="mb-6 text-sm text-gray-500 font-medium">
        Selección artesanal premium.
      </p>
      <Link to="/shop" className="inline-flex items-center gap-2 group/link
text-xs font-bold uppercase tracking-widest text-gray-900">
        Ver más
      <span className="w-8 h-px bg-gray-200 group-hover/link:w-12
transition-all duration-500" />
    </Link>
  </div>
  <img
    src={` ${assetBase}counteer.jpg`}
    alt="Moño"
    className="object-contain w-24 h-24 ml-4 rounded-2xl group-
hover:scale-110 transition-transform duration-500 drop-shadow-2xl"
    onError={(e) => {
      e.target.onerror = null;
      e.target.src = 'https://via.placeholder.com/100x100.png?text=Moño';
    }}
  >

```

/>

</div>

{/ Tarjeta 2: Descuento */}*

<div className="group flex items-center justify-between flex-1 p-8 bg-purple-50/50 border border-purple-100/50 rounded-[2.5rem] shadow-sm hover:shadow-xl transition-all duration-500 overflow-hidden relative">

<div className="absolute bottom-0 right-0 w-32 h-32 bg-purple-500/5 rounded-full blur-2xl" />

<div className="flex-1 relative z-10">

<h3 className="mb-2 text-2xl font-bold text-gray-900 leading-tight">

20% Descuento

</h3>

<p className="mb-6 text-sm text-gray-500 font-medium">

Colección especial limitada.

</p>

<Link to="/shop" className="inline-flex items-center gap-2 group/link text-xs font-bold uppercase tracking-widest text-gray-900">

Ver más

</Link>

</div>

<img

src={` \${assetBase}discount.jpg`}

```

        alt="Oferta"

        className="object-contain w-24 h-24 ml-4 rounded-2xl group-
hover:scale-110 transition-transform duration-500 drop-shadow-2xl"

        onError={(e) => {
            e.target.onerror = null;

            e.target.src = 'https://via.placeholder.com/100x100.png?text=Oferta';

        }}
    />
</div>
</div>
</section>

{ /* Stats Section - Refined for Minimalist Premium Look */ }

<div className="relative py-12 bg-white">
    <div className="px-4 mx-auto max-w-7xl sm:px-6 lg:px-8">
        <div className="grid grid-cols-2 gap-8 md:grid-cols-4 border-y border-
gray-100 py-12">
            <div className="text-center md:border-r border-gray-50 last:border-0 px-
4">
                <div className="text-4xl font-bold text-gray-900 tracking-tighter mb-
1">500+ </div>
                <div className="text-[10px] font-bold uppercase tracking-wider text-
gray-400">Clientes Reales</div>
            </div>
            <div className="text-center md:border-r border-gray-50 last:border-0 px-
4">

```



```
<div className="text-4xl font-bold text-gray-900 tracking-tighter mb-1">100%</div>
```

```
<div className="text-[10px] font-bold uppercase tracking-wider text-gray-400">Artesanal</div>
```

```
</div>
```

```
<div className="text-center md:border-r border-gray-50 last:border-0 px-4">
```

```
<div className="text-4xl font-bold text-gray-900 tracking-tighter mb-1">24h</div>
```

```
<div className="text-[10px] font-bold uppercase tracking-wider text-gray-400">Envío Rápido</div>
```

```
</div>
```

```
<div className="text-center px-4">
```

```
<div className="text-4xl font-bold text-gray-900 tracking-tighter mb-1">5.0</div>
```

```
<div className="text-[10px] font-bold uppercase tracking-wider text-gray-400">Satisfacción</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
{/* Features Section - Enhanced with better spacing and design */}
```

```
<div className="py-16 bg-white border border-gray-200">
```

```
<div className="px-4 mx-auto max-w-7xl sm:px-6 lg:px-8">
```

```
<div className="mb-16 lg:text-center">
```

```
<h2 className="text-base font-semibold tracking-widest text-black uppercase">Nuestros Beneficios</h2>
```

```
<p className="mt-2 text-3xl font-extrabold leading-8 tracking-tight text-gray-900 sm:text-4xl">
```

```
  Por qué elegirnos
```

```
</p>
```

```
<p className="max-w-2xl mt-4 text-xl text-gray-500 lg:mx-auto">
```

```
  Comprometidos con la calidad y la satisfacción del cliente
```

```
</p>
```

```
</div>
```

```
<div className="mt-10">
```

```
  <div className="grid grid-cols-1 gap-12 sm:grid-cols-2 lg:grid-cols-4">
```

```
    {features.map((feature) => (
```

```
      <div key={feature.id} className="text-center group">
```

```
        <div className="flex items-center justify-center w-16 h-16 mx-auto transition-all duration-300 rounded-full bg-indigo-500/10 group-hover:bg-indigo-500/100/10 border border-gray-200">
```

```
          {feature.icon}
```

```
        </div>
```

```
        <h3 className="mt-6 text-lg font-medium text-gray-900">{feature.title}</h3>
```

```
        <p className="mt-2 text-base text-gray-500">
```

```
          {feature.description}
```

```
        </p>
```

```
      </div>
```

```

    )}}
  </div>
</div>
</div>
</div>

{ /* Categories Section - Enhanced with gradient overlays and better design */ }
{ /* Categories Section - Premium Design */ }
<div className="py-20 bg-white">
  <div className="px-4 mx-auto max-w-7xl sm:px-6 lg:px-8">
    <div className="flex flex-col items-center mb-12 text-center">
      <span className="mb-2 text-xs font-bold tracking-widest text-indigo-500 uppercase">Colecciones</span>
      <h2 className="text-3xl font-extrabold tracking-tight text-gray-900 sm:text-4xl">
        Nuestras Categorías
      </h2>
      <div className="w-12 h-1 mt-4 bg-black rounded-full" />
    </div>

    <div className="grid grid-cols-1 gap-6 mt-10 sm:grid-cols-2 lg:grid-cols-4">
      {categoryList.length > 0 ? (
        categoryList.slice(0, 8).map((category) => (
          <Link

```

```

    key={category.id}

    to={` /shop?category=${encodeURIComponent(category.name)} `}

    className="relative flex flex-col justify-end w-full overflow-hidden
transition-transform duration-500 transform bg-gray-100 group h-96 rounded-2xl
hover:-translate-y-2 hover:shadow-2xl"

>

{ /* Category Image */}

<img

    src={category.imageUrl || `${assetBase}clasicos.jpg`}

    alt={category.name}

    className="absolute inset-0 object-cover w-full h-full transition-
transform duration-700 ease-out group-hover:scale-105"

    onError={(e) => {

        e.target.onerror = null;

        e.target.src = 'https://via.placeholder.com/500x500.png?text=' +
encodeURIComponent(category.name);

    }}

/>

{ /* Gradient Overlay */}

<div className="absolute inset-0 transition-opacity duration-500
opacity-80 bg-gradient-to-t from-black/90 via-black/20 to-transparent group-
hover:opacity-100"> </div>

{ /* Content */}

```

```

    <div className="relative z-10 flex flex-col items-start p-6 transition-
transform duration-500 transform translate-y-4 md:p-8 group-hover:translate-y-
0">

```

```

    <h3 className="mb-1 text-2xl font-bold tracking-tight text-white
capitalize">{category.name}</h3>

```

```

    <div className="w-0 h-0.5 bg-white mb-3 transition-all duration-500
group-hover:w-8" />

```

```

    <span className="inline-flex items-center text-xs font-semibold
tracking-widest text-white/80 uppercase transition-colors duration-300 group-
hover:text-white">

```

Explorar

```

    <svg className="w-4 h-4 ml-2 transition-transform duration-300
transform group-hover:translate-x-1" fill="none" viewBox="0 0 24 24"
stroke="currentColor">

```

```

    <path strokeLinecap="round" strokeLinejoin="round"
strokeWidth="2.5" d="M14 5l7 7m0 0l-7 7m7-7H3" />

```

```

    </svg>

```

```

    </span>

```

```

    </div>

```

```

    </Link>

```

```

  ))

```

```

): categoriesLoading ? (

```

```

    <div className="flex items-center justify-center w-full h-64 col-span-4">

```

```

    <div className="w-12 h-12 border-b-2 border-indigo-600 rounded-full
animate-spin"> </div>

```

```

    </div>

```

```

): (

```

```
<div className="flex flex-col items-center justify-center w-full col-span-4
py-16 bg-gray-50 rounded-2xl">
```

```
<div className="mb-4 text-5xl"> 📁 </div>
```

```
<h3 className="mb-2 text-xl font-semibold text-gray-900">No hay
categorías</h3>
```

```
<p className="mb-6 text-gray-500">Actualmente no hay categorías
disponibles en la tienda.</p>
```

```
<Link
```

```
to="/shop"
```

```
className="inline-flex items-center px-6 py-3 text-sm font-semibold
text-white transition-all bg-black rounded-lg hover:bg-gray-800"
```

```
>
```

```
Explorar productos
```

```
</Link>
```

```
</div>
```

```
})
```

```
</div>
```

```
</div>
```

```
</div>
```

```
{/* Featured Products Section - Enhanced with better card design */}
```

```
<div className="py-16 bg-white border border-gray-200">
```

```
<div className="px-4 mx-auto max-w-7xl sm:px-6 lg:px-8">
```

```
<div className="mb-16 lg:text-center">
```

```
<h2 className="text-base font-semibold tracking-wide text-black
uppercase tracking-widest uppercase">Productos Destacados</h2>
```

```
<p className="mt-2 text-3xl font-extrabold leading-8 tracking-tight text-gray-900 sm:text-4xl">
```

```
  Lo más vendido
```

```
</p>
```

```
</div>
```

```
{loading ? (
```

```
  <div className="flex items-center justify-center h-64">
```

```
    <div className="w-12 h-12 border-b-2 border-indigo-600 rounded-full animate-spin"> </div>
```

```
  </div>
```

```
): error ? (
```

```
  <div className="relative px-4 py-3 text-red-700 bg-red-100 border border-red-400 rounded" role="alert">
```

```
    <strong className="font-bold">Error! </strong>
```

```
    <span className="block sm:inline">{error}</span>
```

```
  </div>
```

```
): (
```

```
  <div className="grid grid-cols-1 gap-6 mt-10 sm:grid-cols-2 lg:grid-cols-3 xl:grid-cols-4">
```

```
    {featuredProducts.map((product) => (
```

```
      <ProductCard
```

```
        key={product.id}
```

```
        product={product}
```

```
        getPrimaryImageUrl={getPrimaryImageUrl}
```

```
        onAddToCart={handleAddToCart}
```

```

        onAddToWishlist={handleAddToWishlist}

      />
    )))
  </div>
})

```

```

<div className="mt-16 text-center">

  <Link

    to="/shop"

    className="inline-flex items-center px-8 py-4 text-base font-medium
text-gray-900 transition-all duration-300 border border-transparent rounded-lg
shadow-2xl shadow-sm bg-black text-white hover:from-indigo-400 hover:to-
purple-500 hover:shadow-xl"

    >

      Ver todos los productos

    </Link>

  </div>

</div>

</div>

```

```

{/* Testimonials Section - Premium Aesthetic Redesign */}

<section className="relative py-24 overflow-hidden bg-gray-50/50">

  {/* Decorative elements */}

  <div className="absolute top-0 right-0 w-64 h-64 bg-indigo-500/5
rounded-full blur-3xl -translate-y-1/2 translate-x-1/2" />

```



```

<div className="absolute bottom-0 left-0 w-96 h-96 bg-purple-500/5
rounded-full blur-3xl translate-y-1/2 -translate-x-1/2" />

<div className="relative px-4 mx-auto max-w-7xl sm:px-6 lg:px-8">
  <div className="mb-16 text-center">
    <span className="inline-block px-3 py-1 mb-4 text-xs font-bold tracking-
widest text-indigo-600 uppercase bg-indigo-100 rounded-full">
      Testimonios
    </span>
    <h2 className="text-4xl font-bold tracking-tight text-gray-900 sm:text-
5xl">
      Lo que dicen de nosotros
    </h2>
    <p className="max-w-2xl mx-auto mt-4 text-lg text-gray-500">
      Nuestra mayor satisfacción es ver a nuestros clientes felices con sus
accesorios únicos.
    </p>
  </div>

  <div className="relative max-w-4xl mx-auto">
    {/* Quote Icon Background */}
    <div className="absolute top-0 left-0 -translate-x-6 -translate-y-8
opacity-10">
      <svg width="120" height="95" viewBox="0 0 120 95" fill="none"
className="text-gray-900">
        <path d="M34.2857 0C15.3524 0 0 15.3524 0
34.2857V94.2857H60V34.2857H25.7143C25.7143 25.7143 31.2857 20.1429 39.8571

```

```

20.1429L34.2857 0ZM94.2857 0C75.3524 0 60 15.3524 60
34.2857V94.2857H120V34.2857H85.7143C85.7143 25.7143 91.2857 20.1429
99.8571 20.1429L94.2857 0Z" fill="currentColor"/>

```

```

</svg>

```

```

</div>

```

```

{/* Slider Container */}

```

```

<div className="relative z-10 overflow-hidden rounded-3xl">

```

```

  <div

```

```

    className="flex transition-transform duration-700 cubic-bezier(0.4, 0,
0.2, 1)"

```

```

    style={{ transform: `translateX(-${currentTestimonial * 100}%)` }}

```

```

    onTouchStart={handleTouchStart}

```

```

    onTouchMove={handleTouchMove}

```

```

    onTouchEnd={handleTouchEnd}

```

```

  >

```

```

  {testimonials.map((t) => (

```

```

    <div key={t.id} className="flex-shrink-0 w-full p-2">

```

```

      <div className="bg-white/80 backdrop-blur-md border border-white
shadow-xl rounded-3xl p-8 md:p-12">

```

```

        <div className="flex flex-col md:flex-row md:items-center gap-8">

```

```

          {/* Avatar & Info */}

```

```

          <div className="flex flex-col items-center md:items-start text-
center md:text-left flex-shrink-0">

```

```

            <div className="relative mb-4 group">

```

```

        <div className="absolute inset-0 bg-indigo-500 rounded-full
blur opacity-20 group-hover:opacity-40 transition-opacity" />

        <img

            src={`https://ui-
avatars.com/api/?name=${encodeURIComponent(t.name)}&background=111827&
color=ffffff&size=128&bold=true`}

            alt={t.name}

            className="relative w-24 h-24 rounded-full border-4 border-
white shadow-lg object-cover"

        />

        <div className="absolute bottom-0 right-0 bg-indigo-600 text-
white rounded-full p-1.5 border-2 border-white shadow-md">

            <svg className="w-3 h-3" fill="currentColor" viewBox="0 0 20
20">

                <path d="M16.707 5.293a1 1 0 010 1.414l-8 8a1 1 0 01-1.414
0l-4-4a1 1 0 011.414-1.414L8 12.586l7.293-7.293a1 1 0 011.414 0z" />

            </svg>

        </div>

    </div>

    <h3 className="text-xl font-bold text-gray-900">{t.name}</h3>

    <p className="text-xs font-semibold text-indigo-600 uppercase
tracking-widest mb-2">{t.role}</p>

    <div className="flex gap-1 text-amber-400">

        {[...Array(5)].map((_, i) => (

            <svg key={i} className={`w-4 h-4 ${i < t.rating ? 'fill-current' :
'text-gray-300'}`} viewBox="0 0 20 20">

```

```

        <path d="M9.049 2.927c.3-.921 1.603-.921 1.902 0l1.07
3.292a1 1 0 00.9569h3.462c.969 0 1.371 1.24588 1.81l-2.8 2.034a1 1 0 00-.364
1.118l1.07 3.292c.3.921-.755 1.688-1.54 1.118l-2.8-2.034a1 1 0 00-1.175 0l-2.8
2.034c-.784.57-.38-1.81.588-1.81h3.461a1 1 0 00.951-.69l1.07-3.292z" />

    </svg>

  )})

</div>

</div>

{ /* Content */ }

<div className="flex-1">

  <h4 className="text-2xl font-bold text-gray-900 mb-
4"> "{t.title}" </h4>

  <p className="text-lg text-gray-600 leading-relaxed italic mb-6">
    {t.text}
  </p>

  <div className="flex items-center gap-2 text-xs text-gray-400
font-medium bg-gray-50 self-start px-3 py-1 rounded-full">

    <svg className="w-3.5 h-3.5" fill="none" viewBox="0 0 24 24"
stroke="currentColor">

      <path strokeLinecap="round" strokeLinejoin="round"
strokeWidth="2" d="M8 7V3m8 4V3m-9 8h10M5 21h14a2 2 0 00-2V7a2 2 0 00-
2-2H5a2 2 0 00-2 2v12a2 2 0 00 2z" />

    </svg>

    Publicado {t.date}

  </div>

</div>

```

```
</div>
```

```
</div>
```

```
</div>
```

```
)))
```

```
</div>
```

```
</div>
```

```
{/* Navigation Controls */}
```

```
<div className="flex items-center justify-between mt-10">
```

```
<div className="flex gap-2">
```

```
{testimonials.map((_, index) => (
```

```
<button
```

```
key={index}
```

```
onClick={() => setCurrentTestimonial(index)}
```

```
className={`h-1.5 rounded-full transition-all duration-300 ${
```

```
index === currentTestimonial ? 'w-8 bg-indigo-600 shadow-sm' : 'w-2 bg-indigo-200 hover:bg-indigo-300'
```

```
}}}
```

```
aria-label={`Ir al testimonio ${index + 1}}`
```

```
/>
```

```
)))
```

```
</div>
```

```
<div className="flex gap-3">
```

```
<button
```

```

        onClick={() => setCurrentTestimonial(prev => prev === 0 ?
testimonials.length - 1 : prev - 1)}

        className="p-3 bg-white border border-gray-100 rounded-2xl text-
gray-400 hover:text-indigo-600 hover:border-indigo-100 hover:shadow-lg
transition-all active:scale-95"

        >

        <svg className="w-6 h-6" fill="none" viewBox="0 0 24 24"
stroke="currentColor" strokeWidth="2.5">

            <path strokeLinecap="round" strokeLinejoin="round" d="M15 19l-7-7
7-7" />

        </svg>

    </button>

    <button

        onClick={() => setCurrentTestimonial(prev => (prev + 1) %
testimonials.length)}

        className="p-3 bg-gray-900 text-white rounded-2xl hover:bg-indigo-
600 hover:shadow-lg transition-all active:scale-95 shadow-indigo-200 shadow-xl"

        >

        <svg className="w-6 h-6" fill="none" viewBox="0 0 24 24"
stroke="currentColor" strokeWidth="2.5">

            <path strokeLinecap="round" strokeLinejoin="round" d="M9 5l7 7-7
7" />

        </svg>

    </button>

</div>

</div>

</div>

```

</div>

</section>

{/ Sobre Nosotros Section - Editorial Design */}*

<section className="py-24 bg-gray-50/30">

<div className="px-4 mx-auto max-w-7xl sm:px-6 lg:px-8">

<div className="flex flex-col md:flex-row md:items-end justify-between mb-16 gap-6">

<div className="max-w-2xl">

Nuestra Esencia

<h2 className="text-4xl md:text-5xl font-bold text-gray-900 leading-tight">

Artesanía que cuenta
 una historia

</h2>

</div>

<p className="max-w-md text-lg text-gray-500 leading-relaxed font-medium">

Desde 2015, transformamos telas premium en accesorios que celebran los momentos más especiales de tu vida.

</p>

</div>

```

<div className="grid grid-cols-1 md:grid-cols-3 gap-10">

  {/* Card 1: Historia */}

  <div className="group relative bg-white rounded-[2rem] overflow-hidden
shadow-sm hover:shadow-2xl transition-all duration-500 hover:-translate-y-2
border border-gray-100 flex flex-col">

    <div className="h-72 overflow-hidden relative">

      <img

        src={` ${assetBase}trayectoria.jpg`}

        alt="Nuestra Trayectoria"

        className="object-cover w-full h-full transition-transform duration-700
group-hover:scale-110"

      />

      <div className="absolute inset-0 bg-gradient-to-t from-black/60 via-
transparent to-transparent opacity-0 group-hover:opacity-100 transition-opacity
duration-500" />

      <span className="absolute top-4 left-4 inline-flex items-center px-4 py-
2 text-[10px] font-bold uppercase tracking-widest text-white bg-indigo-600/90
backdrop-blur-md rounded-full">

        Desde 2015

      </span>

    </div>

    <div className="p-8 flex flex-col flex-1">

      <h3 className="text-2xl font-bold text-gray-900 mb-4">Nuestra
Trayectoria</h3>

      <p className="text-gray-500 leading-relaxed mb-6 flex-1">

        Nacimos como un sueño artesanal y hoy somos referentes de elegancia
en cada accesorio de moda que creamos.

```


</p>

<Link

to="/blog"

className="inline-flex items-center text-xs font-bold uppercase
tracking-widest text-indigo-600 hover:text-indigo-700 transition-colors group/link"

>

Descubrir más

<svg className="w-4 h-4 ml-2 group-hover/link:translate-x-1
transition-transform" fill="none" stroke="currentColor" viewBox="0 0 24 24"
strokeWidth="2.5">

<path strokeLinecap="round" strokeLinejoin="round" d="M17 8l4
4m0 0l-4 4m4-4H3"></path>

</svg>

</Link>

</div>

</div>

{/* Card 2: Valores */}

<div className="group relative bg-white rounded-[2rem] overflow-hidden
shadow-sm hover:shadow-2xl transition-all duration-500 hover:-translate-y-2
border border-gray-100 flex flex-col">

<div className="h-72 overflow-hidden relative">

<img

src={` \${assetBase}compromiso.jpg`}

alt="Compromiso"

className="object-cover w-full h-full transition-transform duration-700
group-hover:scale-110"

/>

<div *className*="absolute inset-0 bg-gradient-to-t from-black/60 via-transparent to-transparent opacity-0 group-hover:opacity-100 transition-opacity duration-500" />

Hecho a Mano

</div>

<div *className*="p-8 flex flex-col flex-1">

<h3 *className*="text-2xl font-bold text-gray-900 mb-4">Calidad sin Compromiso</h3>

<p *className*="text-gray-500 leading-relaxed mb-6 flex-1">

Utilizamos solo materiales premium y técnicas milenarias para asegurar que cada pieza sea una obra de arte.

</p>

<Link

to="/blog"

className="inline-flex items-center text-xs font-bold uppercase tracking-widest text-purple-600 hover:text-purple-700 transition-colors group/link"

>

Nuestros procesos

<svg *className*="w-4 h-4 ml-2 group-hover/link:translate-x-1 transition-transform" *fill*="none" *stroke*="currentColor" *viewBox*="0 0 24 24" *strokeWidth*="2.5">

```
      <path strokeLinecap="round" strokeLinejoin="round" d="M17 8l4
4m0 0l-4 4m4-4H3"> </path>
```

```
    </svg>
```

```
  </Link>
```

```
</div>
```

```
</div>
```

```
{/* Card 3: Equipo */}
```

```
  <div className="group relative bg-white rounded-[2rem] overflow-hidden
shadow-sm hover:shadow-2xl transition-all duration-500 hover:-translate-y-2
border border-gray-100 flex flex-col">
```

```
    <div className="h-72 overflow-hidden relative">
```

```
      <img
```

```
        src={` ${assetBase}talento.jpg`}
```

```
        alt="Equipo"
```

```
        className="object-cover w-full h-full transition-transform duration-700
group-hover:scale-110"
```

```
      />
```

```
      <div className="absolute inset-0 bg-gradient-to-t from-black/60 via-
transparent to-transparent opacity-0 group-hover:opacity-100 transition-opacity
duration-500" />
```

```
      <span className="absolute top-4 left-4 inline-flex items-center px-4 py-
2 text-[10px] font-bold uppercase tracking-widest text-white bg-gray-900/90
backdrop-blur-md rounded-full">
```

```
        Talento Local
```

```
      </span>
```

```
    </div>
```

```

<div className="p-8 flex flex-col flex-1">

  <h3 className="text-2xl font-bold text-gray-900 mb-4">Corazón de
  Artesano</h3>

  <p className="text-gray-500 leading-relaxed mb-6 flex-1">

    Detrás de cada moño hay un equipo apasionado que pone su alma en
    cada costura y cada detalle.

  </p>

  <Link

    to="/blog"

    className="inline-flex items-center text-xs font-bold uppercase
    tracking-widest text-gray-900 hover:text-black transition-colors group/link"

    >

      Conoce al equipo

      <svg className="w-4 h-4 ml-2 group-hover/link:translate-x-1
      transition-transform" fill="none" stroke="currentColor" viewBox="0 0 24 24"
      strokeWidth="2.5">

        <path strokeLinecap="round" strokeLinejoin="round" d="M17 8l4
        4m0 0l-4 4m4-4H3"> </path>

      </svg>

    </Link>

  </div>

</div>

</div>

<div className="mt-20 text-center">

  <Link

```

```

    to="/blog"

    className="inline-flex items-center gap-3 px-10 py-5 bg-gray-900 text-
white rounded-2xl font-bold hover:bg-black hover:shadow-2xl hover:shadow-gray-
200 transition-all active:scale-95 group"

    >

    Nuestra Historia Completa

    <svg className="w-5 h-5 group-hover:translate-x-1 transition-transform"
fill="none" viewBox="0 0 24 24" stroke="currentColor" strokeWidth="2.5">

        <path strokeLinecap="round" strokeLinejoin="round" d="M13 7l5 5m0
0l-5 5m5-5H6" />

    </svg>

</Link>

</div>

</div>

</section>

```

```

{ /* Contact Section - Premium Redesign */ }

<section className="py-24 bg-white">

    <div className="px-4 mx-auto max-w-7xl sm:px-6 lg:px-8">

        <div className="flex flex-col lg:flex-row gap-16">

            { /* Left Side: Info */ }

            <div className="lg:w-1/3">

                <span className="inline-block px-3 py-1 mb-4 text-xs font-bold
tracking-widest text-indigo-600 uppercase bg-indigo-50 rounded-full">

                    Contacto

                </span>

```

```

<h2 className="text-4xl font-bold tracking-tight text-gray-900 mb-6">
  Hablemos de tu próximo moño favorito
</h2>

<p className="text-lg text-gray-500 mb-10 leading-relaxed">
  ¿Tienes una idea especial o necesitas ayuda con tu pedido? Estamos aquí
  para escucharte y asesorarte.
</p>

<div className="space-y-8">
  <div className="flex items-start gap-4 group">
    <div className="flex-shrink-0 w-12 h-12 bg-gray-50 rounded-2xl flex
  items-center justify-center text-gray-600 group-hover:bg-indigo-600 group-
  hover:text-white transition-all duration-300 shadow-sm">
      <svg className="w-5 h-5" fill="none" viewBox="0 0 24 24"
  stroke="currentColor" strokeWidth="2">
        <path strokeLinecap="round" strokeLinejoin="round" d="M3 8l7.89
  5.26a2 2 0 002.22 0L21 8M5 19h14a2 2 0 002-2V7a2 2 0 00-2-2H5a2 2 0 00-2
  2v10a2 2 0 002 2z" />
      </svg>
    </div>
    <div>
      <h4 className="text-sm font-bold text-gray-900 uppercase tracking-
  wider mb-1">Escríbenos</h4>
      <p className="text-gray-500 font-
  medium">contacto@psgshop.com</p>
    </div>
  </div>
</div>

```

```

<div className="flex items-start gap-4 group">

  <div className="flex-shrink-0 w-12 h-12 bg-gray-50 rounded-2xl flex
items-center justify-center text-gray-600 group-hover:bg-indigo-600 group-
hover:text-white transition-all duration-300 shadow-sm">

    <svg className="w-5 h-5" fill="none" viewBox="0 0 24 24"
stroke="currentColor" strokeWidth="2">

      <path strokeLinecap="round" strokeLinejoin="round" d="M3 5a2 2 0
0 12-2h3.28a1 1 0 01.948.684l1.498 4.493a1 1 0 01-.502 1.21l-2.257 1.13a11.042
11.042 0 005.516 5.516l1.13-2.257a1 1 0 011.21-.502l4.493 1.498a1 1 0
01.684.949V19a2 2 0 01-2 2h-1C9.716 21 3 14.284 3 6V5z" />

    </svg>

  </div>

  <div>

    <h4 className="text-sm font-bold text-gray-900 uppercase tracking-
wider mb-1">Llámanos</h4>

    <p className="text-gray-500 font-medium">+57 (300) 123-
4567</p>

  </div>

</div>

<div className="flex items-start gap-4 group">

  <div className="flex-shrink-0 w-12 h-12 bg-gray-50 rounded-2xl flex
items-center justify-center text-gray-600 group-hover:bg-indigo-600 group-
hover:text-white transition-all duration-300 shadow-sm">

    <svg className="w-5 h-5" fill="none" viewBox="0 0 24 24"
stroke="currentColor" strokeWidth="2">

```

```

        <path strokeLinecap="round" strokeLinejoin="round" d="M17.657
16.657L13.414 20.9a1.998 1.998 0 01-2.827 0l-4.244-4.243a8 8 0 1111.314 0z" />

        <path strokeLinecap="round" strokeLinejoin="round" d="M15 11a3
3 0 11-6 0 3 3 0 016 0z" />

    </svg>

</div>

<div>

    <h4 className="text-sm font-bold text-gray-900 uppercase tracking-
wider mb-1">Visítanos</h4>

    <p className="text-gray-500 font-medium">Bucaramanga,
Santander, Colombia</p>

</div>

</div>

</div>

</div>

```

```

{ /* Right Side: Form */

```

```

    <div className="lg:w-2/3">

        <div className="bg-gray-50/50 p-8 md:p-12 rounded-[2.5rem] border
border-gray-100 shadow-sm relative overflow-hidden group">

            <div className="absolute top-0 right-0 w-96 h-96 bg-indigo-500/5
rounded-full blur-3xl translate-x-1/2 -translate-y-1/2 group-hover:bg-indigo-
500/10 transition-colors duration-700" />

```

```

{state.succeeded ? (

```

```

    <div className="text-center py-12 relative z-10">

```



```

    <div className="w-20 h-20 bg-indigo-600 text-white rounded-full
flex items-center justify-center mx-auto mb-6 shadow-xl shadow-indigo-200">

        <svg className="w-10 h-10" fill="none" viewBox="0 0 24 24"
stroke="currentColor" strokeWidth="3">

            <path strokeLinecap="round" strokeLinejoin="round" d="M5 13l4
4L19 7" />

        </svg>

    </div>

    <h3 className="text-2xl font-bold text-gray-900 mb-2">¡Mensaje
enviado con éxito!</h3>

    <p className="text-gray-500 mb-8">Gracias por contactarnos. Te
responderemos en menos de 24 horas.</p>

    <button

        onClick={() => window.location.reload()}

        className="px-8 py-3 bg-gray-900 text-white rounded-2xl font-bold
hover:bg-gray-800 transition-all active:scale-95 shadow-xl shadow-gray-200"

        >

        Enviar otro mensaje

    </button>

</div>

):(

<form onSubmit={handleSubmit} className="relative z-10 space-y-
6">

    <div className="grid grid-cols-1 md:gap-8 gap-6 md:grid-cols-2">

        <div className="space-y-2">

            <label htmlFor="name" className="text-xs font-bold text-gray-400
uppercase tracking-wider ml-1">

```

```

    Nombre Completo

</label>

<input
  type="text"
  name="name"
  id="name"
  required
  value={formData.name}
  onChange={handleInputChange}
  autoComplete="name"

  className="w-full bg-white px-5 py-4 rounded-2xl border-2
border-transparent shadow-sm focus:border-indigo-600 focus:ring-0 transition-all
outline-none text-gray-900 placeholder:text-gray-300 font-medium"

  placeholder="Ej. María Pérez"
/>
</div>

<div className="space-y-2">

  <label htmlFor="email" className="text-xs font-bold text-gray-400
uppercase tracking-wider ml-1">
    Correo Electrónico

  </label>

  <input
    type="email"
    name="email"
    id="email"

```

```

    required

    value={formData.email}

    onChange={handleInputChange}

    autoComplete="email"

    className="w-full bg-white px-5 py-4 rounded-2xl border-2
border-transparent shadow-sm focus:border-indigo-600 focus:ring-0 transition-all
outline-none text-gray-900 placeholder:text-gray-300 font-medium"

    placeholder="tu@correo.com"

  />

  <ValidationError prefix="Email" field="email" errors={state.errors}
className="text-xs text-red-500 mt-1 ml-1" />

</div>

</div>

<div className="space-y-2">

  <label htmlFor="subject" className="text-xs font-black text-gray-
400 uppercase tracking-[0.2em] ml-1">

    Asunto

  </label>

  <input

    type="text"

    name="subject"

    id="subject"

    required

    value={formData.subject}

    onChange={handleInputChange}

```

```
        className="w-full bg-white px-5 py-4 rounded-2xl border-2
border-transparent shadow-sm focus:border-indigo-600 focus:ring-0 transition-all
outline-none text-gray-900 placeholder:text-gray-300 font-medium"
```

```
        placeholder="Ej. Pedido personalizado"
```

```
    />
```

```
</div>
```

```
<div className="space-y-2">
```

```
    <label htmlFor="message" className="text-xs font-black text-gray-
400 uppercase tracking-[0.2em] ml-1">
```

```
        Tu Mensaje
```

```
    </label>
```

```
    <textarea
```

```
        id="message"
```

```
        name="message"
```

```
        rows={5}
```

```
        required
```

```
        value={formData.message}
```

```
        onChange={handleInputChange}
```

```
        className="w-full bg-white px-5 py-4 rounded-2xl border-2
border-transparent shadow-sm focus:border-indigo-600 focus:ring-0 transition-all
outline-none text-gray-900 placeholder:text-gray-300 font-medium resize-none"
```

```
        placeholder="Cuéntanos cómo podemos ayudarte..."
```

```
    />
```

```
    <ValidationErrors prefix="Message" field="message"
errors={state.errors} className="text-xs text-red-500 mt-1 ml-1" />
```

```
</div>
```

```
<div className="pt-2">
```

```
<button
```

```
  type="submit"
```

```
  disabled={state.submitting}
```

```
  className={`group w-full relative h-16 bg-gray-900 text-white
rounded-2xl font-bold text-lg overflow-hidden transition-all active:scale-[0.98]
shadow-2xl shadow-gray-200 disabled:opacity-70 disabled:cursor-not-allowed`}
  >
```

```
    <div className="absolute inset-0 bg-indigo-600 translate-y-full
group-hover:translate-y-0 transition-transform duration-300" />
```

```
    <span className="relative z-10 flex items-center justify-center
gap-2">
```

```
      {state.submitting ? (
```

```
        <>
```

```
          <svg className="animate-spin h-5 w-5 text-white" fill="none"
viewBox="0 0 24 24">
```

```
            <circle className="opacity-25" cx="12" cy="12" r="10"
stroke="currentColor" strokeWidth="4"> </circle>
```

```
            <path className="opacity-75" fill="currentColor" d="M4 12a8
8 0 0 18-8V0C5.373 0 0 5.373 0 12h4zm2 5.291A7.962 7.962 0 0 14 12H0c0 3.042
1.135 5.824 3 7.938l3-2.647z"> </path>
```

```
          </svg>
```

```
          Enviando...
```

```
        </>
```

```
      ) : (
```

```

    </>

    Enviar Mensaje

    <svg className="w-5 h-5 group-hover:translate-x-1 transition-
transform" fill="none" viewBox="0 0 24 24" stroke="currentColor"
strokeWidth="2.5">

        <path strokeLinecap="round" strokeLinejoin="round" d="M13
7 15 5m0 0l-5 5m5-5H6" />

    </svg>

</>

})

</span>

</button>

</div>

</form>

})

</div>

</div>

</div>

</div>

</section>

</div>

);

};

export default Home;

```

 src/assets/pages/ModernAdminDashboard.jsx

```
import React, { useState, useEffect, useRef } from 'react';

import { useAuth } from '../context/AuthContext';

import { db } from '../firebase';

import {
  collection,
  getDocs,
  query,
  orderBy,
  limit,
  addDoc,
  updateDoc,
  deleteDoc,
  doc
} from "firebase/firestore";

import { getAllUsers, updateUserRole as updateUserServiceRole } from
'../services/userService';

import { getAllReviews, deleteReview, updateReview } from
'../services/reviewService';

import { getAllOrders, updateOrderStatus, getOrderStatusText,
getStatusBadgeClass } from '../services/orderService';

import { getSalesData, getSalesByCategory, getOrderStatusDistribution,
getTopSellingProducts, getUserRegistrationData, getRevenueByPaymentMethod }
from '../services/analyticsService';

import { getCategories, createCategory, updateCategory, deleteCategory } from
'../services/categoryService';
```

```

import CouponManager from '../components/CouponManager';
import OrderDetailsModal from '../components/OrderDetailsModal';
import Loader from '../components/Loader';
import { signOut } from 'firebase/auth';
import { auth } from '../firebase';
import { useNavigate } from 'react-router-dom';
import {
  FiMenu, FiX, FiHome, FiShoppingBag, FiUsers, FiMessageSquare,
  FiTag, FiPackage, FiBarChart2, FiPlus, FiLogOut, FiSun, FiMoon,
  FiList, FiCheck, FiChevronRight, FiGrid, FiActivity, FiStar, FiImage, FiCamera,
  FiRefreshCw,
  FiLayers, FiTrash2
} from 'react-icons/fi';
import DashboardAnalytics from '../components/admin/DashboardAnalytics';
import AdminProducts from '../components/admin/AdminProducts';
import AdminUsers from '../components/admin/AdminUsers';
import AdminReviews from '../components/admin/AdminReviews';
import AdminOrders from '../components/admin/AdminOrders';
import AdminCategories from '../components/admin/AdminCategories';
import AdminProductModal from '../components/admin/AdminProductModal';

const ModernAdminDashboard = () => {
  const { currentUser, isAdmin } = useAuth();
  const navigate = useNavigate();

```


// URL del servidor (local o producción en Vercel)

```
const API_BASE_URL = window.location.hostname === 'localhost' ||  
window.location.hostname === '127.0.0.1'
```

```
  ? 'http://localhost:3001'
```

```
  : '/api';
```

```
const [sidebarOpen, setSidebarOpen] = useState(false);
```

```
const [activeSection, setActiveSection] = useState('analytics');
```

```
const [theme, setTheme] = useState('light');
```

```
const [stats, setStats] = useState({
```

```
  totalProducts: 0,
```

```
  totalOrders: 0,
```

```
  totalUsers: 0,
```

```
  totalReviews: 0,
```

```
  totalRevenue: 0,
```

```
  recentOrders: [],
```

```
  recentUsers: [],
```

```
  salesData: [],
```

```
  salesByCategory: [],
```

```
  orderStatusData: [],
```

```
  topSellingProducts: [],
```

```
  userRegistrationData: [],
```

```
  paymentMethodData: [],
```

```
  dateRange: 'last30days'
```

```
});
```

```
const [loading, setLoading] = useState(false);
```

// Products state

```
const [products, setProducts] = useState([]);  
const [productsLoading, setProductsLoading] = useState(false);  
const [newProduct, setNewProduct] = useState({  
  name: "", description: "", price: "", imageUrls: [], primaryImageIndex: 0, category: "",  
  material: "", color: "", size: "", style: "", stock: 0, rating: 0  
});  
const [editingProduct, setEditingProduct] = useState(null);  
const [productModalOpen, setProductModalOpen] = useState(false);  
const [modalMode, setModalMode] = useState('add');  
const [stripeSyncing, setStripeSyncing] = useState(false);
```

// Users state

```
const [users, setUsers] = useState([]);  
const [usersLoading, setUsersLoading] = useState(false);
```

// Reviews state

```
const [reviews, setReviews] = useState([]);  
const [reviewsLoading, setReviewsLoading] = useState(false);  
const [editingReview, setEditingReview] = useState(null);  
const [reviewModalOpen, setReviewModalOpen] = useState(false);
```

// Orders state

```
const [orders, setOrders] = useState([]);
```

```
const [ordersLoading, setOrdersLoading] = useState(false);
const [selectedOrder, setSelectedOrder] = useState(null);
const [orderModalOpen, setOrderModalOpen] = useState(false);
const [orderStatusUpdating, setOrderStatusUpdating] = useState({});
```

// Search states

```
const [productSearchTerm, setProductSearchTerm] = useState("");
const [reviewSearchTerm, setReviewSearchTerm] = useState("");
const [orderSearchTerm, setOrderSearchTerm] = useState("");
const [userSearchTerm, setUserSearchTerm] = useState("");
const [userRoleFilter, setUserRoleFilter] = useState('all');
```

// Categories state

```
const [categories, setCategories] = useState([]);
const [categoriesLoading, setCategoriesLoading] = useState(false);
const [newCategory, setNewCategory] = useState({ name: "", description: "" });
const [editingCategory, setEditingCategory] = useState(null);
const [categoryModalOpen, setCategoryModalOpen] = useState(false);
const [categoryModalMode, setCategoryModalMode] = useState('add');
const [categorySearchTerm, setCategorySearchTerm] = useState("");
const [categoryImage, setCategoryImage] = useState(null);
const [categoryImagePreview, setCategoryImagePreview] = useState(null);
const [editingCategoryImage, setEditingCategoryImage] = useState(null);
const [editingCategoryImagePreview, setEditingCategoryImagePreview] =
useState(null);
```

```

const [isProfileMenuOpen, setIsProfileMenuOpen] = useState(false);
const profileMenuRef = useRef(null);
const [error, setError] = useState(null);
const [successMessage, setSuccessMessage] = useState('');
const [uploading, setUploading] = useState(false);

// Close dropdown on click outside
useEffect(() => {
  const handleClickOutside = (event) => {
    if (profileMenuRef.current && !profileMenuRef.current.contains(event.target)) {
      setIsProfileMenuOpen(false);
    }
  };
  document.addEventListener('mousedown', handleClickOutside);
  return () => document.removeEventListener('mousedown', handleClickOutside);
}, []);

const toggleTheme = () => {
  const newTheme = theme === 'light' ? 'dark' : 'light';
  setTheme(newTheme);
  localStorage.setItem('adminDashboardTheme', newTheme);
};

```

```

useEffect(() => {
  const savedTheme = localStorage.getItem('adminDashboardTheme');
  if(savedTheme) {
    setTheme(savedTheme);
  } else if(window.matchMedia('(prefers-color-scheme: dark)').matches) {
    setTheme('dark');
  }
}, []);

```

```

useEffect(() => {
  if(theme === 'dark') {
    document.documentElement.classList.add('dark');
  } else {
    document.documentElement.classList.remove('dark');
  }
}, [theme]);

```

```

const fetchCategories = async () => {
  try {
    setCategoriesLoading(true);
    const categoriesList = await getCategories();
    setCategories(categoriesList);
  } catch (error) {
    setError('Error al cargar las categorías');
  }
}

```

```

    } finally{
      setCategoriesLoading(false);
    }
  };

```

```

const fileToBase64 = (file) => {
  return new Promise((resolve, reject) => {
    const reader = new FileReader();
    reader.readAsDataURL(file);
    reader.onload = () => resolve(reader.result);
    reader.onerror = error => reject(error);
  });
};

```

```

const handleCategoryImageUpload = async (e, isEditing = false) => {
  try{
    const file = e.target.files[0];
    if (!file) return;
    if (file.size > 1024 * 1024) { alert("La imagen debe ser menor a 1MB."); return; }
    const base64Data = await fileToBase64(file);
    if (isEditing) {
      setEditingCategoryImage(base64Data);
      setEditingCategoryImagePreview(base64Data);
    } else {

```

```

    setCategoryImage(base64Data);
    setCategoryImagePreview(base64Data);
  }
} catch (err) { setError("Error al procesar la imagen."); }
};

```

```

const clearCategoryImage = (isEditing = false) => {
  if(isEditing) {
    setEditingCategoryImage(null);
    setEditingCategoryImagePreview(null);
    if(editingCategory?.imageUrl) setEditingCategory({ ...editingCategory, imageUrl:
null });
  } else {
    setCategoryImage(null);
    setCategoryImagePreview(null);
    setNewCategory({ ...newCategory, imageUrl: null });
  }
};

```

```

const createCategoryHandler = async (e) => {
  e.preventDefault();
  if(!newCategory.name) { alert("Introduce un nombre."); return; }
  try{
    const categoryData = { ...newCategory };
    if(categoryImage) categoryData.imageUrl = categoryImage;

```

```

    await createCategory(categoryData);

    setNewCategory({ name: "", description: "" });

    setCategoryImage(null);

    setCategoryImagePreview(null);

    showSuccessMessage("Categoría creada exitosamente!");

    setCategoryModalOpen(false);

    fetchCategories();

  } catch (err) { setError("Error al crear la categoría."); }

};

const updateCategoryHandler = async (e) => {

  e.preventDefault();

  try{

    const categoryData = { ...editingCategory };

    if (editingCategoryImage) categoryData.imageUrl = editingCategoryImage;

    await updateCategory(editingCategory.id, categoryData);

    showSuccessMessage("Categoría actualizada!");

    setCategoryModalOpen(false);

    setEditingCategory(null);

    fetchCategories();

  } catch (err) { setError("Error al actualizar."); }

};

const deleteCategoryHandler = async (id, name) => {

```



```

    if(!window.confirm(`¿Eliminar "${name}"?`)) return;

    try {
      await deleteCategory(id);

      setCategories(categories.filter(c => c.id !== id));

      showSuccessMessage("Categoría eliminada.");
    } catch (err) { setError("Error al eliminar."); }
  };

```

```

const openAddCategoryModal = () => {
  setNewCategory({ name: "", description: "" });
  setCategoryImage(null);
  setCategoryImagePreview(null);
  setCategoryModalMode('add');
  setCategoryModalOpen(true);
};

```

```

const openEditCategoryModal = (category) => {
  setEditingCategory({ ...category });
  setEditingCategoryImage(null);
  setEditingCategoryImagePreview(null);
  setCategoryModalMode('edit');
  setCategoryModalOpen(true);
};

```

```

const handleLogout = async () => {

  try{ await signOut(auth); navigate('/login'); } catch (error) { setError('Error al
cerrar sesión'); }

};

const showSuccessMessage = (message) => {

  setSuccessMessage(message);

  setTimeout(() => setSuccessMessage(''), 3000);

};

const formatCurrency = (amt) => new Intl.NumberFormat('es-CO', { style:
'currency', currency: 'COP', minimumFractionDigits: 0 }).format(amt || 0);

const formatDate = (date) => {

  if(!date) return 'N/A';

  const d = date.toDate ? date.toDate() : new Date(date);

  return d.toLocaleDateString('es-CO', { day: '2-digit', month: 'short', year: 'numeric'
});

};

const fetchDashboardStats = async () => {

  try{

    setLoading(true);

    const endDate = new Date();

    let startDate = new Date();

    switch (stats.dateRange) {

```

```

    case 'last7days': startDate.setDate(startDate.getDate() - 7); break;
    case 'last30days': startDate.setDate(startDate.getDate() - 30); break;
    case 'last90days': startDate.setDate(startDate.getDate() - 90); break;
    case 'lastYear': startDate.setFullYear(startDate.getFullYear() - 1); break;
    default: startDate.setDate(startDate.getDate() - 30);
  }

  const productsSnapshot = await getDocs(collection(db, 'products'));
  const ordersSnapshot = await getDocs(collection(db, 'orders'));

  let totalRev = 0; ordersSnapshot.docs.forEach(d => totalRev +=
d.data().totalAmount || 0);

  const usersList = await getAllUsers();
  const reviewsList = await getAllReviews();

  const salesData = await getSalesData(startDate, endDate);
  const salesByCategory = await getSalesByCategory();
  const orderStatusData = await getOrderStatusDistribution();
  const topSellingProducts = await getTopSellingProducts(10);

  const userRegistrationData = await getUserRegistrationData(startDate,
endDate);

  const paymentMethodData = await getRevenueByPaymentMethod();

  setStats({
    totalProducts: productsSnapshot.size,
    totalOrders: ordersSnapshot.size,
    totalUsers: usersList.length,
    totalReviews: reviewsList.length,
  });

```

```

    totalRevenue: totalRev,
    recentOrders: [], // can be filled if needed
    recentUsers: usersList.slice(0, 5),
    salesData: salesData.salesData,
    salesByCategory,
    orderStatusData,
    topSellingProducts,
    userRegistrationData: userRegistrationData.registrationData,
    paymentMethodData,
    dateRange: stats.dateRange
  });
} catch (e) { setError('Error en estadísticas.');
```

finally { setLoading(false); }

```

};

```

```

const handleDateRangeChange = (range) => setStats(prev => ({ ...prev,
dateRange: range }));

```

```

const fetchProducts = async () => {
  try {
    setProductsLoading(true);
    const snap = await getDocs(collection(db, 'products'));
    setProducts(snap.docs.map(d => ({ id: d.id, ...d.data() })));
  } catch (err) { setError('Error en productos.');
```

finally { setProductsLoading(false); }

```

};

```

```

const fetchUsers = async () => { try{ setUsersLoading(true); setUsers(await
getAllUsers()); } catch (err) { setError('Error en usuarios.')} finally{
setUsersLoading(false); } };

const fetchReviews = async () => { try{ setReviewsLoading(true); setReviews(await
getAllReviews()); } catch (err) { setError('Error en reviews.')} finally{
setReviewsLoading(false); } };

const fetchOrders = async () => { try{ setOrdersLoading(true); setOrders(await
getAllOrders()); } catch (err) { setError('Error en pedidos.')} finally{
setOrdersLoading(false); } };

useEffect(() => {
  if(isAdmin) {
    switch (activeSection) {
      case 'analytics': fetchDashboardStats(); break;
      case 'products': fetchProducts(); fetchCategories(); break;
      case 'users': fetchUsers(); break;
      case 'reviews': fetchReviews(); break;
      case 'orders': fetchOrders(); break;
      case 'categories': fetchCategories(); break;
      default: fetchDashboardStats();
    }
  }
}, [activeSection, isAdmin, stats.dateRange]);

const handleProductChange = (e) => setNewProduct({ ...newProduct,
[e.target.name]: e.target.value });

```

```

const handleEditProductChange = (e) => setEditingProduct({ ...editingProduct,
[e.target.name]: e.target.value });

const handleImageSelect = async (isEditing = false) => {
  const fileInput = document.createElement('input');
  fileInput.type = 'file'; fileInput.accept = 'image/*'; fileInput.multiple = true;
  fileInput.onChange = async (e) => {
    const files = Array.from(e.target.files);
    if(!files.length) return;

    const currentImages = isEditing ? editingProduct.imageUrls :
newProduct.imageUrls;

    if(currentImages.length + files.length > 4) { setError("Máximo 4 imágenes.");
return; }

    setUploading(true);
    const imageDataArray = [];
    for (const file of files) {
      const base64 = await fileToBase64(file);

      imageDataArray.push({ name: file.name, type: file.type, data: base64,
timestamp: Date.now() });
    }

    if(isEditing) setEditingProduct({ ...editingProduct, imageUrls:
[...editingProduct.imageUrls, ...imageDataArray] });

    else setNewProduct({ ...newProduct, imageUrls: [...newProduct.imageUrls,
...imageDataArray] });

    setUploading(false);
  };
};

```

```

    fileInput.click();
  };

const removeImage = (idx, isEditing = false) => {
  if(isEditing) {
    const imgs = [...editingProduct.imageUrls]; imgs.splice(idx, 1);
    setEditingProduct({ ...editingProduct, imageUrls: imgs });
  } else {
    const imgs = [...newProduct.imageUrls]; imgs.splice(idx, 1);
    setNewProduct({ ...newProduct, imageUrls: imgs });
  }
};

const setPrimaryImage = (idx, isEditing = false) => {
  if(isEditing) setEditingProduct({ ...editingProduct, primaryImageIndex: idx });
  else setNewProduct({ ...newProduct, primaryImageIndex: idx });
};

const createProduct = async (e) => {
  e.preventDefault();

  try {
    await addDoc(collection(db, "products"), { ...newProduct, price:
parseFloat(newProduct.price), stock: parseInt(newProduct.stock), rating:
parseFloat(newProduct.rating), createdAt: new Date() });

    showSuccessMessage("Producto creado!");
  }

```

```

    setProductModalOpen(false);

    fetchProducts();

    // Auto-Sync to Stripe

    syncProductsToStripe();

  } catch (err) { setError("Error al crear producto."); }

};

```

```

const updateProduct = async (e) => {

  e.preventDefault();

  try{

    await updateDoc(doc(db, "products", editingProduct.id), { ...editingProduct,
    price: parseFloat(editingProduct.price), stock: parseInt(editingProduct.stock), rating:
    parseFloat(editingProduct.rating), updatedAt: new Date() });

    showSuccessMessage("Producto actualizado!");

    setProductModalOpen(false);

    fetchProducts();

    // Auto-Sync to Stripe

    syncProductsToStripe();

  } catch (err) { setError("Error al actualizar."); }

};

```

```

const deleteProduct = async (id, name) => {

  if(!window.confirm(`¿Eliminar ${name}?`)) return;

  try{ await deleteDoc(doc(db, "products", id)); fetchProducts();
  showSuccessMessage("Eliminado."); } catch (err) { setError("Error."); }

};

```



```
};
```

```
const openAddProductModal = () => { setModalMode('add');  
setProductModalOpen(true); };
```

```
const openEditProductModal = (p) => { setEditingProduct({ ...p });  
setModalMode('edit'); setProductModalOpen(true); };
```

```
const syncProductsToStripe = async () => {
```

```
  setStripeSyncing(true);
```

```
  setError(null);
```

```
  try {
```

```
    // Limpiamos los productos para no enviar datos binarios innecesarios (solo lo  
    que Stripe necesita)
```

```
    const cleanProducts = products.map(p => ({
```

```
      id: p.id,
```

```
      name: p.name,
```

```
      price: p.price,
```

```
      description: p.description || "",
```

```
      imageUrl: p.imageUrl || ""
```

```
    }));
```

```
const response = await fetch(`${API_BASE_URL}/sync-products`, {
```

```
  method: 'POST',
```

```
  headers: { 'Content-Type': 'application/json' },
```

```
  body: JSON.stringify({ products: cleanProducts }),
```

```

});

const data = await response.json();

if (!data.success) throw new Error(data.error || 'Error en sincronización');

showSuccessMessage(`Sincronización exitosa: ${data.created} creados,
${data.updated} actualizados.`);

} catch (err) {

  setError("Error al sincronizar con Stripe: " + err.message);

} finally {

  setStripeSyncing(false);

}

};

const updateUserRole = async (userId, newRole) => {

  try { await updateUserServiceRole(userId, newRole); fetchUsers();
showSuccessMessage("Rol actualizado."); } catch (err) { setError("Error."); }

};

const saveEditedReview = async (e) => {

  e.preventDefault();

  try {

    await updateReview(editingReview.id, { rating: editingReview.rating, comment:
editingReview.comment });

    setReviewModalOpen(false);

```

```

    fetchReviews();

    showSuccessMessage("Reseña actualizada.");

    } catch (err) { setError("Error."); }

};

```

```

const deleteReviewHandler = async (id) => {

    if(!window.confirm("¿Eliminar reseña?")) return;

    try{ await deleteReview(id); fetchReviews(); showSuccessMessage("Eliminado."); }
    catch (err) { setError("Error."); }

};

```

```

const openEditReviewModal = (r) => { setEditingReview({ ...r });
setReviewModalOpen(true); };

```

```

const openOrderDetails = (o) => { setSelectedOrder(o);
setOrderModalOpen(true); };

```

```

const closeOrderDetails = () => { setOrderModalOpen(false);
setSelectedOrder(null); };

```

```

const deleteOrder = async (id) => {

    if(!window.confirm("¿Eliminar pedido?")) return;

    try{ await deleteDoc(doc(db, "orders", id)); fetchOrders();
showSuccessMessage("Eliminado."); } catch (err) { setError("Error."); }

};

```

```

const updateOrderStatusHandler = async (id, status) => {

    try{

```

```

    setOrderStatusUpdating(prev => ({ ...prev, [id]: true }));

    await updateOrderStatus(id, status);

    fetchOrders();

    } catch (err) { setError("Error."); } finally { setOrderStatusUpdating(prev => ({
...prev, [id]: false })); }

};

```

```

const navItems = [
  { id: 'analytics', label: 'Analytics', icon: <FiActivity /> },
  { id: 'products', label: 'Productos', icon: <FiGrid /> },
  { id: 'categories', label: 'Categorías', icon: <FiLayers /> },
  { id: 'users', label: 'Clientes', icon: <FiUsers /> },
  { id: 'reviews', label: 'Feedback', icon: <FiStar /> },
  { id: 'coupons', label: 'Cupones', icon: <FiTag /> },
  { id: 'orders', label: 'Órdenes', icon: <FiPackage /> },
];

```

```

const renderContent = () => {
  switch (activeSection) {
    case 'analytics':
      return <DashboardAnalytics
        stats={stats}
        loading={loading}
        theme={theme}
        handleDateRangeChange={handleDateRangeChange}

```

```

    formatCurrency={formatCurrency}
    formatDate={formatDate}
    getStatusBadgeClass={getStatusBadgeClass}
    getOrderStatusText={getOrderStatusText}
  />;
case 'products':
  return <AdminProducts
    products={products}
    productsLoading={productsLoading}
    productSearchTerm={productSearchTerm}
    setProductSearchTerm={setProductSearchTerm}
    theme={theme}
    formatCurrency={formatCurrency}
    openAddProductModal={openAddProductModal}
    openEditProductModal={openEditProductModal}
    deleteProduct={deleteProduct}
    syncProductsToStripe={syncProductsToStripe}
    stripeSyncing={stripeSyncing}
  />;
case 'users':
  return <AdminUsers
    users={users}
    usersLoading={usersLoading}
    userSearchTerm={userSearchTerm}

```

```

    setUserSearchTerm={setUserSearchTerm}
    userRoleFilter={userRoleFilter}
    setUserRoleFilter={setUserRoleFilter}
    theme={theme}
    updateUserRole={updateUserRole}
  />;
case 'reviews':
  return <AdminReviews
    reviews={reviews}
    reviewsLoading={reviewsLoading}
    reviewSearchTerm={reviewSearchTerm}
    setReviewSearchTerm={setReviewSearchTerm}
    theme={theme}
    formatDate={formatDate}
    openEditReviewModal={openEditReviewModal}
    deleteReviewHandler={deleteReviewHandler}
  />;
case 'orders':
  return <AdminOrders
    orders={orders}
    ordersLoading={ordersLoading}
    orderSearchTerm={orderSearchTerm}
    setOrderSearchTerm={setOrderSearchTerm}
    theme={theme}

```

```

    formatDate={formatDate}
    formatCurrency={formatCurrency}
    getStatusBadgeClass={getStatusBadgeClass}
    getOrderStatusText={getOrderStatusText}
    orderStatusUpdating={orderStatusUpdating}
    updateOrderStatusHandler={updateOrderStatusHandler}
    openOrderDetails={openOrderDetails}
    deleteOrder={deleteOrder}
  />;
case 'categories':
  return <AdminCategories
    categories={categories}
    categoriesLoading={categoriesLoading}
    categorySearchTerm={categorySearchTerm}
    setCategorySearchTerm={setCategorySearchTerm}
    theme={theme}
    openAddCategoryModal={openAddCategoryModal}
    openEditCategoryModal={openEditCategoryModal}
    deleteCategoryHandler={deleteCategoryHandler}
  />;
case 'coupons':
  return (
    <div className="w-full">
      <CouponManager theme={theme} />
    </div>
  );

```

```

    </div>

    );

    default:

        return <div className="p-10 font-black text-center uppercase text-slate-400">Section selection required.</div>;

    }

};

const isDark = theme === 'dark';

const bgMain = isDark ? 'bg-[#111218]' : 'bg-slate-50';

const sidebarBg = isDark ? 'bg-[#1a1b26] border-slate-800' : 'bg-white border-slate-100 shadow-xl';

const headerBg = isDark ? 'bg-[#1a1b26]/80 border-slate-800' : 'bg-white/80 border-slate-100 shadow-sm';

const textTitle = isDark ? 'text-slate-100' : 'text-slate-900';

const textSub = isDark ? 'text-slate-400' : 'text-slate-500';

return (

    <div className={`flex overflow-hidden w-full min-h-screen ${bgMain}`} style={{
    fontFamily: 'Inter, sans-serif' }}>

        {/* SUCCESS/ERROR TOASTS */}

        <div className="fixed top-8 right-8 z-[200] space-y-4">

            {successMessage && (

                <div className="flex gap-3 items-center px-6 py-4 text-white bg-emerald-600 rounded-2xl shadow-2xl duration-500 animate-in slide-in-from-right-10">

```



```

        <FiCheck className="text-xl" /> <span className="text-sm font-black
tracking-widest uppercase">{successMessage}</span>

    </div>

    )}

    {error && (

        <div className="flex gap-3 items-center px-6 py-4 text-white bg-rose-600
rounded-2xl shadow-2xl duration-500 animate-in slide-in-from-right-10">

            <FiX className="text-xl" /> <span className="text-sm font-black
tracking-widest uppercase">{error}</span>

        </div>

    )}

</div>

{ /* MOBILE OVERLAY */

    {sidebarOpen && <div className="fixed inset-0 z-[140] bg-black/60
backdrop-blur-sm lg:hidden" onClick={() => setSidebarOpen(false)} />}

    { /* LUXURY SIDEBAR */

        <aside className={`fixed lg:static inset-y-0 left-0 z-[150] w-72 ${sidebarBg}
border-r transform transition-transform duration-500 ease-out lg:translate-x-0
${sidebarOpen ? 'translate-x-0' : '-translate-x-full'}}>

            <div className="flex flex-col px-8 pt-10 pb-10 h-full">

                <div className="flex justify-between items-center mb-16">

                    <div className="flex gap-3 items-center">

                        <div className="flex justify-center items-center w-10 h-10 italic font-
black text-white bg-black rounded-xl shadow-2xl">P</div>

```

```

    <h1 className={`text-xl font-black tracking-tighter ${textTitle}`}>PSG
ADMIN</h1>

</div>

    <button onClick={() => setSidebarOpen(false)} className="p-2 lg:hidden
text-slate-400"><FiX size={24} /> </button>

</div>

<nav className="flex-1 space-y-1">

    <span className="block text-[10px] font-black text-slate-400 uppercase
tracking-[0.3em] mb-4">Command Center</span>

    {navItems.map((item) => (
        <button
            key={item.id}
            onClick={() => { setActiveSection(item.id); setSidebarOpen(false); }}
            className={`flex items-center w-full px-5 py-4 rounded-2xl transition-all
duration-300 group ${
                activeSection === item.id
                    ? 'bg-indigo-600 text-white shadow-xl shadow-indigo-500/20
active:scale-95'
                    : `text-slate-400 hover:bg-indigo-500/5 ${isDark ? 'hover:text-slate-100'
: 'hover:text-indigo-600'}`
            }}
        >

        <span className={`mr-4 transition-transform group-hover:scale-110
${activeSection === item.id ? 'text-white' : 'text-slate-500'}`}>{item.icon}</span>

        <span className="text-sm font-bold tracking-
tight">{item.label}</span>

```

```

        {activeSection === item.id && <FiChevronRight className="ml-auto
opacity-50" />}

        </button>

    )}}

</nav>

</div>

</aside>

{/* MAIN VIEWPORT */}

<div className="flex flex-col flex-1 min-h-0">

    {/* LUXURY TOP NAV */}

    <header className={`flex sticky top-0 justify-between items-center px-4 py-4
border-b backdrop-blur-md md:px-8 md:py-5 z-[100] ${headerBg}`}>

        <div className="flex gap-6 items-center">

            <button onClick={() => setSidebarOpen(true)} className="p-2 lg:hidden
text-slate-500"><FiMenu size={24} /> </button>

            <div>

                <h2 className={`text-sm font-black uppercase tracking-[0.2em]
${textTitle}`}>

                    {navItems.find(i => i.id === activeSection)?.label || 'Dashboard'}

                </h2>

                <div className="flex items-center gap-2 mt-0.5">

                    <FiHome className="text-slate-400" size={12}/>

                    <FiChevronRight className="text-slate-300" size={10}/>

```

```

        <span className="text-[10px] text-slate-400 font-bold uppercase
tracking-widest">Admin</span>

    </div>

</div>

</div>

<div className="flex gap-6 items-center">

    <button onClick={toggleTheme} className={`p-4 rounded-2xl transition-
all active:scale-90 ${isDark ? 'text-amber-400 bg-slate-800' : 'text-indigo-600
shadow-sm bg-slate-100'}}>

        {theme === 'dark' ? <FiSun /> : <FiMoon />}

    </button>

    <div className="relative" ref={profileMenuRef}>

        <div

            onClick={() => setIsProfileMenuOpen(!isProfileMenuOpen)}

            className="flex gap-4 items-center p-1 rounded-2xl transition-all
cursor-pointer hover:bg-slate-500/10 hover:scale-105 active:scale-95"

            >

                <div className={`w-12 h-12 rounded-2xl flex items-center justify-center
font-black text-white shadow-2xl transition-all ${isProfileMenuOpen ? 'ring-4 ring-
indigo-500/20' : ''} ${isDark ? 'bg-indigo-600' : 'bg-black'}}>

                    {currentUser?.email?.charAt(0).toUpperCase()}

                </div>

                <div className="hidden text-left md:block">

                    <p className={`font-bold tracking-tight text-[12px]
${textTitle}}>{currentUser?.email?.split('@')[0]}</p>

```

```
<p className="text-[10px] font-bold text-slate-400 tracking-wider">Administrator</p>
```

```
</div>
```

```
</div>
```

```
{/* LUXURY DROPDOWN */}
```

```
{isProfileMenuOpen && (
```

```
<div className={`absolute right-0 mt-4 w-64 rounded-3xl border shadow-2xl overflow-hidden z-[150] animate-in slide-in-from-top-4 duration-300 ${isDark ? 'bg-[#1a1b26] border-slate-800 shadow-black' : 'bg-white border-slate-100'}}>
```

```
<div className="p-2 space-y-1">
```

```
<button
```

```
onClick={() => { navigate('/'); setIsProfileMenuOpen(false); }}
```

```
className={`flex items-center w-full gap-4 px-5 py-4 rounded-2xl transition-all text-sm font-bold tracking-tight ${isDark ? 'text-slate-300 hover:bg-slate-800 hover:text-white' : 'text-slate-600 hover:bg-slate-50 hover:text-black'}}`
```

```
>
```

```
<FiHome className="text-indigo-500" size={16} />
```

```
<span>Volver a la Web</span>
```

```
</button>
```

```
<div className={`h-[1px] mx-4 ${isDark ? 'bg-slate-800' : 'bg-slate-100'}} />
```

```
<button
```

```
onClick={() => { handleLogout(); setIsProfileMenuOpen(false); }}
```

```
className="flex gap-4 items-center px-5 py-4 w-full text-sm font-bold tracking-tight text-rose-500 rounded-2xl transition-all hover:bg-rose-500/10"
```

```

    >
    <FiLogOut size={16} />
    <span>Cerrar Sesión</span>
  </button>
</div>
</div>
)}
</div>
</div>
</header>

```

```

{/* MAIN SCROLLABLE AREA */}
<main className="overflow-auto flex-1 p-4 md:p-10 lg:p-14">
  <div className="max-w-[1600px] mx-auto">
    {isAdmin ? renderContent() : (
      <div className="flex flex-col justify-center items-center py-20">
        <FiX className="mb-4 text-rose-500" size={48} />
        <p className="text-xl font-black tracking-widest uppercase text-slate-400">Acceso Denegado</p>
        <button onClick={() => navigate('/')} className="px-10 py-4 mt-8 text-xs font-black tracking-widest text-white uppercase bg-black rounded-2xl">Volver al Home</button>
      </div>
    )}
  </div>

```

</main>

</div>

{/ MODALS INTEGRATION */}*

<AdminProductModal

isOpen={productModalOpen}

onClose={() => setProductModalOpen(false)}

theme={theme}

modalMode={modalMode}

newProduct={newProduct}

editingProduct={editingProduct}

categories={categories}

handleProductChange={handleProductChange}

handleEditProductChange={handleEditProductChange}

createProduct={createProduct}

updateProduct={updateProduct}

handleImageSelect={handleImageSelect}

uploading={uploading}

removeImage={removeImage}

setPrimaryImage={setPrimaryImage}

/>

{/ LUXURY REVIEW MODAL */}*

{reviewModalOpen && editingReview && (

```
<div className="fixed inset-0 z-[200] flex items-center justify-center p-6 bg-black/80 backdrop-blur-sm animate-in fade-in duration-300">
```

```
<div className={`w-full max-w-xl rounded-[2.5rem] border shadow-2xl relative animate-in zoom-in-95 duration-300 ${isDark ? 'bg-[#1a1b26] border-slate-800' : 'bg-white border-slate-100'} p-12`}>
```

```
<div className="flex justify-between items-center mb-10">
```

```
<div>
```

```
<h3 className={`text-2xl font-black tracking-tight ${textTitle}`}>Moderación de Reseña</h3>
```

```
<p className={textSub}>Ajusta la calificación o el contenido del feedback.</p>
```

```
</div>
```

```
<button
```

```
onClick={() => setReviewModalOpen(false)}
```

```
className={`p-3 rounded-2xl transition-all duration-300 active:scale-95 group ${isDark ? 'bg-slate-800 text-slate-400 hover:bg-rose-500/10 hover:text-rose-500' : 'bg-slate-50 text-slate-500 hover:bg-rose-500/10 hover:text-rose-500'}}`
```

```
>
```

```
<FiX size={20} className="transition-transform group-hover:rotate-90"/>
```

```
</button>
```

```
</div>
```

```
<form onSubmit={saveEditedReview} className="space-y-8">
```

```
<div className="flex flex-col items-center">
```

```
<span className="text-[10px] font-black uppercase text-slate-400 tracking-widest mb-4">Nivel de Calificación</span>
```

```
<div className="flex gap-3">
```



```

    {[1, 2, 3, 4, 5].map((s) => (
      <button key={s} type="button" onClick={() =>
setEditingReview({...editingReview, rating: s})} className="transition-transform
active:scale-75">

        <FiStar size={32} className={` ${s <= editingReview.rating ? 'fill-
amber-400 text-amber-400 shadow-amber-500/20' : 'text-slate-200 fill-slate-200'} }`
        />

      </button>

    )]}

  </div>

</div>

<div className="space-y-3">

  <label className="text-[10px] font-black uppercase text-slate-400
tracking-widest px-1">Comentario Editado</label>

  <textarea name="comment" rows={5} value={editingReview.comment}
onChange={(e) => setEditingReview({...editingReview, comment: e.target.value})}
className={`w-full px-6 py-4 rounded-2xl border transition-all ${isDark ? 'text-
white bg-slate-900 border-slate-700' : 'bg-slate-50 border-slate-200 text-slate-
900'} }` />

  </div>

  <button type="submit" className="py-5 w-full text-xs font-black
tracking-widest text-white uppercase bg-black rounded-2xl transition-all hover:bg-
indigo-600">Guardar Cambios</button>

</form>

</div>

</div>

)}

```

```

    { /* LUXURY CATEGORY MODAL */ }

    {categoryModalOpen && (

      <div className="fixed inset-0 z-[200] flex items-center justify-center p-6 bg-
black/80 backdrop-blur-sm animate-in fade-in duration-300">

        <div className={`w-full max-w-xl rounded-[2.5rem] border shadow-2xl
animate-in zoom-in-95 duration-300 ${isDark ? 'bg-[#1a1b26] border-slate-800' :
'bg-white border-slate-100'} p-12`}>

          <div className="flex justify-between items-center mb-10">

            <div>

              <h3 className={`text-2xl font-black tracking-tight
${textTitle}`}>{categoryModalMode === 'add' ? 'Nueva Categoría' : 'Editar
Categoría'}</h3>

              <p className={textSub}>Define los parámetros de tu categoría.</p>

            </div>

            <button

              onClick={() => setCategoryModalOpen(false)}

              className={`p-3 rounded-2xl transition-all duration-300 active:scale-95
group ${isDark ? 'bg-slate-800 text-slate-400 hover:bg-rose-500/10 hover:text-
rose-500' : 'bg-slate-50 text-slate-500 hover:bg-rose-500/10 hover:text-rose-500'}}

            >

              <FiX size={20} className="transition-transform group-hover:rotate-
90"/>

            </button>

          </div>

          <form onSubmit={categoryModalMode === 'add' ? createCategoryHandler
: updateCategoryHandler} className="space-y-8">

            <div className="space-y-3">

```

```

    <label className="text-[10px] font-black uppercase text-slate-400
tracking-widest px-1">Nombre de la Categoría</label>

    <input type="text" name="name" value={categoryModalMode ===
'add' ? newCategory.name : editingCategory?.name}
onChange={categoryModalMode === 'add' ? (e) =>
setNewCategory({...newCategory, name: e.target.value}) : (e) =>
setEditingCategory({...editingCategory, name: e.target.value})} className={`w-full
px-6 py-4 rounded-2xl border ${isDark ? 'text-white bg-slate-900 border-slate-700'
: 'bg-slate-50 border-slate-200 text-slate-900'}} required />

  </div>

  <div className="space-y-3">

    <label className="text-[10px] font-black uppercase text-slate-400
tracking-widest px-1">Descripción Conceptual</label>

    <textarea name="description" rows={3} value={categoryModalMode
=== 'add' ? newCategory.description : editingCategory?.description}
onChange={categoryModalMode === 'add' ? (e) =>
setNewCategory({...newCategory, description: e.target.value}) : (e) =>
setEditingCategory({...editingCategory, description: e.target.value})} className={`w-
full px-6 py-4 rounded-2xl border ${isDark ? 'text-white bg-slate-900 border-slate-
700' : 'bg-slate-50 border-slate-200 text-slate-900'}} />

  </div>

  <div className="space-y-3">

    <label className="text-[10px] font-black uppercase text-slate-400
tracking-widest px-1">Visual Identitario</label>

    <div className="flex gap-6 items-center">

      {{{categoryModalMode === 'add' && categoryImagePreview} ||
(categoryModalMode === 'edit' && (editingCategoryImagePreview ||
editingCategory?.imageUrl)) ? (

        <div className="overflow-hidden relative w-24 h-24 rounded-2xl
shadow-2xl group">

```

```

      <img src={categoryModalMode === 'add' ? categoryImagePreview :
(editingCategoryImagePreview || editingCategory?.imageUrl)} alt="Preview"
className="object-cover w-full h-full" />

      <button type="button" onClick={() =>
clearCategoryImage(categoryModalMode === 'edit')} className="flex absolute
inset-0 justify-center items-center text-white opacity-0 transition-opacity bg-rose-
600/60 group-hover:opacity-100"> <FiTrash2/> </button>

    </div>

  ) : (

    <label className={`w-24 h-24 rounded-2xl border-2 border-dashed
flex flex-col items-center justify-center cursor-pointer transition-all ${isDark ? 'bg-
slate-900 border-slate-700 hover:border-indigo-600' : 'bg-slate-50 border-slate-
200 hover:border-indigo-600'}}>

      <FiCamera className="text-slate-300" size={24}/>

      <input type="file" className="hidden" accept="image/*"
onChange={(e) => handleCategoryImageUpload(e, categoryModalMode ===
'edit')} />

    </label>

  )}

  <p className="text-[10px] text-slate-400 font-bold max-w-
[150px]">Recomendado: 800x800px. Máximo 1MB.</p>

</div>

</div>

  <button type="submit" className="py-5 w-full text-xs font-black
tracking-widest text-white uppercase bg-black rounded-2xl shadow-2xl transition-
all hover:bg-indigo-600">

    {categoryModalMode === 'add' ? 'Crear Categoría' : 'Guardar Cambios'}

  </button>

```

```
    </form>
  </div>
</div>
})
```

```
<OrderDetailsModal
  isOpen={orderModalOpen}
  onClose={closeOrderDetails}
  order={selectedOrder}
  formatDate={formatDate}
  formatCurrency={formatCurrency}
  getOrderStatusText={getOrderStatusText}
  getStatusBadgeClass={getStatusBadgeClass}
  theme={theme}
/>
</div>
);
};

export default ModernAdminDashboard;

📁 src/assets/pages/NotFound(Error404)

import React from 'react';
import { useNavigate, Link } from 'react-router-dom';
import { FiHome, FiTrendingUp } from 'react-icons/fi';
```

```

const NotFound = () => {

  const navigate = useNavigate();

  return (

    <div className="min-h-screen bg-gray-50 flex items-center justify-center p-6 pt-32 pb-24">

      <div className="max-w-xl w-full text-center space-y-8 animate-[fadeIn_0.6s_ease-out]">

        {/* Visual Asset Section: Floating 404 with bows/moños theme context if possible */}

        <div className="relative inline-block group">

          <div className="absolute -inset-4 bg-gradient-to-r from-pink-100 to-purple-100 rounded-full blur-3xl opacity-50 group-hover:opacity-100 transition-opacity duration-1000"></div>

          <div className="relative">

            <h1 className="text-[12rem] font-black leading-none bg-clip-text text-transparent bg-gradient-to-b from-gray-900 to-gray-500 tracking-tighter select-none">

              404

            </h1>

            <div className="absolute top-1/2 left-1/2 -translate-x-1/2 -translate-y-1/2">

              {/* SVG Icon: Bow or Moño simplified concept */}

              <div className="w-24 h-24 bg-white/40 backdrop-blur-sm rounded-2xl rotate-12 flex items-center justify-center p-4 border border-white/50 shadow-2xl">

```

```
<svg viewBox="0 0 24 24" className="w-full h-full text-pink-500 fill-current animate-pulse">
```

```
<path d="M7 18c-1.1 0-1.99 9-1.99 25 5.9 22 7 22s2-.9 2-2-.9-2-2-2zM1 2v2h2l3.6 7.59-1.35 2.45c-.16.28-.25.61-.25.96 0 1.1 9 2 2h12v-2H7.42c-.14 0-.25-.11-.25-.25l.03-.12.9-1.63h7.45c.75 0 1.41-.41 1.75-1.03l3.58-6.49c.08-.14.12-.31.12-.48 0-.55-.45-1-1-1H5.21l-.94-2H1zm16 16c-1.1 0-1.99 9-1.99 25 5.9 22 7 22s2-.9 2-2-.9-2-2-2z" />
```

```
</svg>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
{/* Content Section */}
```

```
<div className="space-y-4 relative">
```

```
<h2 className="text-3xl font-bold text-gray-900 tracking-tight">¡Vaya! Se ha perdido el moño...</h2>
```

```
<p className="text-lg text-gray-500 max-w-md mx-auto leading-relaxed">
```

```
Parece que la página que buscas no está en stock o ha cambiado su ubicación. No te preocupes, ¡tenemos mucho más para ver!
```

```
</p>
```

```
</div>
```

```
{/* CTA Section */}
```

```
<div className="flex flex-col sm:flex-row items-center justify-center gap-4 pt-4">
```

```
<Link
```

```

    to="/"

    className="w-full sm:w-auto inline-flex items-center justify-center gap-2
px-8 py-4 bg-gray-900 text-white font-semibold rounded-2xl hover:bg-gray-800
transform active:scale-95 transition-all shadow-xl shadow-gray-200"

  >

    <FiHome className="text-lg" />

    Regresar al Inicio

  </Link>

  <button

    onClick={() => navigate('/shop')}

    className="w-full sm:w-auto inline-flex items-center justify-center gap-2
px-8 py-4 bg-white text-gray-700 font-semibold rounded-2xl border border-gray-
200 hover:border-gray-900 hover:bg-gray-50 transform active:scale-95 transition-
all"

  >

    <FiTrendingUp className="text-lg text-pink-500" />

    Ver Novedades

  </button>

</div>

{ /* Navigation Support Links */ }

<div className="pt-12 grid grid-cols-1 sm:grid-cols-3 gap-6 opacity-60">

  <Link to="/contact" className="text-sm font-medium hover:text-gray-900
hover:underline">Soporte</Link>

  <Link to="/blog" className="text-sm font-medium hover:text-gray-900
hover:underline">Blog</Link>

```



```
      <Link to="/profile" className="text-sm font-medium hover:text-gray-900
hover:underline">Mi Cuenta</Link>
```

```
    </div>
```

```
  </div>
```

```
</div>
```

```
);
```

```
};
```

```
export default NotFound;
```

 **src/assets/pages/Orders.jsx**

```
import { useCart } from '../context/CartContext';
```

```
import { getOrdersByUserId } from '../services/orderService';
```

```
import OrderDetailsModal from '../components/OrderDetailsModal';
```

```
import { createReview, getUserReviewForProduct, updateReview, deleteReview }
from '../services/reviewService';
```

```
import Swal from 'sweetalert2';
```

```
const Orders = () => {
```

```
  const { currentUser } = useAuth();
```

```
  const { clearCart } = useCart();
```

```
  const navigate = useNavigate();
```

```
  const getPrimaryImageUrl = (product) => {
```

```
    if (product.imageUrls && product.imageUrls.length > 0) {
```

```

    const valid = product.imageUrls.filter(img => img && (typeof img === 'string'
    || (img.data && typeof img.data === 'string')));

    if(valid.length > 0) {

        const safe = Math.min(product.primaryImageIndex || 0, valid.length - 1);

        const img = valid[safe];

        if(typeof img === 'string') return img;

        if(img && img.data) return img.data;

    }

}

return product.imageUrl || 'https://via.placeholder.com/100x100.png?text=PSG';

};

```

```

const [orders, setOrders] = useState([]);

const [productImages, setProductImages] = useState({}); // Cache para imágenes faltantes

const [loading, setLoading] = useState(true);

const [error, setError] = useState(null);

const [selectedOrder, setSelectedOrder] = useState(null);

const [isModalOpen, setIsModalOpen] = useState(false);

const [reviewingProduct, setReviewingProduct] = useState(null);

const getOrderStatusText = (status) => {

    switch (status) {

        case 'pending': return 'Pendiente';

        case 'processing': return 'Procesando';

    }

}

```

```

    case 'shipped': return 'Enviado';
    case 'delivered': return 'Entregado';
    case 'cancelled': return 'Cancelado';
    default: return status;
  }
};

```

```

const getStatusBadge = (status) => {
  switch (status) {
    case 'pending': return 'bg-amber-50 text-amber-700 border border-amber-200';
    case 'processing': return 'bg-blue-50 text-blue-700 border border-blue-200';
    case 'shipped': return 'bg-indigo-50 text-indigo-700 border border-indigo-200';
    case 'delivered': return 'bg-green-50 text-green-700 border border-green-200';
    case 'cancelled': return 'bg-red-50 text-red-700 border border-red-200';
    default: return 'bg-gray-100 text-gray-700 border border-gray-200';
  }
};

```

```

const formatDate = (date) => {
  if(!date) return 'N/A';
  if(date instanceof Date) return date.toLocaleDateString('es-CO');
  if(date.toDate) return date.toDate().toLocaleDateString('es-CO');
  return 'N/A';
};

```

```

const formatCurrency = (amount) => new Intl.NumberFormat('es-CO', { style:
'currency', currency: 'COP', minimumFractionDigits: 0 }).format(amount || 0);

const openOrderDetails = (order) => { setSelectedOrder(order);
setIsModalOpen(true); };

const closeOrderDetails = () => { setIsModalOpen(false); setSelectedOrder(null); };

const openReviewForm = async (product) => {
  try{
    const existingReview = await getUserReviewForProduct(currentUser.uid,
product.id);
    if (existingReview) {
      setReviewingProduct({ id: existingReview.id, productId: product.id,
productName: product.name, rating: existingReview.rating, comment:
existingReview.comment, isEditing: true });
    } else{
      setReviewingProduct({ productId: product.id, productName: product.name,
rating: 0, comment: '', isEditing: false });
    }
  } catch (error) {
    setReviewingProduct({ productId: product.id, productName: product.name,
rating: 0, comment: '', isEditing: false });
  }
};

const handleReviewChange = (e) => {

```

```

const { name, value } = e.target;

setReviewingProduct(prev => ({ ...prev, [name]: value }));

};

const handleRatingClick = (rating) => setReviewingProduct(prev => ({ ...prev,
rating }));

const submitReview = async () => {

  if(reviewingProduct.rating === 0) { Swal.fire({ title: 'Error', text: 'Por favor
selecciona una calificación', icon: 'error', confirmButtonText: 'Aceptar' }); return; }

  if(reviewingProduct.comment.trim() === '') { Swal.fire({ title: 'Error', text: 'Por
favor escribe un comentario', icon: 'error', confirmButtonText: 'Aceptar' }); return; }

  try{

    const reviewData = { productId: reviewingProduct.productId, userId:
currentUser.uid, userEmail: currentUser.email, userName: currentUser.displayName
|| currentUser.email.split('@')[0], rating: reviewingProduct.rating, comment:
reviewingProduct.comment.trim() };

    if(reviewingProduct.isEditing) {

      await updateReview(reviewingProduct.id, reviewData);

      Swal.fire({ title: 'Reseña actualizada', text: 'Tu reseña ha sido actualizada
correctamente', icon: 'success', confirmButtonText: 'Aceptar' });

    } else{

      await createReview(reviewData);

      Swal.fire({ title: 'Reseña enviada', text: 'Tu reseña ha sido enviada
correctamente', icon: 'success', confirmButtonText: 'Aceptar' });

    }

    setReviewingProduct(null);

```

```

    } catch (error) {

        Swal.fire({ title: 'Error', text: 'Hubo un error al procesar tu reseña. Por favor
        intenta nuevamente.', icon: 'error', confirmButtonText: 'Aceptar' });

    }

};

const deleteReviewHandler = async () => {

    if(!reviewingProduct || !reviewingProduct.isEditing) return;

    Swal.fire({ title: '¿Estás seguro?', text: 'Esta acción eliminará tu reseña
    permanentemente', icon: 'warning', showCancelButton: true, confirmButtonColor:
    '#d33', cancelButtonColor: '#6b7280', confirmButtonText: 'Sí, eliminar',
    cancelButtonText: 'Cancelar' }).then(async (result) => {

        if(result.isConfirmed) {

            try{

                await deleteReview(reviewingProduct.id);

                Swal.fire({ title: 'Reseña eliminada', text: 'Tu reseña ha sido eliminada
                correctamente', icon: 'success', confirmButtonText: 'Aceptar' });

                setReviewingProduct(null);

            } catch (error) {

                Swal.fire({ title: 'Error', text: 'Hubo un error al eliminar tu reseña. Por favor
                intenta nuevamente.', icon: 'error', confirmButtonText: 'Aceptar' });

            }

        }

    });

};

```

```

useEffect(() => {

  // Check if coming from a successful payment (Universal detection for
  HashRouter or standard routes)

  const currentPath = window.location.hash || window.location.search;

  if(currentPath.includes('payment=success')) {

    // Find orderId in path

    const urlParams = new URLSearchParams(currentPath.split('?')[1]);

    const orderId = urlParams.get('orderId');

    const finalizeOrder = async () => {

      if(orderId) {

        try{

          await updateDoc(doc(db, 'orders', orderId), {

            paymentStatus: 'paid',

            updatedAt: new Date()

          });

        } catch (e) {

          console.error("Error updating order status:", e);

        }

      }

      clearCart();

      Swal.fire({

        title: '¡Pago Exitoso!',

        text: 'Tu pedido ha sido procesado correctamente con Stripe.',

        icon: 'success',

```

```

    confirmButtonText: 'Aceptar',
    timer: 5000
  });

  // Clean up the URL to avoid repeated alerts

  const cleanUrl =
window.location.href.replace(/[/?&]payment=success(&orderId=[\w-]+)?/, '');

  window.history.replaceState({}, document.title, cleanUrl);

  fetchUserOrders(); // Refresh list
};

finalizeOrder();
}

```

```

const fetchUserOrders = async () => {
  if(!currentUser) { navigate('/login'); return; }

  try{
    const orderList = await getOrdersByUserId(currentUser.uid);
    setOrders(orderList);

    // Cargar las imágenes de los productos que tengan imageUrl null

    const missingImgIds = [];
    orderList.forEach(order => {
      order.items?.forEach(item => {
        if(!item.imageUrl && item.id) {
          missingImgIds.push(item.id);
        }
      });
    });
  } catch (error) {
    console.log(error);
  }
}

```



```

    }
  });
});

if(missingImgIds.length > 0) {
  const uniqueIds = [...new Set(missingImgIds)];
  const imagesMap = { ...productImages };

  for(const id of uniqueIds) {
    if(!imagesMap[id]) {
      try {
        const prodRef = doc(db, 'products', id);
        const prodSnap = await getDoc(prodRef);
        if(prodSnap.exists()) {
          imagesMap[id] = getPrimaryImageUrl(prodSnap.data());
        }
      } catch (e) {
        console.warn(`No se pudo cargar imagen para producto ${id}`, e);
      }
    }
  }

  setProductImages(imagesMap);
}

} catch (err) {

```

```

        setError('No se pudieron cargar tus órdenes. Por favor, intenta nuevamente.');
```

```

    } finally {

        setLoading(false);

    }

};

fetchUserOrders();

}, [currentUser, navigate]);

if(loading) {

    return (

        <div className="min-h-screen bg-gray-50 flex items-center justify-center"
        style={{ fontFamily: 'Inter, sans-serif' }}>

            <div className="w-8 h-8 border-2 border-gray-900 border-t-transparent
            rounded-full animate-spin" />

            </div>

        );

    }

    return (

        <div className="min-h-screen bg-gray-50" style={{ fontFamily: 'Inter, sans-serif'
        }}>

            <div className="px-4 py-10 mx-auto max-w-4xl sm:px-6 lg:px-8">

                {/* Header */}

                <div className="flex items-center justify-between mb-8">

```

```

<div>

  <h1 className="text-2xl font-bold tracking-tight text-gray-900">Mis
Pedidos</h1>

  <p className="mt-1 text-sm text-gray-500">Consulta el estado de tus
pedidos recientes</p>

</div>

{orders.length > 0 && (

  <span className="inline-flex items-center px-3 py-1 rounded-full text-xs
font-medium bg-gray-100 text-gray-700">

    {orders.length} pedido{orders.length !== 1 ? 's' : ''}

  </span>

  )}

</div>

{error && (

  <div className="mb-6 flex gap-3 p-4 bg-red-50 border border-red-200
rounded-xl">

    <svg className="w-5 h-5 text-red-500 flex-shrink-0 mt-0.5"
fill="currentColor" viewBox="0 0 20 20"> <path fillRule="evenodd" d="M10 18a8 8
0 100-16 8 8 0 00 16zM8.707 7.293a1 1 0 00-1.414 1.414L8.586 10l-1.293 1.293a1
1 0 101.414 1.414L10 11.414l1.293 1.293a1 1 0 001.414-1.414L11.414 10l1.293-
1.293a1 1 0 00-1.414-1.414L10 8.586 8.707 7.293z" clipRule="evenodd" /> </svg>

    <p className="text-sm text-red-700">{error}</p>

  </div>

  )}

{orders.length === 0 ? (

```

```

    <div className="text-center py-16 bg-white rounded-2xl border border-
gray-200 shadow-sm">

    <div className="flex items-center justify-center w-14 h-14 mx-auto
rounded-full bg-gray-100 mb-4">

    <svg className="w-7 h-7 text-gray-400" fill="none" viewBox="0 0 24 24"
stroke="currentColor">

    <path strokeLinecap="round" strokeLinejoin="round" strokeWidth="1.5"
d="M9 5H7a2 2 0 00-2 2v12a2 2 0 002 2h10a2 2 0 002 2V7a2 2 0 00-2 2h-2M9
5a2 2 0 002 2h2a2 2 0 002 2M9 5a2 2 0 012 2h2a2 2 0 012 2" />

    </svg>

    </div>

    <h3 className="text-base font-semibold text-gray-900">No tienes
pedidos</h3>

    <p className="mt-1 text-sm text-gray-500">Aún no has realizado ningún
pedido.</p>

    <Link to="/shop" className="inline-flex items-center mt-6 px-5 py-2.5
text-sm font-medium text-white bg-black hover:bg-gray-800 rounded-lg
transition-colors duration-150">

    Comprar ahora

    </Link>

    </div>

  ): (

    <div className="space-y-4">

    {orders.map((order) => (

    <div key={order.id} className="bg-white border border-gray-200
rounded-2xl shadow-sm overflow-hidden hover:border-gray-300 transition-colors
duration-150">

    {/* Order Header */}

```

```

    <div className="px-6 py-4 border-b border-gray-100 flex flex-col
sm:flex-row sm:items-center sm:justify-between gap-3">

      <div className="flex items-center gap-3">

        <div className="flex items-center justify-center w-10 h-10 rounded-
xl bg-gray-100 flex-shrink-0">

          <svg className="w-5 h-5 text-gray-600" fill="none" viewBox="0 0
24 24" stroke="currentColor">

            <path strokeLinecap="round" strokeLinejoin="round"
strokeWidth="1.5" d="M5 8h14M5 8a2 2 0 110-4h14a2 2 0 110 4M5 8v10a2 2 0
002 2h10a2 2 0 002-2V8m-9 4h4" />

          </svg>

        </div>

      <div>

        <h3 className="text-sm font-semibold text-gray-900">Pedido
#{order.id.substring(0, 8).toUpperCase()}</h3>

        <p className="text-xs text-gray-500">Realizado el
{formatDate(order.createdAt)}</p>

      </div>

    </div>

    <div className="flex items-center gap-2">

      <span className={`inline-flex items-center px-2.5 py-1 rounded-full
text-xs font-medium ${getStatusBadge(order.orderStatus)}}`>

        {getOrderStatusText(order.orderStatus)}

      </span>

      <button

        onClick={() => openOrderDetails(order)}


```

```
        className="inline-flex items-center px-3 py-1.5 text-xs font-medium
text-gray-700 bg-white border border-gray-300 rounded-lg hover:bg-gray-50
transition-colors duration-150"
```

```
>
```

```
Ver detalles
```

```
</button>
```

```
</div>
```

```
</div>
```

```
{/* Order Summary */}
```

```
<div className="px-6 py-4">
```

```
<div className="grid grid-cols-2 sm:grid-cols-4 gap-4 mb-5">
```

```
{[
```

```
{ label: 'Total', value: formatCurrency(order.totalAmount) },
```

```
{ label: 'Productos', value: `${order.items?.length || 0}
```

```
ítem${(order.items?.length || 0) !== 1 ? 's' : ''}`,
```

```
{ label: 'Método de Pago', value: order.paymentMethod || 'N/A' },
```

```
{ label: 'Estado de Pago', value: order.paymentStatus || 'N/A' },
```

```
].map(({ label, value }) => (
```

```
<div key={label} className="bg-gray-50 rounded-xl p-3">
```

```
<p className="text-xs text-gray-500 mb-0.5">{label}</p>
```

```
<p className="text-sm font-semibold text-gray-900
capitalize">{value}</p>
```

```
</div>
```

```
)))
```

```
</div>
```

```

    {/* Products */}

    <div>

        <h4 className="text-xs font-semibold text-gray-500 uppercase
tracking-widest mb-3">Productos</h4>

        <div className="space-y-2">

            {order.items?.map((item, index) => (

                <div key={index} className="flex items-center justify-between
gap-4 p-3 rounded-xl border border-gray-100 hover:border-gray-200 transition-
colors">

                    <div className="flex items-center gap-3">

                        <div className="w-12 h-12 rounded-lg overflow-hidden border
border-gray-200 flex-shrink-0">

                            <img

                                src={item.imageUrl || productImages[item.id] ||
'https://via.placeholder.com/48x48.png?text=PSG'}

                                alt={item.name}

                                className="object-cover w-full h-full"

                                onError={(e) => { e.target.onerror = null; e.target.src =
'https://via.placeholder.com/48x48.png?text=PSG'; }}

                            />

                        </div>

                        <div>

                            <p className="text-sm font-medium text-gray-
900">{item.name}</p>

                            <p className="text-xs text-gray-500">Cant: {item.quantity} ·
{formatCurrency(item.price * item.quantity)}</p>

```

```

        </div>

    </div>

    <button
        onClick={() => openReviewForm(item)}

        className="inline-flex items-center px-3 py-1.5 text-xs font-
medium text-gray-700 bg-white border border-gray-300 rounded-lg hover:bg-
gray-900 hover:text-white hover:border-gray-900 transition-all duration-150
whitespace-nowrap flex-shrink-0"

        >

        Dejar reseña

    </button>

</div>

    )})

</div>

</div>

</div>

    )})

</div>

    })

</div>

```

```

{/* Review Modal */

{reviewingProduct && (

    <div className="fixed inset-0 z-50 flex items-center justify-center p-4 bg-
black/40 backdrop-blur-sm">

```



```

    <div className="relative w-full max-w-md bg-white rounded-2xl shadow-
2xl border border-gray-200 overflow-hidden">

    <div className="px-6 py-5 border-b border-gray-100 flex items-center
justify-between">

    <div>

    <h3 className="text-base font-semibold text-gray-
900">{reviewingProduct.isEditing ? 'Editar reseña' : 'Dejar una reseña'}</h3>

    <p className="text-xs text-gray-500 mt-
0.5">{reviewingProduct.productName}</p>

    </div>

    <button onClick={() => setReviewingProduct(null)} className="w-8 h-8
flex items-center justify-center rounded-full text-gray-400 hover:bg-gray-100
hover:text-gray-600 transition-colors">X</button>

    </div>

    <div className="px-6 py-5">

    <div className="mb-4">

    <label className="block text-xs font-semibold text-gray-600 uppercase
tracking-wide mb-2">Calificación</label>

    <div className="flex items-center gap-1">

    {[1, 2, 3, 4, 5].map((star) => (

    <button key={star} type="button" onClick={() =>
handleRatingClick(star)} className="text-2xl focus:outline-none transition-
transform hover:scale-110">

    {star <= reviewingProduct.rating ? <span className="text-yellow-
400">★</span> : <span className="text-gray-300">☆</span>}

    </button>

    )))

```

```

        <span className="ml-2 text-xs text-gray-500">
            {reviewingProduct.rating > 0 ? `${reviewingProduct.rating}/5` :
'Selecciona una calificación'}
        </span>
    </div>
</div>
<div className="mb-5">
    <label htmlFor="review-comment" className="block text-xs font-
semibold text-gray-600 uppercase tracking-wide mb-2">Comentario</label>
    <textarea
        id="review-comment" name="comment" rows={4}
        value={reviewingProduct.comment} onChange={handleReviewChange}
        className="block w-full px-3 py-2.5 text-sm text-gray-900 bg-white
border border-gray-300 rounded-lg focus:outline-none focus:ring-2 focus:ring-
black focus:border-transparent resize-none placeholder-gray-400"
        placeholder="Comparte tu experiencia con este producto..."
    />
</div>
<div className="flex gap-2 justify-end">
    {reviewingProduct.isEditing && (
        <button onClick={deleteReviewHandler} className="px-4 py-2 text-sm
font-medium text-red-600 bg-white border border-red-200 rounded-lg hover:bg-
red-50 transition-colors">Eliminar</button>
    )}
    <button onClick={() => setReviewingProduct(null)} className="px-4 py-
2 text-sm font-medium text-gray-700 bg-white border border-gray-300 rounded-
lg hover:bg-gray-50 transition-colors">Cancelar</button>

```

```

        <button onClick={submitReview} className="px-4 py-2 text-sm font-
medium text-white bg-black rounded-lg hover:bg-gray-800 transition-colors">
            {reviewingProduct.isEditing ? 'Actualizar' : 'Enviar reseña'}
        </button>
    </div>
</div>
</div>
</div>
))

```

```

<OrderDetailsModal
    order={selectedOrder} isOpen={isModalOpen} onClose={closeOrderDetails}
    formatDate={formatDate} formatCurrency={formatCurrency}
    getOrderStatusText={getOrderStatusText}
    getStatusBadgeClass={getStatusBadgeClass}
/>
</div>
);
};

```

export default Orders;

 **src/assets/pages/ProductDetail.jsx**

```

import React, { useState, useEffect } from 'react';
import { useParams, Link, useNavigate } from 'react-router-dom';
// Removed duplicate Navbar import since it's already rendered in App.jsx

```

```

import { getProductById, getProductsByCategory } from
'../services/productService';

import { useCart } from '../context/CartContext';

import { useWishlist } from '../context/WishlistContext';

import { useAuth } from '../context/AuthContext';

import StarRating from '../components/StarRating';

import Swal from 'sweetalert2';

import { getReviewsByProductId } from '../services/reviewService';

```

```

const ProductDetail = () => {
  const { id } = useParams();
  const [product, setProduct] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const [quantity, setQuantity] = useState(1);
  const [selectedImageIndex, setSelectedImageIndex] = useState(0);
  const { addToCart } = useCart();
  const { addToWishlist } = useWishlist();
  const { currentUser } = useAuth();
  const navigate = useNavigate();

```

// Review states

```

const [reviews, setReviews] = useState([]);
const [reviewLoading, setReviewLoading] = useState(true);

```

// Related products states

```
const [relatedProducts, setRelatedProducts] = useState([]);
```

```
const [relatedLoading, setRelatedLoading] = useState(true);
```

```
useEffect(() => {
```

```
  const fetchProduct = async () => {
```

```
    try {
```

```
      setLoading(true);
```

```
      const productData = await getProductById(id);
```

```
      setProduct(productData);
```

```
      // Reset selected image index when product changes
```

```
      setSelectedImageIndex(0);
```

```
      // Fetch related products based on category
```

```
      if(productData.category) {
```

```
        try {
```

```
          setRelatedLoading(true);
```

```
          const related = await getProductsByCategory(productData.category);
```

```
          // Filter out the current product and limit to 4 related products
```

```
          const filteredRelated = related
```

```
            .filter(p => p.id !== productData.id)
```

```
            .slice(0, 4);
```

```
          setRelatedProducts(filteredRelated);
```

```
        } catch (err) {
```

```

        console.error('Error fetching related products:', err);
      } finally {
        setRelatedLoading(false);
      }
    }
  } catch (err) {
    setError('Failed to load product');
    console.error('Error fetching product:', err);
  } finally {
    setLoading(false);
  }
};

if(id) {
  fetchProduct();
}
}, [id]);

```

// Fetch reviews when product loads

```

useEffect(() => {
  const fetchReviews = async () => {
    if(id) {
      try {
        setReviewLoading(true);

```

```

    const reviewsData = await getReviewsByProductId(id);
    setReviews(reviewsData);
  } catch (err) {
    console.error('Error fetching reviews:', err);
  } finally {
    setReviewLoading(false);
  }
}
};

fetchReviews();
}, [id]);

// Get the primary image URL for a product
const getPrimaryImageUrl = (product) => {
  if(product.imageUrls && product.imageUrls.length > 0) {
    // Filter out empty images
    const validImages = product.imageUrls.filter(image =>
      image && (typeof image === 'string' || (image.data && typeof image.data
        === 'string'))
    );
    if(validImages.length > 0) {
      const primaryIndex = product.primaryImageIndex || 0;
      // Ensure primaryIndex is within bounds
      const safeIndex = Math.min(primaryIndex, validImages.length - 1);

```

```

const primaryImage = validImages[safeIndex];

// Handle both string URLs and base64 data objects
if(typeof primaryImage === 'string') {
  return primaryImage;
} else if(primaryImage && primaryImage.data) {
  return primaryImage.data;
}
}
}

return product.imageUrl ||
'https://via.placeholder.com/600x600.png?text= Moño';
};

```

```

const handleAddToCart = () => {

// Check if user is logged in
if(!currentUser) {
  Swal.fire({
    title: 'Inicia sesión',
    text: 'Debes iniciar sesión para agregar productos al carrito',
    icon: 'warning',
    confirmButtonText: 'Iniciar sesión',
    showCancelButton: true,
    cancelButtonText: 'Cancelar'
  }).then((result) => {

```



```

    if(result.isConfirmed) {
        // Redirect to login page
        navigate('/login');
    }
});

return;
}

if(product) {
    // Check if requested quantity exceeds stock
    const stockLimit = product.stock !== undefined ? product.stock : Infinity;
    if(stockLimit === 0) {
        Swal.fire({
            title: 'Producto agotado',
            text: 'Este producto no está disponible actualmente.',
            icon: 'error',
            confirmButtonText: 'Aceptar'
        });
        return;
    }

    // Get the primary product image as selected in the admin panel
    const mainImage = getPrimaryImageUrl(product);

```

```

addToCart({
  ...product,
  imageUrl: mainImage, // Ensure we use the primary image selected in admin panel
  quantity: 1
});

```

```

Swal.fire({
  title: 'Producto agregado',
  text: 'Producto agregado correctamente al carrito',
  icon: 'success',
  confirmButtonText: 'Aceptar'
}).then(() => {
  // Optionally redirect to cart after successful addition
  // navigate('/cart');
});
}
};

```

// Added wishlist handler with authentication check

```

const handleAddToWishlist = () => {
  // Check if user is logged in
  if(!currentUser) {
    Swal.fire({
      title: 'Inicia sesión',

```

```

    text: 'Debes iniciar sesión para agregar productos a tu lista de deseos',
    icon: 'warning',
    confirmButtonText: 'Iniciar sesión',
    showCancelButton: true,
    cancelButtonText: 'Cancelar'
  }).then((result) => {
    if(result.isConfirmed) {
      // Redirect to login page
      navigate('/login');
    }
  });
  return;
}

if(product) {
  // Get the primary product image as selected in the admin panel
  const mainImage = getPrimaryImageUrl(product);

  addToWishlist({
    ...product,
    imageUrl: mainImage // Ensure we use the primary image selected in admin
    panel
  });

  Swal.fire({

```

```

    title: 'Producto agregado',
    text: 'Producto agregado correctamente a tu lista de deseos',
    icon: 'success',
    confirmButtonText: 'Aceptar'
  });
}
};

```

```

// Get all valid image URLs for the product
const getProductImages = () => {
  if(product?.imageUrls && Array.isArray(product.imageUrls)) {
    // Filter out empty or invalid images
    const validImages = product.imageUrls.filter(image =>
      image && (typeof image === 'string' || (image.data && typeof image.data
        === 'string'))
    );

    if(validImages.length > 0) {
      // Extract URLs from either string URLs or base64 data objects
      const imageUrls = validImages.map(image => {
        if(typeof image === 'string') {
          return image;
        } else if(image && image.data) {
          return image.data;
        }
      }
    }
  }
}

```

```

    return 'https://via.placeholder.com/600x600.png?text=Moño';
  });
  return imageUrls;
}

return ['https://via.placeholder.com/600x600.png?text=Moño'];
}

// Fallback to single imageUrl if imageUrls array doesn't exist
if(product?.imageUrl) {
  return [product.imageUrl];
}

return ['https://via.placeholder.com/600x600.png?text=Moño'];
};

// Get the currently selected image URL
const getSelectedImageUrl = () => {
  const images = getProductImages();
  return images[selectedImageIndex] || images[0];
};

// Format date for display
const formatDate = (date) => {
  if(!date) return 'N/A';
  if(date instanceof Date) {

```

```

    return date.toLocaleDateString('es-CO');
  }
  if(date.toDate) {
    return date.toDate().toLocaleDateString('es-CO');
  }
  return 'N/A';
};

// Handle adding related product to cart
const handleAddRelatedToCart = (product) => {
  // Check if user is logged in
  if(!currentUser) {
    Swal.fire({
      title: 'Inicia sesión',
      text: 'Debes iniciar sesión para agregar productos al carrito',
      icon: 'warning',
      confirmButtonText: 'Iniciar sesión',
      showCancelButton: true,
      cancelButtonText: 'Cancelar'
    }).then((result) => {
      if(result.isConfirmed) {
        // Redirect to login page
        navigate('/login');
      }
    })
  }
}

```

```
});  
  
    return;  
}
```

```
// Check if product has stock
```

```
const stockLimit = product.stock !== undefined ? product.stock : Infinity;
```

```
if(stockLimit === 0) {
```

```
    Swal.fire({
```

```
        title: 'Producto agotado',
```

```
        text: 'Este producto no está disponible actualmente.',
```

```
        icon: 'error',
```

```
        confirmButtonText: 'Aceptar'
```

```
    });
```

```
    return;  
}
```

```
// Get the primary product image as selected in the admin panel
```

```
const mainImage = getPrimaryImageUrl(product);
```

```
addToCart({
```

```
    ...product,
```

```
    imageUrl: mainImage, // Ensure we use the primary image selected in admin panel
```

```
    quantity: 1
```

```
});
```

```

Swal.fire({
  title: 'Producto agregado',
  text: 'Producto agregado correctamente al carrito',
  icon: 'success',
  confirmButtonText: 'Aceptar'
});
};

```

```

if(loading) {
  return (
    // Removed duplicate Navbar since it's already rendered in App.jsx
    <div>
      <div className="flex items-center justify-center h-48 sm:h-64">
        <div className="w-8 h-8 sm:w-12 sm:h-12 border-b-2 border-gray-900
rounded-full animate-spin"> </div>
      </div>
    </div>
  );
}

```

```

if(error || !product) {
  return (
    // Removed duplicate Navbar since it's already rendered in App.jsx
    <div>

```



```

    <div className="px-4 py-6 sm:py-8 mx-auto max-w-7xl sm:px-6 lg:px-8">
      <div className="relative px-3 py-2 text-xs sm:text-sm sm:px-4 sm:py-3 text-
red-700 bg-red-100 border border-red-400 rounded" role="alert">
        <strong className="font-bold">Error! </strong>
        <span className="block sm:inline">{error || 'Product not found'}</span>
      </div>
    </div>
  </div>
);
}

```

```
const images = getProductImages();
```

```

return (
  // Removed duplicate Navbar since it's already rendered in App.jsx
  <div className="bg-white min-h-screen">
    <div className="max-w-7xl mx-auto px-4 py-6 sm:py-8 sm:px-6 lg:px-8">
      <nav className="flex mb-4 sm:mb-6" aria-label="Breadcrumb">
        <ol className="flex items-center space-x-1 sm:space-x-2">
          <li>
            <Link to="/home" className="text-xs sm:text-sm text-gray-500
hover:text-black uppercase tracking-widest transition-colors duration-
200">Home</Link>
          </li>
          <li>

```

```

    <span className="text-xs sm:text-sm text-gray-500">/</span>
  </li>
  <li>
    <Link to="/shop" className="text-xs sm:text-sm text-gray-500
hover:text-black uppercase tracking-widest transition-colors duration-
200">Shop</Link>
  </li>
  <li>
    <span className="text-xs sm:text-sm text-gray-500">/</span>
  </li>
  <li>
    <span className="text-xs sm:text-sm text-gray-900 font-medium
truncate max-w-[100px] sm:max-w-none">{product.name}</span>
  </li>
</ol>
</nav>

<div className="lg:grid lg:grid-cols-2 lg:gap-x-12 lg:items-start">
  {/* Image gallery */}
  <div className="flex flex-col-reverse">
    {/* Image selector */}
    {images.length > 1 && (
      <div className="w-full max-w-2xl mx-auto mt-6 lg:max-w-none">
        <div className="grid grid-cols-4 gap-2 sm:gap-4" aria-
orientation="horizontal">
          {images.map((image, index) => (

```

```

<button
  key={index}

  className={`relative h-16 sm:h-20 bg-gray-100 rounded-lg flex
items-center justify-center text-xs sm:text-sm font-medium uppercase text-gray-
900 cursor-pointer hover:bg-gray-100 focus:outline-none transition-all duration-
200 ${

    selectedIndex === index ? 'ring-2 ring-indigo-500' : 'border
border-white/10'

  }}

  onClick={() => setSelectedImageIndex(index)}
>

  <span className="sr-only">Imagen {index + 1}</span>

  <span className="absolute inset-0 overflow-hidden rounded-md">

    <img

      src={image}

      alt={`Imagen ${index + 1}`}

      className="object-cover w-full h-full"

      onError={(e) => {

        e.target.onerror = null;

        e.target.src =
'https://via.placeholder.com/100x100.png?text=Moño';

      }}

    />

  </span>

  {selectedIndex === index && (

```

```
        <span className="absolute inset-0 rounded-md pointer-events-
none ring-2 ring-offset-2 ring-indigo-500" aria-hidden="true"> </span>
```

```
    }}
```

```
  </button>
```

```
  )}}
```

```
</div>
```

```
</div>
```

```
}}
```

```
{/* Main image */}
```

```
<div className="w-full aspect-w-1 aspect-h-1">
```

```
  <img
```

```
    src={getSelectedImageUrl()}>
```

```
    alt={product.name}>
```

```
    className="object-contain object-center w-full h-full"
```

```
    onError={(e) => {
```

```
      e.target.onerror = null;
```

```
      e.target.src = 'https://via.placeholder.com/600x600.png?text= Moño';
```

```
    }}
```

```
  />
```

```
</div>
```

```
</div>
```

```
{/* Product info */}
```

```
<div className="px-4 mt-8 sm:px-0 sm:mt-10 lg:mt-0">
```

```

    <h1 className="text-2xl sm:text-3xl font-bold tracking-tight text-gray-
900">{product.name}</h1>

    <div className="mt-3 sm:mt-4 flex flex-wrap items-center gap-2 sm:gap-
4">

        {product.rating > 0 && (

            <div className="flex items-center">

                <StarRating rating={product.rating} size="md" />

                <span className="ml-2 text-xs sm:text-sm text-gray-
500">({product.rating})</span>

            </div>

        )}

        <span className="text-xs sm:text-sm text-gray-500">SKU:
{product.id?.substring(0, 8)}</span>

    </div>

    <div className="mt-4 sm:mt-6">

        <h2 className="sr-only">Product information</h2>

        <p className="text-2xl sm:text-3xl font-bold text-
black">${parseFloat(product.price).toLocaleString('es-CO')}</p>

    </div>

    <div className="mt-4 sm:mt-6">

        <h3 className="sr-only">Description</h3>

        <div className="prose prose-indigo text-gray-600">

```

```
<p className="text-sm sm:text-base leading-relaxed">{product.description}</p>
```

```
</div>
```

```
</div>
```

```
<div className="mt-6 sm:mt-8 border-t border-b border-white/10 py-4 sm:py-6">
```

```
<div className="grid grid-cols-1 sm:grid-cols-2 gap-3 sm:gap-4">
```

```
<div className="flex items-center">
```

```
<span className="text-xs sm:text-sm text-gray-500 font-medium">Categoría:</span>
```

```
<span className="ml-2 inline-flex items-center px-2 py-1 sm:px-3 sm:py-1 rounded-full text-xs sm:text-sm font-medium bg-gradient-to-r from-indigo-900/50 to-purple-900/50 text-gray-800">
```

```
{product.category}
```

```
</span>
```

```
</div>
```

```
{product.stock !== undefined && (
```

```
<div className="flex items-center">
```

```
<span className="text-xs sm:text-sm text-gray-500 font-medium">Disponibilidad:</span>
```

```
<span className={`ml-2 text-xs sm:text-sm font-medium ${product.stock > 5 ? 'text-green-600' : product.stock > 0 ? 'text-yellow-600' : 'text-red-600'}`}>
```

```
{product.stock > 0 ? `${product.stock} en stock` : 'Agotado'}
```

```
</span>
```

```
</div>
```

```
}}
```

```
{product.material && (
```

```
<div className="flex items-center">
```

```
<span className="text-xs sm:text-sm text-gray-500 font-  
medium">Material:</span>
```

```
<span className="ml-2 text-xs sm:text-sm text-gray-  
900">{product.material}</span>
```

```
</div>
```

```
}}
```

```
{product.color && (
```

```
<div className="flex items-center">
```

```
<span className="text-xs sm:text-sm text-gray-500 font-  
medium">Color:</span>
```

```
<span className="ml-2 text-xs sm:text-sm text-gray-  
900">{product.color}</span>
```

```
</div>
```

```
}}
```

```
{product.size && (
```

```
<div className="flex items-center">
```

```
<span className="text-xs sm:text-sm text-gray-500 font-  
medium">Tamaño:</span>
```

```

        <span className="ml-2 text-xs sm:text-sm text-gray-
900">{product.size}</span>

        </div>

    )}

    {product.style && (

        <div className="flex items-center">

            <span className="text-xs sm:text-sm text-gray-500 font-
medium">Estilo:</span>

            <span className="ml-2 text-xs sm:text-sm text-gray-
900">{product.style}</span>

            </div>

        )}

    </div>

</div>

```

```

<form className="mt-6 sm:mt-8">

    <div className="flex flex-wrap gap-3 sm:gap-4">

        <button

            type="button"

            onClick={handleAddToCart}

            className="flex-1 min-w-[150px] sm:min-w-[200px] px-4 sm:px-6 py-
2.5 sm:py-3 border border-transparent text-sm sm:text-base font-medium
rounded-lg text-gray-900 bg-black text-white hover:from-indigo-400 hover:to-
purple-500 focus:outline-none focus:ring-2 focus:ring-offset-2 focus:ring-indigo-
500 shadow-xl shadow-sm hover:shadow-2xl shadow-sm transition-all duration-
300 transform hover:-translate-y-0.5"

```



```

        disabled={product.stock === 0}
      >
        {product.stock === 0 ? 'Agotado' : 'Agregar al Carrito'}
      </button>

    <button
      type="button"
      onClick={handleAddToWishlist}
      className="flex items-center justify-center px-4 sm:px-5 py-2.5 sm:py-3 border border-slate-600 rounded-lg shadow-2xl shadow-sm text-sm sm:text-base font-medium text-gray-600 bg-white border border-gray-200 hover:bg-white focus:outline-none focus:ring-2 focus:ring-offset-2 focus:ring-indigo-500 transition-all duration-300"
    >
      <svg className="flex-shrink-0 w-4 h-4 sm:w-5 sm:h-5 text-gray-500 hover:text-red-500 transition-colors duration-200" fill="none" viewBox="0 0 24 24" stroke="currentColor">
        <path strokeLinecap="round" strokeLinejoin="round" strokeWidth="2" d="M4.318 6.318a4.5 4.5 0 00 6.364 12 20.364 17.682 7.682 4.5 4.5 0 00-6.364-6.364L12 7.636l-1.318-1.318a4.5 4.5 0 00-6.364 0z" />
      </svg>
      <span className="sr-only">Agregar a favoritos</span>
    </button>
  </div>
</form>
</div>
</div>

```

```

{ /* Reviews Section */}

<div className="mt-12 sm:mt-20">

  <div className="border-t border-white/10 pt-8 sm:pt-12">

    <h2 className="text-xl sm:text-2xl font-bold text-gray-900 mb-6 sm:mb-8">Reseñas de Clientes</h2>

    { /* Reviews List - Only showing existing reviews, no form */}

    <div>

      {reviewLoading ? (

        <div className="flex justify-center py-8 sm:py-12">

          <div className="w-8 h-8 sm:w-10 sm:h-10 border-b-2 border-indigo-600 rounded-full animate-spin"></div>

        </div>

      ) : reviews.length === 0 ? (

        <div className="text-center py-8 sm:py-12 bg-white border border-gray-200 rounded-2xl shadow-2xl shadow-sm shadow-sm p-6 sm:p-8">

          <svg className="mx-auto h-10 w-10 sm:h-12 sm:w-12 text-gray-400" fill="none" viewBox="0 0 24 24" stroke="currentColor">

            <path strokeLinecap="round" strokeLinejoin="round" strokeWidth="2" d="M8 10h.01M12 10h.01M16 10h.01M9 16H5a2 2 0 0 1-2-2V6a2 2 0 0 1-2-2h14a2 2 0 0 12-2v8a2 2 0 0 1-2 2h-5l-5 5v-5z" />

          </svg>

          <h3 className="mt-2 text-base sm:text-lg font-medium text-gray-900">No hay reseñas aún</h3>

          <p className="mt-1 text-sm text-gray-500">

            Este producto aún no tiene reseñas.

          </p>

```

```

    </p>
  </div>
) : (
  <div className="space-y-4 sm:space-y-6">
    {reviews.map((review) => (
      <div key={review.id} className="bg-white border border-gray-200
rounded-2xl shadow-2xl shadow-sm shadow-sm p-4 sm:p-6 border border-gray-
200 hover:shadow-xl shadow-sm transition-shadow duration-300">
        <div className="flex flex-wrap items-center justify-between gap-
2">
          <div>
            <h4 className="text-base sm:text-lg font-semibold text-gray-
900">{review.userName}</h4>
            <div className="mt-1 flex items-center">
              <StarRating rating={review.rating} size="sm" />
              <span className="ml-2 text-xs sm:text-sm text-gray-
500">{review.rating}/5</span>
            </div>
          </div>
          <p className="text-xs sm:text-sm text-gray-
500">{formatDate(review.createdAt)}</p>
        </div>
        <div className="mt-3 sm:mt-4">
          <p className="text-sm text-gray-600 leading-
relaxed">{review.comment}</p>
        </div>
      </div>
    )
  )
}
</div>

```

```

    )}
  </div>

  )}
</div>

</div>

</div>

```

```

{/* Related Products Section */}
<div className="mt-12 sm:mt-20">

  <div className="border-t border-white/10 pt-8 sm:pt-12">

    <h2 className="text-xl sm:text-2xl font-bold text-gray-900 mb-6 sm:mb-8">Productos Relacionados</h2>

```

```

    {relatedLoading ? (

      <div className="flex items-center justify-center h-24 sm:h-32">

        <div className="w-6 h-6 sm:w-8 sm:h-8 border-b-2 border-indigo-600 rounded-full animate-spin"></div>

      </div>

    ) : relatedProducts.length === 0 ? (

      <div className="text-center py-6 sm:py-8 bg-white border border-gray-200 rounded-2xl shadow-2xl shadow-sm shadow-sm p-4 sm:p-6">

        <svg className="mx-auto h-10 w-10 sm:h-12 sm:w-12 text-gray-400 fill="none" viewBox="0 0 24 24" stroke="currentColor">

          <path strokeLinecap="round" strokeLinejoin="round" strokeWidth="2" d="M20 7l-8-4-8 4m8-4v10l-8 4m0-10L4 7m8 4v10M4 7v10l8 4" />

        </svg>

```

```
<h3 className="mt-2 text-base sm:text-lg font-medium text-gray-900">No hay productos relacionados</h3>
```

```
<p className="mt-1 text-sm text-gray-500">
```

```
  No encontramos otros productos en la misma categoría.
```

```
</p>
```

```
</div>
```

```
): (
```

```
<div className="grid grid-cols-1 gap-4 sm:gap-6 sm:grid-cols-2 lg:grid-cols-4">
```

```
  {relatedProducts.map((relatedProduct) => (
```

```
    <div key={relatedProduct.id} className="overflow-hidden transition-all duration-300 bg-white border border-gray-200 rounded-lg shadow-xl shadow-sm hover:shadow-2xl shadow-sm">
```

```
      <Link to={`/product/${relatedProduct.id}`}>
```

```
        <div className="w-full overflow-hidden aspect-w-1 aspect-h-1">
```

```
          <img
```

```
            src={getPrimaryImageUrl(relatedProduct)}
```

```
            alt={relatedProduct.name}
```

```
            className="object-cover w-full h-32 sm:h-48"
```

```
            onError={(e) => {
```

```
              e.target.onerror = null;
```

```
              e.target.src =
```

```
              'https://via.placeholder.com/300x300.png?text=Moño';
```

```
            }}
```

```
          />
```

```
        </div>
```

```

</Link>

<div className="p-3 sm:p-4">

  <h3 className="text-base sm:text-lg font-medium text-gray-900
truncate">{relatedProduct.name}</h3>


  {relatedProduct.rating > 0 && (
    <div className="mt-1">
      <StarRating rating={relatedProduct.rating} size="sm" />
    </div>
  )}

  <div className="mt-2">
    <span className="text-base sm:text-lg font-semibold text-
black">${parseFloat(relatedProduct.price).toLocaleString('es-CO')}</span>
  </div>

  <button
    className="w-full mt-3 sm:mt-4 px-3 py-2 text-xs sm:text-sm font-
medium text-gray-900 transition-all duration-300 border border-transparent
rounded-md shadow-2xl shadow-sm shadow-sm bg-black text-white hover:from-
indigo-400 hover:to-purple-500 focus:outline-none focus:ring-2 focus:ring-offset-2
focus:ring-indigo-500"
    onClick={(e) => {
      e.preventDefault();
      handleAddRelatedToCart(relatedProduct);
    }}
  >

```

```

        >
        Agregar al Carrito
      </button>
    </div>
  </div>
  )))
</div>
  })
</div>
</div>
</div>
</div>
);
};

export default ProductDetail;

```

 **src/assets/pages/Profile.jsx**

```

import React, { useState, useEffect } from 'react';
import { useAuth } from '../context/AuthContext';
import { useNavigate } from 'react-router-dom';
import { db, auth } from '../firebase';

import { doc, getDoc, setDoc, updateDoc, arrayUnion, arrayRemove } from
'firebase/firestore';

import { updatePassword, EmailAuthProvider, reauthenticateWithCredential } from
'firebase/auth';

import Swal from 'sweetalert2';

```

```
import {  
  FaRegUser,  
  FaShieldHalved,  
  FaMapLocationDot,  
  FaCamera,  
  FaLaptopCode,  
  FaRegTrashCan,  
  FaPen,  
  FaCheck,  
  FaPlus  
} from 'react-icons/fa6';
```

```
const Profile = () => {  
  const { currentUser, userRole, refreshUserData } = useAuth();  
  const navigate = useNavigate();
```

```
  // Profile state
```

```
  const [profileData, setProfileData] = useState({  
    displayName: "",  
    email: "",  
    phone: "",  
    profileImage: ""  
  });
```


// Security state

```
const [securityData, setSecurityData] = useState({  
  currentPassword: "",  
  newPassword: "",  
  confirmNewPassword: ""  
});
```

// Addresses state

```
const [addresses, setAddresses] = useState([]);  
const [newAddress, setNewAddress] = useState({  
  id: Date.now(),  
  name: "",  
  street: "",  
  city: "",  
  state: "",  
  zipCode: "",  
  country: 'Colombia'  
});
```

// Editing address state

```
const [editingAddress, setEditingAddress] = useState(null);  
const [editAddressData, setEditAddressData] = useState({  
  id: null,  
  name: "",
```

```
street: "",
city: "",
state: "",
zipCode: "",
country: 'Colombia'
});
```

```
const [activeTab, setActiveTab] = useState('profile');
const [loading, setLoading] = useState(true);
const [saving, setSaving] = useState(false);
const [securitySaving, setSecuritySaving] = useState(false);
const [addressSaving, setAddressSaving] = useState(false);
```

```
// Load user profile data
```

```
useEffect(() => {
  const loadProfileData = async () => {
    if(!currentUser) {
      navigate('/login');
      return;
    }

    try{
      const userDocRef = doc(db, 'users', currentUser.uid);
      const userDocSnap = await getDoc(userDocRef);
```

```

if(userDocSnap.exists()) {
  const userData = userDocSnap.data();
  setProfileData({
    displayName: userData.displayName || currentUser.displayName || "",
    email: currentUser.email,
    phone: userData.phone || "",
    profileImage: userData.profileImage || currentUser.photoURL || ""
  });

  setAddresses(userData.addresses || []);
} else {
  // If no profile exists, initialize with basic data from Firebase user
  setProfileData({
    displayName: currentUser.displayName || "",
    email: currentUser.email,
    phone: "",
    profileImage: currentUser.photoURL || ""
  });
  setAddresses([]);
}
} catch (error) {
  console.error('Error loading profile:', error);
  Swal.fire({

```

```

        title: 'Error',
        text: 'Error al cargar el perfil',
        icon: 'error',
        confirmButtonText: 'Aceptar',
        confirmButtonColor: '#000'
    });

    } finally {
        setLoading(false);
    }
};

loadProfileData();
}, [currentUser, navigate]);

// Handle profile input changes
const handleProfileChange = (e) => {
    const { name, value } = e.target;
    setProfileData(prev => ({
        ...prev,
        [name]: value
    }));
};

// Handle security input changes

```

```

const handleSecurityChange = (e) => {
  const { name, value } = e.target;
  setSecurityData(prev => ({
    ...prev,
    [name]: value
  }));
};

```

// Handle address input changes for new address

```

const handleAddressChange = (e) => {
  const { name, value } = e.target;
  setNewAddress(prev => ({
    ...prev,
    [name]: value
  }));
};

```

// Handle address input changes for editing address

```

const handleEditAddressChange = (e) => {
  const { name, value } = e.target;
  setEditAddressData(prev => ({
    ...prev,
    [name]: value
  }));
};

```

```
};
```

```
// Handle profile image upload
```

```
const handleImageUpload = (e) => {  
  const file = e.target.files[0];  
  if(file) {  
    if(file.size > 2 * 1024 * 1024) {  
      Swal.fire({  
        title: 'Archivo muy grande',  
        text: 'La imagen debe ser menor a 2MB',  
        icon: 'warning',  
        confirmButtonText: 'Aceptar',  
        confirmButtonColor: '#000'  
      });  
      return;  
    }  
  }
```

```
  if(!file.type.match('image.*')) {  
    Swal.fire({  
      title: 'Tipo de archivo inválido',  
      text: 'Por favor selecciona una imagen',  
      icon: 'warning',  
      confirmButtonText: 'Aceptar',  
      confirmButtonColor: '#000'
```

```

    });

    return;
  }

  const reader = new FileReader();
  reader.onload = (e) => {
    setProfileData(prev => ({
      ...prev,
      profileImage: e.target.result
    }));
  };
  reader.readAsDataURL(file);
}

};

// Save profile data

const handleSaveProfile = async (e) => {
  e.preventDefault();
  setSaving(true);

  try {
    const userDocRef = doc(db, 'users', currentUser.uid);
    const userDocSnap = await getDoc(userDocRef);

```

```

let userDataToSave = {
  ...profileData,
  updatedAt: new Date()
};

if (userDocSnap.exists()) {
  const existingData = userDocSnap.data();

  if (existingData.hasOwnProperty('role')) userDataToSave.role =
existingData.role;

  if (existingData.hasOwnProperty('createdAt')) userDataToSave.createdAt =
existingData.createdAt;

  if (existingData.hasOwnProperty('addresses')) userDataToSave.addresses =
existingData.addresses;
} else {
  userDataToSave.role = 'customer';
  userDataToSave.createdAt = new Date();
  userDataToSave.addresses = [];
}

await setDoc(userDocRef, userDataToSave);

console.log('Profile saved, refreshing user data');
refreshUserData();

Swal.fire({

```



```

    title: 'Perfil actualizado',
    text: 'Tu perfil se ha actualizado correctamente',
    icon: 'success',
    confirmButtonText: 'Aceptar',
    confirmButtonColor: '#000'
  });
} catch (error) {
  console.error('Error saving profile:', error);
  Swal.fire({
    title: 'Error',
    text: 'Error al guardar el perfil: ' + error.message,
    icon: 'error',
    confirmButtonText: 'Aceptar',
    confirmButtonColor: '#000'
  });
} finally {
  setSaving(false);
}
};

```

// Change password

```

const handleChangePassword = async (e) => {
  e.preventDefault();
  setSecuritySaving(true);

```

```
if(securityData.newPassword !== securityData.confirmNewPassword) {  
  Swal.fire({  
    title: 'Error',  
    text: 'Las contraseñas nuevas no coinciden',  
    icon: 'error',  
    confirmButtonText: 'Aceptar',  
    confirmButtonColor: '#000'  
  });  
  setSecuritySaving(false);  
  return;  
}
```

```
if(securityData.newPassword.length < 6) {  
  Swal.fire({  
    title: 'Error',  
    text: 'La nueva contraseña debe tener al menos 6 caracteres',  
    icon: 'error',  
    confirmButtonText: 'Aceptar',  
    confirmButtonColor: '#000'  
  });  
  setSecuritySaving(false);  
  return;  
}
```

```

try{
  const credential = EmailAuthProvider.credential(
    currentUser.email,
    securityData.currentPassword
  );
  await reauthenticateWithCredential(currentUser, credential);
  await updatePassword(currentUser, securityData.newPassword);

  Swal.fire({
    title: 'Contraseña actualizada',
    text: 'Tu contraseña se ha actualizado correctamente',
    icon: 'success',
    confirmButtonText: 'Aceptar',
    confirmButtonColor: '#000'
  });

  setSecurityData({
    currentPassword: '',
    newPassword: '',
    confirmNewPassword: ''
  });
} catch (error) {
  console.error('Error changing password:', error);
}

```

```

let errorMessage = 'Error al cambiar la contraseña';

switch (error.code) {
  case 'auth/wrong-password':
    errorMessage = 'La contraseña actual es incorrecta';
    break;
  case 'auth/too-many-requests':
    errorMessage = 'Demasiados intentos. Por favor, inténtalo más tarde';
    break;
  default:
    errorMessage = 'Error al cambiar la contraseña: ' + error.message;
}

Swal.fire({
  title: 'Error',
  text: errorMessage,
  icon: 'error',
  confirmButtonText: 'Aceptar',
  confirmButtonColor: '#000'
});
} finally {
  setSecuritySaving(false);
}
};

```

```

// Add new address

const handleAddAddress = async (e) => {

  e.preventDefault();

  setAddressSaving(true);

  if(!newAddress.name || !newAddress.street || !newAddress.city ||
!newAddress.state || !newAddress.zipCode) {

    Swal.fire({

      title: 'Error',

      text: 'Por favor completa todos los campos de la dirección',

      icon: 'error',

      confirmButtonText: 'Aceptar',

      confirmButtonColor: '#000'

    });

    setAddressSaving(false);

    return;
  }

  try{

    const userDocRef = doc(db, 'users', currentUser.uid);

    const newAddressWithId = { ...newAddress, id: Date.now() };

    if (addresses.length === 0 || newAddress.isDefault) {

      newAddressWithId.isDefault = true;

```

```

    const updatedAddresses = addresses.map(addr => ({ ...addr, isDefault: false
  }));

  setAddresses([...updatedAddresses, newAddressWithId]);

} else {
  setAddresses([...addresses, newAddressWithId]);
}

```

```

await updateDoc(userDocRef, {
  addresses: arrayUnion(newAddressWithId)
});

```

```

Swal.fire({
  title: 'Dirección agregada',
  text: 'La dirección se ha agregado correctamente',
  icon: 'success',
  confirmButtonText: 'Aceptar',
  confirmButtonColor: '#000'
});

```

```

setNewAddress({
  id: Date.now(),
  name: "",
  street: "",
  city: "",
  state: "",

```

```

        zipCode: '',
        country: 'Colombia'
    });
} catch (error) {
    console.error('Error adding address:', error);
    Swal.fire({
        title: 'Error',
        text: 'Error al agregar la dirección: ' + error.message,
        icon: 'error',
        confirmButtonText: 'Aceptar',
        confirmButtonColor: '#000'
    });
} finally {
    setAddressSaving(false);
}
};

```

```

const startEditingAddress = (address) => {
    setEditingAddress(address.id);
    setEditAddressData({...address});
};

```

```

const cancelEditingAddress = () => {
    setEditingAddress(null);
}

```

```
setEditAddressData({
  id: null,
  name: "",
  street: "",
  city: "",
  state: "",
  zipCode: "",
  country: 'Colombia'
});
};
```

```
const saveEditedAddress = async (e) => {
  e.preventDefault();
  setAddressSaving(true);

  if(!editAddressData.name || !editAddressData.street || !editAddressData.city ||
!editAddressData.state || !editAddressData.zipCode) {
    Swal.fire({
      title: 'Error',
      text: 'Por favor completa todos los campos de la dirección',
      icon: 'error',
      confirmButtonText: 'Aceptar',
      confirmButtonColor: '#000'
    });
    setAddressSaving(false);
  }
}
```



```

    return;
  }

  try{

    const userDocRef = doc(db, 'users', currentUser.uid);

    const addressToRemove = addresses.find(addr => addr.id ===
editAddressData.id);

    if(addressToRemove) {

      await updateDoc(userDocRef, {

        addresses: arrayRemove(addressToRemove)

      });

    }

    const updatedAddressWithId = { ...editAddressData };

    await updateDoc(userDocRef, {

      addresses: arrayUnion(updatedAddressWithId)

    });

    setAddresses(addresses.map(addr => addr.id === editAddressData.id ?
updatedAddressWithId : addr));

    Swal.fire({

      title: 'Dirección actualizada',

      text: 'La dirección se ha actualizado correctamente',

```

```

        icon: 'success',
        confirmButtonText: 'Aceptar',
        confirmButtonColor: '#000'
    });

    setEditingAddress(null);
    setEditAddressData({
        id: null,
        name: "",
        street: "",
        city: "",
        state: "",
        zipCode: "",
        country: 'Colombia'
    });
} catch (error) {
    console.error('Error updating address:', error);
    Swal.fire({
        title: 'Error',
        text: 'Error al actualizar la dirección: ' + error.message,
        icon: 'error',
        confirmButtonText: 'Aceptar',
        confirmButtonColor: '#000'
    });
}

```

```

    } finally{
      setAddressSaving(false);
    }
  };

```

```

const handleDeleteAddress = async (addressId) => {
  Swal.fire({
    title: '¿Estás seguro?',
    text: 'Esta acción no se puede deshacer',
    icon: 'warning',
    showCancelButton: true,
    confirmButtonText: 'Sí, eliminar',
    cancelButtonText: 'Cancelar',
    confirmButtonColor: '#ef4444',
    cancelButtonColor: '#6b7280'
  }).then(async (result) => {
    if (result.isConfirmed) {
      try{
        const userDocRef = doc(db, 'users', currentUser.uid);
        const addressToDelete = addresses.find(addr => addr.id === addressId);

        if(addressToDelete) {
          await updateDoc(userDocRef, { addresses: arrayRemove(addressToDelete)
        });

        setAddresses(addresses.filter(addr => addr.id !== addressId));

```

```

    Swal.fire({
      title: 'Eliminada',
      text: 'La dirección fue eliminada',
      icon: 'success',
      confirmButtonText: 'Aceptar',
      confirmButtonColor: '#000'

    });
  }
} catch (error) {
  console.error('Error deleting address:', error);
  Swal.fire({
    title: 'Error',
    text: 'Error al eliminar la dirección: ' + error.message,
    icon: 'error',
    confirmButtonText: 'Aceptar',
    confirmButtonColor: '#000'

  });
}
}

});
};

```

```

const handleSetDefaultAddress = async (addressId) => {

```

```

try{

  const userDocRef = doc(db, 'users', currentUser.uid);
  const updatedAddresses = addresses.map(addr => ({
    ...addr,
    isDefault: addr.id === addressId
  }));
  setAddresses(updatedAddresses);
  await updateDoc(userDocRef, { addresses: updatedAddresses });

  Swal.fire({
    title: 'Actualizado',
    text: 'La dirección se estableció como predeterminada',
    icon: 'success',
    confirmButtonText: 'Aceptar',
    confirmButtonColor: '#000',
    timer: 1500
  });
} catch (error) {
  console.error('Error setting default address:', error);
  Swal.fire({
    title: 'Error',
    text: 'Error al establecer: ' + error.message,
    icon: 'error',
    confirmButtonText: 'Aceptar',
  });
}

```

```

        confirmButtonColor: '#000'

    });

}

};

if (loading) {

    return (

        <div className="flex items-center justify-center min-h-[60vh] bg-[#fdfdfd]">

            <div className="w-8 h-8 border-2 border-t-black border-gray-200 rounded-
full animate-spin"> </div>

        </div>

    );

}

return (

    <div className="min-h-screen bg-[#fdfdfd] pb-24 font-inter text-gray-900"
style={{ fontFamily: 'Inter, sans-serif' }}>

        {/* Header Compacto */}

        <div className="bg-white border-b border-gray-100">

            <div className="px-5 py-8 mx-auto max-w-5xl">

                <h1 className="text-2xl font-bold tracking-tight text-black sm:text-
3xl">Configuración</h1>

                <p className="mt-1 text-sm text-gray-500">Administra tu información
personal y direcciones de envío.</p>

            </div>

```

```
</div>
```

```
<div className="px-5 mx-auto max-w-5xl mt-8">
```

```
<div className="flex flex-col md:flex-row gap-8">
```

```
{/* Sidebar Tabs */}
```

```
<aside className="w-full md:w-64 flex-shrink-0">
```

```
<nav className="flex flex-row md:flex-col gap-2 overflow-x-auto pb-4  
md:pb-0 scrollbar-hide">
```

```
<button
```

```
onClick={() => setActiveTab('profile')}
```

```
className={`flex items-center gap-3 px-4 py-3 rounded-xl text-sm font-  
medium transition-colors whitespace-nowrap ${
```

```
activeTab === 'profile'
```

```
? 'bg-gray-100 text-black'
```

```
: 'text-gray-500 hover:text-black hover:bg-gray-50'
```

```
}}`
```

```
>
```

```
<FaRegUser size={16} />
```

```
Perfil Personal
```

```
</button>
```

```
<button
```

```
onClick={() => setActiveTab('security')}
```

```
className={`flex items-center gap-3 px-4 py-3 rounded-xl text-sm font-  
medium transition-colors whitespace-nowrap ${
```

```

        activeTab === 'security'
        ? 'bg-gray-100 text-black'
        : 'text-gray-500 hover:text-black hover:bg-gray-50'
    }}
  >
    <FaShieldHalved size={16} />
    Seguridad
  </button>
  <button
    onClick={() => setActiveTab('addresses')}
    className={`flex items-center gap-3 px-4 py-3 rounded-xl text-sm font-
medium transition-colors whitespace-nowrap ${
      activeTab === 'addresses'
      ? 'bg-gray-100 text-black'
      : 'text-gray-500 hover:text-black hover:bg-gray-50'
    }}
  >
    <FaMapLocationDot size={16} />
    Mis Direcciones
  </button>
</nav>
</aside>

{/* Content Area */}
<main className="flex-1 min-w-0">

```



```

{ /* — Perfil Tab — */
{activeTab === 'profile' && (
  <div className="space-y-8 animate-fadeIn">
    <div className="bg-white border border-gray-100 shadow-sm
rounded-2xl p-6 sm:p-8">
      <h2 className="text-lg font-bold text-black mb-6">Información
Personal</h2>

      <form onSubmit={handleSaveProfile} className="space-y-8">
        { /* Img Upload */
        <div className="flex items-center gap-6">
          <div className="relative group">
            {profileData.profileImage ? (
              <img
                src={profileData.profileImage}
                alt="Profile"
                className="object-cover w-20 h-20 rounded-full bg-gray-100
border border-gray-200"
              />
            ) : (
              <div className="flex items-center justify-center w-20 h-20 bg-
gray-100 text-gray-400 rounded-full border border-gray-200">
                <span className="text-2xl font-bold text-gray-500">
                  {profileData.displayName?.charAt(0).toUpperCase() ||
profileData.email?.charAt(0).toUpperCase() || 'U'}

```

```

        </span>
      </div>
    })

    <label className="absolute inset-0 flex items-center justify-center
bg-black/50 text-white rounded-full opacity-0 group-hover:opacity-100 transition-
opacity cursor-pointer">

      <FaCamera size={18} />

      <input
        type="file"
        className="hidden"
        accept="image/*"
        onChange={handleImageUpload}
      />
    </label>
  </div>

  <div>
    <h3 className="text-sm font-semibold text-black">Foto de
Perfil</h3>

    <p className="text-xs text-gray-500 mt-0.5">JPG, GIF o PNG. Max
2MB.</p>
  </div>
</div>

{ /* Inputs */ }

<div className="grid grid-cols-1 gap-6 sm:grid-cols-2">

  <div>

```

```

        <label htmlFor="displayName" className="block text-xs font-
semibold text-gray-700 uppercase tracking-widest mb-2">
            Nombre Completo
        </label>
        <input
            type="text"
            name="displayName"
            id="displayName"
            value={profileData.displayName}
            onChange={handleProfileChange}
            className="w-full px-4 py-3 bg-gray-50 border-none rounded-xl
focus:ring-2 focus:ring-black focus:bg-white transition-colors sm:text-sm"
        />
    </div>

```

```

    <div>
        <label htmlFor="email" className="block text-xs font-semibold
text-gray-700 uppercase tracking-widest mb-2">
            Correo Electrónico
        </label>
        <input
            type="email"
            name="email"
            id="email"
            value={profileData.email}

```

disabled

className="w-full px-4 py-3 bg-gray-100 text-gray-500 border-none rounded-xl cursor-not-allowed sm:text-sm"

/>

</div>

<div>

<label *htmlFor*="phone" *className*="block text-xs font-semibold text-gray-700 uppercase tracking-widest mb-2">

Teléfono

</label>

<input

type="tel"

name="phone"

id="phone"

value={profileData.phone}

onChange={handleProfileChange}

className="w-full px-4 py-3 bg-gray-50 border-none rounded-xl focus:ring-2 focus:ring-black focus:bg-white transition-colors sm:text-sm"

/>

</div>

<div>

<label *htmlFor*="role" *className*="block text-xs font-semibold text-gray-700 uppercase tracking-widest mb-2">

Rol Actual

```

</label>

<input
  type="text"
  name="role"
  id="role"
  value={userRole === 'admin' ? 'Administrador' : 'Cliente'}
  disabled

  className="w-full px-4 py-3 bg-gray-100 text-gray-500 border-
none rounded-xl cursor-not-allowed sm:text-sm capitalize"
/>
</div>
</div>

<div className="pt-4 flex justify-end">
  <button
    type="submit"
    disabled={saving}

    className="px-6 py-3 text-sm font-semibold text-white bg-black
rounded-xl hover:bg-gray-800 focus:outline-none focus:ring-2 focus:ring-offset-2
focus:ring-black disabled:opacity-50 transition-colors"
  >

    {saving ? 'Guardando...' : 'Guardar Cambios'}

  </button>
</div>
</form>

```

```

    </div>

  </div>

)}

{ /* — Seguridad Tab — */
{activeTab === 'security' && (
  <div className="space-y-8 animate-fadeIn">

    <div className="bg-white border border-gray-100 shadow-sm
rounded-2xl p-6 sm:p-8">

      <h2 className="text-lg font-bold text-black mb-6">Cambiar
Contraseña</h2>

      <form onSubmit={handleChangePassword} className="space-y-6">

        <div>

          <label htmlFor="currentPassword" className="block text-xs font-
semibold text-gray-700 uppercase tracking-widest mb-2">
            Contraseña Actual

          </label>

          <input

            type="password"

            name="currentPassword"

            id="currentPassword"

            value={securityData.currentPassword}

            onChange={handleSecurityChange}

            className="w-full md:w-1/2 px-4 py-3 bg-gray-50 border-none
rounded-xl focus:ring-2 focus:ring-black focus:bg-white transition-colors sm:text-
sm"

```

```

/>
</div>

<div className="grid grid-cols-1 gap-6 sm:grid-cols-2">
  <div>
    <label htmlFor="newPassword" className="block text-xs font-
semibold text-gray-700 uppercase tracking-widest mb-2">
      Nueva Contraseña
    </label>
    <input
      type="password"
      name="newPassword"
      id="newPassword"
      value={securityData.newPassword}
      onChange={handleSecurityChange}
      className="w-full px-4 py-3 bg-gray-50 border-none rounded-xl
focus:ring-2 focus:ring-black focus:bg-white transition-colors sm:text-sm"
    />
  </div>

  <div>
    <label htmlFor="confirmNewPassword" className="block text-xs
font-semibold text-gray-700 uppercase tracking-widest mb-2">
      Repetir Nueva Contraseña
    </label>

```

```

<input
  type="password"
  name="confirmNewPassword"
  id="confirmNewPassword"
  value={securityData.confirmNewPassword}
  onChange={handleSecurityChange}
  className="w-full px-4 py-3 bg-gray-50 border-none rounded-xl
focus:ring-2 focus:ring-black focus:bg-white transition-colors sm:text-sm"
/>
</div>
</div>

<div className="pt-4 flex justify-end">
  <button
    type="submit"
    disabled={securitySaving}
    className="px-6 py-3 text-sm font-semibold text-white bg-black
rounded-xl hover:bg-gray-800 focus:outline-none focus:ring-2 focus:ring-offset-2
focus:ring-black disabled:opacity-50 transition-colors"
  >
    {securitySaving ? 'Actualizando...' : 'Actualizar Contraseña'}
  </button>
</div>
</form>
</div>

```



```

    <div className="bg-white border border-gray-100 shadow-sm
rounded-2xl p-6 sm:p-8">

        <h2 className="text-lg font-bold text-black mb-6">Dispositivos
Activos</h2>

        <div className="flex items-center gap-4 bg-gray-50 border border-
gray-100 p-4 rounded-xl">

            <div className="w-10 h-10 rounded-full bg-white flex items-center
justify-center text-black border border-gray-200">

                <FaLaptopCode size={18} />

            </div>

            <div className="flex-1">

                <h3 className="text-sm font-bold text-black">Este
dispositivo</h3>

                <p className="text-xs text-gray-500 mt-0.5">Última actividad:
Ahora mismo</p>

            </div>

            <span className="px-3 py-1 text-[10px] font-bold tracking-widest
uppercase bg-green-100 text-green-700 rounded-lg">

                Activo

            </span>

        </div>

        <p className="text-xs text-gray-500 mt-4 leading-relaxed">

            Asegúrate de cambiar tu contraseña regularmente. Si notas actividad
sospechosa, te recomendamos actualizarla de inmediato.

        </p>

    </div>

```

```

</div>

)}}

{/* — Addresses Tab — */}

{activeTab === 'addresses' && (

  <div className="space-y-8 animate-fadeIn">

    <div className="bg-white border border-gray-100 shadow-sm
rounded-2xl p-6 sm:p-8">

      <h2 className="text-lg font-bold text-black mb-6">

        {editingAddress ? 'Editar Dirección' : 'Agregar Nueva Dirección'}

      </h2>

      <form onSubmit={editingAddress ? saveEditedAddress :
handleAddAddress}>

        <div className="grid grid-cols-1 gap-6 sm:grid-cols-2">

          <div className="sm:col-span-2">

            <label htmlFor="addressName" className="block text-xs font-
semibold text-gray-700 uppercase tracking-widest mb-2">

              Nombre de la dirección

            </label>

            <input

              type="text"

              name="name"

              id="addressName"

              placeholder="Ej: Mi Casa, Oficina..."

              value={editingAddress ? editAddressData.name :
newAddress.name}

```

```

        onChange={editingAddress ? handleEditAddressChange :
handleAddressChange}

        className="w-full px-4 py-3 bg-gray-50 border-none rounded-xl
focus:ring-2 focus:ring-black focus:bg-white transition-colors sm:text-sm"

    />

</div>

<div className="sm:col-span-2">

    <label htmlFor="addressStreet" className="block text-xs font-
semibold text-gray-700 uppercase tracking-widest mb-2">

        Calle / Número / Detalle

    </label>

    <input

        type="text"

        name="street"

        id="addressStreet"

        placeholder="Ej: Carrera 12 #34-56 Apto 7B"

        value={editingAddress ? editAddressData.street :
newAddress.street}

        onChange={editingAddress ? handleEditAddressChange :
handleAddressChange}

        className="w-full px-4 py-3 bg-gray-50 border-none rounded-xl
focus:ring-2 focus:ring-black focus:bg-white transition-colors sm:text-sm"

    />

</div>

```

```

<div>

  <label htmlFor="addressCity" className="block text-xs font-
semibold text-gray-700 uppercase tracking-widest mb-2">

    Ciudad

  </label>

  <input

    type="text"

    name="city"

    id="addressCity"

    value={editingAddress ? editAddressData.city : newAddress.city}

    onChange={editingAddress ? handleEditAddressChange :
handleAddressChange}

    className="w-full px-4 py-3 bg-gray-50 border-none rounded-xl
focus:ring-2 focus:ring-black focus:bg-white transition-colors sm:text-sm"

  />

</div>

```

```

<div>

  <label htmlFor="addressState" className="block text-xs font-
semibold text-gray-700 uppercase tracking-widest mb-2">

    Departamento

  </label>

  <input

    type="text"

    name="state"

    id="addressState"

```

```

        value={editingAddress ? editAddressData.state : newAddress.state}

        onChange={editingAddress ? handleEditAddressChange :
handleAddressChange}

        className="w-full px-4 py-3 bg-gray-50 border-none rounded-xl
focus:ring-2 focus:ring-black focus:bg-white transition-colors sm:text-sm"

    />
</div>

<div>

    <label htmlFor="addressZipCode" className="block text-xs font-
semibold text-gray-700 uppercase tracking-widest mb-2">

        Código Postal

    </label>

    <input

        type="text"

        name="zipCode"

        id="addressZipCode"

        value={editingAddress ? editAddressData.zipCode :
newAddress.zipCode}

        onChange={editingAddress ? handleEditAddressChange :
handleAddressChange}

        className="w-full px-4 py-3 bg-gray-50 border-none rounded-xl
focus:ring-2 focus:ring-black focus:bg-white transition-colors sm:text-sm"

    />

</div>

```

```

    <div>

        <label htmlFor="addressCountry" className="block text-xs font-
semibold text-gray-700 uppercase tracking-widest mb-2">

            País

        </label>

        <input

            type="text"

            name="country"

            id="addressCountry"

            value="Colombia"

            disabled

            className="w-full px-4 py-3 bg-gray-100 text-gray-500 border-
none rounded-xl cursor-not-allowed sm:text-sm"

        />

    </div>

</div>

<div className="pt-6 flex gap-3 justify-end border-t border-gray-100
mt-8">

    {editingAddress && (

        <button

            type="button"

            onClick={cancelEditingAddress}

            className="px-6 py-3 text-sm font-semibold text-gray-600 bg-
gray-100 rounded-xl hover:bg-gray-200 transition-colors"

        >

```

```

        Cancelar
      </button>
    })
    <button
      type="submit"
      disabled={addressSaving}
      className="px-6 py-3 text-sm font-semibold text-white bg-black
rounded-xl hover:bg-gray-800 disabled:opacity-50 transition-colors"
    >
      {addressSaving ? 'Guardando...' : (editingAddress ? 'Guardar
Cambios' : 'Agregar Dirección')}
    </button>
  </div>
</form>
</div>

<div className="mt-8">
  <h2 className="text-lg font-bold text-black mb-6">Mis Direcciones
Guardadas</h2>
  {addresses.length === 0 ? (
    <div className="bg-white border border-gray-100 border-dashed
rounded-2xl p-12 text-center text-gray-500 shadow-sm">
      <FaMapLocationDot size={32} className="mx-auto mb-4 text-gray-
300" />
      <p className="text-sm font-medium text-black">Aún no tienes
direcciones</p>

```

```

        <p className="text-xs mt-1">Empieza a agregar tus lugares
frecuentes para comprar más rápido.</p>

    </div>

): (

    <div className="grid grid-cols-1 md:grid-cols-2 gap-4">

        {addresses.map((address) => (

            <div key={address.id} className={`relative p-5 rounded-2xl border
transition-all ${

                address.isDefault ? 'border-black bg-black text-white shadow-md' :
'border-gray-200 bg-white hover:border-gray-300 shadow-sm'

            }}>

                <div className="flex justify-between items-start mb-3">

                    <h3 className={`text-base font-bold ${address.isDefault ? 'text-
white' : 'text-black'}}>

                        {address.name}

                    </h3>

                    {address.isDefault && (

                        <span className="flex items-center gap-1 text-[10px] font-
bold tracking-widest uppercase bg-white text-black px-2.5 py-1 rounded-md">

                            <FaCheck size={10} /> Predeterminado

                        </span>

                    )}

                </div>

                <div className={`text-sm space-y-1 ${address.isDefault ? 'text-
gray-300' : 'text-gray-500'}}>

                    <p>{address.street}</p>

```



```

    <p>{address.city}, {address.state} {address.zipCode}</p>
    <p>{address.country}</p>
  </div>

```

```

    <div className="flex items-center gap-2 mt-6 pt-4 border-t
border-current border-opacity-10">
      <button
        onClick={() => startEditingAddress(address)}
        className={`flex items-center gap-1.5 text-xs font-semibold px-
3 py-1.5 rounded-lg transition-colors ${
          address.isDefault ? 'bg-white/10 hover:bg-white/20 text-white' :
'bg-gray-100 hover:bg-gray-200 text-gray-700'
        }}
      >
        <FaPen size={10} /> Editar
      </button>

      {!address.isDefault && (
        <button
          onClick={() => handleSetDefaultAddress(address.id)}
          className="flex items-center gap-1.5 text-xs font-semibold
px-3 py-1.5 rounded-lg bg-gray-100 hover:bg-gray-200 text-gray-700 transition-
colors"
        >
          Hacer principal
        </button>
      )}
    </div>

```

```

    })

    <button
      onClick={() => handleDeleteAddress(address.id)}
      className={`ml-auto flex items-center justify-center w-8 h-8
rounded-lg transition-colors ${
        address.isDefault ? 'hover:bg-red-500/20 text-red-300' :
'hover:bg-red-50 text-red-500'
      }}
      aria-label="Eliminar dirección"
    >
      <FaRegTrashCan size={14} />
    </button>
  </div>
</div>
)}}
</div>
)}
</div>
</div>
)}

</main>
</div>
</div>

```

```

    </div>

  );
};

export default Profile;

```

 **src/assets/pages/shop.jsx**

```

import React, { useState, useEffect, useMemo } from 'react';

import { Link } from 'react-router-dom';

import { getProducts } from '../services/productService';

import StarRating from '../components/StarRating';

import { useCart } from '../context/CartContext';

import { useWishlist } from '../context/WishlistContext';

import { useAuth } from '../context/AuthContext';

import Swal from 'sweetalert2';

```

```

/*————— Skeleton card —————*/

const SkeletonCard = () => (

  <div className="bg-white border border-gray-100 rounded-2xl overflow-hidden
animate-pulse">

    <div className="aspect-square bg-gray-100" />

    <div className="p-4 space-y-3">

      <div className="h-3 bg-gray-100 rounded w-1/3" />

      <div className="h-4 bg-gray-100 rounded w-4/5" />

      <div className="h-3 bg-gray-100 rounded w-1/2" />

      <div className="h-5 bg-gray-100 rounded w-1/4 mt-1" />

      <div className="h-9 bg-gray-100 rounded-xl mt-2" />

```

```

    </div>

</div>

);

/*————— Product Card————— */

const ProductCard = ({ product, getPrimaryImageUrl, onAddToCart,
onAddToWishlist }) => {

  const [wishlisted, setWishlisted] = useState(false);

  const handleWishlist = (e) => {

    e.preventDefault();

    e.stopPropagation();

    setWishlisted(true);

    onAddToWishlist(product);

    setTimeout(() => setWishlisted(false), 2000);

  };

  const outOfStock = product.stock === 0;

  const lowStock = product.stock !== undefined && product.stock > 0 &&
product.stock <= 5;

  return (

    <div className="group relative bg-white border border-gray-100 rounded-2xl
overflow-hidden shadow-sm hover:shadow-xl hover:-translate-y-1 transition-all
duration-300 flex flex-col">

      {/* Image container */}

```

```
<Link to={`/${product}/${product.id}`} className="block relative overflow-hidden aspect-square bg-gray-50 flex-shrink-0">
```

```
<img
```

```
  src={getPrimaryImageUrl(product)}
```

```
  alt={product.name}
```

```
  className="object-cover w-full h-full group-hover:scale-105 transition-transform duration-500 ease-out"
```

```
  onError={(e) => { e.target.onerror = null; e.target.src =  
'https://via.placeholder.com/400x400.png?text=PSG'; }}
```

```
/>
```

```
{/* Gradient overlay on hover */}
```

```
<div className="absolute inset-0 bg-gradient-to-t from-black/30 via-transparent to-transparent opacity-0 group-hover:opacity-100 transition-opacity duration-300" />
```

```
{/* Quick-view label on hover */}
```

```
<div className="absolute inset-x-0 bottom-0 flex items-center justify-center pb-4 opacity-0 group-hover:opacity-100 translate-y-2 group-hover:translate-y-0 transition-all duration-300">
```

```
<span className="px-4 py-1.5 bg-white/90 backdrop-blur-sm text-gray-900 text-xs font-semibold rounded-full shadow">
```

```
  Ver detalles →
```

```
</span>
```

```
</div>
```

```
{/* Badges */}
```

```

<div className="absolute top-3 left-3 flex flex-col gap-1.5">
  {outOfStock && (
    <span className="inline-flex items-center px-2.5 py-1 rounded-full text-[10px] font-bold uppercase tracking-wide bg-red-600 text-white shadow-sm">
      Agotado
    </span>
  )}
  {lowStock && (
    <span className="inline-flex items-center px-2.5 py-1 rounded-full text-[10px] font-bold uppercase tracking-wide bg-amber-500 text-white shadow-sm">
      Solo {product.stock} left
    </span>
  )}
</div>

```

```

{/* Wishlist button */}

<button
  className={`absolute top-3 right-3 w-9 h-9 rounded-full flex items-center justify-center shadow-sm transition-all duration-200
    ${wishlist
      ? 'bg-red-500 text-white scale-110'
      : 'bg-white/80 backdrop-blur-sm text-gray-500 hover:bg-white hover:text-red-500 opacity-0 group-hover:opacity-100 translate-y-1 group-hover:translate-y-0'}
  }
  onClick={handleWishlist}

```

```

    aria-label="Agregar a favoritos"

    >

    <svg className="w-4 h-4" fill={wishlist ? 'currentColor' : 'none'}
viewBox="0 0 24 24" stroke="currentColor" strokeWidth="2">

        <path strokeLinecap="round" strokeLinejoin="round" d="M4.318 6.318a4 4
0 00 6.364L12 20.364l7.682-7.682a4 4 0 00-6.364-6.364L12 7.636l-1.318-1.318a4 4
0 00-6.364 0z" />

    </svg>

</button>

</Link>

{ /* Info */ }

<div className="p-4 flex flex-col flex-1">

    { /* Category */ }

    <span className="text-[10px] font-bold uppercase tracking-widest text-
indigo-500 mb-1">

        {product.category || 'General'}

    </span>

    { /* Name */ }

    <h3 className="text-sm font-semibold text-gray-900 leading-snug mb-2
line-clamp-2 flex-1">

        {product.name}

    </h3>

    { /* Rating */ }

```

```

{product.rating > 0 && (
  <div className="mb-2">
    <StarRating rating={product.rating} size="sm" />
  </div>
)}

{ /* Price + Add to cart */ }

<div className="flex items-center justify-between mt-auto pt-3 border-t
border-gray-50">
  <span className="text-base font-extrabold text-gray-900 tracking-tight">
    ${parseFloat(product.price).toLocaleString('es-CO')}
  </span>
  <button
    className={`flex items-center gap-1.5 px-3.5 py-2 rounded-xl text-xs font-
semibold transition-all duration-150
    ${outOfStock
      ? 'bg-gray-100 text-gray-400 cursor-not-allowed'
      : 'bg-gray-900 text-white hover:bg-gray-700 active:scale-95 shadow-sm
hover:shadow'}
    `}
    disabled={outOfStock}
    onClick={(e) => { e.preventDefault(); onAddToCart(product); }}
  >
    {outOfStock ? (
      'Agotado'

```



```

    ): (
      <>
        <svg className="w-3.5 h-3.5" fill="none" viewBox="0 0 24 24"
stroke="currentColor" strokeWidth="2.2">
          <path strokeLinecap="round" strokeLinejoin="round" d="M3 3h2l.4
2M7 13h10l4-8H5.4M7 13L5.4 5M7 13l-2.293 2.293c-.63.63-.184 1.707.707
1.707H17m0 0a2 2 0 100 4 2 2 0 000-4zm-8 2a2 2 0 11-4 0 2 2 0 014 0z" />
        </svg>
        Agregar
      </>
    )}
  </button>
</div>
</div>
</div>
);
};

```

```

/*————— Main Shop —————*/

```

```

const Shop = () => {
  const [products, setProducts] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const [filter, setFilter] = useState('all');
  const [searchTerm, setSearchTerm] = useState('');

```

```
const [sortBy, setSortBy] = useState('default');
```

```
const { addToCart } = useCart();
```

```
const { addToWishlist } = useWishlist();
```

```
const { currentUser } = useAuth();
```

```
useEffect(() => {
```

```
  const fetchProducts = async () => {
```

```
    try{
```

```
      setLoading(true);
```

```
      const productList = await getProducts();
```

```
      setProducts(productList);
```

```
    } catch (err) {
```

```
      setError('No se pudieron cargar los productos. Intenta de nuevo.');
```

```
    } finally {
```

```
      setLoading(false);
```

```
    }
```

```
  };
```

```
  fetchProducts();
```

```
}, []);
```

```
const categories = useMemo(() => ['all', ...new Set(products.map(p =>  
p.category).filter(Boolean))], [products]);
```

```
const filteredProducts = useMemo(() => {
```

```
  let result = filter === 'all' ? products : products.filter(p => p.category === filter);
```

```

    if(searchTerm) {
      const term = searchTerm.toLowerCase();
      result = result.filter(p =>
        p.name.toLowerCase().includes(term) ||
        (p.description && p.description.toLowerCase().includes(term)) ||
        (p.category && p.category.toLowerCase().includes(term)) ||
        (p.material && p.material.toLowerCase().includes(term)) ||
        (p.color && p.color.toLowerCase().includes(term))
      );
    }

    if(sortBy === 'price-asc') result = [...result].sort((a, b) => a.price - b.price);
    if(sortBy === 'price-desc') result = [...result].sort((a, b) => b.price - a.price);
    if(sortBy === 'rating') result = [...result].sort((a, b) => (b.rating || 0) - (a.rating || 0));
    if(sortBy === 'name') result = [...result].sort((a, b) => a.name.localeCompare(b.name));

    return result;
  }, [products, filter, searchTerm, sortBy]);

const getPrimaryImageUrl = (product) => {
  if(product.imageUrls && product.imageUrls.length > 0) {

```

```

    const validImages = product.imageUrls.filter(img => img && (typeof img ===
'string' || (img.data && typeof img.data === 'string')));

    if(validImages.length > 0) {

        const safeIndex = Math.min(product.primaryImageIndex || 0,
validImages.length - 1);

        const img = validImages[safeIndex];

        if(typeof img === 'string') return img;

        if(img && img.data) return img.data;

    }

}

return product.imageUrl || 'https://via.placeholder.com/400x400.png?text=PSG';

};

```

```

const requireAuth = (action) => {

    if(!currentUser) {

        Swal.fire({

            title: 'Inicia sesión',

            text: 'Debes iniciar sesión para continuar',

            icon: 'warning',

            confirmButtonText: 'Iniciar sesión',

            confirmButtonColor: '#111827',

            showCancelButton: true,

            cancelButtonText: 'Cancelar',

        }).then(result => { if(result.isConfirmed) window.location.href = '/psg-
official/#/login'; });

    }

};

```

```

    return false;
  }
  return true;
};

```

```

const handleAddToCart = (product) => {
  if(!requireAuth()) return;

  const stockLimit = product.stock !== undefined ? product.stock : Infinity;
  if(stockLimit === 0) {

    Swal.fire({ title: 'Producto agotado', text: 'Este producto no está disponible actualmente.', icon: 'error', confirmButtonColor: '#111827' });

    return;
  }

  addToCart({ ...product, imageUrl: getPrimaryImageUrl(product), quantity: 1 });

  Swal.fire({ title: '¡Agregado!', text: 'Producto agregado al carrito correctamente.', icon: 'success', confirmButtonColor: '#111827', timer: 1800, showConfirmButton: false });
};

```

```

const handleAddToWishlist = (product) => {
  if(!requireAuth()) return;

  addToWishlist({ ...product, imageUrl: getPrimaryImageUrl(product) });

  Swal.fire({ title: '¡Guardado!', text: 'Producto agregado a tu lista de deseos.', icon: 'success', confirmButtonColor: '#111827', timer: 1800, showConfirmButton: false });
};

```

```

return (

  <div style={{ fontFamily: 'Inter, sans-serif' }} className="min-h-screen bg-gray-50/50">

    <div className="px-4 py-10 mx-auto max-w-7xl sm:px-6 lg:px-8">

      {/* — Page Header — */}

      <div className="mb-8">

        <h1 className="text-3xl font-extrabold tracking-tight text-gray-900">Nuestra Colección</h1>

        <p className="mt-1 text-sm text-gray-500">

          {!loading && `${filteredProducts.length} producto${filteredProducts.length !== 1 ? 's' : ''} disponible${filteredProducts.length !== 1 ? 's' : ''}}

        </p>

      </div>

      {/* — Filters bar — */}

      <div className="flex flex-col gap-4 mb-8">

        {/* Category pills */}

        <div className="flex flex-wrap gap-2">

          {categories.map(cat => (

            <button

              key={cat}

              onClick={() => setFilter(cat)}

              className={`px-4 py-1.5 rounded-full text-xs font-semibold border transition-all duration-150

                ${filter === cat

```

```

        ? 'bg-gray-900 text-white border-gray-900 shadow-sm'
        : 'bg-white text-gray-600 border-gray-200 hover:border-gray-400
hover:text-gray-900'
    }}
  >
    {cat === 'all' ? 'Todos' : cat.charAt(0).toUpperCase() + cat.slice(1)}
  </button>
)}}
</div>

```

```

{/* Search + Sort row */}

<div className="flex flex-col gap-3 sm:flex-row sm:items-center sm:justify-
between">
  {/* Search */}
  <div className="relative sm:w-72">
    <input
      type="text"
      id="product-search"
      placeholder="Buscar productos..."
      className="w-full py-2.5 pl-10 pr-9 text-sm text-gray-700 bg-white
border border-gray-200 rounded-xl focus:outline-none focus:ring-2 focus:ring-
gray-900 focus:border-transparent transition-all placeholder-gray-400 shadow-sm"
      value={searchTerm}
      onChange={(e) => setSearchTerm(e.target.value)}
    />

```

```

    <svg className="absolute left-3.5 top-1/2 -translate-y-1/2 w-4 h-4 text-
gray-400" fill="none" viewBox="0 0 24 24" stroke="currentColor">

```

```

    <path strokeLinecap="round" strokeLinejoin="round" strokeWidth="2"
d="M21 21l-6-6m2-5a7 7 0 11-14 0 7 7 0 0114 0z" />

```

```

</svg>

```

```

{searchTerm && (

```

```

    <button

```

```

        onClick={() => setSearchTerm("")}

```

```

        className="absolute right-3 top-1/2 -translate-y-1/2 w-4 h-4 text-
gray-400 hover:text-gray-700 transition-colors"

```

```

    >

```

```

    <svg fill="none" viewBox="0 0 24 24" stroke="currentColor"
strokeWidth="2.5">

```

```

    <path strokeLinecap="round" strokeLinejoin="round" d="M6 18L18
6M6 6l12 12" />

```

```

</svg>

```

```

</button>

```

```

  )}

```

```

</div>

```

```

{ /* Sort */ }

```

```

<div className="relative">

```

```

  <select

```

```

    id="sort-filter"

```

```

    value={sortBy}

```

```

    onChange={(e) => setSortBy(e.target.value)}

```



```
        className="appearance-none pl-9 pr-8 py-2.5 text-sm text-gray-700
bg-white border border-gray-200 rounded-xl focus:outline-none focus:ring-2
focus:ring-gray-900 focus:border-transparent transition-all cursor-pointer shadow-
sm"
```

```
>
```

```
<option value="default">Orden predeterminado</option>
```

```
<option value="price-asc">Precio: menor a mayor</option>
```

```
<option value="price-desc">Precio: mayor a menor</option>
```

```
<option value="rating">Mejor valorados</option>
```

```
<option value="name">Nombre A-Z</option>
```

```
</select>
```

```
<svg className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4 text-
gray-400 pointer-events-none" fill="none" viewBox="0 0 24 24"
stroke="currentColor">
```

```
<path strokeLinecap="round" strokeLinejoin="round" strokeWidth="2"
d="M3 7h18M7 12h10M11 17h4" />
```

```
</svg>
```

```
<svg className="absolute right-2.5 top-1/2 -translate-y-1/2 w-3.5 h-3.5
text-gray-400 pointer-events-none" fill="none" stroke="currentColor" viewBox="0
0 24 24">
```

```
<path strokeLinecap="round" strokeLinejoin="round" strokeWidth="2.5"
d="M19 9l-7 7-7-7" />
```

```
</svg>
```

```
</div>
```

```
</div>
```

```
{/* Active search tag */}
```

```

{searchTerm && !loading && (
  <div className="flex items-center gap-2">
    <span className="text-sm text-gray-500">{filteredProducts.length}
    resultado{filteredProducts.length !== 1 ? 's' : ''} para</span>
    <span className="inline-flex items-center gap-1.5 px-2.5 py-0.5 bg-gray-
    100 rounded-full text-sm font-medium text-gray-800">
      "{searchTerm}"
      <button onClick={() => setSearchTerm("")} className="text-gray-400
      hover:text-gray-700">
        <svg className="w-3 h-3" fill="none" viewBox="0 0 24 24"
        stroke="currentColor" strokeWidth="2.5">
          <path strokeLinecap="round" strokeLinejoin="round" d="M6 18L18
          6M6 6l12 12" />
        </svg>
      </button>
    </span>
  </div>
)}
</div>

{ /* — Grid — */
  {loading ? (
    <div className="grid grid-cols-1 gap-6 sm:grid-cols-2 lg:grid-cols-3 xl:grid-
    cols-4">
      {Array.from({ length: 8 }).map((_, i) => <SkeletonCard key={i} />)}
    </div>

```

```

) : error ? (

  <div className="flex flex-col items-center justify-center py-20 gap-4">

    <div className="w-14 h-14 rounded-full bg-red-50 flex items-center justify-center">

      <svg className="w-7 h-7 text-red-400" fill="none" viewBox="0 0 24 24" stroke="currentColor">

        <path strokeLinecap="round" strokeLinejoin="round" strokeWidth="1.8" d="M12 9v2m0 4h.01m-6.938 4h13.856c1.54 0 2.502-1.667 1.732-3L13.732 4c-.77-1.333-2.694-1.333-3.464 0L3.34 16c-.77 1.333 1.333 1.92 3 1.732 3z" />

      </svg>

    </div>

    <p className="text-sm text-gray-600">{error}</p>

    <button

      onClick={() => window.location.reload()}

      className="px-4 py-2 text-sm font-medium text-white bg-gray-900 rounded-xl hover:bg-gray-700 transition-colors"

    >

      Reintentar

    </button>

  </div>

) : filteredProducts.length === 0 ? (

  <div className="flex flex-col items-center justify-center py-20 gap-4 text-center">

    <div className="w-16 h-16 rounded-full bg-gray-100 flex items-center justify-center">

      <svg className="w-8 h-8 text-gray-400" fill="none" viewBox="0 0 24 24" stroke="currentColor">

```

```

    <path strokeLinecap="round" strokeLinejoin="round" strokeWidth="1.5"
d="M9.172 16.172a4 4 0 015.656 0M9 10h.01M15 10h.01M21 12a9 9 0 11-18 0 9 9
0 0118 0z" />

```

```

</svg>

```

```

</div>

```

```

<div>

```

```

    <h3 className="text-base font-semibold text-gray-900">No se
encontraron productos</h3>

```

```

    <p className="mt-1 text-sm text-gray-500">

```

```

    {searchTerm ? `No hay coincidencias para "${searchTerm}".` : 'No hay
productos disponibles en esta categoría.'}

```

```

</p>

```

```

</div>

```

```

<button

```

```

    onClick={() => { setSearchTerm(''); setFilter('all'); setSortBy('default'); }}

```

```

    className="mt-1 inline-flex items-center px-5 py-2.5 text-sm font-
semibold text-white bg-gray-900 rounded-xl hover:bg-gray-700 transition-colors
shadow-sm"

```

```

>

```

```

    Ver todos los productos

```

```

</button>

```

```

</div>

```

```

): (

```

```

    <div className="grid grid-cols-1 gap-6 sm:grid-cols-2 lg:grid-cols-3 xl:grid-
cols-4">

```

```

    {filteredProducts.map((product) => (

```

```

    <ProductCard

```

```

      key={product.id}
      product={product}
      getPrimaryImageUrl={getPrimaryImageUrl}
      onAddToCart={handleAddToCart}
      onAddToWishlist={handleAddToWishlist}
    />
  )))
</div>
})

{ /* — Bottom count — */
  {!loading && !error && filteredProducts.length > 0 && (
    <p className="mt-10 text-center text-xs text-gray-400">
      Mostrando {filteredProducts.length} de {products.length} productos
    </p>
  )}

</div>
</div>
);
};
export default Shop;

```

📁 src/assets/pages/wishlist.jsx

```
import React from 'react';

import { Link } from 'react-router-dom';

import { useWishlist } from '../context/WishlistContext';

const Wishlist = () => {

  const { items, removeFromWishlist, loading } = useWishlist();

  const getPrimaryImageUrl = (product) => {

    if(product.imageUrls && product.imageUrls.length > 0) {

      const validImages = product.imageUrls.filter(img => img && (typeof img === 'string' || (img.data && typeof img.data === 'string')));

      if(validImages.length > 0) {

        const safeIndex = Math.min(product.primaryImageIndex || 0, validImages.length - 1);

        const primaryImage = validImages[safeIndex];

        if(typeof primaryImage === 'string') return primaryImage;

        if(primaryImage && primaryImage.data) return primaryImage.data;

      }

    }

    return product.imageUrl || 'https://via.placeholder.com/300x300.png?text=PSG';

  };

  if(loading) {

    return (
```

```

    <div className="min-h-screen bg-gray-50 flex items-center justify-center"
    style={{ fontFamily: 'Inter, sans-serif' }}>

      <div className="w-8 h-8 border-2 border-gray-900 border-t-transparent
      rounded-full animate-spin" />

    </div>

  );
}

return (
  <div style={{ fontFamily: 'Inter, sans-serif' }}>
    <div className="px-4 py-10 mx-auto max-w-7xl sm:px-6 lg:px-8">
      {/* Header */}
      <div className="flex items-center justify-between mb-8">
        <div>
          <h1 className="text-2xl font-bold tracking-tight text-gray-900">Lista de
          Deseos</h1>
          <p className="mt-1 text-sm text-gray-500">
            {items.length > 0 ? `${items.length} producto${items.length !== 1 ? 's' : ''}
            guardado${items.length !== 1 ? 's' : ''}` : 'Tu lista está vacía'}
          </p>
        </div>
      </div>
      {items.length > 0 && (
        <Link to="/shop" className="text-sm font-medium text-gray-600
        hover:text-black transition-colors duration-150 flex items-center gap-1">
          <svg className="w-4 h-4 fill="none" viewBox="0 0 24 24"
          stroke="currentColor"> <path strokeLinecap="round" strokeLinejoin="round"
          strokeWidth="2" d="M12 6v6m0 0v6m0-6h6m-6 0H6" /> </svg>

```

```

    Agregar productos
  </Link>
})
</div>

```

```

{items.length === 0 ? (

  <div className="text-center py-16 bg-white rounded-2xl border border-
gray-200 shadow-sm">

    <div className="flex items-center justify-center w-14 h-14 mx-auto
rounded-full bg-gray-100 mb-4">

      <svg className="w-7 h-7 text-gray-400" fill="none" viewBox="0 0 24 24"
stroke="currentColor">

        <path strokeLinecap="round" strokeLinejoin="round" strokeWidth="1.5"
d="M4.318 6.318a4.5 4.5 0 00 6.364L12 20.364l7.682-7.682a4.5 4.5 0 00-6.364-
6.364L12 7.636l-1.318-1.318a4.5 4.5 0 00-6.364 0z" />

      </svg>

    </div>

    <h3 className="text-base font-semibold text-gray-900">Tu lista de
deseos está vacía</h3>

    <p className="mt-1 text-sm text-gray-500">Empieza a guardar productos
que te gusten</p>

    <Link to="/shop" className="inline-flex items-center mt-6 px-5 py-2.5
text-sm font-medium text-white bg-black hover:bg-gray-800 rounded-lg
transition-colors duration-150">

      Explorar Productos
    </Link>

  </div>

```



```

): (
    <div className="grid grid-cols-1 gap-6 sm:grid-cols-2 lg:grid-cols-3 xl:grid-
cols-4">
        {items.map((item) => (
            <div key={item.id} className="group bg-white border border-gray-200
rounded-2xl overflow-hidden hover:shadow-md hover:border-gray-300 transition-
all duration-200">
                <Link to={`/product/${item.id}`}>
                    <div className="relative overflow-hidden aspect-square bg-gray-50">
                        <img
                            src={getPrimaryImageUrl(item)}
                            alt={item.name}
                            className="object-cover w-full h-full group-hover:scale-105
transition-transform duration-300"
                            onError={(e) => { e.target.onerror = null; e.target.src =
'https://via.placeholder.com/300x300.png?text=PSG'; }}
                        />
                    </div>
                </Link>
                <div className="p-4">
                    <div className="mb-1">
                        <span className="text-[10px] font-semibold uppercase tracking-
widest text-gray-400">{item.category}</span>
                    </div>
                    <h3 className="text-sm font-semibold text-gray-900 leading-snug
mb-1 line-clamp-2">{item.name}</h3>

```

```

    {item.rating > 0 && (
      <div className="flex items-center gap-1 mb-2">
        {[...Array(5)].map((_, i) => (
          <svg key={i} className={`w-3.5 h-3.5 ${i < Math.floor(item.rating) ?
'text-yellow-400' : 'text-gray-300'}`} fill="currentColor" viewBox="0 0 20 20">
            <path d="M9.049 2.927c.3-.921 1.603-.921 1.902 0l1.07 3.292a1 1
0 00.956 9.69h3.462c.969 0 1.371 1.245 1.811-2.8 2.034a1 1 0 00-.364 1.118l1.07
3.292c.3.921-.755 1.688-1.54 1.118l-2.8-2.034a1 1 0 00-1.175 0l-2.8 2.034c-.784 57-
1.838-.197-1.539-1.118l1.07-3.292a1 1 0 00-.364-1.118L2.98 8.72c-.783-.57-.38-
1.811.588-1.811h3.461a1 1 0 00.951-.69l1.07-3.292z" />
          </svg>
        )]}
        <span className="text-xs text-gray-500 ml-1">{item.rating}</span>
      </div>
    )}

    <p className="text-base font-bold text-gray-900 mb-
3">${parseFloat(item.price).toLocaleString('es-CO')}</p>

    <div className="flex gap-2">
      <Link
        to={` /product/${item.id}`}
        className="flex-1 py-2 text-center text-xs font-medium text-white
bg-black hover:bg-gray-800 rounded-lg transition-colors duration-150"
      >
        Ver detalles
      </Link>
    </div>

```

```

    <button
      onClick={() => removeFromWishlist(item.id)}

      className="flex items-center justify-center w-9 h-9 text-gray-400
bg-white border border-gray-300 rounded-lg hover:border-red-300 hover:text-
red-500 transition-all duration-150"

      aria-label="Eliminar de favoritos"
    >

      <svg className="h-4 w-4" fill="none" viewBox="0 0 24 24"
stroke="currentColor">

        <path strokeLinecap="round" strokeLinejoin="round"
strokeWidth="2" d="M19 7l-.867 12.142A2 2 0 0116.138 21H7.862a2 2 0 01-1.995-
1.858L5 7m5 4v6m4-6v6m1-10V4a1 1 0 00-1-1h-4a1 1 0 00-1 1v3M4 7h16" />

      </svg>

    </button>

  </div>

</div>

</div>

  )}

</div>

)}

</div>

</div>

);

};

export default Wishlist;

```

📁 src/assets/routes/AdminRoute.jsx

```
import React from 'react';

import { Navigate, Outlet } from 'react-router-dom';

import { useAuth } from '../context/AuthContext';

const AdminRoute = ({ children }) => {

  const { currentUser, isAdmin, loading, firebaseError } = useAuth();

  if (loading) {

    // Muestra un loader mientras se carga la información del usuario

    return <div className="min-h-screen flex items-center justify-center bg-gray-50">

      <div className="animate-spin rounded-full h-12 w-12 border-b-2 border-gray-900"></div>

    </div>;

  }

  // Handle Firebase configuration errors

  if (firebaseError) {

    return (

      <div className="min-h-screen flex items-center justify-center bg-gray-50">

        <div className="max-w-md w-full space-y-8">

          <div className="bg-yellow-100 border border-yellow-400 text-yellow-700 px-4 py-3 rounded relative" role="alert">
```

```

        <strong className="font-bold">Configuración incompleta!
    </strong>

    <span className="block sm:inline">{firebaseError}</span>

    <p className="mt-2 text-sm">
        Por favor, verifica tu archivo .env con las credenciales de Firebase.
    </p>
</div>
</div>
</div>
);
}

if(!currentUser) {
    // Si no ha iniciado sesión, redirige a la página de login
    return <Navigate to="/login" />;
}

if(!isAdmin) {
    // Si ha iniciado sesión pero no es admin, redirige a la home
    return <Navigate to="/home" />;
}

// Si es admin, muestra el contenido de la ruta protegida
return children ? children : <Outlet />;
};

```

export default AdminRoute;

 **src/assets/routes/AppRoutes.jsx**

import React from 'react';

import { Routes, Route } from 'react-router-dom';

import Home from '../pages/Home';

import Login from '../components/login';

import Register from '../components/Register';

import ResetPassword from '../components/ResetPassword';

import ModernAdminDashboard from '../pages/ModernAdminDashboard';

import ProductDetail from '../pages/ProductDetail';

import Cart from '../pages/Cart';

import Checkout from '../pages/Checkout';

import CheckoutSuccess from '../pages/CheckoutSuccess';

import CheckoutCancel from '../pages/CheckoutCancel';

import Shop from '../pages/Shop';

import Wishlist from '../pages/Wishlist';

import Profile from '../pages/Profile';

import StarRatingTest from '../pages/StarRatingTest';

import Orders from '../pages/Orders';

import Blog from '../pages/Blog';

import BlogPost from '../pages/BlogPost';

import Footer from '../components/footer';

import AdminRoute from './AdminRoute';

```
import ProtectedRoute from './protectedRoute';
```

```
const AppRoutes = () => {
```

```
  return (
```

```
    <div className="flex flex-col min-h-screen">
```

```
      <main className="flex-grow">
```

```
        <Routes>
```

```
          <Route path="/" element={<Home />} />
```

```
          <Route path="/home" element={<Home />} />
```

```
          <Route path="/login" element={<Login />} />
```

```
          <Route path="/register" element={<Register />} />
```

```
          <Route path="/reset-password" element={<ResetPassword />} />
```

```
          <Route path="/shop" element={<Shop />} />
```

```
          <Route path="/product/:id" element={<ProductDetail />} />
```

```
          <Route path="/cart" element={<Cart />} />
```

```
          <Route path="/wishlist" element={<Wishlist />} />
```

```
          <Route path="/profile" element={<ProtectedRoute> <Profile  
/> </ProtectedRoute>} />
```

```
          <Route path="/orders" element={<ProtectedRoute> <Orders  
/> </ProtectedRoute>} />
```

```
          <Route path="/blog" element={<Blog />} />
```

```
          <Route path="/blog/:id" element={<BlogPost />} />
```

```
          <Route path="/admin/*"
```

```
            element={<AdminRoute> <ModernAdminDashboard /> </AdminRoute>} />
```

```
          <Route path="/star-rating-test" element={<StarRatingTest />} />
```

```

    </Routes>

  </main>

  <Footer />

</div>

);

};

export default AppRoutes;

```

 **src/assets/routes/protectedRoute.jsx**

```

import React from 'react';

import { Navigate, Outlet } from 'react-router-dom';

import { useAuth } from '../context/AuthContext';

const ProtectedRoute = ({ children }) => {

  const { currentUser, loading, firebaseError } = useAuth();

  if(loading) {

    return <div className="min-h-screen flex items-center justify-center">

      <div className="animate-spin rounded-full h-12 w-12 border-b-2 border-
gray-900"> </div>

    </div>;

  }

  // Handle Firebase configuration errors

  if(firebaseError) {

    return (

```



```

    <div className="min-h-screen flex items-center justify-center bg-gray-
50">

      <div className="max-w-md w-full space-y-8">

        <div className="bg-yellow-100 border border-yellow-400 text-
yellow-700 px-4 py-3 rounded relative" role="alert">

          <strong className="font-bold">Configuración incompleta!
</strong>

          <span className="block sm:inline">{firebaseError}</span>

          <p className="mt-2 text-sm">

            Por favor, verifica tu archivo .env con las credenciales de Firebase.

          </p>

        </div>

      </div>

    </div>

  );
}

```

```

    return currentUser ? (children ? children : <Outlet />) : <Navigate to="/login" />;
  };

```

```

export default ProtectedRoute;

```

 **src/assets/context/authcontext.jsx**

```

import React, { createContext, useContext } from 'react';

```

```

// Create AuthContext

```

```

export const AuthContext = createContext();

```

```

export const useAuth = () => {
  return useContext(AuthContext);
};

export default AuthContext;

```

src/assets/context/cartcontext.jsx

```

import React, { createContext, useContext, useReducer, useEffect, useState, useRef,
useCallback } from 'react';

import { db } from '../firebase';

import { doc, setDoc, getDoc, collection, getDocs, query, where } from
'firebase/firestore';

import { useAuth } from '../context/AuthContext';

```

```

const CartContext = createContext();

```

```

const cartReducer = (state, action) => {
  switch (action.type) {
    case 'SET_CART':
      return {
        ...state,
        items: action.payload.items || [],
        coupon: action.payload.coupon || null,
      };

    case 'ADD_TO_CART':

```

```

const existingItem = state.items.find(item => item.id === action.payload.id);
if(existingItem) {
  // Check stock limit if available
  const newQuantity = existingItem.quantity + (action.payload.quantity || 1);
  const stockLimit = action.payload.stock !== undefined ? action.payload.stock :
Infinity;

```

```

  if(newQuantity > stockLimit) {
    // Don't exceed stock limit
    return state;
  }

```

```

  return {
    ...state,
    items: state.items.map(item =>
      item.id === action.payload.id
        ? { ...item, quantity: newQuantity }
        : item
    ),
  };
}

```

```

// Check stock for new item
const initialQuantity = action.payload.quantity || 1;

```

```
const itemStock = action.payload.stock !== undefined ? action.payload.stock :
Infinity;
```

```
if (initialQuantity > itemStock) {
```

```
  // Don't exceed stock limit
```

```
  return state;
```

```
}
```

```
return {
```

```
  ...state,
```

```
  items: [...state.items, { ...action.payload, quantity: initialQuantity }],
```

```
};
```

```
case 'REMOVE_FROM_CART':
```

```
return {
```

```
  ...state,
```

```
  items: state.items.filter(item => item.id !== action.payload),
```

```
};
```

```
case 'UPDATE_QUANTITY':
```

```
  // Find the item to get stock info
```

```
const itemToUpdate = state.items.find(item => item.id === action.payload.id);
```

```
if (!itemToUpdate) return state;
```

```
const stockLimit = itemToUpdate.stock !== undefined ? itemToUpdate.stock :
Infinity;
```

```
// Don't allow quantity to exceed stock
```

```
const newQuantity = Math.min(action.payload.quantity, stockLimit);
```

```
if(newQuantity <= 0) {
```

```
  return {
```

```
    ...state,
```

```
    items: state.items.filter(item => item.id !== action.payload.id),
```

```
  };
```

```
}
```

```
return {
```

```
  ...state,
```

```
  items: state.items.map(item =>
```

```
    item.id === action.payload.id
```

```
      ? { ...item, quantity: newQuantity }
```

```
      : item
```

```
  ),
```

```
};
```

```
case 'CLEAR_CART':
```

```
  return {
```

```
    ...state,
```

```

    items: [],
    coupon: null,
  };

  case 'APPLY_COUPON':
    return {
      ...state,
      coupon: action.payload,
    };

  case 'REMOVE_COUPON':
    return {
      ...state,
      coupon: null,
    };

  default:
    return state;
}

};

export const CartProvider = ({ children }) => {
  const [state, dispatch] = useReducer(cartReducer, { items: [], coupon: null });
  const [loading, setLoading] = useState(true);

```

```

const { currentUser } = useAuth();

const initialCartLoad = useRef(true);

// Get the primary image URL for a product

const getPrimaryImageUrl = (product) => {
  if(product.imageUrls && product.imageUrls.length > 0) {
    // Filter out empty images

    const validImages = product.imageUrls.filter(image =>
      image && (typeof image === 'string' || (image.data && typeof image.data
        === 'string'))
    );

    if(validImages.length > 0) {
      const primaryIndex = product.primaryImageIndex || 0;
      // Ensure primaryIndex is within bounds

      const safeIndex = Math.min(primaryIndex, validImages.length - 1);

      const primaryImage = validImages[safeIndex];

      // Handle both string URLs and base64 data objects

      if(typeof primaryImage === 'string') {
        return primaryImage;
      } else if(primaryImage && primaryImage.data) {
        return primaryImage.data;
      }
    }
  }
}

```

```
    return product.imageUrl ||  
    'https://via.placeholder.com/600x600.png?text= Moño';  
  }  
};
```

```
// Load cart from Firestore when user logs in
```

```
const loadCartFromFirestore = useCallback(async () => {
```

```
  // Always reset loading state when starting
```

```
  setLoading(true);
```

```
  if(!currentUser) {
```

```
    console.log('No user logged in, clearing cart');
```

```
    // If no user, clear local cart
```

```
    dispatch({ type: 'SET_CART', payload: { items: [], coupon: null } });
```

```
    setLoading(false);
```

```
    return;
```

```
  }
```

```
  try{
```

```
    console.log('Attempting to load cart from Firestore for user:', currentUser.uid);
```

```
    const userCartRef = doc(db, 'carts', currentUser.uid);
```

```
    const docSnap = await getDoc(userCartRef);
```

```
    if(docSnap.exists()) {
```

```
      const cartData = docSnap.data();
```

```
      console.log('Cart data loaded from Firestore:', cartData);
```



```

    dispatch({ type: 'SET_CART', payload: { items: cartData.items || [], coupon:
cartData.coupon || null } });
  } else {
    console.log('No cart found for user, initializing empty cart');
    // No cart exists for this user, initialize with empty cart
    dispatch({ type: 'SET_CART', payload: { items: [], coupon: null } });
  }
} catch (error) {
  console.error('Error loading cart from Firestore:', error);
  console.error('Error details:', error.code, error.message);
  // Fallback to empty cart
  dispatch({ type: 'SET_CART', payload: { items: [], coupon: null } });
} finally {
  setLoading(false);
}
}, [currentUser]);

// Load cart when user changes
useEffect(() => {
  console.log('CartProvider useEffect triggered with currentUser:', currentUser);
  loadCartFromFirestore();
}, [currentUser, loadCartFromFirestore]);

// Save cart to Firestore whenever cart changes and user is logged in
useEffect(() => {

```

```

// Don't save during initial load or if no user
if(!currentUser || loading || initialCartLoad.current) {
  if(initialCartLoad.current) {
    console.log('Skipping save during initial cart load');
    initialCartLoad.current = false;
  }
  return;
}

```

```

const saveCartToFirestore = async () => {
  console.log('Saving cart to Firestore:', {
    userId: currentUser.uid,
    itemCount: state.items.length,
    items: state.items,
    coupon: state.coupon
  });
}

```

```

try{
  const userCartRef = doc(db, 'carts', currentUser.uid);
  // Add a unique timestamp to force update
  const dataToSave = {
    userId: currentUser.uid,
    items: state.items.map(item => ({
      ...item,

```

```

        lastUpdated: Date.now()
      })),
      coupon: state.coupon,
      updatedAt: new Date().toISOString(),
      timestamp: Date.now()
    };

    await setDoc(userCartRef, dataToSave, { merge: true });
    console.log('Successfully saved cart to Firestore with data:', dataToSave);

    // Verify the save was successful by reading it back
    const verificationSnap = await getDoc(userCartRef);
    if(verificationSnap.exists()) {
      console.log('Verification: Cart successfully saved and can be read back');
    } else {
      console.error('Verification: Failed to verify cart save');
    }
  } catch (error) {
    console.error('Error saving cart to Firestore:', error);
    console.error('Error details:', error.code, error.message);
  }
};

// Debounce saves to prevent multiple rapid saves

```

```

const timeoutId = setTimeout(() => {
  saveCartToFirestore();
}, 500);

return () => clearTimeout(timeoutId);
}, [state.items, state.coupon, currentUser, loading]);

const addToCart = (product) => {
  console.log('Adding to cart:', product);
  // Ensure we use the primary image selected in the admin panel
  const productWithPrimaryImage = {
    ...product,
    imageUrl: getPrimaryImageUrl(product)
  };
  dispatch({ type: 'ADD_TO_CART', payload: productWithPrimaryImage });
};

const removeFromCart = async (productId) => {
  console.log('Removing from cart:', productId);
  dispatch({ type: 'REMOVE_FROM_CART', payload: productId });

  // Also remove from Firestore
  if(currentUser) {
    try{

```

```

const userCartRef = doc(db, 'carts', currentUser.uid);

// We'll update the entire cart instead of trying to remove a field

// The useEffect will handle saving the updated cart
} catch (error) {

  console.error('Error removing item from Firestore cart:', error);

}

}

};

const updateQuantity = (productId, quantity) => {

  console.log('Updating quantity:', productId, quantity);

  if(quantity <= 0) {

    removeFromCart(productId);

    return;

  }

  dispatch({ type: 'UPDATE_QUANTITY', payload: { id: productId, quantity } });

};

const clearCart = async () => {

  dispatch({ type: 'CLEAR_CART' });

// Also clear from Firestore

  if(currentUser) {

    try{

```

```

const userCartRef = doc(db, 'carts', currentUser.uid);

await setDoc(userCartRef, {
  userId: currentUser.uid,
  items: [],
  coupon: null,
  updatedAt: new Date()
});
} catch (error) {
  console.error('Error clearing cart in Firestore:', error);
}
}
};

const getTotalItems = () => {
  return state.items.reduce((total, item) => total + item.quantity, 0);
};

const getTotalPrice = () => {
  const subtotal = state.items.reduce((total, item) => total + (item.price *
item.quantity), 0);

  // Apply coupon discount if available
  if(state.coupon) {
    if(state.coupon.discountType === 'percentage') {
      return subtotal * (1 - state.coupon.discountValue / 100);
    }
  }
}

```

```

    } else if (state.coupon.discountType === 'fixed') {
      return Math.max(0, subtotal - state.coupon.discountValue);
    }
  }

  return subtotal;
};

const applyCoupon = async (couponCode) => {
  try {
    // Query Firestore for the coupon
    const couponsRef = collection(db, 'coupons');
    const q = query(couponsRef, where('code', '==', couponCode));
    const querySnapshot = await getDocs(q);

    if (querySnapshot.empty) {
      throw new Error('Cupón no válido');
    }

    const couponDoc = querySnapshot.docs[0];
    const couponData = { id: couponDoc.id, ...couponDoc.data() };

    // Check if coupon is active
    if (!couponData.isActive) {

```

```

    throw new Error('Este cupón no está activo');
  }

  // Check if coupon has expired
  const currentDate = new Date();
  if (couponData.expiryDate && currentDate > couponData.expiryDate.toDate()) {
    throw new Error('Este cupón ha expirado');
  }

  // Apply coupon
  dispatch({ type: 'APPLY_COUPON', payload: couponData });
  return couponData;
} catch (error) {
  console.error('Error applying coupon:', error);
  throw error;
}
};

const removeCoupon = () => {
  dispatch({ type: 'REMOVE_COUPON' });
};

return (
  <CartContext.Provider

```



```

    value={{
      items: state.items,
      coupon: state.coupon,
      loading,
      addToCart,
      removeFromCart,
      updateQuantity,
      clearCart,
      getTotalItems,
      getTotalPrice,
      applyCoupon,
      removeCoupon,
    }}
  >
  {children}
</CartContext.Provider>
);
};

```

```

export const useCart = () => {
  const context = useContext(CartContext);
  if(!context) {
    throw new Error('useCart must be used within a CartProvider');
  }
}

```

```

    return context;
};

📁 src/assets/context/wishlistcontext.jsx

import React, { createContext, useContext, useState, useRef, useCallback, useEffect }
from 'react';

import { db } from '../firebase';

import { doc, setDoc, getDoc } from 'firebase/firestore';

import { useAuth } from '../context/AuthContext';

const WishlistContext = createContext();

export const useWishlist = () => useContext(WishlistContext);

export const WishlistProvider = ({ children }) => {
  const [wishlist, setWishlist] = useState([]);
  const [loading, setLoading] = useState(true);
  const { currentUser } = useAuth();
  const initialWishlistLoad = useRef(true);

  // Get the primary image URL for a product
  const getPrimaryImageUrl = (product) => {
    if(product.imageUrls && product.imageUrls.length > 0) {
      // Filter out empty images
      const validImages = product.imageUrls.filter(image =>

```

```

    image && (typeof image === 'string' || (image.data && typeof image.data
=== 'string'))
  );

  if (validImages.length > 0) {
    const primaryIndex = product.primaryImageIndex || 0;
    // Ensure primaryIndex is within bounds
    const safeIndex = Math.min(primaryIndex, validImages.length - 1);
    const primaryImage = validImages[safeIndex];

    // Handle both string URLs and base64 data objects
    if (typeof primaryImage === 'string') {
      return primaryImage;
    } else if (primaryImage && primaryImage.data) {
      return primaryImage.data;
    }
  }

  return product.imageUrl ||
'https://via.placeholder.com/600x600.png?text= Moño';
};

```

```

// Load wishlist from Firestore when user logs in
const loadWishlistFromFirestore = useCallback(async () => {
  // Always reset loading state when starting
  setLoading(true);

```

```

if(!currentUser) {
  // If no user, clear local wishlist
  setWishlist([]);
  setLoading(false);
  initialWishlistLoad.current = false; // Set to false even when no user
  return;
}

try {
  const userWishlistRef = doc(db, 'wishlists', currentUser.uid);
  const docSnap = await getDoc(userWishlistRef);

  if(docSnap.exists()) {
    const wishlistData = docSnap.data();
    setWishlist(wishlistData.items || []);
  } else {
    // No wishlist exists for this user, initialize with empty wishlist
    setWishlist([]);
  }
} catch (error) {
  console.error('Error loading wishlist from Firestore:', error);
  // Fallback to empty wishlist
  setWishlist([]);
}

```

```

    } finally {
      setLoading(false);
      initialWishlistLoad.current = false; // Set to false after loading
    }
  }, [currentUser]);

// Load wishlist when user changes
useEffect(() => {
  loadWishlistFromFirestore();
}, [currentUser, loadWishlistFromFirestore]);

// Save wishlist to Firestore whenever wishlist changes and user is logged in
useEffect(() => {
  // Don't save during initial load or if no user
  if(!currentUser || loading || initialWishlistLoad.current) {
    return;
  }

  const saveWishlistToFirestore = async () => {
    try {
      const userWishlistRef = doc(db, 'wishlists', currentUser.uid);
      // Add a unique timestamp to force update
      const dataToSave = {
        userId: currentUser.uid,

```

```

      items: wishlist.map(item => ({
        ...item,
        lastUpdated: Date.now()
      })),
      updatedAt: new Date().toISOString(),
      timestamp: Date.now()
    };

    await setDoc(userWishlistRef, dataToSave, { merge: true });

    // Verify the save was successful by reading it back
    const verificationSnap = await getDoc(userWishlistRef);
    if(!verificationSnap.exists()) {
      console.error('Verification: Failed to verify wishlist save');
    }
  } catch (error) {
    console.error('Error saving wishlist to Firestore:', error);
  }
};

// Debounce saves to prevent multiple rapid saves
const timeoutId = setTimeout(() => {
  saveWishlistToFirestore();
}, 500);

```

```
    return () => clearTimeout(timeoutId);  
  }, [wishlist, currentUser, loading]);
```

```
const addToWishlist = (product) => {  
  setWishlist(prevWishlist => {  
    // Check if product already exists in wishlist  
    const exists = prevWishlist.find(item => item.id === product.id);  
    if (exists) {  
      return prevWishlist;  
    }  
    // Ensure we use the primary image selected in the admin panel  
    const productWithPrimaryImage = {  
      ...product,  
      imageUrl: getPrimaryImageUrl(product)  
    };  
    return [...prevWishlist, productWithPrimaryImage];  
  });  
};
```

```
const removeFromWishlist = (productId) => {  
  setWishlist(prevWishlist => prevWishlist.filter(item => item.id !== productId));  
};
```

```

const isWishlist = (productId) => {
  return wishlist.some(item => item.id === productId);
};

const clearWishlist = async () => {
  setWishlist([]);

  // Also clear from Firestore
  if(currentUser) {
    try{
      const userWishlistRef = doc(db, 'wishlists', currentUser.uid);
      await setDoc(userWishlistRef, {
        userId: currentUser.uid,
        items: [],
        updatedAt: new Date()
      });
    } catch (error) {
      console.error('Error clearing wishlist in Firestore:', error);
    }
  }
};

const getTotalItems = () => {
  return wishlist.length;
};

```



```

};

// Expose a function to manually trigger loading
const refreshWishlist = () => {
  loadWishlistFromFirestore();
};

return (
  <WishlistContext.Provider
    value={{
      items: wishlist,
      loading,
      addToWishlist,
      removeFromWishlist,
      isInWishlist,
      clearWishlist,
      getTotalItems,
      refreshWishlist
    }}
  >
    {children}
  </WishlistContext.Provider>
);
};

```

src/assets/hooks/UseScrollTop.jsx

```
import { useEffect } from 'react';  
import { useLocation } from 'react-router-dom';
```

```
const useScrollToTop = () => {  
  const { pathname } = useLocation();
```

```
  useEffect(() => {
```

```
    window.scrollTo(0, 0);  
  }, [pathname]);
```

```
  return null;
```

```
};
```

```
export default useScrollToTop;
```

src/assets/services/analyticsService.js

```
import { db } from '../firebase';
```

```
import { collection, getDocs, query, orderBy, where } from 'firebase/firestore';
```

```
import { formatCurrency } from './orderService';
```

```
// Get sales data by date range
```

```
export const getSalesData = async (startDate, endDate) => {
```

```
  try{
```

```

const ordersCollection = collection(db, 'orders');

const q = query(
  ordersCollection,
  where('createdAt', '>=', startDate),
  where('createdAt', '<=', endDate),
  orderBy('createdAt', 'desc')
);

const orderSnapshot = await getDocs(q);
const orders = orderSnapshot.docs.map(doc => ({
  id: doc.id,
  ...doc.data()
}));

// Group orders by date

const salesByDate = {};
let totalRevenue = 0;
let totalOrders = 0;

orders.forEach(order => {
  const date = order.createdAt.toDate().toISOString().split('T')[0];
  if(!salesByDate[date]) {
    salesByDate[date] = {
      date,
      totalSales: 0,

```

```
    orderCount: 0,  
    revenue: 0  
  };  
}
```

```
const orderTotal = order.totalAmount || 0;  
salesByDate[date].totalSales += orderTotal;  
salesByDate[date].orderCount += 1;  
salesByDate[date].revenue += orderTotal;  
totalRevenue += orderTotal;  
totalOrders += 1;  
});
```

```
// Convert to array and sort by date
```

```
const salesData = Object.values(salesByDate).sort((a, b) =>  
  new Date(a.date) - new Date(b.date)  
);
```

```
return {  
  salesData,  
  totalRevenue,  
  totalOrders,  
  averageOrderValue: totalOrders > 0 ? totalRevenue / totalOrders : 0  
};
```

```

    } catch (error) {
      console.error('Error fetching sales data:', error);
      throw error;
    }
  };

  // Get sales data by category
  export const getSalesByCategory = async () => {
    try{
      // Get all products with their categories
      const productsCollection = collection(db, 'products');
      const productSnapshot = await getDocs(productsCollection);
      const products = productSnapshot.docs.map(doc => ({
        id: doc.id,
        ...doc.data()
      }));

      // Get all orders
      const ordersCollection = collection(db, 'orders');
      const orderSnapshot = await getDocs(ordersCollection);
      const orders = orderSnapshot.docs.map(doc => ({
        id: doc.id,
        ...doc.data()
      }));
    }
  };

```

// Create a product ID to category mapping

```
const productCategoryMap = {};  
products.forEach(product => {  
  productCategoryMap[product.id] = product.category || 'Sin categoría';  
});
```

// Calculate sales by category

```
const categorySales = {};  
  
orders.forEach(order => {  
  if (order.items && Array.isArray(order.items)) {  
    order.items.forEach(item => {  
      const category = productCategoryMap[item.productId] || 'Sin categoría';  
      if (!categorySales[category]) {  
        categorySales[category] = {  
          category,  
          sales: 0,  
          quantity: 0,  
          revenue: 0  
        };  
      }  
    }  
  }  
});
```

```
const itemTotal = (item.price || 0) * (item.quantity || 0);
```

```

    categorySales[category].sales += itemTotal;
    categorySales[category].quantity += item.quantity || 0;
    categorySales[category].revenue += itemTotal;
  });
}
});

// Convert to array and sort by revenue
const salesByCategory = Object.values(categorySales).sort((a, b) =>
  b.revenue - a.revenue
);

return salesByCategory;
} catch (error) {
  console.error('Error fetching sales by category:', error);
  throw error;
}
};

// Get order status distribution
export const getOrderStatusDistribution = async () => {
  try{
    const ordersCollection = collection(db, 'orders');
    const orderSnapshot = await getDocs(ordersCollection);

```

```
const orders = orderSnapshot.docs.map(doc => ({
  id: doc.id,
  ...doc.data()
}));
```

```
const statusDistribution = {
  pending: 0,
  processing: 0,
  shipped: 0,
  delivered: 0,
  cancelled: 0
};
```

```
orders.forEach(order => {
  const status = order.orderStatus || 'pending';
  if (status in statusDistribution) {
    statusDistribution[status]++;
  }
});
```

// Convert to array format for charts

```
const statusData = Object.entries(statusDistribution).map(([status, count]) => ({
  status,
  count
```



```

    });

    return statusData;
  } catch (error) {
    console.error('Error fetching order status distribution:', error);
    throw error;
  }
};

```

```

// Get top selling products
export const getTopSellingProducts = async (limit = 10) => {
  try{
    // Get all products
    const productsCollection = collection(db, 'products');
    const productSnapshot = await getDocs(productsCollection);
    const products = productSnapshot.docs.map(doc => ({
      id: doc.id,
      ...doc.data()
    }));
  }
};

```

```

// Get all orders
const ordersCollection = collection(db, 'orders');
const orderSnapshot = await getDocs(ordersCollection);
const orders = orderSnapshot.docs.map(doc => ({

```

```

    id: doc.id,
    ...doc.data()
  });

// Create a product ID to name mapping
const productNameMap = {};
products.forEach(product => {
  productNameMap[product.id] = product.name || 'Producto sin nombre';
});

// Calculate sales by product
const productSales = {};

orders.forEach(order => {
  if (order.items && Array.isArray(order.items)) {
    order.items.forEach(item => {
      const productId = item.productId;
      if (!productSales[productId]) {
        productSales[productId] = {
          productId,
          productName: productNameMap[productId] || 'Producto desconocido',
          sales: 0,
          quantity: 0,
          revenue: 0
        };
      }
      productSales[productId].sales += item.sales;
      productSales[productId].quantity += item.quantity;
      productSales[productId].revenue += item.revenue;
    });
  }
});

```

```

    };
  }

  const itemTotal = (item.price || 0) * (item.quantity || 0);
  productSales[productId].sales += itemTotal;
  productSales[productId].quantity += item.quantity || 0;
  productSales[productId].revenue += itemTotal;
});
}
});

// Convert to array, sort by revenue, and limit
const topSellingProducts = Object.values(productSales)
  .sort((a, b) => b.revenue - a.revenue)
  .slice(0, limit);

return topSellingProducts;
} catch (error) {
  console.error('Error fetching top selling products:', error);
  throw error;
}
};

// Get user registration data

```

```

export const getUserRegistrationData = async (startDate, endDate) => {
  try {
    const usersCollection = collection(db, 'users');
    const q = query(
      usersCollection,
      where('createdAt', '>=', startDate),
      where('createdAt', '<=', endDate),
      orderBy('createdAt', 'desc')
    );
    const userSnapshot = await getDocs(q);
    const users = userSnapshot.docs.map(doc => ({
      id: doc.id,
      ...doc.data()
    }));

    // Group users by registration date
    const registrationsByDate = {};
    let totalUsers = 0;

    users.forEach(user => {
      const date = user.createdAt.toDate().toISOString().split('T')[0];
      if (!registrationsByDate[date]) {
        registrationsByDate[date] = {
          date,

```

```

        registrations: 0
    };
}

registrationsByDate[date].registrations += 1;
totalUsers += 1;
});

// Convert to array and sort by date
const registrationData = Object.values(registrationsByDate).sort((a, b) =>
    new Date(a.date) - new Date(b.date)
);

return {
    registrationData,
    totalUsers
};
} catch (error) {
    console.error('Error fetching user registration data:', error);
    throw error;
}
};

// Get revenue by payment method

```

```

export const getRevenueByPaymentMethod = async () => {
  try{
    const ordersCollection = collection(db, 'orders');
    const orderSnapshot = await getDocs(ordersCollection);
    const orders = orderSnapshot.docs.map(doc => ({
      id: doc.id,
      ...doc.data()
    }));

    const paymentMethodRevenue = {};

    orders.forEach(order => {
      const paymentMethod = order.paymentMethod || 'Desconocido';
      const revenue = order.totalAmount || 0;

      if(!paymentMethodRevenue[paymentMethod]) {
        paymentMethodRevenue[paymentMethod] = {
          method: paymentMethod,
          revenue: 0,
          count: 0
        };
      }

      paymentMethodRevenue[paymentMethod].revenue += revenue;
    });
  }
}

```

```

    paymentMethodRevenue[paymentMethod].count += 1;
  });

  // Convert to array
  const revenueByPaymentMethod = Object.values(paymentMethodRevenue);

  return revenueByPaymentMethod;
} catch (error) {
  console.error('Error fetching revenue by payment method:', error);
  throw error;
}
};

```

src/assets/services/categoryservice.js

```

import { db, storage } from '../firebase';

import { collection, getDocs, addDoc, updateDoc, deleteDoc, doc } from
'firebase/firestore';

import { ref, uploadBytes, getDownloadURL } from "firebase/storage";

// Get all categories
export const getCategories = async () => {
  try {
    const querySnapshot = await getDocs(collection(db, 'categories'));
    const categories = [];
    querySnapshot.forEach((doc) => {
      categories.push({

```

```

    id: doc.id,
    ...doc.data()
  });
});

return categories;
} catch (error) {
  console.error('Error fetching categories:', error);
  throw error;
}
};

// Create a new category

export const createCategory = async (categoryData) => {
  try{
    // If there's an image, ensure it's properly formatted
    const dataToSave = { ...categoryData };
    if(dataToSave.imageUrl && dataToSave.imageUrl.startsWith('data:')) {
      // Keep base64 image as is for Firestore storage
      // Firestore can handle base64 strings
    }

    const docRef = await addDoc(collection(db, 'categories'), {
      ...dataToSave,
      createdAt: new Date()
    });
  }
};

```



```

});

return { id: docRef.id, ...dataToSave, createdAt: new Date() };
} catch (error) {
  console.error('Error creating category:', error);
  throw error;
}
};

// Update a category

export const updateCategory = async (categoryId, categoryData) => {
  try{
    const categoryDoc = doc(db, 'categories', categoryId);

    // If there's an image, ensure it's properly formatted
    const dataToUpdate = { ...categoryData };
    if(dataToUpdate.imageUrl && dataToUpdate.imageUrl.startsWith('data:')) {
      // Keep base64 image as is for Firestore storage
      // Firestore can handle base64 strings
    }

    const updatedData = { ...dataToUpdate, updatedAt: new Date() };
    await updateDoc(categoryDoc, updatedData);
    return { id: categoryId, ...updatedData };
  } catch (error) {

```

```

    console.error('Error updating category:', error);

    throw error;
  }
};

```

// Delete a category

```

export const deleteCategory = async (categoryId) => {
  try{
    const categoryDoc = doc(db, 'categories', categoryId);
    await deleteDoc(categoryDoc);
    return categoryId;
  } catch (error) {
    console.error('Error deleting category:', error);
    throw error;
  }
};

```

// Upload category image

```

export const uploadCategoryImage = async (file, categoryId) => {
  try{
    const storageRef = ref(storage, `categories/${categoryId}/${file.name}`);
    const snapshot = await uploadBytes(storageRef, file);
    const downloadURL = await getDownloadURL(snapshot.ref);
    return downloadURL;
  }
};

```

```

    } catch (error) {

      console.error('Error uploading category image:', error);

      throw error;

    }

  };

```

 **src/assets/services/orderservice.js**

```

import { db } from '../firebase';

import { collection, getDocs, doc, getDoc, addDoc, updateDoc, query, where,
orderBy } from 'firebase/firestore';

// Format currency

export const formatCurrency = (amount) => {

  return new Intl.NumberFormat('es-CO', {

    style: 'currency',

    currency: 'COP',

    minimumFractionDigits: 0

  }).format(amount || 0);

};

// Format date for display

export const formatDate = (date) => {

  if(!date) return 'N/A';

  if(date instanceof Date) {

    return date.toLocaleDateString('es-CO');

  }

}

```

```
    if(date.toDate) {  
        return date.toDate().toLocaleDateString('es-CO');  
    }  
    return 'N/A';  
};
```

// Get status text in Spanish

```
export const getOrderStatusText = (status) => {  
    switch (status) {  
        case 'pending':  
            return 'Pendiente';  
        case 'processing':  
            return 'Procesando';  
        case 'shipped':  
            return 'Enviado';  
        case 'delivered':  
            return 'Entregado';  
        case 'cancelled':  
            return 'Cancelado';  
        default:  
            return status;  
    }  
};
```

```

// Get status badge class
export const getStatusBadgeClass = (status, theme = 'light') => {
  if(theme === 'dark') {
    switch (status) {
      case 'pending':
        return 'bg-yellow-900 text-yellow-200';
      case 'processing':
        return 'bg-blue-900 text-blue-200';
      case 'shipped':
        return 'bg-indigo-900 text-indigo-200';
      case 'delivered':
        return 'bg-green-900 text-green-200';
      case 'cancelled':
        return 'bg-red-900 text-red-200';
      default:
        return 'bg-gray-700 text-gray-200';
    }
  } else {
    switch (status) {
      case 'pending':
        return 'bg-yellow-100 text-yellow-800';
      case 'processing':
        return 'bg-blue-100 text-blue-800';
      case 'shipped':

```

```

        return 'bg-indigo-100 text-indigo-800';
    case 'delivered':
        return 'bg-green-100 text-green-800';
    case 'cancelled':
        return 'bg-red-100 text-red-800';
    default:
        return 'bg-gray-100 text-gray-800';
    }
}
};

```

```

// Create a new order
export const createOrder = async (orderData) => {
  try{
    const ordersCollection = collection(db, 'orders');
    const docRef = await addDoc(ordersCollection, {
      ...orderData,
      createdAt: new Date(),
      status: 'pending'
    });
    return {
      id: docRef.id,
      ...orderData,
      createdAt: new Date(),

```

```

    status: 'pending'
  };
} catch (error) {
  console.error('Error creating order:', error);
  throw error;
}
};

// Get orders by user ID
export const getOrdersByUserId = async (userId) => {
  try{
    const ordersCollection = collection(db, 'orders');

    const q = query(ordersCollection, where('userId', '==', userId),
      orderBy('createdAt', 'desc'));

    const orderSnapshot = await getDocs(q);

    const orderList = orderSnapshot.docs.map(doc => ({
      id: doc.id,
      ...doc.data()
    }));

    return orderList;
  } catch (error) {
    // Check if it's a missing index error

    if(error.code === 'failed-precondition' ||
      error.code === 'invalid-argument' ||

```

```

    (error.message && (error.message.includes('index') ||
error.message.includes('query requires an index')))) {

    console.warn('Missing index for orders query, using fallback method. Create the
required index for better performance.');
```

console.warn('Index creation link:', error.message.match(/https:\V\^[^"]*/)?.[0] ||
'Check Firebase Console');

// Fallback to unordered query and sort in memory

```

    try{

    const ordersCollection = collection(db, 'orders');

    const q = query(ordersCollection, where('userId', '=', userId));

    const orderSnapshot = await getDocs(q);

    const orderList = orderSnapshot.docs.map(doc => ({

        id: doc.id,

        ...doc.data()

    }));

// Sort in memory by createdAt (descending)

    const sortedOrders = orderList.sort((a, b) => {

        // Handle different date formats

        let dateA, dateB;

        // For Firestore timestamps

        if(a.createdAt && typeof a.createdAt.toDate === 'function') {

            dateA = a.createdAt.toDate();

        }

```



```

// For JavaScript Date objects
else if(a.createdAt instanceof Date) {
    dateA = a.createdAt;
}

// For string dates
else if(typeof a.createdAt === 'string') {
    dateA = new Date(a.createdAt);
}

// For numeric timestamps
else if(typeof a.createdAt === 'number') {
    dateA = new Date(a.createdAt);
}

// Default
else {
    dateA = new Date(0);
}

// Same for dateB
if(b.createdAt && typeof b.createdAt.toDate === 'function') {
    dateB = b.createdAt.toDate();
} else if(b.createdAt instanceof Date) {
    dateB = b.createdAt;
} else if(typeof b.createdAt === 'string') {
    dateB = new Date(b.createdAt);
}

```

```

    } else if(typeof b.createdAt === 'number') {
        dateB = new Date(b.createdAt);
    } else {
        dateB = new Date(0);
    }

    return dateB - dateA;
});

return sortedOrders;
} catch (fallbackErr) {
    console.error('Error fetching orders without index:', fallbackErr);
    throw fallbackErr;
}
} else {
    console.error('Error fetching orders:', error);
    throw error;
}
}
};

// Get order by ID
export const getOrderById = async (id) => {
    try{

```

```

const orderDoc = doc(db, 'orders', id);
const orderSnapshot = await getDoc(orderDoc);
if(orderSnapshot.exists()) {
  return {
    id: orderSnapshot.id,
    ...orderSnapshot.data()
  };
} else {
  throw new Error('Order not found');
}
} catch (error) {
  console.error('Error fetching order:', error);
  throw error;
}
};

// Update order status
export const updateOrderStatus = async (id, status) => {
  try{
    const orderDoc = doc(db, 'orders', id);
    await updateDoc(orderDoc, {
      orderStatus: status,
      updatedAt: new Date()
    });
  }
};

```

```

    return { id, status };
  } catch (error) {
    console.error('Error updating order status:', error);
    throw error;
  }
};

// Get all orders (for admin)
export const getAllOrders = async () => {
  try {
    const ordersCollection = collection(db, 'orders');
    const q = query(ordersCollection, orderBy('createdAt', 'desc'));
    const orderSnapshot = await getDocs(q);
    const orderList = orderSnapshot.docs.map(doc => ({
      id: doc.id,
      ...doc.data()
    }));
    return orderList;
  } catch (error) {
    // Check if it's a missing index error
    if (error.code === 'failed-precondition' ||
        error.code === 'invalid-argument' ||
        (error.message && (error.message.includes('index') ||
            error.message.includes('query requires an index')))) {

```

```
console.warn('Missing index for all orders query, using fallback method. Create the required index for better performance.');
```

```
console.warn('Index creation link:', error.message.match(/https:\V\^[^"]*/)?.[0] || 'Check Firebase Console');
```

```
// Fallback to unordered query and sort in memory
```

```
try{
```

```
const ordersCollection = collection(db, 'orders');
```

```
const orderSnapshot = await getDocs(ordersCollection);
```

```
const orderList = orderSnapshot.docs.map(doc => ({
```

```
  id: doc.id,
```

```
  ...doc.data()
```

```
}));
```

```
// Sort in memory by createdAt (descending)
```

```
const sortedOrders = orderList.sort((a, b) => {
```

```
// Handle different date formats
```

```
let dateA, dateB;
```

```
// For Firestore timestamps
```

```
if(a.createdAt && typeof a.createdAttoDate === 'function') {
```

```
  dateA = a.createdAttoDate();
```

```
}
```

```
// For JavaScript Date objects
```

```
else if(a.createdAt instanceof Date) {
```

```
  dateA = a.createdAt;
```

```

}

// For string dates

else if(typeof a.createdAt === 'string') {
    dateA = new Date(a.createdAt);
}

// For numeric timestamps

else if(typeof a.createdAt === 'number') {
    dateA = new Date(a.createdAt);
}

// Default

else {
    dateA = new Date(0);
}


// Same for dateB

if(b.createdAt && typeof b.createdAt.toDate === 'function') {
    dateB = b.createdAt.toDate();
} else if(b.createdAt instanceof Date) {
    dateB = b.createdAt;
} else if(typeof b.createdAt === 'string') {
    dateB = new Date(b.createdAt);
} else if(typeof b.createdAt === 'number') {
    dateB = new Date(b.createdAt);
} else {

```

```

        dateB = new Date(0);
    }

    return dateB - dateA;
});

return sortedOrders;
} catch (fallbackErr) {
    console.error('Error fetching all orders without index:', fallbackErr);
    throw fallbackErr;
}
} else {
    if (error.code === 'permission-denied') {
        console.error('Firebase Permission Denied: Check Firestore rules for "orders"
collection and ensure your user document in "users" collection has role: "admin".',
error);
    } else {
        console.error('Error fetching all orders:', error);
    }
    throw error;
}
}
};

```

 src/assets/services/productservice.js

```
import { db } from '../firebase';

import { collection, getDocs, doc, getDoc, addDoc, updateDoc, deleteDoc, query,
where } from 'firebase/firestore';

// Get all products

export const getProducts = async () => {
  try {
    const productsCollection = collection(db, 'products');
    const productSnapshot = await getDocs(productsCollection);
    const productList = productSnapshot.docs.map(doc => ({
      id: doc.id,
      ...doc.data()
    }));
    return productList;
  } catch (error) {
    console.error('Error fetching products:', error);
    throw error;
  }
};

// Get product by ID

export const getProductById = async (id) => {
  try {
    const productDoc = doc(db, 'products', id);
```



```

const productSnapshot = await getDoc(productDoc);
if(productSnapshot.exists()) {
  return {
    id: productSnapshot.id,
    ...productSnapshot.data()
  };
} else {
  throw new Error('Product not found');
}
} catch (error) {
  console.error('Error fetching product:', error);
  throw error;
}
};

```

// Get products by category

```

export const getProductsByCategory = async (category) => {
  try{
    const productsCollection = collection(db, 'products');
    const q = query(productsCollection, where('category', '==', category));
    const productSnapshot = await getDocs(q);
    const productList = productSnapshot.docs.map(doc => ({
      id: doc.id,
      ...doc.data()
    }

```

```

    });

    return productList;
  } catch (error) {
    console.error('Error fetching products by category:', error);
    throw error;
  }
};

// Create a new product

export const createProduct = async (productData) => {
  try{
    const productsCollection = collection(db, 'products');
    const docRef = await addDoc(productsCollection, productData);
    return {
      id: docRef.id,
      ...productData
    };
  } catch (error) {
    console.error('Error creating product:', error);
    throw error;
  }
};

// Update a product

```

```

export const updateProduct = async (id, productData) => {
  try{
    const productDoc = doc(db, 'products', id);
    await updateDoc(productDoc, productData);
    return {
      id,
      ...productData
    };
  } catch (error) {
    console.error('Error updating product:', error);
    throw error;
  }
};

```

// Delete a product

```

export const deleteProduct = async (id) => {
  try{
    const productDoc = doc(db, 'products', id);
    await deleteDoc(productDoc);
    return id;
  } catch (error) {
    console.error('Error deleting product:', error);
    throw error;
  }
}

```

```
};
```

 **src/assets/services/reviewservice.js**

```
import { db } from './firebase';
```

```
import { collection, getDocs, doc, getDoc, addDoc, updateDoc, deleteDoc, query,  
where, orderBy } from 'firebase/firestore';
```

```
// Create a new review
```

```
export const createReview = async (reviewData) => {
```

```
  try{
```

```
    const reviewsCollection = collection(db, 'reviews');
```

```
    const docRef = await addDoc(reviewsCollection, {
```

```
      ...reviewData,
```

```
      createdAt: new Date(),
```

```
      updatedAt: new Date()
```

```
    });
```

```
    return {
```

```
      id: docRef.id,
```

```
      ...reviewData,
```

```
      createdAt: new Date(),
```

```
      updatedAt: new Date()
```

```
    };
```

```
  } catch (error) {
```

```
    console.error('Error creating review:', error);
```

```
    throw error;
```

```
  }
```

```
};
```

```
// Get reviews by product ID
```

```
export const getReviewsByProductId = async (productId) => {
```

```
  try{
```

```
    const reviewsCollection = collection(db, 'reviews');
```

```
    const q = query(reviewsCollection, where('productId', '=', productId),  
orderBy('createdAt', 'desc'));
```

```
    const reviewSnapshot = await getDocs(q);
```

```
    const reviewList = reviewSnapshot.docs.map(doc => ({
```

```
      id: doc.id,
```

```
      ...doc.data()
```

```
    }));
```

```
    return reviewList;
```

```
  } catch (error) {
```

```
    // Check if it's a missing index error
```

```
    if(error.code === 'failed-precondition' || (error.message &&  
error.message.includes('index')))) {
```

```
      console.warn('Missing index for reviews query, using fallback method. Create  
the required index for better performance.');
```

```
    // Fallback to unordered query and sort in memory
```

```
    try{
```

```
      const reviewsCollection = collection(db, 'reviews');
```

```
      const q = query(reviewsCollection, where('productId', '=', productId));
```

```
      const reviewSnapshot = await getDocs(q);
```

```

const reviewList = reviewSnapshot.docs.map(doc => ({
  id: doc.id,
  ...doc.data()
}));

// Sort in memory by createdAt (descending)
const sortedReviews = reviewList.sort((a, b) => {
  // Handle different date formats
  let dateA, dateB;

  // For Firestore timestamps
  if(a.createdAt && typeof a.createdAt.toDate === 'function') {
    dateA = a.createdAt.toDate();
  }

  // For JavaScript Date objects
  else if(a.createdAt instanceof Date) {
    dateA = a.createdAt;
  }

  // For string dates
  else if(typeof a.createdAt === 'string') {
    dateA = new Date(a.createdAt);
  }

  // For numeric timestamps
  else if(typeof a.createdAt === 'number') {

```

```

    dateA = new Date(a.createdAt);
}

// Default

else {
    dateA = new Date(0);
}

// Same for dateB

if(b.createdAt && typeof b.createdAt.toDate === 'function') {
    dateB = b.createdAt.toDate();
} else if(b.createdAt instanceof Date) {
    dateB = b.createdAt;
} else if(typeof b.createdAt === 'string') {
    dateB = new Date(b.createdAt);
} else if(typeof b.createdAt === 'number') {
    dateB = new Date(b.createdAt);
} else {
    dateB = new Date(0);
}

return dateB - dateA;
});

return sortedReviews;

```

```

    } catch (fallbackErr) {
      console.error('Error fetching reviews without index:', fallbackErr);
      throw fallbackErr;
    }
  } else {
    console.error('Error fetching reviews:', error);
    throw error;
  }
}
};

```

// Get reviews by user ID and product ID (to check if user already reviewed a product)

```

export const getUserReviewForProduct = async (userId, productId) => {
  try{
    const reviewsCollection = collection(db, 'reviews');
    const q = query(
      reviewsCollection,
      where('userId', '==', userId),
      where('productId', '==', productId)
    );
    const reviewSnapshot = await getDocs(q);

    if(reviewSnapshot.empty) {
      return null;
    }
  }
};

```



```

    }

    // Return the first (and should be only) review
    const reviewDoc = reviewSnapshot.docs[0];

    return {
      id: reviewDoc.id,
      ...reviewDoc.data()
    };
  } catch (error) {
    console.error('Error fetching user review for product:', error);
    throw error;
  }
};

// Get all reviews (for admin)
export const getAllReviews = async () => {
  try {
    const reviewsCollection = collection(db, 'reviews');
    const q = query(reviewsCollection, orderBy('createdAt', 'desc'));
    const reviewSnapshot = await getDocs(q);
    const reviewList = reviewSnapshot.docs.map(doc => ({
      id: doc.id,
      ...doc.data()
    }));
  }

```

```

    return reviewList;
  } catch (error) {

    // Check if it's a missing index error

    if(error.code === 'failed-precondition' || (error.message &&
error.message.includes('index')))) {

      console.warn('Missing index for all reviews query, using fallback method. Create
the required index for better performance.');
```

// Fallback to unordered query and sort in memory

```

    try{

      const reviewsCollection = collection(db, 'reviews');
      const reviewSnapshot = await getDocs(reviewsCollection);
      const reviewList = reviewSnapshot.docs.map(doc => ({
        id: doc.id,
        ...doc.data()
      }));

      // Sort in memory by createdAt (descending)

      const sortedReviews = reviewList.sort((a, b) => {

        // Handle different date formats

        let dateA, dateB;

        // For Firestore timestamps

        if(a.createdAt && typeof a.createdAt.toDate === 'function') {
          dateA = a.createdAt.toDate();
        }

```

```

// For JavaScript Date objects

else if(a.createdAt instanceof Date) {

    dateA = a.createdAt;

}

// For string dates

else if(typeof a.createdAt === 'string') {

    dateA = new Date(a.createdAt);

}

// For numeric timestamps

else if(typeof a.createdAt === 'number') {

    dateA = new Date(a.createdAt);

}

// Default

else {

    dateA = new Date(0);

}


// Same for dateB

if(b.createdAt && typeof b.createdAt.toDate === 'function') {

    dateB = b.createdAt.toDate();

} else if(b.createdAt instanceof Date) {

    dateB = b.createdAt;

} else if(typeof b.createdAt === 'string') {

    dateB = new Date(b.createdAt);

}

```

```

    } else if(typeof b.createdAt === 'number') {
        dateB = new Date(b.createdAt);
    } else {
        dateB = new Date(0);
    }

    return dateB - dateA;

});

return sortedReviews;
} catch (fallbackErr) {
    console.error('Error fetching all reviews without index:', fallbackErr);
    throw fallbackErr;
}
} else {
    console.error('Error fetching all reviews:', error);
    throw error;
}
}
};

// Update a review
export const updateReview = async (id, reviewData) => {
    try{

```

```

const reviewDoc = doc(db, 'reviews', id);

await updateDoc(reviewDoc, {
  ...reviewData,
  updatedAt: new Date()
});

return {
  id,
  ...reviewData,
  updatedAt: new Date()
};
} catch (error) {
  console.error('Error updating review:', error);
  throw error;
}
};

```

// Delete a review

```

export const deleteReview = async (id) => {
  try{
    const reviewDoc = doc(db, 'reviews', id);
    await deleteDoc(reviewDoc);
    return id;
  } catch (error) {
    console.error('Error deleting review:', error);
  }
}

```

```
    throw error;
  }
};
```

src/assets/services/userservice.js

```
import { db } from '../firebase';

import { collection, getDocs, doc, getDoc, updateDoc, query, where, orderBy } from
'firebase/firestore';

// Get user by ID
export const getUserById = async (id) => {
  try {
    const userDoc = doc(db, 'users', id);
    const userSnapshot = await getDoc(userDoc);
    if (userSnapshot.exists()) {
      return {
        id: userSnapshot.id,
        ...userSnapshot.data()
      };
    } else {
      throw new Error('User not found');
    }
  } catch (error) {
    console.error('Error fetching user:', error);
    throw error;
  }
}
```

```
};
```

```
// Update user profile
```

```
export const updateUserProfile = async (id, userData) => {
```

```
  try{
```

```
    const userDoc = doc(db, 'users', id);
```

```
    await updateDoc(userDoc, userData);
```

```
    return {
```

```
      id,
```

```
      ...userData
```

```
    };
```

```
  } catch (error) {
```

```
    console.error('Error updating user profile:', error);
```

```
    throw error;
```

```
  }
```

```
};
```

```
// Get all users (for admin)
```

```
export const getAllUsers = async () => {
```

```
  try{
```

```
    const usersCollection = collection(db, 'users');
```

```
    const userSnapshot = await getDocs(usersCollection);
```

```
    const userList = userSnapshot.docs.map(doc => ({
```

```
      id: doc.id,
```

```

    ...doc.data()

  });

  return userList;
} catch (error) {

  console.error('Error fetching users:', error);

  throw error;
}
};

// Update user role (for admin)
export const updateUserRole = async (id, role) => {
  try{
    const userDoc = doc(db, 'users', id);

    await updateDoc(userDoc, { role });

    return { id, role };
  } catch (error) {

    console.error('Error updating user role:', error);

    throw error;
  }
};

```

 **src/assets/utils/createsamplecoupon.js**

```

import { db } from '../firebase';

import { collection, addDoc } from 'firebase/firestore';

```



```

// Create a sample coupon for testing

const createSampleCoupon = async () => {
  try{
    const couponData = {
      code: 'DESCUENTO10',
      discountType: 'percentage',
      discountValue: 10,
      isActive: true,
      createdAt: new Date()
    };

    const docRef = await addDoc(collection(db, 'coupons'), couponData);
    console.log('Sample coupon created with ID: ', docRef.id);
    return docRef.id;
  } catch (error) {
    console.error('Error creating sample coupon: ', error);
    throw error;
  }
};

export default createSampleCoupon;

```

 src/assets/utils/firebaseadmin.js

// Utility functions for Firebase admin operations

import { getAuth } from "firebase/auth";

import { app } from "../firebase"; // Fixed import path

// Function to create an admin user in Firebase

export const createAdminUser = async (email, password) => {

try{

const auth = getAuth(app);

// Note: In a real application, you would use Firebase Admin SDK on the server

// For demonstration purposes, we'll just show how it would work

console.log("In a production environment, this would create an admin user with:", { email, password });

console.log("You would need to use Firebase Admin SDK on a server to create users programmatically");

// Return a success message

return { success: true, message: "Admin user creation simulated successfully" };

} catch (error) {

console.error("Error creating admin user:", error);

return { success: false, message: error.message };

}

};

// Function to set custom claims for a user (admin role)

```

export const setAdminRole = async (uid) => {
  try{
    // Note: This would also require Firebase Admin SDK on the server

    console.log("In a production environment, this would set admin role for user:",
uid);

    console.log("You would need to use Firebase Admin SDK on a server to set
custom claims");

    // Return a success message

    return { success: true, message: "Admin role assignment simulated successfully" };
  } catch (error) {
    console.error("Error setting admin role:", error);
    return { success: false, message: error.message };
  }
};

```

 **src/assets/components/Admin/AdminCategories.jsx**

```

import React from 'react';

import Loader from '../Loader';

import { FiList, FiPlus, FiEdit3, FiTrash2, FiSearch, FiLayers, FilmImage } from 'react-
icons/fi';

const AdminCategories = ({
  categories,
  categoriesLoading,
  categorySearchTerm,

```

```

    setCategorySearchTerm,
    theme,
    openAddCategoryModal,
    openEditCategoryModal,
    deleteCategoryHandler
  }) => {
    if(categoriesLoading) {
      return (
        <div className="flex flex-col items-center justify-center min-h-[40vh] space-y-4">
          <div className="w-12 h-12 border-4 border-indigo-600 border-b-transparent rounded-full animate-spin"> </div>
          <p className="text-[10px] font-black uppercase tracking-[0.3em] text-indigo-600">Organizando Secciones...</p>
        </div>
      );
    }

    const isDark = theme === 'dark';
    const filteredCategories = categorySearchTerm
      ? categories.filter(category =>
          category.name.toLowerCase().includes(categorySearchTerm.toLowerCase()) ||
          (category.description &&
            category.description.toLowerCase().includes(categorySearchTerm.toLowerCase()))
        )
      : categories;

```

// Luxury UI Tokens

```
const bgCard = isDark ? 'bg-[#1a1b26] border-slate-800 shadow-2xl' : 'bg-white border-slate-100 shadow-sm';
```

```
const textTitle = isDark ? 'text-slate-100' : 'text-slate-900';
```

```
const textSub = isDark ? 'text-slate-400' : 'text-slate-500';
```

```
return (
```

```
<div className="space-y-10 animate-in fade-in slide-in-from-bottom-4 duration-500">
```

```
  {/* Header & Search Zone */}
```

```
<div className={` ${bgCard} p-8 md:p-10 rounded-[2.5rem] border flex flex-col xl:flex-row xl:items-center justify-between gap-8 relative overflow-hidden`>
```

```
<div className="absolute top-0 right-0 w-64 h-64 bg-indigo-500/5 rounded-full -mr-32 -mt-32 blur-3xl"></div>
```

```
<div className="relative">
```

```
<div className="flex items-center gap-3 mb-2">
```

```
<div className="w-8 h-1 bg-indigo-600 rounded-full"></div>
```

```
<span className={`text-[10px] font-black uppercase tracking-[0.4em] text-indigo-600`} >Structure Hub</span>
```

```
</div>
```

```
<h2 className={`text-3xl font-black tracking-tight ${textTitle}`} >Categorías de Marca</h2>
```

```
<p className={` ${textSub} mt-1 text-sm`} >Define la arquitectura de navegación y agrupamiento de tus productos.</p>
```

```

</div>

<div className="flex flex-col md:flex-row items-center gap-4 relative">
  <div className="relative w-full md:w-80 group">
    <FiSearch className={`absolute left-5 top-1/2 -translate-y-1/2 transition-
colors ${isDark ? 'text-slate-500 group-focus-within:text-indigo-400' : 'text-slate-
400 group-focus-within:text-indigo-600'}} />
    <input
      type="text"
      placeholder="Buscar categorías..."
      value={categorySearchTerm}
      onChange={(e) => setCategorySearchTerm(e.target.value)}
      className={`w-full pl-14 pr-6 py-4 rounded-2xl border transition-all
duration-300 outline-none focus:ring-4 ${
        isDark ? 'bg-slate-900/50 border-slate-700 focus:ring-indigo-500/20 text-
white' : 'bg-slate-50 border-slate-200 focus:ring-indigo-500/10 text-slate-900'
      }}
    />
  </div>
  <button
    onClick={openAddCategoryModal}
    className="flex items-center justify-center gap-2 px-8 py-4 bg-black text-
white rounded-2xl font-black text-sm transition-all hover:bg-gray-800
hover:shadow-2xl active:scale-95 w-full md:w-auto"
  >
    <FiPlus size={20} /> Nueva Categoría
  </button>
</div>

```

```

    </button>

  </div>

</div>

{/* Categories Grid */}

<div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 xl:grid-cols-4
gap-8">

  {filteredCategories.map((cat) => (

    <div

      key={cat.id}

      className={` ${bgCard} group rounded-[2.5rem] border overflow-hidden
transition-all duration-500 hover:-translate-y-2`}

      >

        <div className="relative h-60 w-full overflow-hidden">

          {cat.imageUrl ? (

            <img

              src={cat.imageUrl}

              alt={cat.name}

              className="w-full h-full object-cover transition-transform duration-700
group-hover:scale-110"

              />

            ) : (

              <div className={`w-full h-full flex items-center justify-center ${isDark ?
'bg-slate-900' : 'bg-slate-50'}`}>

                <FilmImage className={textSub} size={40} opacity={0.2} />

              </div>

            )}

          )}

        </div>

```

```
}}
```

```
<div className="absolute inset-0 bg-gradient-to-t from-black/80 via-transparent to-transparent opacity-80"> </div>
```

```
<div className="absolute top-4 right-4 flex gap-2 transition-all duration-300">
```

```
<button
```

```
  onClick={() => openEditCategoryModal(cat)}
```

```
  className="p-2.5 rounded-xl bg-white/90 backdrop-blur-md text-indigo-600 hover:bg-indigo-600 hover:text-white shadow-lg transition-all active:scale-90 border border-white/20"
```

```
    title="Editar"
```

```
>
```

```
  <FiEdit3 size={15} />
```

```
</button>
```

```
<button
```

```
  onClick={() => deleteCategoryHandler(cat.id, cat.name)}
```

```
  className="p-2.5 rounded-xl bg-white/90 backdrop-blur-md text-rose-600 hover:bg-rose-600 hover:text-white shadow-lg transition-all active:scale-90 border border-white/20"
```

```
    title="Eliminar"
```

```
>
```

```
  <FiTrash2 size={15} />
```

```
</button>
```

```
</div>
```



```

<div className="absolute bottom-6 left-6 right-6">
  <div className="flex items-center gap-2 mb-1">
    <div className="w-4 h-0.5 bg-indigo-500 rounded-full"> </div>
    <span className="text-[10px] font-black uppercase tracking-[0.3em]
text-white/60">Categoría</span>
  </div>
  <h3 className="text-xl font-black text-white tracking-
tight">{cat.name}</h3>
</div>
</div>

<div className="p-8">
  <p className={`text-sm leading-relaxed line-clamp-2 ${textSub}`}>
    {cat.description || 'Sin descripción detallada para esta categoría.'}
  </p>
  <div className="mt-6 flex items-center gap-3">
    <FiLayers className="text-indigo-500" size={14} />
    <span className="text-[10px] font-black uppercase tracking-widest
text-slate-400">Total: Dinámico</span>
  </div>
</div>
</div>
)}}

```

```

    {filteredCategories.length === 0 && (
      <div className="col-span-full py-24 text-center">
        <div className={`w-20 h-20 rounded-full flex items-center justify-center
mx-auto mb-6 ${isDark ? 'bg-slate-800' : 'bg-slate-50'}}>
          <FiList size={32} className="opacity-20" />
        </div>
        <h3 className={`text-lg font-black ${textTitle}`}>Sin Resultados</h3>
        <p className={` ${textSub} text-sm mt-2`} >No pudimos encontrar
categorías que coincidan con tu búsqueda.</p>
      </div>
    )}
  </div>
</div>
);
};

```

```

export default AdminCategories;

```

```

 src/assets/components/Admin/AdminOrders.jsx

```

```

import React from 'react';

```

```

import Loader from '../Loader';

```

```

import { FiPackage, FiShoppingBag, FiTrash2, FiSearch, FiTruck, FiRefreshCw,
FiExternalLink } from 'react-icons/fi';

```

```

const AdminOrders = ({
  orders,

```

```

ordersLoading,
orderSearchTerm,
setOrderSearchTerm,
theme,
formatDate,
formatCurrency,
getStatusBadgeClass,
getOrderStatusText,
orderStatusUpdating,
updateOrderStatusHandler,
openOrderDetails,
deleteOrder
}) => {
  if(ordersLoading) {
    return (
      <div className="flex flex-col items-center justify-center min-h-[40vh] space-y-4">
        <div className="w-12 h-12 border-4 border-indigo-600 border-b-transparent rounded-full animate-spin"> </div>
        <p className="text-[10px] font-black uppercase tracking-[0.3em] text-indigo-600">Rastreando Logística...</p>
      </div>
    );
  }
}

```

```

const isDark = theme === 'dark';

const filteredOrders = orderSearchTerm
  ? orders.filter(order =>
    (order.id && order.id.toLowerCase().includes(orderSearchTerm.toLowerCase()))
  ||
    (order.userEmail &&
order.userEmail.toLowerCase().includes(orderSearchTerm.toLowerCase())) ||
    (order.orderStatus &&
order.orderStatus.toLowerCase().includes(orderSearchTerm.toLowerCase()))
  )
  : orders;

```

// Luxury UI Tokens

```

const bgCard = isDark ? 'bg-[#1a1b26] border-slate-800 shadow-2xl' : 'bg-white
border-slate-100 shadow-sm';

```

```

const textTitle = isDark ? 'text-slate-100' : 'text-slate-900';

```

```

const textSub = isDark ? 'text-slate-400' : 'text-slate-500';

```

```

return (

```

```

  <div className="space-y-10 animate-in fade-in slide-in-from-bottom-4
duration-500">

```

```

    {/* Header & Filter Zone */}

```

```

    <div className={` ${bgCard} p-6 md:p-10 rounded-3xl md:rounded-[2.5rem]
border flex flex-col xl:flex-row xl:items-center justify-between gap-6 md:gap-8
relative overflow-hidden`} >

```

```
<div className="absolute top-0 right-0 w-64 h-64 bg-indigo-500/5
rounded-full -mr-32 -mt-32 blur-3xl"></div>
```

```
<div className="relative">
  <div className="flex items-center gap-3 mb-2">
    <div className="w-8 h-1 bg-indigo-600 rounded-full"></div>
    <span className={`text-[10px] font-black uppercase tracking-[0.4em] text-
indigo-600`}>Logistics Control</span>
  </div>
  <h2 className={`text-2xl md:text-3xl font-black tracking-tight
${textTitle}`}>Flujo de Pedidos</h2>
  <p className={` ${textSub} mt-1 text-xs md:text-sm`}>Supervisa el ciclo de
vida de cada venta, desde el pago hasta la entrega final.</p>
</div>
```

```
<div className="flex flex-col md:flex-row items-center gap-4 relative">
  <div className="relative w-full md:w-80 group">
    <FiSearch className={`absolute left-5 top-1/2 -translate-y-1/2 transition-
colors ${isDark ? 'text-slate-500 group-focus-within:text-indigo-400' : 'text-slate-
400 group-focus-within:text-indigo-600'}} />
    <input
      type="text"
      placeholder="Buscar pedido..."
      value={orderSearchTerm}
      onChange={(e) => setOrderSearchTerm(e.target.value)}
      className={`w-full pl-14 pr-6 py-4 rounded-2xl border transition-all
duration-300 outline-none focus:ring-4 ${
```

```
isDark ? 'bg-slate-900/50 border-slate-700 focus:ring-indigo-500/20 text-
white' : 'bg-slate-50 border-slate-200 focus:ring-indigo-500/10 text-slate-900'
```

```
}}
```

```
/>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
{/* Orders Content Table / Cards */}
```

```
<div className={` ${bgCard} rounded-3xl md:rounded-[2.5rem] border
overflow-hidden`} >
```

```
{/* Desktop Table View */}
```

```
<div className="hidden md:block overflow-x-auto">
```

```
<table className="w-full text-left border-collapse">
```

```
<thead>
```

```
<tr className={`text-[10px] font-black uppercase tracking-[0.2em]
${textSub} border-b border-inherit`} >
```

```
<th className="px-10 py-6">Pedido & Comprador</th>
```

```
<th className="px-10 py-6">Fecha Venta</th>
```

```
<th className="px-10 py-6">Valor Total</th>
```

```
<th className="px-10 py-6">Estado Logístico</th>
```

```
<th className="px-10 py-6 text-right pr-14">Detalles</th>
```

```
</tr>
```

```
</thead>
```

```
<tbody className={`divide-y ${isDark ? 'divide-slate-800/10' : 'divide-slate-
50'}}>
```

```

{filteredOrders.map((o) => (
  <tr key={o.id} className="group hover:bg-slate-500/5 transition-all
duration-300">
    <td className="px-10 py-6">
      <div className="flex items-center gap-5">
        <div className={`w-12 h-12 rounded-2xl flex items-center justify-
center font-black shadow-inner ${
  isDark ? 'bg-slate-800 text-indigo-400' : 'bg-indigo-50 text-indigo-
600'
}}}>
          <FiPackage size={20} />
        </div>
        <div className="flex flex-col">
          <span className={`font-black text-sm tracking-tight
${textTitle}`}>{o.userEmail || 'Invitado'}</span>
          <span className="text-[10px] text-slate-400 font-bold uppercase
tracking-widest mt-0.5">ORD-{o.id.substring(0, 8).toUpperCase()}</span>
        </div>
      </div>
    </td>
    <td className="px-10 py-6">
      <span className={`text-[11px] font-black uppercase tracking-widest
${textSub}`}>
        {formatDate(o.createdAt)}
      </span>
    </td>
  )
)

```

```

<td className="px-10 py-6">
  <span className={`font-black text-base text-emerald-500`}>
    {formatCurrency(o.totalAmount || 0)}
  </span>
</td>

<td className="px-10 py-6">
  <div className="flex items-center gap-3">
    <div className={`flex items-center gap-2 px-3 py-1.5 rounded-full
text-[9px] font-black uppercase tracking-widest border
${getStatusBadgeClass(o.orderStatus, theme)} shadow-sm`}>
      <FiTruck size={10} /> {getOrderStatusText(o.orderStatus)}
    </div>

    {orderStatusUpdating[o.id] ? (
      <FiRefreshCw className="animate-spin text-slate-400" size={12}
/>
    ) : (
      <select
        value={o.orderStatus || 'pending'}
        onChange={(e) => updateOrderStatusHandler(o.id, e.target.value)}
        className={`text-[9px] font-black uppercase tracking-widest
rounded-lg px-2 py-1 outline-none transition-all ${
          isDark ? 'bg-slate-800 border-slate-700 text-slate-400 focus:bg-
slate-700' : 'bg-slate-100 border-slate-200 text-slate-600 focus:bg-white'
        }}
      >

```



```

      <option value="pending">Pendiente</option>
      <option value="processing">Procesando</option>
      <option value="shipped">Enviado</option>
      <option value="delivered">Entregado</option>
      <option value="cancelled">Cancelado</option>
    </select>
  }}
</div>
</td>
<td className="px-10 py-6 text-right pr-14">
  <div className="flex items-center justify-end gap-3">
    <button
      onClick={() => openOrderDetails(o)}
      className={`p-3 rounded-2xl transition-all shadow-sm flex items-
center justify-center ${isDark ? 'bg-slate-800 text-blue-400 hover:bg-blue-600
hover:text-white' : 'bg-slate-50 text-blue-600 hover:bg-blue-600 hover:text-white'}}
      title="Ver Detalles"
    >
      <FiExternalLink size={18} />
    </button>
    <button
      onClick={() => deleteOrder(o.id)}
      className={`p-3 rounded-2xl transition-all shadow-sm flex items-
center justify-center ${isDark ? 'bg-slate-800 text-rose-400 hover:bg-rose-600
hover:text-white' : 'bg-slate-50 text-rose-600 hover:bg-rose-600 hover:text-white'}}
      title="Eliminar Registro"

```

```

    >
    <FiTrash2 size={18} />
  </button>
</div>
</td>
</tr>
)}}
</tbody>
</table>
</div>

```

```

{/* Mobile View (Cards) */}

<div className="md:hidden divide-y divide-slate-100 dark:divide-slate-
800/10">
  {filteredOrders.map((o) => (
    <div key={o.id} className="p-6 space-y-4 hover:bg-indigo-500/5
transition-all">
      <div className="flex items-center justify-between">
        <div className="flex items-center gap-4 min-w-0">
          <div className={`w-10 h-10 rounded-xl flex items-center justify-center
shrink-0 shadow-inner ${
            isDark ? 'bg-slate-800 text-indigo-400' : 'bg-indigo-50 text-indigo-600'
          }}>
            <FiPackage size={16} />
          </div>

```

```

    <div className="min-w-0">

      <span className={`font-black text-sm block truncate tracking-tight
${textTitle}`}>{o.userEmail || 'Invitado'}</span>

      <span className="text-[10px] text-slate-400 font-bold uppercase
tracking-widest">ORD-{o.id.substring(0, 8).toUpperCase()}</span>

    </div>

  </div>

  <span className={`text-[10px] font-black uppercase tracking-widest
${textSub} shrink-0`}>
    {formatDate(o.createdAt)}
  </span>
</div>

<div className="flex justify-between items-center bg-slate-500/5 p-4
rounded-2xl">
  <div>

    <p className="text-[9px] font-black uppercase text-slate-400 tracking-
widest mb-1">Monto Total</p>

    <span className={`font-black text-lg text-emerald-500`}>
      {formatCurrency(o.totalAmount || 0)}
    </span>

  </div>

  <div className="text-right">

    <p className="text-[9px] font-black uppercase text-slate-400 tracking-
widest mb-1">Estado</p>

```

```

        <div className={`flex items-center gap-2 px-3 py-1 rounded-full text-[8px] font-black uppercase tracking-widest border ${getStatusBadgeClass(o.orderStatus, theme)} shadow-sm`} >
            {getOrderStatusText(o.orderStatus)}
        </div>
    </div>
</div>

```

```

<div className="flex flex-col gap-3">
    <div className="flex items-center gap-3 p-3 rounded-2xl border border-slate-500/10">
        <span className="text-[9px] font-black uppercase text-slate-400 tracking-widest min-w-[50px]">Status:</span>
        {orderStatusUpdating[o.id] ? (
            <FiRefreshCw className="animate-spin text-indigo-500" size={16} />
        ) : (
            <select
                value={o.orderStatus || 'pending'}
                onChange={(e) => updateOrderStatusHandler(o.id, e.target.value)}
                className={`flex-1 text-[10px] font-black uppercase tracking-widest rounded-xl px-4 py-2 bg-slate-500/5 border-none outline-none ${isDark ? 'text-white' : 'text-slate-900'}}
            >
                <option value="pending">Pendiente</option>
                <option value="processing">Procesando</option>
                <option value="shipped">Enviado</option>
            >

```

```

      <option value="delivered">Entregado</option>
      <option value="cancelled">Cancelado</option>
    </select>
  )}
</div>

```

```

<div className="flex gap-3">
  <button
    onClick={() => openOrderDetails(o)}
    className={`flex-1 flex items-center justify-center gap-2 py-3.5
rounded-2xl font-bold text-xs transition-all ${isDark ? 'bg-slate-800 text-blue-400' :
'bg-blue-50 text-blue-600'}`}
    >
    <FiExternalLink size={16} /> Detalles
  </button>
  <button
    onClick={() => deleteOrder(o.id)}
    className={`flex-1 flex items-center justify-center gap-2 py-3.5
rounded-2xl font-bold text-xs transition-all ${isDark ? 'bg-slate-800/50 text-rose-
400' : 'bg-rose-50 text-rose-600'}`}
    >
    <FiTrash2 size={16} /> Eliminar
  </button>
</div>
</div>
</div>

```

```

    )}
  </div>

  {filteredOrders.length === 0 && (
    <div className="py-24 text-center">
      <div className={`w-20 h-20 rounded-full flex items-center justify-center
mx-auto mb-6 ${isDark ? 'bg-slate-800' : 'bg-slate-50'}}>
        <FiPackage size={32} className="opacity-20" />
      </div>
      <h3 className={`text-lg font-black ${textTitle}`}>Sin órdenes
registradas</h3>
      <p className={` ${textSub} text-sm mt-2`}>No hay transacciones que
coincidan con los filtros aplicados.</p>
    </div>
  )}
</div>
</div>
);
};

export default AdminOrders;

```

📁 **src/assets/components/Admin/AdminProductModal.jsx**

```

const AdminProductModal = ({
  isOpen,
  onClose,
  theme,

```

```

modalMode,
newProduct,
editingProduct,
categories,
handleProductChange,
handleEditProductChange,
createProduct,
updateProduct,
handleImageSelect,
uploading,
removeImage,
setPrimaryImage
}) => {
  if(!isOpen) return null;

  const isDark = theme === 'dark';

  const bgCard = isDark ? 'bg-[#1a1b26] border-slate-800 shadow-2xl' : 'bg-white
border-slate-100 shadow-sm';

  const textTitle = isDark ? 'text-slate-100' : 'text-slate-900';
  const textSub = isDark ? 'text-slate-400' : 'text-slate-500';

  const modalOverlay = isDark ? 'bg-black/80 backdrop-blur-md' : 'bg-slate-900/40
backdrop-blur-md';

  const productData = modalMode === 'add' ? newProduct : editingProduct;

```

```
const onChange = modalMode === 'add' ? handleProductChange :  
handleEditProductChange;
```

```
return (  
  <div className={`flex fixed inset-0 justify-center items-center p-4 duration-300  
z-[200] md:p-6 ${modalOverlay} animate-in fade-in`} >  
    <div  
      className={`w-full max-w-4xl max-h-[95vh] overflow-y-auto rounded-[2rem]  
md:rounded-[2.5rem] border shadow-2xl relative animate-in zoom-in-95 duration-  
300 ${bgCard}`}  
    >  
      <div className="relative p-6 md:p-12">  
  
        {/* Close Button */}  
  
        <button  
          onClick={onClose}  
  
          className={`absolute right-4 md:right-8 top-4 md:top-8 p-3 rounded-2xl  
transition-all active:scale-95 z-10 ${isDark ? 'bg-slate-800 text-slate-400 hover:bg-  
slate-700' : 'bg-slate-100 text-slate-500 hover:bg-slate-200'}`}  
        >  
          <FiX size={18} className="md:w-5 md:h-5" />  
        </button>  
  
        {/* Header */}  
  
        <div className="mb-8 md:mb-12">  
          <div className="flex gap-3 items-center mb-2">
```



```

    <div className="w-8 h-1 bg-indigo-600 rounded-full"> </div>

    <span className={`text-[9px] md:text-[10px] font-black uppercase
tracking-[0.4em] text-indigo-600`} >Catalogue Engineering</span>

</div>

<h3 className={`text-xl md:text-4xl font-black tracking-tight ${textTitle}`} >

    {modalMode === 'add' ? 'Nuevo Ejemplar' : 'Editar Producto'}

</h3>

<p className={`mt-1 text-xs md:text-base ${textSub}`} >Define los
parámetros técnicos y estéticos de tu producto.</p>

</div>

<form onSubmit={modalMode === 'add' ? createProduct : updateProduct}
className="space-y-8 md:space-y-10">

    {/* Basic Info */}

    <div className="grid grid-cols-1 gap-6 md:grid-cols-3">

        <div className="space-y-6 md:space-y-8 md:col-span-2">

            <div className="space-y-3">

                <label className={`block text-[10px] font-black uppercase tracking-
[0.2em] px-1 ${textSub}`} >Nombre de Identidad</label>

                <div className="relative group">

                    <FiType className={`absolute left-5 top-1/2 -translate-y-1/2
transition-colors ${isDark ? 'text-slate-500 group-focus-within:text-indigo-400' :
'text-slate-400 group-focus-within:text-indigo-600'}} />

                    <input

                        type="text"

```

```

      name="name"

      value={productData?.name}

      onChange={onChange}

      className={`w-full pl-14 pr-6 py-4 rounded-2xl border transition-all
duration-300 outline-none focus:ring-4 ${
        isDark ? 'text-white bg-slate-900 border-slate-700 focus:ring-indigo-
500/10' : 'bg-slate-50 border-slate-200 focus:ring-indigo-500/5 text-slate-900'
      }}

      placeholder="Ej: Bolso Luxury Edition"

      required
    />
  </div>
</div>

```

```

<div className="space-y-3">

  <label className={`block text-[10px] font-black uppercase tracking-
[0.2em] px-1 ${textSub}`}>Narrativa del Producto</label>

  <textarea

    name="description"

    value={productData?.description}

    onChange={onChange}

    rows="4"

    className={`w-full px-6 py-4 rounded-2xl border transition-all
duration-300 outline-none focus:ring-4 ${
      isDark ? 'text-white bg-slate-900 border-slate-700 focus:ring-indigo-
500/10' : 'bg-slate-50 border-slate-200 focus:ring-indigo-500/5 text-slate-900'
    }}
  >

```

```

    }}

    placeholder="Detalla los materiales, el origen y la propuesta de
valor..."

  />
</div>
</div>

<div className="space-y-6 md:space-y-8">

  <div className="space-y-3">

    <label className={`block text-[10px] font-black uppercase tracking-
[0.2em] px-1 ${textSub}}`>Inversión (COP)</label>

    <div className="relative group">

      <FiDollarSign className={`absolute left-5 top-1/2 -translate-y-1/2
transition-colors ${isDark ? 'text-slate-500 group-focus-within:text-emerald-400' :
'text-slate-400 group-focus-within:text-emerald-600'}}` />

      <input

        type="number"

        name="price"

        value={productData?.price}

        onChange={onChange}

        className={`w-full pl-14 pr-6 py-4 rounded-2xl border transition-all
duration-300 outline-none focus:ring-4 ${

          isDark ? 'font-bold text-white bg-slate-900 border-slate-700
focus:ring-emerald-500/10' : 'font-bold bg-slate-50 border-slate-200 focus:ring-
emerald-500/5 text-slate-900'

        }}

        placeholder="0.00"

```

```

        required

      />
    </div>
  </div>

  <div className="space-y-3">

    <label className={`block text-[10px] font-black uppercase tracking-[0.2em] px-1 ${textSub}}`>Existencias</label>

    <div className="relative group">

      <FiBox className={`absolute left-5 top-1/2 -translate-y-1/2
transition-colors ${isDark ? 'text-slate-500 group-focus-within:text-amber-400' :
'text-slate-400 group-focus-within:text-amber-600'}} />

      <input

        type="number"

        name="stock"

        value={productData?.stock}

        onChange={onChange}

        className={`w-full pl-14 pr-6 py-4 rounded-2xl border transition-all
duration-300 outline-none focus:ring-4 ${

          isDark ? 'text-white bg-slate-900 border-slate-700 focus:ring-amber-
500/10' : 'bg-slate-50 border-slate-200 focus:ring-amber-500/5 text-slate-900'

        }}

        placeholder="Unidades"

      />
    </div>
  </div>

```

</div>

</div>

{/ Classification Grid */}*

<div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-6">

<div className="space-y-3">

<label className={`block text-[10px] font-black uppercase tracking-[0.2em] px-1 \${textSub}`}>Categoría</label>

<select

name="category"

value={productData?.category}

onChange={onChange}

className={`w-full px-6 py-4 rounded-2xl border transition-all outline-none font-bold text-xs uppercase tracking-widest \${

isDark ? 'bg-slate-900 border-slate-700 text-slate-200' : 'bg-slate-50 border-slate-200 text-slate-700'

}}>

>

<option value="">Clasificación</option>

{categories.map((c) => (

<option key={c.id} value={c.name}>{c.name}</option>

)))

</select>

</div>

```

<div className="space-y-3">

  <label className={`block text-[10px] font-black uppercase tracking-
[0.2em] px-1 ${textSub}`}>Valoración Base</label>

  <div className="relative group">

    <FiStar className={`absolute left-5 top-1/2 text-amber-500 -translate-
y-1/2`} />

    <input
      type="number"
      step="0.1"
      name="rating"
      value={productData?.rating}
      onChange={onChange}
      className={`w-full pl-14 pr-6 py-4 rounded-2xl border transition-all
outline-none ${
        isDark ? 'text-white bg-slate-900 border-slate-700' : 'bg-slate-50
border-slate-200 text-slate-900'
      }}
      placeholder="0.0 - 5.0"
    />
  </div>
</div>

```

```

<div className="space-y-3">

  <label className={`block text-[10px] font-black uppercase tracking-
[0.2em] px-1 ${textSub}`}>Material</label>

  <input

```

```

      type="text"
      name="material"
      value={productData?.material}
      onChange={onChange}
      className={`w-full px-6 py-4 rounded-2xl border transition-all outline-
none ${
        isDark ? 'text-white bg-slate-900 border-slate-700' : 'bg-slate-50
border-slate-200 text-slate-900'
      }}
      placeholder="Ej: Cuero Italiano"
    />
  </div>

```

```

<div className="space-y-3">
  <label className={`block text-[10px] font-black uppercase tracking-
[0.2em] px-1 ${textSub}}`>Color / Estilo</label>
  <input
    type="text"
    name="color"
    value={productData?.color}
    onChange={onChange}
    className={`w-full px-6 py-4 rounded-2xl border transition-all outline-
none ${
      isDark ? 'text-white bg-slate-900 border-slate-700' : 'bg-slate-50
border-slate-200 text-slate-900'
    }}
  />

```

```

        placeholder="Ej: Midnight Blue"

    />

</div>

</div>

{ /* Media System */

<div className="space-y-6">

    <div className="flex justify-between items-center">

        <label className={`block text-[10px] font-black uppercase tracking-[0.2em] px-1 ${textSub}`}>Gestión de Assets Visuales (Máx 4)</label>

        <button

            type="button"

            onClick={() => handleImageSelect(modalMode === 'edit')}

            disabled={uploading || (productData?.imageUrls?.length >= 4)}

            className={`flex items-center gap-2 px-4 md:px-6 py-2.5 md:py-3 rounded-xl border-2 transition-all font-black text-[9px] md:text-[10px] uppercase tracking-widest ${

                isDark

                ? 'text-indigo-400 border-indigo-600/30 hover:bg-indigo-600 hover:text-white'

                : 'text-indigo-600 border-indigo-600 hover:bg-indigo-600 hover:text-white'

            }}

        >

            {uploading ? <FiRefreshCw className="animate-spin" /> : <FiCamera size={14} className="md:w-4 md:h-4" />}}

```



```

      {uploading ? '...' : 'Cargar Assets'}
    </button>
  </div>

  <div className="grid grid-cols-2 gap-4 md:gap-6 md:grid-cols-4">
    {productData?.imageUrls?.map((img, idx) => (
      <div key={idx} className={`relative rounded-2xl md:rounded-3xl
overflow-hidden border-2 group transition-all duration-300 ${isDark ? 'border-
slate-800' : 'border-slate-100'} h-32 md:h-40`}>
        <img
          src={typeof img === 'string' ? img : img.data}
          alt="Product"
          className="object-cover w-full h-full transition-transform group-
hover:scale-110"
        />
        <div className="flex absolute inset-0 flex-col gap-2 justify-center
items-center opacity-0 transition-opacity bg-black/60 group-hover:opacity-100">
          <button
            type="button"
            onClick={() => setPrimaryImage(idx, modalMode === 'edit')}
            className={`px-3 py-1 rounded-lg text-[9px] font-black uppercase
tracking-widest transition-all ${
              productData.primaryImageIndex === idx ? 'bg-emerald-500 text-
white' : 'bg-white text-slate-900 hover:bg-indigo-600 hover:text-white'
            }}
          >

```

```

        {productData.primaryImageIndex === idx ? <FiCheck
className="inline-block mr-1"/> : null}

        {productData.primaryImageIndex === idx ? 'Principal' : 'Fijar
Portada'}

    </button>

    <button

        type="button"

        onClick={() => removeImage(idx, modalMode === 'edit')}

        className="px-3 py-1 rounded-lg text-[9px] font-black uppercase
tracking-widest bg-rose-600 text-white hover:bg-rose-700 transition-all"

    >

        Eliminar

    </button>

</div>

</div>

)}}

{[...Array(Math.max(0, 4 - (productData?.imageUrls?.length || 0)))]}.map((_,
i) => (

    <div key={i} className={`rounded-2xl md:rounded-3xl border-2
border-dashed flex flex-col items-center justify-center gap-3 h-32 md:h-40 ${isDark
? 'border-slate-800 bg-slate-900/40' : 'border-slate-100 bg-slate-50'}`}>

        <FiLayers className="text-slate-300 md:w-6 md:h-6" size={20} />

        <span className="text-[9px] md:text-[10px] font-black uppercase
tracking-widest text-slate-400">Viento</span>

    </div>

)}}

</div>

```

```

</div>

{ /* Actions */}

<div className="flex flex-col md:flex-row gap-4 pt-6 md:pt-8">

  <button

    type="button"

    onClick={onClose}

    className={`order-2 md:order-1 flex-1 py-4 md:py-5 rounded-2xl font-
black text-[9px] md:text-xs uppercase tracking-widest transition-all ${
      isDark ? 'bg-slate-800 text-slate-400 hover:bg-slate-700' : 'bg-slate-100
text-slate-600 hover:bg-slate-200'
    }}

    >

    Cancelar

  </button>

  <button

    type="submit"

    className={`order-1 md:order-2 flex-[2] py-4 md:py-5 ${isDark ? 'bg-
indigo-600' : 'bg-black'} text-white rounded-2xl font-black text-[9px] md:text-xs
uppercase tracking-widest transition-all hover:bg-indigo-500 hover:shadow-2xl
hover:shadow-indigo-500/30 active:scale-[0.98]`}

    >

    {modalMode === 'add' ? 'Lanzar al Mercado' : 'Actualizar Ejemplar'}

  </button>

</div>

</form>

```

```

        </div>

    </div>

</div>

);

};

export default AdminProductModal;

```

 **src/assets/components/Admin/AdminProducts.jsx**

```

import React from 'react';

import Loader from '../Loader';

import { FiShoppingBag, FiPlus, FiEdit3, FiTrash2, FiSearch, FiLayers, FiPackage,
FiActivity, FiRefreshCw } from 'react-icons/fi';

const AdminProducts = ({
    products,
    productsLoading,
    productSearchTerm,
    setProductSearchTerm,
    theme,
    formatCurrency,
    openAddProductModal,
    openEditProductModal,
    deleteProduct,
    syncProductsToStripe,
    stripeSyncing
}) => {

```

```

if(productsLoading) {
  return (
    <div className="flex flex-col items-center justify-center min-h-[40vh] space-y-4">
      <div className="w-12 h-12 border-4 border-indigo-600 border-b-transparent rounded-full animate-spin"></div>
      <p className="text-[10px] font-black uppercase tracking-[0.3em] text-indigo-600">Sincronizando Inventario...</p>
    </div>
  );
}

```

```

const isDark = theme === 'dark';
const filteredProducts = productSearchTerm
  ? products.filter(product =>
    product.name.toLowerCase().includes(productSearchTerm.toLowerCase()) ||
    (product.category &&
    product.category.toLowerCase().includes(productSearchTerm.toLowerCase())) ||
    (product.description &&
    product.description.toLowerCase().includes(productSearchTerm.toLowerCase()))
  )
  : products;

```

// Luxury UI Tokens

```

const bgCard = isDark ? 'bg-[#1a1b26] border-slate-800 shadow-2xl' : 'bg-white border-slate-100 shadow-sm';

```

```

const textTitle = isDark ? 'text-slate-100' : 'text-slate-900';
const textSub = isDark ? 'text-slate-400' : 'text-slate-500';

return (
  <div className="space-y-8 animate-in fade-in slide-in-from-bottom-4
duration-500">

    {/* Header & Search Zone */}

    <div className={` ${bgCard} p-6 md:p-10 rounded-3xl md:rounded-[2.5rem]
border flex flex-col xl:flex-row xl:items-center justify-between gap-6 md:gap-8
relative overflow-hidden`} >

      <div className="absolute top-0 right-0 w-64 h-64 bg-indigo-500/5
rounded-full -mr-32 -mt-32 blur-3xl"> </div>

      <div className="relative">

        <div className="flex items-center gap-3 mb-2">

          <div className="w-8 h-1 bg-indigo-600 rounded-full"> </div>

          <span className={`text-[10px] font-black uppercase tracking-[0.4em] text-
indigo-600`} >Inventory Hub</span>

        </div>

        <h2 className={`text-2xl md:text-3xl font-black tracking-tight
${textTitle}`} >Gestión de Productos</h2>

        <p className={` ${textSub} mt-1 text-xs md:text-sm`} >Control total sobre tu
catálogo, precios y niveles de stock.</p>

      </div>

    <div className="flex flex-col md:flex-row items-center gap-4 relative">

```

```

<div className="relative w-full md:w-80 group">

  <FiSearch className={`absolute left-5 top-1/2 -translate-y-1/2 transition-
colors ${isDark ? 'text-slate-500 group-focus-within:text-indigo-400' : 'text-slate-
400 group-focus-within:text-indigo-600'}} />

  <input

    type="text"

    placeholder="Buscar..."

    value={productSearchTerm}

    onChange={(e) => setProductSearchTerm(e.target.value)}

    className={`w-full pl-14 pr-6 py-4 rounded-2xl border transition-all
duration-300 outline-none focus:ring-4 ${

      isDark ? 'bg-slate-900/50 border-slate-700 focus:ring-indigo-500/20 text-
white' : 'bg-slate-50 border-slate-200 focus:ring-indigo-500/10 text-slate-900'

    }}

  />

</div>

<button

  onClick={openAddProductModal}

  className="flex items-center justify-center gap-2 px-8 py-4 bg-black text-
white rounded-2xl font-black text-sm transition-all hover:bg-gray-800
hover:shadow-2xl active:scale-95 w-full md:w-auto"

  >

    <FiPlus size={20} /> Añadir Producto

  </button>

  <button

    onClick={syncProductsToStripe}

```

```

        disabled={stripeSyncing}

        className={`flex items-center justify-center gap-2 px-8 py-4 ${isDark ? 'bg-indigo-600' : 'bg-emerald-600'} text-white rounded-2xl font-black text-sm transition-all hover:opacity-90 hover:shadow-2xl active:scale-95 w-full md:w-auto disabled:opacity-50`}

        >

        {stripeSyncing ? <FiRefreshCw className="animate-spin" /> : <FiActivity size={20} />}

        {stripeSyncing ? (stripeSyncing === "syncing" ? "Sincronizando..." : "Sincronizado!") : "Sincronizar Stripe"}

        </button>

    </div>

</div>

```

```

    {/ * Main Content Table / Cards */}

    <div className={` ${bgCard} rounded-3xl md:rounded-[2.5rem] border overflow-hidden`} >

        {/ * Desktop Table */}

        <div className="hidden md:block overflow-x-auto">

            <table className="w-full text-left border-collapse">

                <thead>

                    <tr className={`text-[10px] font-black uppercase tracking-[0.2em] ${textSub} border-b border-inherit`} >

                        <th className="px-10 py-6">Detalle del Producto</th>

                        <th className="px-10 py-6">Categoría</th>

                        <th className="px-10 py-6">Inversión</th>

                        <th className="px-10 py-6">Disponibilidad</th>

```



```

    <th className="px-10 py-6 text-right pr-14">Opciones</th>
  </tr>
</thead>

<tbody className={`divide-y ${isDark ? 'divide-slate-800/10' : 'divide-slate-50'}>
  {filteredProducts.map((p) => (
    <tr key={p.id} className="group hover:bg-indigo-500/5 transition-all duration-300">
      <td className="px-10 py-6">
        <div className="flex items-center gap-5">
          <div className={`w-14 h-14 rounded-2xl overflow-hidden shadow-inner flex items-center justify-center ${isDark ? 'bg-slate-800' : 'bg-slate-100'}>
            {p.imageUrl ? (
              <img src={p.imageUrl} alt={p.name} className="w-full h-full object-cover" />
            ) : (
              <FiShoppingBag className={textSub} size={24} />
            )}
          </div>
          <div className="flex flex-col">
            <span className={`font-black text-base tracking-tight ${textTitle}>{p.name}</span>
            <span className="text-[10px] text-slate-400 font-bold uppercase tracking-widest mt-0.5">#{p.id.slice(-6).toUpperCase()}</span>
          </div>
        </div>
      </td>
    </tr>
  )}
</tbody>

```

```

</td>

<td className="px-10 py-6">

  <span className={`inline-flex items-center gap-2 px-4 py-1.5
rounded-full text-[10px] font-black uppercase tracking-widest ${isDark ? 'bg-slate-
800 text-slate-300' : 'bg-slate-100 text-slate-600'}}>

    <FiLayers size={10} /> {p.category || 'General'}

  </span>

</td>

<td className="px-10 py-6">

  <span className={`font-black text-base ${isDark ? 'text-emerald-400' :
'text-emerald-600'}}>

    {formatCurrency(p.price)}

  </span>

</td>

<td className="px-10 py-6">

  <div className="flex items-center gap-3">

    <div className={`w-2 h-2 rounded-full ${p.stock > 10 ? 'bg-
emerald-500' : p.stock > 0 ? 'bg-amber-500' : 'bg-rose-500'}}> </div>

    <span className={`text-sm font-bold ${textTitle}}>{p.stock} <span
className="opacity-40 font-medium">Uni.</span> </span>

  </div>

</td>

<td className="px-10 py-6 text-right pr-14">

  <div className="flex items-center justify-end gap-3 transition-
opacity">

    <button

```

```

        onClick={() => openEditProductModal(p)}

        className={`p-3 rounded-2xl transition-all shadow-sm flex items-
center justify-center ${isDark ? 'bg-slate-800 text-indigo-400 hover:bg-slate-700' :
'bg-slate-50 text-indigo-600 hover:bg-indigo-600 hover:text-white'}}

        title="Editar"

    >

    <FiEdit3 size={18} />

</button>

<button

    onClick={() => deleteProduct(p.id, p.name)}

    className={`p-3 rounded-2xl transition-all shadow-sm flex items-
center justify-center ${isDark ? 'bg-slate-800 text-rose-400 hover:bg-rose-500/20' :
'bg-slate-50 text-rose-600 hover:bg-rose-600 hover:text-white'}}

    title="Eliminar"

    >

    <FiTrash2 size={18} />

</button>

</div>

</td>

</tr>

)}}

</tbody>

</table>

</div>

{ /* Mobile View (Cards) */}

```

```

    <div className="md:hidden divide-y divide-slate-100 dark:divide-slate-800/10">
      {filteredProducts.map((p) => (
        <div key={p.id} className="p-6 space-y-4 hover:bg-indigo-500/5 transition-all">
          <div className="flex items-center gap-4">
            <div className={`w-16 h-16 shrink-0 rounded-2xl overflow-hidden shadow-inner flex items-center justify-center ${isDark ? 'bg-slate-800' : 'bg-slate-100'}}>
              {p.imageUrl ? (
                <img src={p.imageUrl} alt={p.name} className="w-full h-full object-cover" />
              ) : (
                <FiShoppingBag className={textSub} size={24} />
              )}
            </div>
            <div className="flex-1 min-w-0">
              <span className={`font-black text-lg block truncate tracking-tight ${textTitle}`}>{p.name}</span>
              <div className="flex items-center gap-2 mt-1">
                <span className={`inline-flex items-center gap-1 px-2 py-0.5 rounded-full text-[9px] font-black uppercase tracking-wider ${isDark ? 'bg-slate-800 text-slate-300' : 'bg-slate-100 text-slate-600'}}>
                  {p.category || 'General'}
                </span>
                <span className="text-[9px] text-slate-400 font-bold uppercase tracking-widest">#{p.id.slice(-6).toUpperCase()}</span>
              </div>
            </div>
          </div>
        )
      )}
    </div>

```

</div>

</div>

</div>

<div className="flex justify-between items-center bg-slate-500/5 p-4 rounded-2xl">

<div>

<p className="text-[9px] font-black uppercase text-slate-400 tracking-widest mb-1">Inversión</p>

{formatCurrency(p.price)}

</div>

<div className="text-right">

<p className="text-[9px] font-black uppercase text-slate-400 tracking-widest mb-1">Disponibilidad</p>

<div className="flex items-center justify-end gap-2 text-sm font-bold">

<div className={`w-2 h-2 rounded-full \${p.stock > 10 ? 'bg-emerald-500' : p.stock > 0 ? 'bg-amber-500' : 'bg-rose-500'}}></div>

{p.stock} Uni.

</div>

</div>

</div>

```

<div className="flex gap-3">

  <button

    onClick={() => openEditProductModal(p)}

    className={`flex-1 flex items-center justify-center gap-2 py-3.5
rounded-2xl font-bold text-xs transition-all ${isDark ? 'bg-slate-800 text-indigo-
400' : 'bg-indigo-50 text-indigo-600'}}

  >

    <FiEdit3 size={16} /> Editar

  </button>

  <button

    onClick={() => deleteProduct(p.id, p.name)}

    className={`flex-1 flex items-center justify-center gap-2 py-3.5
rounded-2xl font-bold text-xs transition-all ${isDark ? 'bg-slate-800/50 text-rose-
400' : 'bg-rose-50 text-rose-600'}}

  >

    <FiTrash2 size={16} /> Eliminar

  </button>

</div>

</div>

)}}

</div>

```

```

{filteredProducts.length === 0 && (

  <div className="py-24 text-center">

    <div className={`w-20 h-20 rounded-full flex items-center justify-center
mx-auto mb-6 ${isDark ? 'bg-slate-800' : 'bg-slate-50'}}>

```

```

        <FiPackage size={32} className="opacity-20" />

    </div>

    <h3 className={`text-lg font-black ${textTitle}`}>Sin Resultados
Coincidentes</h3>

    <p className={` ${textSub} text-sm mt-2`} >Intenta refinar tus términos de
búsqueda o añade un nuevo ejemplar.</p>

    </div>

    )}

</div>

</div>

);

};

export default AdminProducts;

```

 **src/assets/components/Admin/AdminReviews.jsx**

```

import React from 'react';

import Loader from '../Loader';

import { FiMessageSquare, FiEdit3, FiTrash2, FiSearch, FiStar, FiUser, FiActivity } from
'react-icons/fi';

const AdminReviews = ({
    reviews,
    reviewsLoading,
    reviewSearchTerm,
    setReviewSearchTerm,
    theme,

```

```

formatDate,
openEditReviewModal,
deleteReviewHandler
}) => {
  if(reviewsLoading) {
    return (
      <div className="flex flex-col items-center justify-center min-h-[40vh] space-y-4">
        <div className="w-12 h-12 border-4 border-indigo-600 border-b-transparent rounded-full animate-spin"></div>
        <p className="text-[10px] font-black uppercase tracking-[0.3em] text-indigo-600">Escaneando Opiniones...</p>
      </div>
    );
  }

  const isDark = theme === 'dark';
  const filteredReviews = reviews.filter(review =>
    (review.productName &&
    review.productName.toLowerCase().includes(reviewSearchTerm.toLowerCase())) ||
    (review.createdAt &&
    formatDate(review.createdAt).toLowerCase().includes(reviewSearchTerm.toLowerCase()))
  );

  // Luxury UI Tokens

```



```
const bgCard = isDark ? 'bg-[#1a1b26] border-slate-800 shadow-2xl' : 'bg-white border-slate-100 shadow-sm';
```

```
const textTitle = isDark ? 'text-slate-100' : 'text-slate-900';
```

```
const textSub = isDark ? 'text-slate-400' : 'text-slate-500';
```

```
return (
```

```
<div className="space-y-10 animate-in fade-in slide-in-from-bottom-4 duration-500">
```

```
  {/* Header & Filter Zone */}
```

```
<div className={` ${bgCard} p-6 md:p-10 rounded-3xl md:rounded-[2.5rem] border flex flex-col xl:flex-row xl:items-center justify-between gap-6 md:gap-8 relative overflow-hidden`} >
```

```
<div className="absolute top-0 right-0 w-64 h-64 bg-indigo-500/5 rounded-full -mr-32 -mt-32 blur-3xl"></div>
```

```
<div className="relative">
```

```
<div className="flex items-center gap-3 mb-2">
```

```
<div className="w-8 h-1 bg-indigo-600 rounded-full"></div>
```

```
<span className={`text-[10px] font-black uppercase tracking-[0.4em] text-indigo-600`} >Feedback Moderation</span>
```

```
</div>
```

```
<h2 className={`text-2xl md:text-3xl font-black tracking-tight ${textTitle}`} >Moderación de Reseñas</h2>
```

```
<p className={` ${textSub} mt-1 text-xs md:text-sm`} >Supervisa y gestiona la reputación de tus productos en el mercado.</p>
```

```
</div>
```

```

<div className="flex flex-col md:flex-row items-center gap-4 relative">
  <div className="relative w-full md:w-80 group">
    <FiSearch className={`absolute left-5 top-1/2 -translate-y-1/2 transition-
colors ${isDark ? 'text-slate-500 group-focus-within:text-indigo-400' : 'text-slate-
400 group-focus-within:text-indigo-600'}} />
    <input
      type="text"
      placeholder="Buscar por producto..."
      value={reviewSearchTerm}
      onChange={(e) => setReviewSearchTerm(e.target.value)}
      className={`w-full pl-14 pr-6 py-4 rounded-2xl border transition-all
duration-300 outline-none focus:ring-4 ${
        isDark ? 'bg-slate-900/50 border-slate-700 focus:ring-indigo-500/20 text-
white' : 'bg-slate-50 border-slate-200 focus:ring-indigo-500/10 text-slate-900'
      }}
    />
  </div>
</div>
</div>
</div>

```

```

{/* Reviews Content Table / Cards */}

```

```

<div className={` ${bgCard} rounded-3xl md:rounded-[2.5rem] border
overflow-hidden`}>

```

```

{/* Desktop Table View */}

```

```

<div className="hidden md:block overflow-x-auto">

```

```

<table className="w-full text-left border-collapse">
  <thead>
    <tr className={`text-[10px] font-black uppercase tracking-[0.2em]
    ${textSub} border-b border-inherit`}>
      <th className="px-10 py-6">Calificación y Comentario</th>
      <th className="px-10 py-6">Producto Asociado</th>
      <th className="px-10 py-6">Fecha Registro</th>
      <th className="px-10 py-6 text-right pr-14">Acciones</th>
    </tr>
  </thead>
  <tbody className={`divide-y ${isDark ? 'divide-slate-800/10' : 'divide-slate-
50'}}`>
    {filteredReviews.map((r) => (
      <tr key={r.id} className="group hover:bg-indigo-500/5 transition-all
duration-300">
        <td className="px-10 py-6">
          <div className="flex gap-5">
            <div className={`w-12 h-12 rounded-2xl flex-shrink-0 flex items-
center justify-center font-black shadow-inner ${
isDark ? 'bg-slate-800 text-indigo-400' : 'bg-indigo-50 text-indigo-
600'
}}`>
              <FiMessageSquare size={20} />
            </div>
            <div className="flex flex-col max-w-sm">
              <div className="flex items-center gap-1 mb-2">

```

```

      {[1, 2, 3, 4, 5].map((star) => (
        <FiStar
          key={star}
          size={12}
          className={star <= r.rating ? 'fill-amber-400 text-amber-400' :
'text-slate-300'}
        />
      )]}
    </div>

    <p className={`text-sm leading-relaxed ${textTitle} opacity-80
italic`}>
      "{r.comment ? (r.comment.length > 50 ? r.comment.substring(0, 50)
+ '...' : r.comment) : 'Sin comentarios.'}"
    </p>
  </div>
</div>
</td>

<td className="px-10 py-6">
  <div className="flex flex-col">
    <span className={`font-black text-sm tracking-tight
${textTitle}`}>{r.productName || 'N/A'}</span>
    <span className="text-[10px] text-slate-400 font-bold uppercase
tracking-widest mt-1">Review ID: {r.id.slice(-6).toUpperCase()}</span>
  </div>
</td>

<td className="px-10 py-6">

```

```

        <span className={`text-[11px] font-black uppercase tracking-widest
${textSub}`}>
            {formatDate(r.createdAt)}
        </span>
    </td>
    <td className="px-10 py-6 text-right pr-14">
        <div className="flex items-center justify-end gap-3">
            <button
                onClick={() => openEditReviewModal(r)}
                className={`p-3 rounded-2xl transition-all shadow-sm flex items-
center justify-center ${isDark ? 'bg-slate-800 text-indigo-400 hover:bg-slate-700' :
'bg-slate-50 text-indigo-600 hover:bg-indigo-600 hover:text-white'}`}
                title="Ver / Editar"
            >
                <FiEdit3 size={18} />
            </button>
            <button
                onClick={() => deleteReviewHandler(r.id)}
                className={`p-3 rounded-2xl transition-all shadow-sm flex items-
center justify-center ${isDark ? 'bg-slate-800 text-rose-400 hover:bg-rose-500/20' :
'bg-slate-50 text-rose-600 hover:bg-rose-600 hover:text-white'}`}
                title="Eliminar"
            >
                <FiTrash2 size={18} />
            </button>
        </div>
    </td>

```

```

        </td>
      </tr>
    )}
  </tbody>
</table>
</div>

```

```

{/* Mobile View (Cards) */}

<div className="md:hidden divide-y divide-slate-100 dark:divide-slate-
800/10">
  {filteredReviews.map((r) => (
    <div key={r.id} className="p-6 space-y-4 hover:bg-indigo-500/5
transition-all">
      <div className="flex items-start justify-between">
        <div className="flex gap-4">
          <div className={`w-10 h-10 rounded-xl flex-shrink-0 flex items-center
justify-center font-black shadow-inner ${
            isDark ? 'bg-slate-800 text-indigo-400' : 'bg-indigo-50 text-indigo-600'
          }}>
            <FiMessageSquare size={16} />
          </div>
          <div className="min-w-0">
            <span className={`font-black text-sm block truncate tracking-tight
${textTitle}`}>{r.productName || 'N/A'}</span>
            <span className="text-[9px] text-slate-400 font-bold uppercase
tracking-widest">ID: {r.id.slice(-6).toUpperCase()}</span>

```

```

    </div>

  </div>

  <span className={`text-[9px] font-black uppercase tracking-widest
    ${textSub}`}>
    {formatDate(r.createdAt)}
  </span>
</div>

<div className={`$${isDark ? 'bg-slate-900/40' : 'bg-slate-50'} p-4
rounded-2xl space-y-3`}>
  <div className="flex items-center gap-1">
    {[1, 2, 3, 4, 5].map((star) => (
      <FiStar
        key={star}
        size={14}
        className={star <= r.rating ? 'fill-amber-400 text-amber-400' : 'text-
slate-300'}
      />
    ))}
  </div>

  <p className={`text-sm leading-relaxed ${textTitle} opacity-80 italic`}>
    "{r.comment || 'Sin comentarios.'}"
  </p>
</div>

```

```

<div className="flex gap-3">

  <button

    onClick={() => openEditReviewModal(r)}

    className={`flex-1 flex items-center justify-center gap-2 py-3.5
rounded-2xl font-bold text-xs transition-all ${isDark ? 'bg-slate-800 text-indigo-
400' : 'bg-indigo-50 text-indigo-600'}}

  >

    <FiEdit3 size={16} /> Moderar

  </button>

  <button

    onClick={() => deleteReviewHandler(r.id)}

    className={`flex-1 flex items-center justify-center gap-2 py-3.5
rounded-2xl font-bold text-xs transition-all ${isDark ? 'bg-slate-800/50 text-rose-
400' : 'bg-rose-50 text-rose-600'}}

  >

    <FiTrash2 size={16} /> Eliminar

  </button>

</div>

</div>

)}}

</div>

```

```

{filteredReviews.length === 0 && (

  <div className="py-24 text-center">

    <div className={`w-20 h-20 rounded-full flex items-center justify-center
mx-auto mb-6 ${isDark ? 'bg-slate-800' : 'bg-slate-50'}}>

```



```

        <FiActivity size={32} className="opacity-20" />
      </div>

      <h3 className={`text-lg font-black ${textTitle}`}>Aún no hay reseñas</h3>

      <p className={`${textSub} text-sm mt-2`}>Las opiniones de tus clientes
aparecerán aquí una vez que comiencen a calificar.</p>

    </div>

  )}

</div>

</div>

);

};

```

```
export default AdminReviews;
```

```
 src/assets/components/Admin/AdminUsers.jsx
```

```
import React from 'react';
```

```
import Loader from '../Loader';
```

```
import { FiUsers, FiEdit3, FiSearch, FiShield, FiUser, FiActivity, FiKey } from 'react-
icons/fi';
```

```

const AdminUsers = ({
  users,
  usersLoading,
  userSearchTerm,
  setUserSearchTerm,
  userRoleFilter,
  setUserRoleFilter,

```

```

theme,
updateUserRole
}) => {
  if(usersLoading) {
    return (
      <div className="flex flex-col items-center justify-center min-h-[40vh] space-y-4">
        <div className="w-12 h-12 border-4 border-indigo-600 border-b-transparent rounded-full animate-spin"></div>
        <p className="text-[10px] font-black uppercase tracking-[0.3em] text-indigo-600">Verificando Credenciales...</p>
      </div>
    );
  }
}

```

```

const isDark = theme === 'dark';

const filteredUsers = users.filter(user => {
  const matchesSearch = !userSearchTerm ||
    (user.email &&
user.email.toLowerCase().includes(userSearchTerm.toLowerCase())) ||
    (user.role && user.role.toLowerCase().includes(userSearchTerm.toLowerCase()));

  const matchesRole = userRoleFilter === 'all' ||
    (userRoleFilter === 'admin' && user.role === 'admin') ||
    (userRoleFilter === 'user' && user.role === 'user');

```

```

    return matchesSearch && matchesRole;

  });

  // Luxury UI Tokens

  const bgCard = isDark ? 'bg-[#1a1b26] border-slate-800 shadow-2xl' : 'bg-white border-slate-100 shadow-sm';

  const textTitle = isDark ? 'text-slate-100' : 'text-slate-900';

  const textSub = isDark ? 'text-slate-400' : 'text-slate-500';

  return (

    <div className="space-y-10 animate-in fade-in slide-in-from-bottom-4 duration-500">

      {/* Header & Filters Zone */}

      <div className={` ${bgCard} p-6 md:p-10 rounded-3xl md:rounded-[2.5rem] border flex flex-col xl:flex-row xl:items-center justify-between gap-6 md:gap-8 relative overflow-hidden`} >

        <div className="absolute top-0 right-0 w-64 h-64 bg-indigo-500/5 rounded-full -mr-32 -mt-32 blur-3xl"></div>

        <div className="relative">

          <div className="flex items-center gap-3 mb-2">

            <div className="w-8 h-1 bg-indigo-600 rounded-full"></div>

            <span className={`text-[10px] font-black uppercase tracking-[0.4em] text-indigo-600`} >Identity Management</span>

          </div>


```

```

    <h2 className={`text-2xl md:text-3xl font-black tracking-tight
    ${textTitle}}>Directorio de Usuarios</h2>

    <p className={` ${textSub} mt-1 text-xs md:text-sm`} >Controla los niveles
    de acceso y gestiona la comunidad de PSG Shop.</p>

    </div>

    <div className="flex flex-col md:flex-row items-center gap-4 relative">

        <div className="relative w-full md:w-80 group">

            <FiSearch className={`absolute left-5 top-1/2 -translate-y-1/2 transition-
            colors ${isDark ? 'text-slate-500 group-focus-within:text-indigo-400' : 'text-slate-
            400 group-focus-within:text-indigo-600'}} />

            <input

                type="text"

                placeholder="Buscar email..."

                value={userSearchTerm}

                onChange={(e) => setUserSearchTerm(e.target.value)}

                className={`w-full pl-14 pr-6 py-4 rounded-2xl border transition-all
            duration-300 outline-none focus:ring-4 ${

                isDark ? 'bg-slate-900/50 border-slate-700 focus:ring-indigo-500/20 text-
            white' : 'bg-slate-50 border-slate-200 focus:ring-indigo-500/10 text-slate-900'

                }}

            />

        </div>

        <select

            value={userRoleFilter}

            onChange={(e) => setUserRoleFilter(e.target.value)}

```

```

        className={`w-full md:w-auto px-6 py-4 rounded-2xl border transition-all
duration-300 outline-none focus:ring-4 font-bold text-xs uppercase tracking-
widest ${
        isDark ? 'bg-slate-800 border-slate-700 text-slate-200 focus:ring-indigo-
500/20' : 'bg-white border-slate-200 text-slate-700 focus:ring-indigo-500/10'
        }}
    >

    <option value="all">Ver Todos</option>

    <option value="admin">Administradores</option>

    <option value="user">Clientes</option>

</select>

</div>

</div>

```

```

{/* Users Content Table / Cards */}

<div className={` ${bgCard} rounded-3xl md:rounded-[2.5rem] border
overflow-hidden`} >

    {/* Desktop Table View */}

    <div className="hidden md:block overflow-x-auto">

        <table className="w-full text-left border-collapse">

            <thead>

                <tr className={`text-[10px] font-black uppercase tracking-[0.2em]
${textSub} border-b border-inherit`} >

                    <th className="px-10 py-6">Perfil de Usuario</th>

                    <th className="px-10 py-6">Nivel de Acceso</th>

                    <th className="px-10 py-6">Estado Cuenta</th>

```

```

    <th className="px-10 py-6 text-right pr-14">Ajustes de Rango</th>
  </tr>
</thead>

<tbody className={`divide-y ${isDark ? 'divide-slate-800/10' : 'divide-slate-50'}}>
  {filteredUsers.map((u) => (
    <tr key={u.id} className="group hover:bg-slate-500/5 transition-all duration-300">
      <td className="px-10 py-6">
        <div className="flex items-center gap-5">
          <div className={`w-12 h-12 rounded-2xl flex items-center justify-center font-black text-xs shadow-inner ${
            isDark ? 'bg-slate-800 text-indigo-400' : 'bg-indigo-50 text-indigo-600'
          }}>
            {u.email?.charAt(0).toUpperCase() || <FiUser/>}
          </div>
          <div className="flex flex-col">
            <span className={`font-black text-sm tracking-tight ${textTitle}`}>{u.email}</span>
            <span className="text-[10px] text-slate-400 font-bold uppercase tracking-widest mt-0.5">UID: {u.id.slice(-8).toUpperCase()}</span>
          </div>
        </div>
      </td>
      <td className="px-10 py-6">

```

```

        <span className={`inline-flex items-center gap-2 px-4 py-1.5
rounded-full text-[10px] font-black uppercase tracking-widest ${
        u.role === 'admin'

        ? isDark ? 'bg-indigo-500/10 text-indigo-400 ring-1 ring-indigo-
500/20' : 'bg-indigo-100 text-indigo-700 border border-indigo-200'

        : isDark ? 'bg-slate-800 text-slate-400' : 'bg-slate-100 text-slate-500'
}}>

        {u.role === 'admin' ? <FiShield size={12} /> : <FiUser size={12} />}

        {u.role === 'admin' ? 'Master Admin' : 'Customer'}

    </span>
</td>

<td className="px-10 py-6">

    <div className="flex items-center gap-2">

        <span className="w-2 h-2 rounded-full bg-emerald-500"> </span>

        <span className={`text-[10px] font-black uppercase tracking-widest
${textSub}`}>Activo</span>

    </div>

</td>

<td className="px-10 py-6 text-right pr-14">

    <button

        onClick={() => updateUserRole(u.id, u.role === 'admin' ? 'user' :
'admin')}

        className={`px-5 py-2.5 rounded-xl transition-all duration-300 flex
items-center gap-2 ml-auto text-[10px] font-black uppercase tracking-widest
shadow-sm ${
        isDark

```

```

        ? 'bg-slate-800 text-slate-200 hover:bg-indigo-600 hover:text-
white'

        : 'bg-white text-slate-700 hover:bg-black hover:text-white border
border-slate-100'

    }}

>

    <FiKey size={14} className="opacity-50" />

    {u.role === 'admin' ? 'Degradar' : 'Hacer Admin'}

  </button>

</td>

</tr>

)}}

</tbody>

</table>

</div>

```

```

{/* Mobile View (Cards) */}

<div className="md:hidden divide-y divide-slate-100 dark:divide-slate-
800/10">

  {filteredUsers.map((u) => (

    <div key={u.id} className="p-6 space-y-4 hover:bg-indigo-500/5
transition-all">

      <div className="flex items-center justify-between">

        <div className="flex items-center gap-4">

          <div className={`w-12 h-12 rounded-2xl flex items-center justify-
center font-black text-xs shadow-inner shrink-0 ${

```



```

    isDark ? 'bg-slate-800 text-indigo-400' : 'bg-indigo-50 text-indigo-600'
  }}>

  {u.email?.charAt(0).toUpperCase() || <FiUser/>}

</div>

<div className="min-w-0">

  <span className={`font-black text-sm block truncate tracking-tight
${textTitle}`}>{u.email}</span>

  <span className="text-[10px] text-slate-400 font-bold uppercase
tracking-widest">UID: {u.id.slice(-8).toUpperCase()}</span>

</div>

</div>

<div className="flex items-center gap-1.5 shrink-0">

  <span className="w-1.5 h-1.5 rounded-full bg-emerald-
500"></span>

  <span className={`text-[9px] font-black uppercase tracking-widest
${textSub}`}>Activo</span>

</div>

</div>

<div className="flex items-center justify-between bg-slate-500/5 p-4
rounded-2xl">

  <div>

    <p className="text-[9px] font-black uppercase text-slate-400 tracking-
widest mb-1.5">Nivel de Acceso</p>

    <span className={`inline-flex items-center gap-2 px-3 py-1 rounded-
full text-[9px] font-black uppercase tracking-widest ${
      u.role === 'admin'

```

```

      ? isDark ? 'bg-indigo-500/10 text-indigo-400 ring-1 ring-indigo-
500/20' : 'bg-indigo-100 text-indigo-700 border border-indigo-200'
      : isDark ? 'bg-slate-800 text-slate-400' : 'bg-slate-100 text-slate-500'
    }}>
    {u.role === 'admin' ? <FiShield size={10} /> : <FiUser size={10} />}
    {u.role === 'admin' ? 'Admin' : 'Cliente'}
  </span>
</div>

<button
  onClick={() => updateUserRole(u.id, u.role === 'admin' ? 'user' :
'admin')}
  className={`px-4 py-2.5 rounded-xl transition-all font-black text-[9px]
uppercase tracking-widest flex items-center gap-2 ${
    isDark
      ? 'bg-slate-800 text-indigo-400 border border-slate-700 active:bg-
indigo-600 active:text-white'
      : 'bg-white text-slate-700 border border-slate-200 active:bg-black
active:text-white shadow-sm'
    }}
>
  <FiKey size={12} className="opacity-50" />
  {u.role === 'admin' ? 'Degradar' : 'Elevar'}
</button>
</div>
</div>

```

```

    ))}
  </div>

  {filteredUsers.length === 0 && (
    <div className="py-24 text-center">
      <div className={`w-20 h-20 rounded-full flex items-center justify-center
mx-auto mb-6 ${isDark ? 'bg-slate-800' : 'bg-slate-50'}}>
        <FiUsers size={32} className="opacity-20" />
      </div>
      <h3 className={`text-lg font-black ${textTitle}`}>Usuarios no
encontrados</h3>
      <p className={` ${textSub} text-sm mt-2`}>No hay registros que coincidan
con los criterios de búsqueda actuales.</p>
    </div>
  )}
</div>
</div>
);
};

```

export default AdminUsers;

 **src/assets/components/Admin/DashBoardAnalytics.jsx**

import React from 'react';

import Loader from '../Loader';

import SalesTrendChart from '../charts/SalesTrendChart';

import SalesByCategoryChart from '../charts/SalesByCategoryChart';

```

import OrderStatusChart from '../charts/OrderStatusChart';

import PaymentMethodChart from '../charts/PaymentMethodChart';

import UserRegistrationChart from '../charts/UserRegistrationChart';

import { FiShoppingBag, FiPackage, FiUsers, FiBarChart2, FiActivity, FiTrendingUp,
FiArrowRight } from 'react-icons/fi';

const DashboardAnalytics = ({
  stats,
  loading,
  theme,
  handleDateRangeChange,
  formatCurrency,
  formatDate,
  getStatusBadgeClass,
  getOrderStatusText
}) => {
  if(loading) {
    return (
      <div className="flex flex-col items-center justify-center min-h-[60vh] space-y-4">
        <div className="w-16 h-16 border-4 border-indigo-600 border-b-transparent rounded-full animate-spin"> </div>
        <p className="text-sm font-black uppercase tracking-[0.3em] text-indigo-600 animate-pulse">Sincronizando Inteligencia...</p>
      </div>
    );
  }

```

```
}
```

```
const isDark = theme === 'dark';
```

```
// Luxury UI Tokens
```

```
const bgMain = isDark ? 'bg-[#111218]' : 'bg-slate-50';
```

```
const bgCard = isDark ? 'bg-[#1a1b26] border-slate-800 shadow-2xl' : 'bg-white border-slate-100 shadow-sm';
```

```
const textTitle = isDark ? 'text-slate-100' : 'text-slate-900';
```

```
const textSub = isDark ? 'text-slate-400' : 'text-slate-500';
```

```
const btnBase = "px-6 py-3 text-[10px] font-black uppercase tracking-widest rounded-2xl transition-all duration-300 active:scale-95";
```

```
const btnActive = `${btnBase} bg-indigo-600 text-white shadow-lg shadow-indigo-500/20`;
```

```
const btnInactive = `${btnBase} ${isDark ? 'bg-slate-800 text-slate-400 hover:bg-slate-700 hover:text-white' : 'bg-white text-slate-500 hover:bg-slate-100 border border-slate-100'}`;
```

```
return (
```

```
<div className={`w-full space-y-10 animate-in fade-in slide-in-from-bottom-4 duration-700 ${bgMain}`}>
```

```
{/* Dynamic Header & Period Selector */}
```

```
<div className={`${bgCard} p-10 rounded-[2.5rem] border flex flex-col lg:flex-row lg:items-center justify-between gap-8 relative overflow-hidden`>
```

```

<div className="absolute top-0 right-0 w-64 h-64 bg-indigo-500/5
rounded-full -mr-32 -mt-32 blur-3xl"></div>

<div className="relative">

  <div className="flex items-center gap-3 mb-2">

    <div className="w-8 h-1 bg-indigo-600 rounded-full"></div>

    <span className={`text-[10px] font-black uppercase tracking-[0.4em] text-
indigo-600`} >Intelligence Center</span>

  </div>

  <h2 className={`text-4xl font-black tracking-tight ${textTitle}`} >Análisis
Proyectivo</h2>

  <p className={` ${textSub} mt-1 text-base`} >Visualiza el rendimiento y
crecimiento de tu ecosistema comercial.</p>

</div>

```

```

<div className="flex flex-wrap gap-3 relative">

  {[

    { id: 'last7days', label: '7 Días' },
    { id: 'last30days', label: '30 Días' },
    { id: 'last90days', label: '90 Días' },
    { id: 'lastYear', label: 'Anual' },

  ].map((period) => (

    <button

      key={period.id}

      onClick={() => handleDateRangeChange(period.id)}

      className={stats.dateRange === period.id ? btnActive : btnInactive}

    >

```

```

        {period.label}

      </button>

    )))

  </div>

</div>

{/* Premium KPI Grid */}

<div className="grid w-full grid-cols-1 gap-8 sm:grid-cols-2 lg:grid-cols-4">

  {[

    { label: 'Productos', value: stats.totalProducts, icon: FiShoppingBag, color:
'blue', desc: 'Catálogo total' },

    { label: 'Pedidos', value: stats.totalOrders, icon: FiPackage, color: 'emerald',
desc: 'Flujo de ventas' },

    { label: 'Comunidad', value: stats.totalUsers, icon: FiUsers, color: 'purple', desc:
'Usuarios registrados' },

    { label: 'Ingresos', value: formatCurrency(stats.totalRevenue), icon:
FiTrendingUp, color: 'indigo', desc: 'Valor total bruto' },

  ].map((metric, idx) => (

    <div key={idx} className={` ${bgCard} p-8 rounded-[2rem] border group
hover:scale-[1.02] transition-all duration-300 relative overflow-hidden`} >

      <div className={` absolute bottom-0 right-0 w-24 h-24 bg-${metric.color}-
500/5 rounded-full -mb-12 -mr-12 blur-2xl group-hover:scale-150 transition-
transform`} > </div>

      <div className="flex flex-col gap-6">

        <div className={`w-14 h-14 rounded-2xl flex items-center justify-center
${metric.color} === 'blue' ? 'bg-blue-500/10 text-blue-500' : metric.color ===
'emerald' ? 'bg-emerald-500/10 text-emerald-500' : metric.color === 'purple' ? 'bg-

```

```
purple-500/10 text-purple-500' : 'bg-indigo-500/10 text-indigo-500'} shadow-  
inner`}>
```

```
<metric.icon size={26} />
```

```
</div>
```

```
<div>
```

```
<p className={`text-[10px] font-black uppercase tracking-[0.2em]  
${textSub} mb-1`} >{metric.label}</p>
```

```
<h4 className={`text-2xl font-black tracking-tight  
${textTitle}`} >{metric.value}</h4>
```

```
<p className="text-[10px] text-slate-400 mt-2 font-medium opacity-60  
flex items-center gap-1">
```

```
{metric.desc}
```

```
</p>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
)))
```

```
</div>
```

```
{/* Charts Architecture */}
```

```
<div className="grid w-full grid-cols-1 gap-10 lg:grid-cols-2">
```

```
<div className={` ${bgCard} p-10 rounded-[2.5rem] border`} >
```

```
<div className="flex items-center justify-between mb-8">
```

```
<h3 className={`text-xl font-black tracking-tight ${textTitle}`} >Tendencia  
de Ingresos</h3>
```

```
<FiActivity className="text-indigo-500 opacity-50" size={20} />
```



```

</div>

<div className="w-full h-80">

  <SalesTrendChart data={stats.salesData} theme={theme} />

</div>

</div>

<div className={` ${bgCard} p-10 rounded-[2.5rem] border`} >

  <div className="flex items-center justify-between mb-8">

    <h3 className={`text-xl font-black tracking-tight ${textTitle}`} >Dominio de
    Categorías</h3>

    <div className="px-3 py-1 bg-emerald-500/10 text-emerald-500
    rounded-full text-[9px] font-black uppercase tracking-widest">Share %</div>

  </div>

  <div className="w-full h-80">

    <SalesByCategoryChart data={stats.salesByCategory} theme={theme} />

  </div>

</div>

<div className={` ${bgCard} p-10 rounded-[2.5rem] border`} >

  <div className="flex items-center justify-between mb-8">

    <h3 className={`text-xl font-black tracking-tight ${textTitle}`} >Flujo de
    Operaciones</h3>

    <span className={textSub}> <FiPackage/> </span>

  </div>

  <div className="w-full h-80">

```

```

    <OrderStatusChart data={stats.orderStatusData} theme={theme} />
  </div>
</div>

<div className={` ${bgCard} p-10 rounded-[2.5rem] border`} >
  <div className="flex items-center justify-between mb-8">
    <h3 className={`text-xl font-black tracking-tight ${textTitle}`} >Capital por
Canal</h3>
    <span className={textSub}> <FiBarChart2/> </span>
  </div>
  <div className="w-full h-80">
    <PaymentMethodChart data={stats.paymentMethodData} theme={theme}
/>
  </div>
</div>
</div>

{/* Advanced Intelligence Tables */}

<div className="grid w-full grid-cols-1 gap-10 xl:grid-cols-2">

{/* Top Products Card */}

<div className={` ${bgCard} rounded-[2.5rem] border overflow-hidden`} >
  <div className="px-10 py-8 border-b border-inherit flex items-center
justify-between">
    <h3 className={`text-xl font-black tracking-tight
${textTitle}`} >Performance de Productos</h3>

```

```

    <button className="text-[10px] font-black uppercase tracking-widest text-indigo-500 flex items-center gap-2 hover:gap-3 transition-all">Ver todos
    <FiArrowRight/> </button>

```

```

</div>

```

```

<div className="overflow-x-auto p-4">

```

```

  <table className="w-full text-left">

```

```

    <thead>

```

```

      <tr className={`text-[10px] font-black uppercase tracking-[0.2em] ${textSub}`}>

```

```

        <th className="px-8 py-6">Producto Estrella</th>

```

```

        <th className="px-8 py-6">Volumen</th>

```

```

        <th className="px-8 py-6 text-right pr-12">Valor Bruto</th>

```

```

      </tr>

```

```

    </thead>

```

```

    <tbody className={`divide-y ${isDark ? 'divide-slate-800/10' : 'divide-slate-50'}>

```

```

      {stats.topSellingProducts.map((p, idx) => (

```

```

        <tr key={idx} className="group hover:bg-slate-500/5 transition-all">

```

```

          <td className="px-8 py-6">

```

```

            <div className="flex items-center gap-4">

```

```

              <div className={`w-10 h-10 rounded-xl bg-slate-500/5 flex items-center justify-center font-black text-xs ${textSub}`}>{idx + 1}</div>

```

```

              <span className={`font-bold text-sm ${textTitle} tracking-tight`}>{p.productName}</span>

```

```

            </div>

```

```

          </td>

```

```

        <td className="px-8 py-6">
            <span className={`px-3 py-1 rounded-lg text-xs font-black ${isDark
? 'bg-slate-800 text-slate-300' : 'bg-slate-100 text-slate-600'}}>{p.quantity}
uds</span>
        </td>
        <td className="px-8 py-6 text-right pr-12">
            <span className={`font-black text-sm text-emerald-
500`}>{formatCurrency(p.revenue)}</span>
        </td>
    </tr>
    )}}
</tbody>
</table>
</div>
</div>

```

```

{ /* Recent Traffic Card */ }
<div className={` ${bgCard} rounded-[2.5rem] border overflow-hidden`}>
    <div className="px-10 py-8 border-b border-inherit flex items-center
justify-between">
        <h3 className={`text-xl font-black tracking-tight ${textTitle}`}>Monitoreo
de Pedidos</h3>
        <div className="flex items-center gap-2">
            <span className="w-2 h-2 rounded-full bg-indigo-500 animate-
pulse"></span>
            <span className="text-[10px] font-black text-slate-400 uppercase
tracking-widest">Live Flow</span>

```

```

    </div>

</div>

<div className="overflow-x-auto p-4">
  <table className="w-full text-left">
    <thead>
      <tr className={`text-[10px] font-black uppercase tracking-[0.2em]
${textSub}`}>
        <th className="px-8 py-6">Identificador</th>
        <th className="px-8 py-6">Cliente</th>
        <th className="px-8 py-6 text-right pr-12">Total</th>
      </tr>
    </thead>
    <tbody className={`divide-y ${isDark ? 'divide-slate-800/10' : 'divide-
slate-50'}`}>
      {stats.recentOrders.map((o) => (
        <tr key={o.id} className="group hover:bg-slate-500/5 transition-all">
          <td className="px-8 py-6">
            <div className="flex flex-col">
              <span className={`font-black text-xs ${textTitle} tracking-
widest`}>#{o.id.substring(0, 8).toUpperCase()}</span>
              <span className="text-[10px] text-slate-400 mt-
1">{formatDate(o.createdAt)}</span>
            </div>
          </td>
          <td className="px-8 py-6">

```

```

        <span className={`text-xs font-bold ${textSub}`}>{o.userEmail ||
'Guest Account'}</span>

    </td>

    <td className="px-8 py-6 text-right pr-12">

        <div className="flex flex-col items-end">

            <span className={`font-black text-sm
${textTitle}`}>{formatCurrency(o.totalAmount || 0)}</span>

            <span className={`text-[8px] font-black uppercase tracking-tighter
mt-1 px-2 py-0.5 rounded-full border ${getStatusBadgeClass(o.orderStatus ||
'pending', theme)}}>

                {getOrderStatusText(o.orderStatus || 'pending')}

            </span>

        </div>

    </td>

</tr>

)}}

</tbody>

</table>

</div>

</div>

</div>

```

{/ Expansion Growth Chart */}*

```

<div className={` ${bgCard} p-10 rounded-[2.5rem] border relative overflow-
hidden`}>

```

```
<div className="absolute bottom-0 left-0 w-full h-1 bg-gradient-to-r from-blue-600 via-indigo-600 to-purple-600"> </div>
```

```
<div className="flex items-center justify-between mb-10">
```

```
<div>
```

```
<h3 className={`text-2xl font-black tracking-tight ${textTitle}`}>Tracción de Usuarios</h3>
```

```
<p className={textSub}>Incremento de la base instalada en el tiempo.</p>
```

```
</div>
```

```
<div className="p-4 rounded-[1.5rem] bg-indigo-600/10 text-indigo-600">
```

```
<FiActivity size={24} />
```

```
</div>
```

```
</div>
```

```
<div className="w-full h-80">
```

```
<UserRegistrationChart data={stats.userRegistrationData} theme={theme} />
```

```
</div>
```

```
</div>
```

```
</div>
```

```
);
```

```
};
```

```
export default DashboardAnalytics;
```

```
📁 src/assets/components/Charts/OrderStatusCharts.jsx
```

```
import React from 'react';
```

```
import { BarChart, Bar, XAxis, YAxis, CartesianGrid, Tooltip, ResponsiveContainer, Legend, Cell } from 'recharts';
```

```
const OrderStatusChart = ({ data }) => {
```

```
  // Get status text in Spanish
```

```
  const getStatusText = (status) => {
```

```
    switch (status) {
```

```
      case 'pending':
```

```
        return 'Pendiente';
```

```
      case 'processing':
```

```
        return 'Procesando';
```

```
      case 'shipped':
```

```
        return 'Enviado';
```

```
      case 'delivered':
```

```
        return 'Entregado';
```

```
      case 'cancelled':
```

```
        return 'Cancelado';
```

```
      default:
```

```
        return status;
```

```
    }
```

```
  };
```

```
  // Colors for different statuses
```

```
  const getStatusColor = (status) => {
```

```
    switch (status) {
```

```
      case 'pending':
```

```
        return '#f59e0b';
```



```

    case 'processing':
      return '#3b82f6';
    case 'shipped':
      return '#8b5cf6';
    case 'delivered':
      return '#10b981';
    case 'cancelled':
      return '#ef4444';
    default:
      return '#6b7280';
  }
};

```

// Format data for the chart

```

const formattedData = data.map(item => ({
  name: getStatusText(item.status),
  value: item.count
}));

```

```

return (
  <ResponsiveContainer width="100%" height="100%">
    <BarChart
      data={formattedData}
      margin={{

```

```

    top: 5,
    right: 30,
    left: 20,
    bottom: 5,
  }}
>
<CartesianGrid strokeDasharray="3 3" />
<XAxis dataKey="name" tick={{ fontSize: 12 }} />
<YAxis tick={{ fontSize: 12 }} />
<Tooltip
  formatter={(value) => [value, 'Pedidos']}
  labelFormatter={(label) => `Estado: ${label}`}
/>
<Legend />
<Bar
  dataKey="value"
  name="Número de Pedidos"
>
{formattedData.map((entry, index) => (
  <Cell
    key={`cell-${index}`}
    fill={getStatusColor(data[index].status)}
  />
  )))

```

```

        </Bar>

    </BarChart>

</ResponsiveContainer>

);

};

export default OrderStatusChart;

```

📁 **src/assets/components/Charts/PaymentMethodChart.jsx**

```

import React from 'react';

import { PieChart, Pie, Cell, ResponsiveContainer, Tooltip, Legend } from 'recharts';

const PaymentMethodChart = ({ data }) => {

    // Format data for the chart

    const formattedData = data.map(item => ({

        name: item.method,

        value: item.revenue,

        count: item.count

    }));

    // Colors for the chart

    const COLORS = ['#3b82f6', '#10b981', '#f59e0b', '#ef4444', '#8b5cf6', '#06b6d4',
    '#f97316', '#ec4899'];

    // Format currency for tooltip

    const formatCurrency = (value) => {

        return new Intl.NumberFormat('es-CO', {

```

```

    style: 'currency',
    currency: 'COP',
    minimumFractionDigits: 0
  }).format(value);
};

return (
  <ResponsiveContainer width="100%" height="100%">
    <PieChart>
      <Pie
        data={formattedData}
        cx="50%"
        cy="50%"
        labelLine={true}
        outerRadius={80}
        fill="#8884d8"
        dataKey="value"
        nameKey="name"
        label={({ name, percent }) => `${name}: ${(percent * 100).toFixed(0)}%`}
      >
        {formattedData.map((entry, index) => (
          <Cell key={`cell-${index}`} fill={COLORS[index % COLORS.length]} />
        ))}
      </Pie>
    </PieChart>
  </ResponsiveContainer>
);

```

```

    <Tooltip
      formatter={(value, name, props) => {
        if(name === 'value') {
          return [formatCurrency(value), 'Ingresos'];
        }
        return [value, 'Pedidos'];
      }}
      labelFormatter={(label) => `Método: ${label}`}
    />
    <Legend />
  </PieChart>
</ResponsiveContainer>
);
};

export default PaymentMethodChart;

```

 **src/assets/components/Charts/SalesByCategoryChart.jsx**

```

import React from 'react';
import { PieChart, Pie, Cell, ResponsiveContainer, Tooltip, Legend } from 'recharts';

const SalesByCategoryChart = ({ data }) => {
  // Format data for the chart
  const formattedData = data.map(item => ({
    name: item.category,
    value: item.revenue,
  }));

```

```
    sales: item.sales,  
    quantity: item.quantity  
  });
```

```
// Colors for the chart
```

```
const COLORS = ['#3b82f6', '#10b981', '#f59e0b', '#ef4444', '#8b5cf6', '#06b6d4',  
                '#f97316', '#ec4899'];
```

```
// Format currency for tooltip
```

```
const formatCurrency = (value) => {  
  return new Intl.NumberFormat('es-CO', {  
    style: 'currency',  
    currency: 'COP',  
    minimumFractionDigits: 0  
  }).format(value);  
};
```

```
return (
```

```
<ResponsiveContainer width="100%" height="100%">
```

```
<PieChart>
```

```
<Pie
```

```
  data={formattedData}
```

```
  cx="50%"
```

```
  cy="50%"
```

```
  labelLine={true}
```

```

    outerRadius={80}

    fill="#8884d8"

    dataKey="value"

    nameKey="name"

    label={({ name, percent }) => `${name}: ${(percent * 100).toFixed(0)}%`}
  >

  {formattedData.map((entry, index) => (
    <Cell key={`cell-${index}`} fill={COLORS[index % COLORS.length]} />
  ))}
</Pie>

<Tooltip
  formatter={(value) => formatCurrency(value)}
  labelFormatter={(label) => `Categoría: ${label}`}
/>

<Legend />
</PieChart>
</ResponsiveContainer>

);
};

export default SalesByCategoryChart;

```

 **src/assets/components/Charts/SalesTrendChart.jsx**

```

import React from 'react';

import { LineChart, Line, XAxis, YAxis, CartesianGrid, Tooltip, ResponsiveContainer,
  Legend } from 'recharts';

```

```

const SalesTrendChart = ({ data }) => {

  // Format data for the chart

  const formattedData = data.map(item => ({

    date: item.date,

    Ventas: item.totalSales,

    Pedidos: item.orderCount

  }));


  // Format currency for tooltip

  const formatCurrency = (value) => {

    return new Intl.NumberFormat('es-CO', {

      style: 'currency',

      currency: 'COP',

      minimumFractionDigits: 0

    }).format(value);

  };


  return (

    <ResponsiveContainer width="100%" height="100%">

      <LineChart

        data={formattedData}

        margin={{

          top: 5,

          right: 30,

```



```

    left: 20,

    bottom: 5,
  }}
>
<CartesianGrid strokeDasharray="3 3" />
<XAxis
  dataKey="date"
  tick={{ fontSize: 12 }}
  tickFormatter={{(value) => {
    const date = new Date(value);
    return date.toLocaleDateString('es-CO', { month: 'short', day: 'numeric' });
  }}
/>
<YAxis
  tick={{ fontSize: 12 }}
  tickFormatter={{(value) => formatCurrency(value)}}
/>
<Tooltip
  formatter={{(value, name) => {
    if(name === 'Ventas') {
      return [formatCurrency(value), 'Ventas'];
    }
    return [value, 'Pedidos'];
  }}

```

```

    labelFormatter={(label) => {
      const date = new Date(label);

      return date.toLocaleDateString('es-CO', { weekday: 'long', year: 'numeric',
month: 'long', day: 'numeric' });
    }}
  />
<Legend />
<Line
  type="monotone"
  dataKey="Ventas"
  stroke="#3b82f6"
  activeDot={{ r: 8 }}
  strokeWidth={2}
  name="Ventas"
/>
<Line
  type="monotone"
  dataKey="Pedidos"
  stroke="#10b981"
  strokeWidth={2}
  name="Pedidos"
/>
</LineChart>
</ResponsiveContainer>
);

```

```
};
```

```
export default SalesTrendChart;
```

```
📁 src/assets/components/Charts/UserRegistrationChart
```

```
import React from 'react';
```

```
import { AreaChart, Area, XAxis, YAxis, CartesianGrid, Tooltip, ResponsiveContainer, Legend } from 'recharts';
```

```
const UserRegistrationChart = ({ data }) => {
```

```
  // Format data for the chart
```

```
  const formattedData = data.map(item => ({
```

```
    date: item.date,
```

```
    Registros: item.registrations
```

```
  }));
```

```
  return (
```

```
    <ResponsiveContainer width="100%" height="100%">
```

```
      <AreaChart
```

```
        data={formattedData}
```

```
        margin={{
```

```
          top: 10,
```

```
          right: 30,
```

```
          left: 0,
```

```
          bottom: 0,
```

```
        }}>
```

```
    </>
```

```

<CartesianGrid strokeDasharray="3 3" />
<XAxis
  dataKey="date"
  tick={{ fontSize: 12 }}
  tickFormatter={(value) => {
    const date = new Date(value);
    return date.toLocaleDateString('es-CO', { month: 'short', day: 'numeric' });
  }}
/>
<YAxis tick={{ fontSize: 12 }} />
<Tooltip
  labelFormatter={(label) => {
    const date = new Date(label);
    return date.toLocaleDateString('es-CO', { weekday: 'long', year: 'numeric',
month: 'long', day: 'numeric' });
  }}
/>
<Legend />
<Area
  type="monotone"
  dataKey="Registros"
  stroke="#8b5cf6"
  fill="#8b5cf6"
  fillOpacity={0.3}
  strokeWidth={2}

```

```

    />

  </AreaChart>

</ResponsiveContainer>

);

};

export default UserRegistrationChart;

```

 **src/assets/components/AdminSetup**

```

import React, { useState } from 'react';

import { useNavigate } from 'react-router-dom';

import { auth, db } from '../firebase';

import { createUserWithEmailAndPassword } from 'firebase/auth';

import { doc, setDoc } from 'firebase/firestore';

import { useAuth } from '../../context/AuthContext';

const AdminSetup = () => {

  const [email, setEmail] = useState('');

  const [password, setPassword] = useState('');

  const [confirmPassword, setConfirmPassword] = useState('');

  const [message, setMessage] = useState('');

  const [error, setError] = useState('');

  const [loading, setLoading] = useState(false);

  const navigate = useNavigate();

  const { firebaseError } = useAuth();

```

```

const handleSubmit = async (e) => {
  e.preventDefault();

  // Reset messages
  setMessage('');
  setError('');

  // Check if Firebase is properly configured
  if(firebaseError) {
    setError(firebaseError);
    return;
  }

  // Check if auth and db are available
  if(!auth || !db) {
    setError('Firebase no está configurado correctamente. Por favor, verifica tu
    archivo .env');
    return;
  }

  // Validate passwords match
  if(password !== confirmPassword) {
    setError('Las contraseñas no coinciden');
    return;
  }

```

```

// Validate password strength
if(password.length < 6) {
  setError('La contraseña debe tener al menos 6 caracteres');
  return;
}

setLoading(true);

try{
  // Crear usuario administrador email - password
  const userCredential = await createUserWithEmailAndPassword(auth, email,
password);
  const user = userCredential.user;

  // Guardar información de usuario en la base de datos - rol administrador
  await setDoc(doc(db, 'users', user.uid), {
    email: user.email,
    createdAt: new Date(),
    role: 'admin' // rol administrador
  });

  setMessage(`Usuario admin creado exitosamente: ${user.email}`);
  setEmail('');
  setPassword('');

```

```

setConfirmPassword('');

// redireccionar a la ruta de login despues de 2 segundos

setTimeout(() => {
  navigate('/login');
}, 2000);
} catch (err) {
  setError(`Error creando usuario: ${err.message}`);
  console.error('Error creating admin user:', err);
} finally {
  setLoading(false);
}
};

return (
  <div className="flex items-center justify-center min-h-screen px-4 py-12 bg-gray-50 sm:px-6 lg:px-8">
    <div className="w-full max-w-md space-y-8">
      <div>
        <h2 className="mt-6 text-3xl font-extrabold text-center text-gray-900">
          Configuración de Admin
        </h2>
        <p className="mt-2 text-sm text-center text-gray-600">
          Crear cuenta de administrador
        </p>

```


</div>

{firebaseError && (

<div *className*="relative px-4 py-3 text-yellow-700 bg-yellow-100 border border-yellow-400 rounded" *role*="alert">

<strong *className*="font-bold">Configuración incompleta!

{firebaseError}

</div>

)}

{message && (

<div *className*="relative px-4 py-3 text-green-700 bg-green-100 border border-green-400 rounded" *role*="alert">

<strong *className*="font-bold">Éxito!

{message}

</div>

)}

{error && (

<div *className*="relative px-4 py-3 text-red-700 bg-red-100 border border-red-400 rounded" *role*="alert">

<strong *className*="font-bold">Error!

{error}

</div>

)}

```

<form className="mt-8 space-y-6" onSubmit={handleSubmit}>
  <div className="-space-y-px rounded-md shadow-sm">
    <div>
      <label htmlFor="email-address" className="sr-only">Email</label>
      <input
        id="email-address"
        name="email"
        type="email"
        autoComplete="email"
        required
        className="relative block w-full px-3 py-2 text-gray-900 placeholder-
gray-500 border border-gray-300 rounded-none appearance-none rounded-t-md
focus:outline-none focus:ring-indigo-500 focus:border-indigo-500 focus:z-10
sm:text-sm"
        placeholder="Email de administrador"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
        disabled={firebaseError || loading}
      />
    </div>
    <div>
      <label htmlFor="password" className="sr-only">Contraseña</label>
      <input
        id="password"

```

```

    name="password"

    type="password"

    autoComplete="new-password"

    required

    className="relative block w-full px-3 py-2 text-gray-900 placeholder-
gray-500 border border-gray-300 rounded-none appearance-none focus:outline-
none focus:ring-indigo-500 focus:border-indigo-500 focus:z-10 sm:text-sm"

    placeholder="Contraseña"

    value={password}

    onChange={(e) => setPassword(e.target.value)}

    disabled={firebaseError || loading}

/>

</div>

<div>

    <label htmlFor="confirm-password" className="sr-only">Confirmar
Contraseña</label>

    <input

        id="confirm-password"

        name="confirm-password"

        type="password"

        autoComplete="new-password"

        required

        className="relative block w-full px-3 py-2 text-gray-900 placeholder-
gray-500 border border-gray-300 rounded-none appearance-none rounded-b-md
focus:outline-none focus:ring-indigo-500 focus:border-indigo-500 focus:z-10
sm:text-sm"

```

```

        placeholder="Confirmar Contraseña"
        value={confirmPassword}
        onChange={(e) => setConfirmPassword(e.target.value)}
        disabled={firebaseError || loading}
      />
    </div>
  </div>

```

```

<div>
  <button
    type="submit"
    disabled={loading || firebaseError}
    className="relative flex justify-center w-full px-4 py-2 text-sm font-
medium text-gray-900 bg-indigo-600 border border-transparent rounded-md
group hover:bg-indigo-700 focus:outline-none focus:ring-2 focus:ring-offset-2
focus:ring-indigo-500 disabled:opacity-50"
  >
    {loading ? 'Creando...' : 'Crear Usuario Admin'}
  </button>
</div>
</form>

```

```

<div className="p-4 mt-6 border border-blue-200 rounded-lg bg-blue-50">
  <h3 className="text-sm font-medium text-blue-800">Importante</h3>
  <div className="mt-2 text-sm text-blue-700">

```

```

    <p>
        Después de crear el usuario admin, inicia sesión con las credenciales
        proporcionadas.
    </p>
</div>
</div>
</div>
</div>
);
};
export default AdminSetup;

```

src/assets/components/CouponManager.jsx

```

import React, { useState, useEffect } from 'react';
import { db } from '../firebase';

import { collection, addDoc, getDocs, deleteDoc, doc, updateDoc, query, orderBy }
from "firebase/firestore";

import { FiPlus, FiTrash2, FiEdit3, FiCheckCircle, FiXCircle, FiGrid, FiClock, FiPercent,
FiDollarSign, FiCalendar, FiAlertCircle, FiX } from 'react-icons/fi';

const CouponManager = ({ theme = 'light' }) => {
    const [coupons, setCoupons] = useState([]);
    const [loading, setLoading] = useState(true);
    const [error, setError] = useState(null);
    const [successMessage, setSuccessMessage] = useState("");
    const [isModalOpen, setIsModalOpen] = useState(false);

```

```

// Form state for new or editing coupon

const [formData, setFormData] = useState({
  id: null,
  code: "",
  discountType: 'percentage',
  discountValue: "",
  isActive: true,
  expiryDate: ""
});

const couponsCollectionRef = collection(db, "coupons");

// Fetch all coupons

const fetchCoupons = async () => {
  try {
    const couponsQuery = query(couponsCollectionRef, orderBy("createdAt",
"desc"));

    const data = await getDocs(couponsQuery);

    setCoupons(data.docs.map((doc) => ({ ...doc.data(), id: doc.id })));
  } catch (err) {
    setError("No se pudieron cargar los cupones.");
  }

  setLoading(false);
};

```

```
useEffect(() => {  
  fetchCoupons();  
}, []);
```

```
const showSuccessMessage = (message) => {  
  setSuccessMessage(message);  
  setTimeout(() => setSuccessMessage(''), 3000);  
};
```

```
const handleInputChange = (e) => {  
  const { name, value, type, checked } = e.target;  
  setFormData({  
    ...formData,  
    [name]: type === 'checkbox' ? checked : value  
  });  
};
```

```
const openForm = (coupon = null) => {  
  if(coupon) {  
    setFormData({  
      id: coupon.id,  
      code: coupon.code,  
      discountType: coupon.discountType,
```

```

        discountValue: coupon.discountValue,

        isActive: coupon.isActive,

        expiryDate: coupon.expiryDate ? (coupon.expiryDate.toDate ?
coupon.expiryDate.toDate().toISOString().split('T')[0] : coupon.expiryDate) : ""

    });

    } else {

        setFormData({ id: null, code: "", discountType: 'percentage', discountValue: "",
isActive: true, expiryDate: "" });

    }

    setIsModalOpen(true);

};

const handleSubmit = async (e) => {

    e.preventDefault();

    if(formData.discountType === 'percentage' && (formData.discountValue < 1 ||
formData.discountValue > 100)) {

        setError("Porcentaje inválido."); return;

    }

    try{

        let expiryDateValue = null;

        if (formData.expiryDate) {

            const dateObj = new Date(formData.expiryDate);

            if(!isNaN(dateObj.getTime())) expiryDateValue = dateObj;

        }

    }

```



```

if(formData.id) {

  // Update

  const updateData = {

    code: formData.code.toUpperCase(),

    discountType: formData.discountType,

    discountValue: parseFloat(formData.discountValue),

    isActive: formData.isActive,

    updatedAt: new Date(),

    ...(expiryDateValue && { expiryDate: expiryDateValue })

  };

  await updateDoc(doc(db, "coupons", formData.id), updateData);

  showSuccessMessage("Cupón actualizado.");

} else {

  // Create

  const couponData = {

    code: formData.code.toUpperCase(),

    discountType: formData.discountType,

    discountValue: parseFloat(formData.discountValue),

    isActive: formData.isActive,

    createdAt: new Date(),

    ...(expiryDateValue && { expiryDate: expiryDateValue })

  };

  await addDoc(couponsCollectionRef, couponData);

```

```

    showSuccessMessage("Cupón creado.");
  }

  setIsModalOpen(false);
  fetchCoupons();
} catch (err) {
  setError("Error al procesar el cupón.");
}
};

const deleteCoupon = async (id, code) => {
  if(!window.confirm(`¿Seguro que quieres eliminar "${code}"?`)) return;
  try{
    await deleteDoc(doc(db, "coupons", id));
    showSuccessMessage("Cupón eliminado.");
    fetchCoupons();
  } catch (err) {
    setError("Error al eliminar.");
  }
};

const formatDate = (date) => {
  if(!date) return 'Sin fecha';
  const d = date.toDate ? date.toDate() : new Date(date);

```

```

    return d.toLocaleDateString('es-CO', { day: '2-digit', month: 'short', year: 'numeric'
  });
};

```

```

const isExpired = (expiryDate) => {
  if(!expiryDate) return false;
  const d = expiryDate.toDate ? expiryDate.toDate() : new Date(expiryDate);
  return d < new Date();
};

```

```

const formatCurrency = (amt) => new Intl.NumberFormat('es-CO', { style:
'currency', currency: 'COP', minimumFractionDigits: 0 }).format(amt || 0);

```

// Stats logic

```

const stats = {
  total: coupons.length,
  active: coupons.filter(c => c.isActive && !isExpired(c.expiryDate)).length,
  expired: coupons.filter(c => isExpired(c.expiryDate)).length
};

```

// Dark Mode Tokens (Luxury Style)

```

const isDark = theme === 'dark';

const bgCard = isDark ? 'bg-[#1a1b26] border-slate-800 shadow-2xl' : 'bg-white
border-gray-100 shadow-sm';

const textTitle = isDark ? 'text-slate-100' : 'text-slate-900';

```

```

const textSub = isDark ? 'text-slate-400' : 'text-slate-500';

const modalOverlay = isDark ? 'bg-black/80 backdrop-blur-sm' : 'bg-slate-900/40
backdrop-blur-sm';

return (

  <div className={`p-4 md:p-8 min-h-screen ${isDark ? 'bg-[#111218] text-slate-
200' : 'bg-slate-50 text-slate-900'}`} style={{ fontFamily: 'Inter, sans-serif' }}>

    /* Header with Title & Action */

    <div className="flex flex-col md:flex-row md:items-center justify-between
gap-6 mb-8 md:mb-12">

      <div>

        <h1 className={`text-2xl md:text-3xl font-black tracking-tight
${textTitle}`}>Gestión de Cupones</h1>

        <p className={`${textSub} text-xs md:text-sm`}>Crea, edita y monitorea los
beneficios de tu tienda.</p>

      </div>

      <button

        onClick={() => openForm()}

        className="group relative flex items-center justify-center gap-2 px-8 py-4
bg-indigo-600 text-white rounded-2xl font-bold transition-all hover:bg-indigo-700
hover:shadow-indigo-500/20 hover:shadow-2xl active:scale-95 overflow-hidden w-
full md:w-auto"

        >

        <div className="absolute inset-x-0 bottom-0 h-1 bg-white/20 transform
translate-y-1 group-hover:translate-y-0 transition-transform"></div>

        <FiPlus className="text-lg" /> Nuevo Beneficio

```

```

</button>

</div>

{ /* KPI Section */

  <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-4 md:gap-8
mb-8 md:mb-12">

    {[

      { label: 'Cupones Totales', val: stats.total, icon: FiGrid, color: 'indigo' },

      { label: 'Vigentes Ahora', val: stats.active, icon: FiCheckCircle, color: 'emerald'
    },

      { label: 'Expirados', val: stats.expired, icon: FiClock, color: 'rose' },

    ].map((s, idx) => (

      <div key={idx} className={` ${bgCard} p-6 md:p-8 rounded-3xl md:rounded-
[2rem] border relative overflow-hidden group`} >

        <div className={` absolute top-0 right-0 w-32 h-32 -mr-12 -mt-12 bg-
${s.color}-500/5 rounded-full transition-transform group-hover:scale-150`} > </div>

        <div className="relative flex items-center gap-5 md:gap-6">

          <div className={` p-3 md:p-4 rounded-2xl ${s.color === 'indigo' ? 'bg-
indigo-500/10 text-indigo-500' : s.color === 'emerald' ? 'bg-emerald-500/10 text-
emerald-500' : 'bg-rose-500/10 text-rose-500'}`} >

            <s.icon size={24} className="md:w-[28px] md:h-[28px]" />

          </div>

          <div>

            <span className={` text-[9px] md:text-[11px] font-black uppercase
tracking-[0.2em] ${textSub}`} > {s.label} </span>

            <h4 className={` text-2xl md:text-4xl font-black mt-1
${textTitle}`} > {s.val} </h4>

```

```

    </div>

  </div>

</div>

)}}

</div>

```

```

{/* Main Table / Cards Card */}

```

```

<div className={` ${bgCard} rounded-3xl md:rounded-[2rem] border overflow-
hidden`} >

```

```

  <div className="px-6 md:px-10 py-6 md:py-8 border-b border-inherit bg-
inherit flex items-center justify-between">

```

```

    <h3 className={`text-lg md:text-xl font-bold ${textTitle}`} >Catálogo de
    Cupones</h3>

```

```

    <div className="flex items-center gap-2">

```

```

      <span className="w-2 h-2 rounded-full bg-indigo-500 animate-
pulse"> </span>

```

```

      <span className="text-[9px] md:text-[10px] font-bold text-slate-400
uppercase tracking-widest hidden sm:inline">En tiempo real</span>

```

```

    </div>

```

```

  </div>

```

```

{/* Desktop Table */}

```

```

<div className="hidden md:block overflow-x-auto px-4 py-4">

```

```

  <table className="w-full text-left">

```

```

    <thead>

```

```

    <tr className={`text-[10px] font-black uppercase tracking-[0.2em]
    ${textSub}`}>
      <th className="px-8 py-6">Código de Cupón</th>
      <th className="px-8 py-6">Valor Beneficio</th>
      <th className="px-8 py-6">Estado</th>
      <th className="px-8 py-6">Fecha Expiración</th>
      <th className="px-8 py-6 text-right pr-12">Opciones</th>
    </tr>
  </thead>

  <tbody className={`divide-y ${isDark ? 'divide-slate-800/10' : 'divide-slate-
50'}`}>

    {!!loading && coupons.map((c) => {
      const expired = isExpired(c.expiryDate);
      return (
        <tr key={c.id} className="group hover:bg-indigo-500/5 transition-all
duration-300">
          <td className="px-8 py-6">
            <div className="flex flex-col">
              <span className={`font-black text-lg tracking-tight
${textTitle}`}>{c.code}</span>
              <span className={`text-[10px] font-medium opacity-50`}>ID:
{c.id.slice(-6).toUpperCase()}</span>
            </div>
          </td>
          <td className="px-8 py-6">
            <div className="flex items-center gap-3">

```

```

        <div className={`w-10 h-10 rounded-xl flex items-center justify-
center ${c.discountType === 'percentage' ? 'bg-indigo-500/10 text-indigo-500' :
'bg-emerald-500/10 text-emerald-500'}}>

            {c.discountType === 'percentage' ? <FiPercent /> : <FiDollarSign
/>}

        </div>

        <span className={`font-bold text-base ${textTitle}}>

            {c.discountType === 'percentage' ? `${c.discountValue}%` :
formatCurrency(c.discountValue)}

        </span>

    </div>

</td>

<td className="px-8 py-6">

    {expired ? (

        <span className="inline-flex items-center gap-2 px-4 py-1.5
rounded-full text-[10px] font-black uppercase tracking-widest bg-rose-500/10 text-
rose-500 ring-1 ring-rose-500/20">

            Expirado

        </span>

    ) : c.isActive ? (

        <span className="inline-flex items-center gap-2 px-4 py-1.5
rounded-full text-[10px] font-black uppercase tracking-widest bg-emerald-500/10
text-emerald-500 ring-1 ring-emerald-500/20">

            Vigente

        </span>

    ) : (

```



```

        <span className="inline-flex items-center gap-2 px-4 py-1.5
rounded-full text-[10px] font-black uppercase tracking-widest bg-slate-500/10
text-slate-500 ring-1 ring-slate-500/20">

            Inactivo

        </span>

    )}

</td>

<td className="px-8 py-6">

    <div className="flex flex-col">

        <span className={`text-sm font-bold ${expired ? 'text-rose-400
opacity-60 line-through' : 'textTitle'}}>

            {formatDate(c.expiryDate)}

        </span>

        <span className="text-[10px] text-slate-400 capitalize">{expired ?
'Caducado' : 'Próxima fecha'}</span>

    </div>

</td>

<td className="px-8 py-6 text-right pr-12">

    <div className="flex items-center justify-end gap-3 transition-all">

        <button

            onClick={() => openForm(c)}

            className={`p-3 rounded-2xl transition-all shadow-sm flex items-
center justify-center ${isDark ? 'bg-slate-800 text-indigo-400 hover:bg-slate-700
hover:shadow-indigo-500/10' : 'bg-slate-100 text-indigo-600 hover:bg-indigo-600
hover:text-white'}}

            title="Editar Cupón"

        >

```

```

        <FiEdit3 size={18} />

    </button>

    <button

        onClick={() => deleteCoupon(c.id, c.code)}

        className={`p-3 rounded-2xl transition-all shadow-sm flex items-
center justify-center ${isDark ? 'bg-slate-800 text-rose-400 hover:bg-rose-500/20
hover:shadow-rose-500/10' : 'bg-slate-100 text-rose-600 hover:bg-rose-600
hover:text-white'}`}

        title="Eliminar Cupón"

    >

        <FiTrash2 size={18} />

    </button>

</div>

</td>

</tr>

);

}}

</tbody>

</table>

</div>

{ /* Mobile View (Cards) */ }

<div className="md:hidden divide-y divide-slate-100 dark:divide-slate-
800/10">

    {!loading && coupons.map((c) => {

        const expired = isExpired(c.expiryDate);

```

```

    return (
      <div key={c.id} className="p-6 space-y-4 hover:bg-indigo-500/5
transition-all">
        <div className="flex justify-between items-start">
          <div className="min-w-0">
            <span className={`font-black text-xl block tracking-tight
${textTitle}`}>{c.code}</span>
            <span className="text-[9px] text-slate-400 font-bold uppercase
tracking-widest mt-0.5">ID: {c.id.slice(-6).toUpperCase()}</span>
          </div>
          {expired ? (
            <span className="px-3 py-1 rounded-full text-[8px] font-black
uppercase tracking-widest bg-rose-500/10 text-rose-500">Expirado</span>
          ) : c.isActive ? (
            <span className="px-3 py-1 rounded-full text-[8px] font-black
uppercase tracking-widest bg-emerald-500/10 text-emerald-500">Vigente</span>
          ) : (
            <span className="px-3 py-1 rounded-full text-[8px] font-black
uppercase tracking-widest bg-slate-500/10 text-slate-500">Inactivo</span>
          )}
        </div>

        <div className="flex justify-between items-center bg-slate-500/5 p-4
rounded-2xl">
          <div className="flex items-center gap-3">

```

```

    <div className={`w-8 h-8 rounded-lg flex items-center justify-center
    ${c.discountType === 'percentage' ? 'bg-indigo-500/10 text-indigo-500' : 'bg-
    emerald-500/10 text-emerald-500'}`}>

```

```

        {c.discountType === 'percentage' ? <FiPercent size={14} /> :
    <FiDollarSign size={14} />}

```

```

    </div>

```

```

    <div>

```

```

        <p className="text-[8px] font-black uppercase text-slate-400
    tracking-widest mb-0.5">Valor</p>

```

```

        <span className={`font-black text-base ${textTitle}`}>

```

```

            {c.discountType === 'percentage' ? `${c.discountValue}%` :
    formatCurrency(c.discountValue)}

```

```

        </span>

```

```

    </div>

```

```

</div>

```

```

    <div className="text-right">

```

```

        <p className="text-[8px] font-black uppercase text-slate-400
    tracking-widest mb-0.5">Expiración</p>

```

```

        <span className={`text-[10px] font-bold ${expired ? 'text-rose-400' :
    textTitle}`}>

```

```

            {formatDate(c.expiryDate)}

```

```

        </span>

```

```

    </div>

```

```

</div>

```

```

<div className="flex gap-3">

```

```

    <button

```

```

        onClick={() => openForm(c)}

        className={`flex-1 flex items-center justify-center gap-2 py-3.5
rounded-2xl font-bold text-xs transition-all ${isDark ? 'bg-slate-800 text-indigo-
400' : 'bg-indigo-50 text-indigo-600'}`}

    >

    <FiEdit3 size={16} /> Editar

</button>

<button

    onClick={() => deleteCoupon(c.id, c.code)}

    className={`flex-1 flex items-center justify-center gap-2 py-3.5
rounded-2xl font-bold text-xs transition-all ${isDark ? 'bg-slate-800/50 text-rose-
400' : 'bg-rose-50 text-rose-600'}`}

    >

    <FiTrash2 size={16} /> Eliminar

</button>

</div>

</div>

);

}}

</div>

{loading && <div className="py-20 text-center"> <div className="w-12 h-
12 border-4 border-indigo-500 border-b-transparent rounded-full animate-spin
mx-auto"> </div> </div>}

{!loading && coupons.length === 0 && <div className="py-20 text-center
text-slate-400 font-bold uppercase tracking-widest">Sin resultados
registrados.</div>}

</div>

```

```

    {isModalOpen && (
      <div className={`fixed inset-0 z-[200] flex items-center justify-center p-4
md:p-6 ${modalOverlay} animate-in fade-in duration-300`}>
        <div
          className={`w-full max-w-xl max-h-[95vh] overflow-y-auto rounded-
[2rem] md:rounded-[2.5rem] border shadow-2xl relative animate-in zoom-in-95
duration-300 p-1 md:p-2 ${bgCard}`}
        >
          <div className={`p-6 md:p-10 ${isDark ? 'bg-[#1a1b26]' : 'bg-white'}
rounded-[1.9rem] md:rounded-[2.4rem]`}>
            <div className="flex items-center justify-between mb-8 md:mb-10">
              <div>
                <h3 className={`text-2xl font-black tracking-tight ${textTitle}`}>
                  {formData.id ? 'Refinar Cupón' : 'Propulsar Campaña'}
                </h3>
                <p className={textSub}>Define los parámetros del nuevo
beneficio.</p>
              </div>
              <button
                onClick={() => setIsModalOpen(false)}
                className={`p-3 rounded-2xl transition-colors ${isDark ? 'bg-slate-800
text-slate-400 hover:bg-slate-700' : 'bg-slate-100 text-slate-500 hover:bg-slate-
200'}`}
              >
                <FiX size={20} />
              </button>
            </div>
          </div>
        </div>
      </div>
    )}
  </div>

```

</div>

```
<form onSubmit={handleSubmit} className="space-y-6 md:space-y-8">
  <div className="grid grid-cols-1 md:grid-cols-2 gap-5 md:gap-8">
    <div className="space-y-4">
      <label className={`block text-[10px] font-black uppercase tracking-[0.2em] px-1 ${textSub}}`>Código Promocional</label>
      <input
        name="code"
        required
        value={formData.code}
        onChange={handleInputChange}
        className={`w-full px-6 py-4 rounded-2xl border transition-all duration-300 outline-none focus:ring-4 ${
          isDark ? 'bg-slate-900/50 border-slate-700 focus:ring-indigo-500/20 text-white' : 'bg-white border-slate-200 focus:ring-indigo-500/10 text-slate-900'
        }}
        placeholder="EJ: BLACKFRIDAY"
      />
    </div>
  </div>
```

```
<div className="space-y-4">
  <label className={`block text-[10px] font-black uppercase tracking-[0.2em] px-1 ${textSub}}`>Tipo de Oferta</label>
  <div className={`p-1 flex rounded-2xl ${isDark ? 'bg-slate-900/50 border border-slate-700' : 'bg-slate-100 border border-slate-200'}`>
```

```

    {[ 'percentage', 'fixed' ].map(type => (
      <button
        key={type}
        type="button"
        onClick={() => setFormData({...formData, discountType: type})}
        className={`flex-1 py-3 px-4 rounded-[0.9rem] flex items-center
justify-center gap-2 font-bold text-xs transition-all ${
          formData.discountType === type
            ? 'bg-indigo-600 text-white shadow-xl shadow-indigo-500/20'
            : 'text-slate-400 hover:text-slate-200'
        }}
      >
        {type === 'percentage' ? <FiPercent /> : <FiDollarSign />}
        {type === 'percentage' ? '%' : '$'}
      </button>
    )
  )}
</div>
</div>

```

```

<div className="space-y-4">
  <label className={`block text-[10px] font-black uppercase tracking-
[0.2em] px-1 ${textSub}`}>Valor del Descuento</label>
  <input
    name="discountValue"
    type="number"
  >

```



```

        required

        value={formData.discountValue}

        onChange={handleInputChange}

        className={`w-full px-6 py-4 rounded-2xl border transition-all
duration-300 outline-none focus:ring-4 ${
            isDark ? 'bg-slate-900/50 border-slate-700 focus:ring-indigo-500/20
text-white' : 'bg-white border-slate-200 focus:ring-indigo-500/10 text-slate-900'
        }}

        placeholder={formData.discountType === 'percentage' ? 'Ej: 15%' :
'Ej: 50.000'}

    />

</div>

```

```

<div className="space-y-4">

    <label className={`block text-[10px] font-black uppercase tracking-
[0.2em] px-1 ${textSub}`}>Fecha de Límite</label>

    <div className="relative">

        <FiCalendar className={`absolute left-5 top-1/2 -translate-y-1/2
${textSub}`} />

        <input

            name="expiryDate"

            type="date"

            value={formData.expiryDate}

            onChange={handleInputChange}

            className={`w-full pl-14 pr-6 py-4 rounded-2xl border transition-all
duration-300 outline-none focus:ring-4 ${

```

```
isDark ? 'bg-slate-900/50 border-slate-700 focus:ring-indigo-500/20 text-white' : 'bg-white border-slate-200 focus:ring-indigo-500/10 text-slate-900'
```

```
}}
```

```
/>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<div className={`p-4 md:p-6 rounded-[1.5rem] border ${isDark ? 'bg-indigo-500/5 border-indigo-500/10' : 'bg-indigo-50 border-indigo-100'} flex items-center justify-between`}>
```

```
<div className="flex items-center gap-4">
```

```
<div className={`w-10 h-10 md:w-12 md:h-12 rounded-2xl flex items-center justify-center ${isDark ? 'bg-slate-900 text-indigo-400' : 'bg-white text-indigo-600 shadow-sm'}}>
```

```
<FiCheckCircle size={20} className="md:w-5 md:h-5" />
```

```
</div>
```

```
<div>
```

```
<p className={`text-xs md:text-sm font-black ${textTitle}`}>Vínculo Activo</p>
```

```
<p className="text-[10px] opacity-60">El cupón será canjeable inmediatamente.</p>
```

```
</div>
```

```
</div>
```

```
<input
```

```
name="isActive"
```

```

        type="checkbox"

        checked={formData.isActive}

        onChange={handleInputChange}

        className="w-8 h-8 rounded-xl border-slate-300 text-indigo-600
focus:ring-indigo-500 cursor-pointer"

    />
</div>

<div className="flex gap-4 pt-4 md:pt-6">

    <button

        type="submit"

        className="flex-1 py-4 md:py-5 bg-indigo-600 text-white rounded-
[1.2rem] md:rounded-[1.5rem] font-black text-sm md:text-base transition-all
hover:bg-indigo-700 hover:shadow-2xl hover:shadow-indigo-500/30 active:scale-
95"

        >

            {formData.id ? 'Guardar Cambios' : 'Lanzar Cupón 🔥 '}

        </button>

    </div>

</form>

</div>

</div>

</div>

))

```

```

{/* Global Toast Message */}

```

```

    {successMessage && (
      <div className="fixed bottom-10 right-10 z-[100] p-5 rounded-2xl bg-slate-
900 text-white shadow-2xl flex items-center gap-4 animate-in slide-in-from-right-
10 duration-500">

        <div className="bg-emerald-500 p-2 rounded-xl text-
white"> <FiCheckCircle size={20} /> </div>

        <p className="text-sm font-bold pr-4">{successMessage}</p>

      </div>

    )}
  </div>

);
};

```

export default CouponManager;

 **src/assets/components/Databaselnit.jsx**

```

import React, { useState } from 'react';
import { db } from '../firebase';
import { collection, addDoc, setDoc, doc } from 'firebase/firestore';

```

```

const Databaselnit = () => {
  const [loading, setLoading] = useState(false);
  const [message, setMessage] = useState("");
  const [error, setError] = useState("");

```

```

const initializeDatabase = async () => {
  setLoading(true);

```

```

setError('');

setMessage('');

try{

  // Create sample categories

  const categories = [

    { name: 'Moños Elegantes', description: 'Moños para ocasiones formales' },

    { name: 'Moños Infantiles', description: 'Moños para niños y niñas' },

    { name: 'Moños Deportivos', description: 'Moños para actividades deportivas'

  },

    { name: 'Moños Casuales', description: 'Moños para uso diario' }

  ];

  // Create sample products

  const products = [

    {

      name: 'Moño Clásico Negro',

      description: 'Moño elegante de satén negro, perfecto para ocasiones formales',

      price: 25.99,

      category: 'Moños Elegantes',

      image: 'https://images.unsplash.com/photo-1591047139829-d91aecb6caea?ixlib=rb-4.0.3&ixid=M3wxMjA3fDB8MHxwaG90by1wYWdlfHx8fGVufDB8fHx8fA%3D%3D&auto=format&fit=crop&w=200&q=80',

      stock: 50
    }
  ]
}

```

```

    },
    {
      name: 'Moño Infantil Rojo',
      description: 'Moño color rojo vibrante para niños, cómodo y duradero',
      price: 15.99,
      category: 'Moños Infantiles',
      image: 'https://images.unsplash.com/photo-1614647615808-30460881f755?ixlib=rb-4.0.3&ixid=M3wxMjA3fDB8MHxwaG90by1wYWdlfHx8fGVufDB8fHx8fA%3D%3D&auto=format&fit=crop&w=200&q=80',
      stock: 30
    },
    {
      name: 'Moño Deportivo Azul',
      description: 'Moño transpirable para actividades deportivas, color azul marino',
      price: 19.99,
      category: 'Moños Deportivos',
      image: 'https://images.unsplash.com/photo-1521572163474-6864f9cf17ab?ixlib=rb-4.0.3&ixid=M3wxMjA3fDB8MHxwaG90by1wYWdlfHx8fGVufDB8fHx8fA%3D%3D&auto=format&fit=crop&w=200&q=80',
      stock: 40
    }
  ];

```

// Initialize categories collection

```

    for (const category of categories) {
      await addDoc(collection(db, 'categories'), category);
    }

    // Initialize products collection
    for (const product of products) {
      await addDoc(collection(db, 'products'), product);
    }

    setMessage('Base de datos inicializada exitosamente con categorías y
    productos de ejemplo');
  } catch (err) {
    console.error('Error initializing database:', err);
    setError(`Error al inicializar la base de datos: ${err.message}`);
  } finally {
    setLoading(false);
  }
};

return (
  <div className="min-h-screen flex items-center justify-center bg-gray-50 py-12
  px-4 sm:px-6 lg:px-8">
    <div className="max-w-md w-full space-y-8">
      <div>
        <h2 className="mt-6 text-center text-3xl font-extrabold text-gray-900">

```

Inicializar Base de Datos

</h2>

<p *className*="mt-2 text-center text-sm text-gray-600">

Crea las colecciones iniciales para categorías y productos

</p>

</div>

{error && (

<div *className*="bg-red-100 border border-red-400 text-red-700 px-4 py-3 rounded">

{error}

</div>

)}

{message && (

<div *className*="bg-green-100 border border-green-400 text-green-700 px-4 py-3 rounded">

{message}

</div>

)}

<div>

<button

onClick={initializeDatabase}

disabled={loading}


```
      className="group relative w-full flex justify-center py-2 px-4 border
border-transparent text-sm font-medium rounded-md text-gray-900 bg-indigo-
600 hover:bg-indigo-700 focus:outline-none focus:ring-2 focus:ring-offset-2
focus:ring-indigo-500 disabled:opacity-50"
```

```
>
```

```
  {loading ? 'Inicializando...' : 'Iniciar Base de Datos'}
```

```
</button>
```

```
</div>
```

```
<div className="mt-4 p-4 bg-yellow-50 border border-yellow-200
rounded">
```

```
<h3 className="font-bold mb-2">Instrucciones:</h3>
```

```
<ol className="list-decimal pl-5 space-y-2 text-sm">
```

```
<li>Asegúrate de que Firestore Database esté creado en tu proyecto
Firebase</li>
```

```
<li>Actualiza las reglas de seguridad de Firestore como se indicó
anteriormente</li>
```

```
<li>Haz clic en "Iniciar Base de Datos" para crear las colecciones de
ejemplo</li>
```

```
</ol>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
);
```

```
}
```

```
export default Databaselnit;
```

src/assets/components/Footer.jsx

```
import React from 'react';
```

```
import { Link } from 'react-router-dom';
```

```
const Footer = () => {
```

```
  return (
```

```
    <footer className="bg-white border-t border-gray-200" style={{ fontFamily: 'Inter, sans-serif' }}>
```

```
      <div className="mx-auto max-w-7xl px-4 sm:px-6 lg:px-8 py-12">
```

```
        <div className="grid grid-cols-1 md:grid-cols-4 gap-10">
```

```
          { /* Brand */ }
```

```
          <div className="md:col-span-1">
```

```
            <div className="flex items-center gap-0.5 mb-3">
```

```
              <span className="text-lg font-black tracking-tight text-black">PSG</span>
```

```
              <span className="w-1 h-1 rounded-full bg-black inline-block mx-1" />
```

```
              <span className="text-sm font-light tracking-widest text-gray-500 uppercase">SHOP</span>
```

```
            </div>
```

```
            <p className="text-sm text-gray-500 leading-relaxed">
```

```
              Tu destino para moños elegantes y accesorios de moda de alta calidad.
```

```
            </p>
```

```
          </div>
```

```
          { /* Quick Links */ }
```

```

<div>

    <h4 className="text-xs font-semibold uppercase tracking-widest text-
gray-400 mb-4">Navegación</h4>

    <ul className="space-y-2.5">

        <li><Link to="/home" className="text-sm text-gray-600 hover:text-black
transition-colors duration-150">Inicio</Link> </li>

        <li><Link to="/shop" className="text-sm text-gray-600 hover:text-black
transition-colors duration-150">Tienda</Link> </li>

        <li><Link to="/blog" className="text-sm text-gray-600 hover:text-black
transition-colors duration-150">Sobre Nosotros</Link> </li>

        <li><Link to="/contact" className="text-sm text-gray-600 hover:text-
black transition-colors duration-150">Contacto</Link> </li>

    </ul>

</div>

```

```

{ /* Account */

```

```

<div>

    <h4 className="text-xs font-semibold uppercase tracking-widest text-
gray-400 mb-4">Mi cuenta</h4>

    <ul className="space-y-2.5">

        <li><Link to="/login" className="text-sm text-gray-600 hover:text-black
transition-colors duration-150">Iniciar Sesión</Link> </li>

        <li><Link to="/register" className="text-sm text-gray-600 hover:text-
black transition-colors duration-150">Registrarse</Link> </li>

        <li><Link to="/profile" className="text-sm text-gray-600 hover:text-
black transition-colors duration-150">Mi Perfil</Link> </li>

```

<Link to="/orders" className="text-sm text-gray-600 hover:text-black transition-colors duration-150">Mis Pedidos</Link>

<Link to="/wishlist" className="text-sm text-gray-600 hover:text-black transition-colors duration-150">Lista de Deseos</Link>

</div>

{/* Contact */}

<div>

<h4 className="text-xs font-semibold uppercase tracking-widest text-gray-400 mb-4">Contacto</h4>

<address className="not-italic space-y-2">

<p className="text-sm text-gray-600">psgmoños@gmail.com</p>

<p className="text-sm text-gray-600">(123) 456-7890</p>

<p className="text-sm text-gray-600">Cajamarca, Tolima,
Colombia</p>

</address>

<div className="flex gap-3 mt-5">

<svg className="h-4 w-4" fill="currentColor" viewBox="0 0 24 24"><path fillRule="evenodd" d="M22 12c0-5.523-4.477-10-10-10S2 6.477 2 12c0 4.991 3.657 9.128 8.438 9.878v-6.987h-2.54V12h2.54V9.797c0-2.506 1.492-3.89 3.777-3.89 1.094 0 2.238.195 2.238.195v2.46h-1.26c-1.243 0-1.63.771-1.63 1.562V12h2.773l-.443 2.89h-2.33v6.988C18.343 21.128 22 16.991 22 12z" clipRule="evenodd" /></svg>


```
<a href="#" aria-label="Instagram" className="w-8 h-8 flex items-center justify-center rounded-full border border-gray-200 text-gray-500 hover:border-gray-900 hover:text-gray-900 transition-all duration-150">
```

```
<svg className="h-4 w-4" fill="currentColor" viewBox="0 0 24 24"><path fillRule="evenodd" d="M12.315 2c2.43 0 2.784.013 3.808.06 1.064.049 1.791.218 2.427.465a4.902 4.902 0 011.772 1.153 4.902 4.902 0 011.153 1.772c.247.636.416 1.363.465 2.427.048 1.067.06 1.407.06 4.123v.08c0 2.643-.012 2.987-.06 4.043-.049 1.064-.218 1.791-.465 2.427a4.902 4.902 0 01-1.153 1.772 4.902 4.902 0 01-1.772 1.153c-.636.247-1.363.416-2.427.465-1.067.048-1.407.06-4.123.06h-.08c-2.643 0-2.987-.012-4.043-.06-1.064-.049-1.791-.218-2.427-.465a4.902 4.902 0 01-1.772-1.153 4.902 4.902 0 01-1.153-1.772c-.247-.636-.416-1.363-.465-2.427-.047-1.024-.06-1.379-.06-3.808v-.63c0-2.43-.013-2.784-.06-3.808.049-1.064.218-1.791.465-2.427a4.902 4.902 0 011.153-1.772A4.902 4.902 0 015.45 2.525c.636-.247 1.363-.416 2.427-.465C8.901 2.013 9.256 2 11.685 2h.63zm-.081 1.802h-.468c-2.456 0-2.784.011-3.807.058-.975.045-1.504.207-1.857.344-.467.182-.839.115-.748-.35-.566.683-.748 1.15-.137.353-.382-.344 1.857-.047 1.023-.058 1.351-.058 3.807v.468c0 2.456.011 2.784.058 3.807.045.975.207 1.504.344 1.857.182.466.399.8748 1.15.35.35.683.566 1.15.748.353.137.882-.3 1.857-.344-1.023-.047-1.351-.058-3.807-.058zM12 6.865a5.135 5.135 0 110 10.27 5.135 5.135 0 010-10.27zm0 1.802a3.333 3.333 0 100 6.666 3.333 3.333 0 000-6.666zm5.338-3.205a1.2 1.2 0 110 2.4 1.2 1.2 0 010-2.4z" clipRule="evenodd" /></svg>
```

```
</a>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<div className="mt-10 pt-6 border-t border-gray-100 flex flex-col sm:flex-
row items-center justify-between gap-3">
```

```
<p className="text-xs text-gray-400"> © {new Date().getFullYear()} PSG
SHOP. Todos los derechos reservados.</p>
```

```
<p className="text-xs text-gray-400">Cajamarca, Tolima · Colombia</p>
```

```
</div>
```

```
</div>
```

```
</footer>
```

```
);
```

```
};
```

```
export default Footer;
```

 **src/assets/components/Loader.jsx**

```
import React from 'react';
```

```
import './Loader.css';
```

```
const Loader = ({ text = 'Cargando...', size = 'md' }) => {
```

```
  return (
```

```
    <div className="loader-container">
```

```
      <span className={`loader ${size === 'lg' ? 'loader-lg' : ''}`}></span>
```

```
      {text} && <span className="loader-text">{text}</span>
```

```
    </div>
```

```
  );
```

```
};
```

```
export default Loader;
```

src/assets/components/Login.jsx

```
import React, { useState } from 'react';  
  
import { auth, GoogleAuthProvider } from '../firebase';  
  
import { signInWithEmailAndPassword, signInWithPopup } from 'firebase/auth';  
  
import { useNavigate, Link } from 'react-router-dom';  
  
import { doc, setDoc, getDoc } from 'firebase/firestore';  
  
import { db } from '../firebase';  
  
import { useAuth } from '../context/AuthContext';
```

```
const Login = () => {  
  const [email, setEmail] = useState('');  
  const [password, setPassword] = useState('');  
  const [error, setError] = useState('');  
  const [loading, setLoading] = useState(false);  
  const navigate = useNavigate();  
  const { currentUser } = useAuth();
```

```
  React.useEffect(() => {  
    if(currentUser) navigate('/home');  
  }, [currentUser, navigate]);
```

```
  const handleGoogleSignIn = async () => {  
    setError('');
```

```

setLoading(true);

try{

  const provider = new GoogleAuthProvider();

  const result = await signInWithPopup(auth, provider);

  const user = result.user;

  const userDocRef = doc(db, 'users', user.uid);

  const userSnapshot = await getDoc(userDocRef);

  if(!userSnapshot.exists()) {

    await setDoc(userDocRef, {

      email: user.email,

      displayName: user.displayName || '',

      profileImage: user.photoURL || '',

      createdAt: new Date(),

      role: 'customer'

    });

  }

  navigate('/home');

} catch (err) {

  setError('Error al iniciar sesión con Google. Por favor, inténtalo de nuevo.');
```

finally{

```

  setLoading(false);

}

};
```



```

const handleSubmit = async (e) => {
  e.preventDefault();
  setError('');
  setLoading(true);
  try {
    await signInWithEmailAndPassword(auth, email, password);
    navigate('/home');
  } catch (err) {
    switch (err.code) {
      case 'auth/user-not-found': setError('No se encontró una cuenta con ese correo electrónico.');
```

break;

```

      case 'auth/wrong-password': setError('Contraseña incorrecta.');
```

break;

```

      case 'auth/invalid-email': setError('El correo electrónico no es válido.');
```

break;

```

      case 'auth/user-disabled': setError('Esta cuenta ha sido deshabilitada.');
```

break;

```

      default: setError('Error al iniciar sesión. Por favor, verifica tus credenciales.');
```

}

```

    } finally {
      setLoading(false);
    }
  };

```

```

const inputCls = "block w-full px-4 py-3 rounded-xl border border-gray-200
focus:outline-none focus:ring-2 focus:ring-black focus:border-transparent
transition-all duration-200 text-sm text-gray-900 placeholder-gray-400
disabled:bg-gray-50 disabled:text-gray-400";

```

```

return (

  <div className="flex min-h-screen bg-gray-50 items-center justify-center p-6"
    style={{ fontFamily: 'Inter, sans-serif' }}>

    <div className="w-full max-w-md bg-white rounded-2xl shadow-xl shadow-
      gray-200/50 p-8 md:p-12 border border-gray-100">

      {/* Logo */}

      <div className="flex flex-col items-center mb-10">

        <div className="flex items-center gap-1.5 mb-2">

          <span className="text-3xl font-black tracking-tight text-
            black">PSG</span>

          <span className="w-1.5 h-1.5 rounded-full bg-black inline-block mt-1" />

          <span className="text-base font-light tracking-widest text-gray-500
            uppercase">SHOP</span>

        </div>

        <p className="text-sm text-gray-400 font-medium">Inicia sesión en tu
          cuenta</p>

      </div>

      {error && (

        <div className="mb-6 flex gap-3 items-start p-4 rounded-xl bg-red-50
          border border-red-100 animate-in fade-in slide-in-from-top-2">

          <svg className="w-5 h-5 text-red-500 mt-0.5 flex-shrink-0"
            fill="currentColor" viewBox="0 0 20 20">

            <path fillRule="evenodd" d="M18 10a8 8 0 11-16 0 8 8 0 0116 0zm-7 4a1
              1 0 11-2 0 1 1 0 012 0zm-1-9a1 1 0 00-1 1v4a1 1 0 102 0V6a1 1 0 00-1-1z"
              clipRule="evenodd" />

          </svg>

```

```

    <p className="text-sm text-red-700 font-medium">{error}</p>

  </div>

})

{ /* Google SignIn */

  <button

    onClick={handleGoogleSignIn}

    disabled={loading}

    className="flex items-center justify-center gap-3 w-full py-3 px-4 rounded-
xl border border-gray-200 bg-white text-sm font-semibold text-gray-700 hover:bg-
gray-50 hover:border-gray-300 transition-all duration-200 disabled:opacity-50 mb-
8 active:scale-[0.98]"

    >

      <svg className="w-5 h-5" viewBox="0 0 24 24">

        <path d="M22.56 12.25c0-.78-.07-1.53-.2-2.25H12v4.26h5.92c-.26 1.37-
1.04 2.53-2.21 3.31v2.77h3.57c2.08-1.92 3.28-4.74 3.28-8.09z" fill="#4285F4" />

        <path d="M12 23c2.97 0 5.46-.98 7.28-2.66l-3.57-2.77c-.98.66-2.23 1.06-
3.71 1.06-2.86 0-5.29-1.93-6.16-4.53H2.18v2.84C3.99 20.53 7.7 23 12 23z"
fill="#34A853" />

        <path d="M5.84 14.09c-.22-.66-.35-1.36-.35-2.09s.13-1.43.35-
2.09V7.07H2.18C1.43 8.55 1 10.22 1 12s.43 3.45 1.18 4.93l2.85-2.22.81-.62z"
fill="#FBBC05" />

        <path d="M12 5.38c1.62 0 3.06.56 4.21 1.64l3.15-3.15C17.45 2.09 14.97 1
12 1 7.7 1 3.99 3.47 2.18 7.07l3.66 2.84c.87-2.6 3.3-4.53 6.16-4.53z" fill="#EA4335"
/>

      </svg>

      Continuar con Google

```

</button>

{/ Divider */}*

<div *className*="relative mb-8">

<div *className*="absolute inset-0 flex items-center"> <div *className*="w-full border-t border-gray-100" /> </div>

<div *className*="relative flex justify-center"> o con tu email </div>

</div>

{/ Login Form */}*

<form *onSubmit*={handleSubmit} *className*="space-y-5">

<div>

<label *htmlFor*="email" *className*="block text-xs font-bold text-gray-500 uppercase tracking-widest mb-2">Correo Electrónico</label>

<input *id*="email" *name*="email" *type*="email" *autoComplete*="email" *required* *placeholder*="tu@ejemplo.com" *className*={inputCls} *value*={email} *onChange*={(e) => setEmail(e.target.value)} *disabled*={loading} />

</div>

<div>

<div *className*="flex items-center justify-between mb-2">

<label *htmlFor*="password" *className*="block text-xs font-bold text-gray-500 uppercase tracking-widest">Contraseña</label>

<Link *to*="/reset-password" *name*="reset-password-link" *className*="text-xs font-semibold text-gray-400 hover:text-black transition-colors underline-offset-4 hover:underline">¿La olvidaste?</Link>

```

    </div>

    <input id="password" name="password" type="password"
autoComplete="current-password" required placeholder="....."
className={inputCls} value={password} onChange={(e) =>
setPassword(e.target.value)} disabled={loading} />

    </div>

    <div className="flex items-center gap-3 pt-1">

        <input id="remember-me" name="remember-me" type="checkbox"
className="w-4 h-4 rounded-md border-gray-300 text-black focus:ring-black
focus:ring-offset-0 cursor-pointer" />

        <label htmlFor="remember-me" className="text-sm font-medium text-
gray-600 cursor-pointer select-none">Recordarme en este equipo</label>

    </div>

    <button

        type="submit"

        disabled={loading}

        className="w-full py-3.5 px-4 bg-black text-white text-sm font-bold
rounded-xl hover:bg-gray-800 active:bg-gray-900 active:scale-[0.99] transition-all
duration-200 disabled:opacity-40 shadow-lg shadow-gray-200"

        >

        {loading ? 'Iniciando sesión...' : 'Entrar a mi cuenta'}

    </button>

</form>

<div className="mt-10 text-center">

```

```

    <p className="text-sm text-gray-500 font-medium">
      ¿Aún no tienes cuenta?{' '}
      <Link to="/register" name="register-link" className="font-bold text-black
border-b-2 border-black/10 hover:border-black transition-all pb-0.5">Regístrate
gratis</Link>
    </p>
  </div>

```

```

    <p className="mt-10 text-center text-[10px] font-bold text-gray-300
uppercase tracking-[0.2em]">

```

```

    © {new Date().getFullYear()} PSG SHOP · CALIDAD PREMIUM

```

```

    </p>

```

```

  </div>

```

```

</div>

```

```

);

```

```

};

```

```

export default Login;

```

```

📁 src/assets/components/Navbar.jsx

```

```

import React, { useState, useEffect, useRef } from 'react';

```

```

import { Link, useNavigate, useLocation } from 'react-router-dom';

```

```

import { useAuth } from '../context/AuthContext';

```

```

import { useCart } from '../context/CartContext';

```

```

import { useWishlist } from '../context/WishlistContext';

```

```

import { signOut } from 'firebase/auth';

```

```

import { auth } from '../firebase';

```

```

import {
  FiUser, FiSettings, FiList, FiLogOut, FiMenu, FiX, FiHome,
  FiShoppingBag, FiInfo, FiMail, FiChevronRight, FiGrid
} from "react-icons/fi";

import { FaRegHeart } from "react-icons/fa6";

const Navbar = () => {
  const [isMenuOpen, setIsMenuOpen] = useState(false);
  const [isProfileMenuOpen, setIsProfileMenuOpen] = useState(false);
  const [scrolled, setScrolled] = useState(false);
  const { currentUser, isAdmin, userProfile } = useAuth();
  const { items } = useCart();
  const { items: wishlistItems } = useWishlist();
  const navigate = useNavigate();
  const location = useLocation();
  const profileMenuRef = useRef(null);

  useEffect(() => {
    const handleScroll = () => setScrolled(window.scrollY > 10);
    window.addEventListener('scroll', handleScroll);
    return () => window.removeEventListener('scroll', handleScroll);
  }, []);

  useEffect(() => {

```

```

const handleClickOutside = (event) => {
  if(profileMenuRef.current && !profileMenuRef.current.contains(event.target)) {
    setIsProfileMenuOpen(false);
  }
};

document.addEventListener('mousedown', handleClickOutside);

return () => document.removeEventListener('mousedown', handleClickOutside);
}, []);

```

```

const toggleMenu = () => setIsMenuOpen(v => !v);
const toggleProfileMenu = () => setIsProfileMenuOpen(v => !v);
const closeProfileMenu = () => setIsProfileMenuOpen(false);

```

```

const handleLogout = async () => {
  try{
    await signOut(auth);
    closeProfileMenu();
    navigate('/home');
  } catch (error) {
    console.error('Error signing out:', error);
  }
};

```

```

const cartItemCount = items.reduce((total, item) => total + item.quantity, 0);

```



```

const wishlistItemCount = wishlistItems.length;

const isActiveLink = (path) => location.pathname === path;

const navLinks = [
  { to: '/home', label: 'Inicio', icon: <FiHome /> },
  { to: '/shop', label: 'Tienda', icon: <FiShoppingBag /> },
  { to: '/blog', label: 'Sobre Nosotros', icon: <FiInfo /> },
  { to: '/contact', label: 'Contacto', icon: <FiMail /> },
];

return (
  <>
    <header
      className={`fixed top-0 left-0 right-0 z-[100] transition-all duration-300 ${
        scrolled
          ? 'py-2'
          : 'py-4'
        }`}
    >
      <nav className={`mx-auto px-4 sm:px-6 lg:px-8 transition-all duration-300 ${
        scrolled
          ? 'max-w-5xl'
          : 'max-w-7xl'
        }`} >

```

```

    <div className={`flex items-center justify-between transition-all duration-
300 px-6 rounded-2xl border ${
    scrolled
    ? 'bg-white/80 backdrop-blur-md border-slate-200/50 shadow-lg h-14'
    : 'bg-white border-transparent h-16'
}}>

```

```

    {/* Logo & Mobile Trigger */
    <div className="flex items-center gap-4">
      <button
        className="lg:hidden p-2 text-slate-500 hover:text-black transition-
colors"
        onClick={toggleMenu}
      >
        <FiMenu size={20} />
      </button>

```

```

    <Link to="/home" className="flex items-center gap-2 group select-
none">
      <div className="w-7 h-7 bg-black rounded-lg flex items-center justify-
center text-white shadow-lg transition-transform group-hover:scale-105
active:scale-95">
        <span className="font-bold italic text-[10px]">P</span>
      </div>
      <div className="hidden sm:flex items-center gap-1">

```

```

      <span className="text-sm font-black tracking-tighter text-black">PSG</span>

      <span className="w-1 h-1 rounded-full bg-indigo-500" />

      <span className="text-[10px] font-medium tracking-[0.2em] text-slate-400 uppercase">SHOP</span>

    </div>

  </Link>

</div>

{ /* Desktop Nav */ }

<ul className="hidden lg:flex items-center bg-slate-50/50 border border-slate-100 p-1 rounded-full">

  {navLinks.map(({ to, label }) => (

    <li key={to}>

      <Link

        to={to}

        className={`px-5 py-2 rounded-full text-[11px] font-bold tracking-tight transition-all duration-200 ${

          isActiveLink(to)

            ? 'bg-black text-white shadow-md'

            : 'text-slate-500 hover:text-black hover:bg-slate-100'

        }`}

      >

        {label}

      </Link>

    </li>

```

```

    )}
  </ul>

  {/* Action Icons */}

  <div className="flex items-center gap-1 sm:gap-2">

    {/* Wishlist */}

    <Link

      to="/wishlist"

      className="p-2 rounded-xl transition-all duration-200 relative group
text-slate-600 hover:text-black hover:bg-slate-50"

    >

      <FaRegHeart size={18} strokeWidth={1.5} />

      {wishlistItemCount > 0 && (

        <span className="absolute -top-1 -right-1 flex items-center justify-
center w-4 h-4 bg-black text-white text-[9px] font-bold rounded-full shadow-sm
animate-in zoom-in-50 duration-300">

          {wishlistItemCount}

        </span>

      )}

    </Link>

    {/* Cart */}

    <Link

      to="/cart"

      className="p-2 rounded-xl transition-all duration-200 relative group
text-slate-600 hover:text-black hover:bg-slate-50"

```

```

>

<FiShoppingBag size={18} strokeWidth={2} />

{cartItemCount > 0 && (

  <span className="absolute -top-1 -right-1 flex items-center justify-
center w-4 h-4 bg-indigo-600 text-white text-[9px] font-bold rounded-full shadow-
sm animate-in zoom-in-50 duration-300">

    {cartItemCount}

  </span>

)}

</Link>


<div className="w-px h-6 bg-slate-200 mx-1 hidden sm:block" />


{/* User Profile / Login */}

<div className="relative" ref={profileMenuRef}>

  {currentUser ? (

    <button

      onClick={toggleProfileMenu}

      className="flex items-center gap-2 p-1 pl-1 pr-3 bg-slate-50
rounded-full hover:bg-slate-100 transition-all border border-transparent
hover:border-slate-200 active:scale-95"

    >

      <div className="w-8 h-8 bg-indigo-600 rounded-full flex items-
center justify-center font-bold text-white text-[10px] shadow-sm overflow-
hidden">

        {userProfile?.profileImage ? (

```

```
        <img src={userProfile.profileImage} alt="Profile" className="w-full
h-full object-cover" />
```

```
    ) : (
```

```
        <span>{userProfile?.displayName?.charAt(0).toUpperCase() ||
currentUser.email?.charAt(0).toUpperCase()}</span>
```

```
    )}
```

```
</div>
```

```
    <p className="hidden sm:block text-[11px] font-bold text-slate-700
tracking-tight max-w-[80px] truncate">
```

```
        {userProfile?.displayName?.split(' ')[0] ||
currentUser.email?.split('@')[0]}
```

```
</p>
```

```
</button>
```

```
) : (
```

```
<Link
```

```
    to="/login"
```

```
    className="px-5 py-2.5 bg-black text-white rounded-full text-[11px]
font-bold tracking-tight transition-all hover:bg-indigo-600 shadow-sm active:scale-
95"
```

```
>
```

```
    Entrar
```

```
</Link>
```

```
)}
```

```
{/* Profile Dropdown */}
```

```
{currentUser && isProfileMenuOpen && (
```

```

    <div className="absolute right-0 mt-3 w-56 bg-white rounded-2xl
border border-slate-100 shadow-xl overflow-hidden z-[150] animate-in fade-in
slide-in-from-top-2 duration-200">

      <div className="p-4 bg-slate-50 border-b border-slate-100">

        <p className="text-[11px] font-bold text-slate-900 truncate
tracking-tight">{userProfile?.displayName || currentUser.email?.split('@')[0]}</p>

        <p className="text-[9px] font-medium text-slate-400 truncate
tracking-wider mt-0.5">{currentUser.email}</p>

      </div>

      <div className="p-1">

        {isAdmin && (

          <Link to="/admin" onClick={closeProfileMenu} className="flex
items-center gap-2.5 px-3 py-2 rounded-xl text-[11px] font-bold text-slate-600
hover:bg-black hover:text-white transition-all group">

            <FiGrid className="text-indigo-500 group-hover:text-white"
size={13} />

            <span>Panel Admin</span>

          </Link>

        )}

        <Link to="/profile" onClick={closeProfileMenu} className="flex
items-center gap-2.5 px-3 py-2 rounded-xl text-[11px] font-bold text-slate-600
hover:bg-black hover:text-white transition-all group">

          <FiUser className="text-indigo-500 group-hover:text-white"
size={13} />

          <span>Mi Perfil</span>

        </Link>

```

```

        <Link to="/orders" onClick={closeProfileMenu} className="flex
items-center gap-2.5 px-3 py-2 rounded-xl text-[11px] font-bold text-slate-600
hover:bg-black hover:text-white transition-all group">

        <FiList className="text-indigo-500 group-hover:text-white"
size={13} />

        <span>Pedidos</span>

    </Link>

    <div className="h-[1px] bg-slate-100 my-1 mx-2" />

    <button onClick={handleLogout} className="flex items-center gap-
2.5 w-full px-3 py-2 rounded-xl text-[11px] font-bold text-rose-500 hover:bg-rose-
50 transition-all">

        <FiLogOut size={13} />

        <span>Salir</span>

    </button>

</div>

</div>

)}

</div>

</div>

</div>

</nav>

</header>

```

```

    {/* Spacing for fixed header */}

```

```

    <div className={`transition-all duration-300 ${scrolled ? 'h-16' : 'h-
24'}}`></div>

```



```

{ /* Mobile Drawer Overlay */

{isMenuOpen && (

  <div className="fixed inset-0 z-[140] bg-black/40 backdrop-blur-sm
animate-in fade-in duration-300" onClick={toggleMenu} />

)}}

{ /* Mobile Sidebar */

<aside

  className={`fixed top-0 left-0 h-full w-72 bg-white z-[150] shadow-2xl
transform transition-transform duration-300 ease-out flex flex-col ${

    isMenuOpen ? 'translate-x-0' : '-translate-x-full'

  }}

  >

    <div className="p-6 flex items-center justify-between border-b border-slate-
50">

      <Link to="/home" onClick={toggleMenu} className="flex items-center gap-
2">

        <div className="w-7 h-7 bg-black rounded-lg flex items-center justify-
center text-white font-bold italic text-[10px]">P</div>

        <span className="text-sm font-black tracking-tighter">PSG
SHOP</span>

      </Link>

      <button onClick={toggleMenu} className="p-2 text-slate-400 hover:text-
black transition-colors">

        <FiX size={20} />

      </button>

```

</div>

<nav className="flex-1 overflow-auto p-4">

{/* Mobile Quick Actions */}

<div className="flex gap-2 mb-6">

<Link to="/wishlist" onClick={toggleMenu} className="flex-1 flex items-center justify-center gap-2 py-3 bg-slate-50 rounded-xl relative">

<FaRegHeart size={16} className="text-slate-600" />

Favoritos

{wishlistItemCount > 0 && (

{wishlistItemCount}

)}

</Link>

<Link to="/cart" onClick={toggleMenu} className="flex-1 flex items-center justify-center gap-2 py-3 bg-slate-50 rounded-xl relative">

<FiShoppingBag size={16} className="text-slate-600" />

Carrito

{cartItemCount > 0 && (

{cartItemCount}

```

    </span>
  )}
</Link>
</div>

```

```

    <span className="block text-[9px] font-bold text-slate-300 uppercase
tracking-[0.2em] mb-4 px-2">Menu</span>

    <ul className="space-y-1">
      {navLinks.map(({ to, label, icon }) => (
        <li key={to}>
          <Link
            to={to}
            onClick={toggleMenu}
            className={`flex items-center gap-3 px-4 py-3 rounded-xl text-[11px]
font-bold tracking-tight transition-all ${
              isActiveLink(to)
                ? 'bg-black text-white shadow-lg'
                : 'text-slate-600 hover:bg-slate-50'
            }}
          >
            <span className={isActiveLink(to) ? 'text-indigo-400' : 'text-slate-
400'}>{icon}</span>
            <span>{label}</span>
          </Link>
        </li>

```

```

    )}
  </ul>
</nav>

<div className="p-6 border-t border-slate-50">
  {currentUser ? (
    <div className="space-y-3">
      <div className="p-3 bg-slate-50 rounded-2xl flex items-center gap-
3">
        <div className="w-9 h-9 bg-indigo-600 rounded-full flex items-
center justify-center text-white font-bold text-[10px]">
          {currentUser.email?.charAt(0).toUpperCase()}
        </div>
        <div className="min-w-0">
          <p className="text-[11px] font-bold text-slate-900 truncate
tracking-tight">{userProfile?.displayName || currentUser.email?.split('@')[0]}</p>
          <p className="text-[9px] font-medium text-slate-400 truncate
tracking-wider">Cliente</p>
        </div>
      </div>
    </div>
    {isAdmin && (
      <Link to="/admin" onClick={toggleMenu} className="block w-full
py-3 bg-indigo-600 text-white rounded-xl text-[10px] font-bold tracking-widest
uppercase text-center shadow-md">
        Panel Admin
      </Link>
    )}
  )}

```

```

    })

    <button onClick={() => { handleLogout(); setIsMenuOpen(false); }}
className="w-full py-3 text-rose-500 font-bold text-[11px] tracking-tight
hover:bg-rose-50 rounded-xl transition-all">

        Terminar Sesión

    </button>

</div>

): (

    <Link to="/login" onClick={toggleMenu} className="block w-full py-4
bg-black text-white rounded-xl text-[11px] font-bold tracking-widest uppercase
text-center shadow-lg">

        Iniciar Sesión

    </Link>

    })

</div>

</aside>

</>

);

};

export default Navbar;

```

 **src/assets/components/OrderDetailModal.jsx**

```

const OrderDetailsModal = ({ order, isOpen, onClose, formatDate, formatCurrency,
getOrderStatusText, getStatusBadgeClass }) => {

    const modalRef = useRef(null);

    useEffect(() => {

```

```

    if(isOpen && modalRef.current) modalRef.current.scrollTop = 0;
  }, [isOpen]);

```

```

useEffect(() => {
  if(!isOpen) return;

  const onKey = (e) => { if(e.key === 'Escape') onClose(); };

  document.addEventListener('keydown', onKey);

  return () => document.removeEventListener('keydown', onKey);
}, [isOpen, onClose]);

```

```

if(!isOpen || !order) return null;

```

```

const Section = ({ title, children }) => (
  <div className="bg-gray-50 rounded-xl p-4">
    <p className="text-[10px] font-semibold uppercase tracking-widest text-gray-400 mb-3">{title}</p>
    {children}
  </div>
);

```

```

const InfoRow = ({ label, value, className = '' }) => (
  <div>
    <p className="text-xs text-gray-400 mb-0.5">{label}</p>
    <p className={`text-sm font-medium text-gray-900 ${className}`}>{value}</p>
  </div>
);

```

```

    </div>

    );

    return (

    <div

        className="fixed inset-0 z-[250] flex items-start justify-center p-3 sm:p-6
        overflow-y-auto bg-black/70 backdrop-blur-md animate-in fade-in duration-300"

        onClick={{(e) => { if(e.target === e.currentTarget) onClose(); }}}

    >

    <div

        ref={modalRef}

        className="relative w-full max-w-2xl mx-auto my-4 bg-white rounded-
        [2.5rem] shadow-2xl border border-gray-100 overflow-hidden animate-in zoom-in-
        95 duration-300"

        style={{ fontFamily: 'Inter, sans-serif' }}

    >

        {/* Header */}

        <div className="flex items-center justify-between px-6 py-5 border-b
        border-gray-100">

            <div>

                <h3 className="text-base font-bold text-gray-900">Detalles del
                Pedido</h3>

                <p className="text-xs text-gray-400 mt-0.5">#{order.id.substring(0,
                8).toUpperCase()}</p>

            </div>

            <button

```

```

onClick={onClose}

className="w-8 h-8 flex items-center justify-center rounded-full text-gray-
400 hover:bg-gray-100 hover:text-gray-600 transition-colors"

>

<svg className="w-4 h-4" fill="none" viewBox="0 0 24 24"
stroke="currentColor">

  <path strokeLinecap="round" strokeLinejoin="round" strokeWidth="2"
d="M6 18L18 6M6 6l12 12" />

</svg>

</button>

</div>

```

```

{/* Body */}

<div className="px-6 py-5 space-y-4 max-h-[75vh] overflow-y-auto">

{/* Order Info */}

<Section title="Información del Pedido">

  <div className="grid grid-cols-2 sm:grid-cols-4 gap-3">

    <InfoRow label="Fecha" value={formatDate(order.createdAt)} />

    <div>

      <p className="text-xs text-gray-400 mb-1">Estado</p>

      <span className={`inline-flex items-center px-2.5 py-1 rounded-full
text-xs font-medium ${getStatusBadgeClass(order.orderStatus || 'pending')}}>

        {getOrderStatusText(order.orderStatus || 'pending')}

      </span>

    </div>

  </div>

```



```

        <InfoRow label="Método de Pago" value={order.paymentMethod || 'N/A'}
/>

        <InfoRow label="Estado Pago" value={order.paymentStatus || 'N/A'}
className="capitalize" />

    </div>

</Section>

{/* Customer */}

<Section title="Cliente">

    <div className="grid grid-cols-1 sm:grid-cols-2 gap-3">

        <InfoRow label="Correo" value={order.userEmail || 'N/A'} />

        <InfoRow label="ID Usuario" value={order.userId?.substring(0, 8) || 'N/A'}
/>

    </div>

</Section>

{/* Shipping Address */}

{order.address && (

    <Section title="Dirección de Envío">

        <div className="text-sm space-y-0.5">

            <p className="font-medium text-gray-900">{order.address.name}</p>

            <p className="text-gray-500">{order.address.street}</p>

            <p className="text-gray-500">{order.address.city}, {order.address.state}
{order.address.zipCode}</p>

            <p className="text-gray-500">{order.address.country}</p>

        </div>

```

```

</Section>

)}}

{ /* Coupon */
{order.coupon && (
  <div className="bg-green-50 border border-green-200 rounded-xl p-4">
    <p className="text-[10px] font-semibold uppercase tracking-widest text-
green-600 mb-3">Cupón aplicado</p>
    <div className="grid grid-cols-2 gap-3">
      <InfoRow label="Código" value={order.coupon.code} />
      <div>
        <p className="text-xs text-gray-400 mb-0.5">Descuento</p>
        <p className="text-sm font-semibold text-green-700">
          {order.coupon.discountType === 'percentage'
            ? `${order.coupon.discountValue}%`
            : formatCurrency(order.coupon.discountValue)}
        </p>
      </div>
    </div>
  </div>
)}

```

```

{ /* Products */
<Section title="Productos">
  <div className="space-y-2">

```

```

    {order.items?.map((item, index) => (
      <div key={index} className="flex items-center gap-3 p-3 bg-white
border border-gray-100 rounded-xl hover:border-gray-200 transition-colors">
        <div className="flex-shrink-0 w-12 h-12 rounded-lg overflow-hidden
border border-gray-200">
          <img
            src={item.imageUrl ||
'https://via.placeholder.com/48x48.png?text=PSG'}
            alt={item.name}
            className="w-full h-full object-cover"
            onError={(e) => { e.target.onerror = null; e.target.src =
'https://via.placeholder.com/48x48.png?text=PSG'; }}
          />
        </div>
        <div className="flex-1 min-w-0">
          <div className="flex items-start justify-between gap-2">
            <p className="text-sm font-medium text-gray-900
truncate">{item.name}</p>
            <p className="text-sm font-semibold text-gray-900 flex-shrink-
0">{formatCurrency(item.price * item.quantity)}</p>
          </div>
          <p className="text-xs text-gray-400 mt-0.5">Cant: {item.quantity} ·
Unidad: {formatCurrency(item.price)}</p>
        </div>
      </div>
    )))
  </div>

```

```

</Section>

{ /* Summary */ }

<Section title="Resumen">

  <div className="space-y-2">

    <div className="flex items-center justify-between text-sm">

      <span className="text-gray-500">Subtotal</span>

      <span className="font-medium text-gray-900">{formatCurrency(order.subtotal || 0)}</span>

    </div>

    {order.coupon && (

      <div className="flex items-center justify-between text-sm">

        <span className="text-gray-500">Descuento</span>

        <span className="font-medium text-green-600">-{formatCurrency(order.discount || 0)}</span>

      </div>

    )}

    <div className="flex items-center justify-between text-sm">

      <span className="text-gray-500">Envío</span>

      <span className="font-medium text-gray-900">Incluido</span>

    </div>

    <div className="flex items-center justify-between pt-3 border-t border-gray-200">

      <span className="text-sm font-semibold text-gray-900">Total</span>

      <span className="text-base font-bold text-gray-900">{formatCurrency(order.totalAmount || 0)}</span>

    </div>

  </div>

</Section>

```

```

    </div>

  </div>

</Section>

</div>

  {/* Footer */}

  <div className="px-6 py-4 border-t border-gray-100 flex justify-end">

    <button

      type="button"

      onClick={onClose}

      className="px-5 py-2.5 text-sm font-medium text-white bg-black
rounded-xl hover:bg-gray-800 transition-colors focus:outline-none focus:ring-2
focus:ring-offset-2 focus:ring-black"

      >

        Cerrar

    </button>

  </div>

</div>

</div>

);

};

export default OrderDetailsModal;

src/assets/components/ProductsReview.jsx

import React, { useState, useEffect } from 'react';

import { useAuth } from '../context/AuthContext';

```

```

import { getReviewsByProductId } from '../services/reviewService';

import StarRating from './StarRating';

import Swal from 'sweetalert2';

const ProductReviews = ({ productId }) => {

  const { currentUser } = useAuth();

  const [reviews, setReviews] = useState([]);

  const [loading, setLoading] = useState(true);

  const [reviewForm, setReviewForm] = useState({ rating: 0, comment: '' });

  const [submitting, setSubmitting] = useState(false);

  useEffect(() => {

    const fetchReviews = async () => {

      try {

        setLoading(true);

        const data = await getReviewsByProductId(productId);

        setReviews(data);

      } catch (err) {

        console.error('Error fetching reviews:', err);

      } finally {

        setLoading(false);

      }

    };

    if(productId) fetchReviews();
  }

```

```
}, [productId]);
```

```
const handleReviewChange = (e) => {  
  const { name, value } = e.target;  
  setReviewForm(prev => ({ ...prev, [name]: value }));  
};
```

```
const handleRatingClick = (rating) => setReviewForm(prev => ({ ...prev, rating }));
```

```
const handleSubmitReview = async (e) => {  
  e.preventDefault();  
  if(!currentUser) {  
    Swal.fire({ title: 'Inicia sesión', text: 'Debes iniciar sesión para dejar una reseña',  
      icon: 'warning', confirmButtonText: 'Iniciar sesión', showCancelButton: true,  
      cancelButtonText: 'Cancelar' });  
    return;  
  }  
  if(reviewForm.rating === 0) {  
    Swal.fire({ title: 'Error', text: 'Por favor selecciona una calificación', icon: 'error',  
      confirmButtonText: 'Aceptar' });  
    return;  
  }  
  if(reviewForm.comment.trim() === '') {  
    Swal.fire({ title: 'Error', text: 'Por favor escribe un comentario', icon: 'error',  
      confirmButtonText: 'Aceptar' });  
    return;  
  }  
}
```

```

    }

    setSubmitting(true);

    try{

      // Note: review submission handled externally; reset form here

      setReviewForm({ rating: 0, comment: '' });

      Swal.fire({ title: 'Reseña enviada', text: 'Tu reseña ha sido enviada correctamente', icon: 'success', confirmButtonText: 'Aceptar' });

    } catch (error) {

      Swal.fire({ title: 'Error', text: 'Hubo un error al enviar tu reseña. Por favor intenta nuevamente.', icon: 'error', confirmButtonText: 'Aceptar' });

    } finally{

      setSubmitting(false);

    }

  };

```

```

const formatDate = (date) => {

  if(!date) return 'N/A';

  if(date instanceof Date) return date.toLocaleDateString('es-CO');

  if(date.toDate) return date.toDate().toLocaleDateString('es-CO');

  return 'N/A';

};

```

```

const avgRating = reviews.length > 0 ? (reviews.reduce((s, r) => s + r.rating, 0) /
reviews.length).toFixed(1) : 0;

```



```

if (loading) {
  return (
    <div className="flex justify-center py-10">
      <div className="w-7 h-7 border-2 border-gray-900 border-t-transparent
rounded-full animate-spin" />
    </div>
  );
}

return (
  <div className="mt-10" style={{ fontFamily: 'Inter, sans-serif' }}>
    {/* Header */}
    <div className="flex items-center justify-between mb-6">
      <div>
        <h3 className="text-lg font-bold text-gray-900">Reseñas de
Clientes</h3>
        {reviews.length > 0 && (
          <p className="text-sm text-gray-500 mt-0.5">{reviews.length}
reseña{reviews.length !== 1 ? 's' : ''} · Promedio {avgRating}★</p>
        )}
      </div>
    </div>
    {reviews.length === 0 ? (
      <div className="py-10 text-center bg-gray-50 rounded-2xl border border-
gray-100">

```

```

    <svg className="w-10 h-10 text-gray-300 mx-auto mb-3" fill="none"
viewBox="0 0 24 24" stroke="currentColor">

      <path strokeLinecap="round" strokeLinejoin="round" strokeWidth="1.5"
d="M8 12h.01M12 12h.01M16 12h.01M21 12c0 4.418-4.03 8-9 8a9.863 9.863 0 01-
4.255-.949L3 20l1.395-3.72C3.512 15.042 3 13.574 3 12c0-4.418 4.03-8 9-8s9 3.582
9 8z" />

    </svg>

    <p className="text-sm font-medium text-gray-900">Sin reseñas aún</p>

    <p className="text-sm text-gray-400 mt-0.5">Sé el primero en dejar una
reseña</p>

  </div>

) : (

  <div className="space-y-3">

    {reviews.map((review) => (

      <div key={review.id} className="bg-white border border-gray-200
rounded-2xl p-5 hover:border-gray-300 transition-colors">

        <div className="flex items-start justify-between gap-4">

          <div className="flex items-center gap-3">

            <div className="flex items-center justify-center w-9 h-9 rounded-full
bg-gray-900 text-white text-xs font-bold flex-shrink-0">

              {review.userName?.charAt(0).toUpperCase() || 'U'}

            </div>

            <div>

              <p className="text-sm font-semibold text-gray-
900">{review.userName}</p>

              <StarRating rating={review.rating} size="sm" />

            </div>

          </div>

        </div>

      )
    )}

  </div>

```

```

        </div>

        <p className="text-xs text-gray-400 flex-shrink-0">{formatDate(review.createdAt)}</p>

    </div>

    <p className="mt-3 text-sm text-gray-600 leading-relaxed pl-12">{review.comment}</p>

</div>

    )}

</div>

})

</div>

);

};

export default ProductReviews;

```

 **src/assets/components/Register.jsx**

```

import React, { useState } from 'react';

import { useNavigate, Link } from 'react-router-dom';

import { createUserWithEmailAndPassword, GoogleAuthProvider, signInWithPopup } from 'firebase/auth';

import { auth, db } from '../firebase';

import { doc, setDoc, getDoc } from 'firebase/firestore';

import { useAuth } from '../context/AuthContext';

const Register = () => {

    const [email, setEmail] = useState("");

```

```

const [password, setPassword] = useState('');
const [confirmPassword, setConfirmPassword] = useState('');
const [error, setError] = useState('');
const [message, setMessage] = useState('');
const [loading, setLoading] = useState(false);
const navigate = useNavigate();
const { firebaseError } = useAuth();

const handleGoogleSignUp = async () => {
  setError(''); setMessage(''); setLoading(true);
  try {
    const provider = new GoogleAuthProvider();
    const result = await signInWithPopup(auth, provider);
    const user = result.user;
    const userDocRef = doc(db, 'users', user.uid);
    const userSnapshot = await getDoc(userDocRef);
    if (!userSnapshot.exists()) {
      await setDoc(userDocRef, { email: user.email, displayName: user.displayName ||
'', profileImage: user.photoURL || '', createdAt: new Date(), role: 'customer' });
    }
    navigate('/home');
  } catch (err) {
    setError('Error al registrarse con Google. Por favor, inténtalo de nuevo.');
```

} *finally* { setLoading(false); }
 };

```

const handleSubmit = async (e) => {
  e.preventDefault();
  setError(''); setMessage('');
  if(firebaseError) { setError(firebaseError); return; }
  if(!auth || !db) { setError('Firebase no está configurado correctamente. '); return; }
  if(password !== confirmPassword) { setError('Las contraseñas no coinciden');
return; }

  if(password.length < 6) { setError('La contraseña debe tener al menos 6
caracteres'); return; }

  setLoading(true);

  try{
    const userCredential = await createUserWithEmailAndPassword(auth, email,
password);

    const user = userCredential.user;

    await setDoc(doc(db, 'users', user.uid), { email: user.email, displayName: '',
profileImage: '', createdAt: new Date(), role: 'customer' });

    setMessage('Cuenta creada exitosamente. Redirigiendo...');

    setTimeout(() => navigate('/home'), 2000);
  } catch (err) {
    switch (err.code) {

      case 'auth/email-already-in-use': setError('Este correo electrónico ya está
registrado. '); break;

      case 'auth/invalid-email': setError('El correo electrónico no es válido. '); break;

      case 'auth/operation-not-allowed': setError('El registro está deshabilitado.
Habilita la autenticación en Firebase. '); break;

```

```

    case 'auth/weak-password': setError('La contraseña es muy débil.');
```

break;

```

    default: setError(`Error al crear la cuenta: ${err.message}`);
  }
} finally { setLoading(false); }
};
```

```

const inputCls = "block w-full px-4 py-3 text-sm text-gray-900 bg-white border
border-gray-200 rounded-xl focus:outline-none focus:ring-2 focus:ring-black
focus:border-transparent transition-all duration-200 placeholder-gray-400
disabled:bg-gray-50 disabled:text-gray-400";
```

```

return (
  <div className="flex min-h-screen bg-gray-50 items-center justify-center p-6"
  style={{ fontFamily: 'Inter, sans-serif' }}>
    <div className="w-full max-w-md bg-white rounded-2xl shadow-xl shadow-
gray-200/50 p-8 md:p-12 border border-gray-100">
      { /* Logo */ }
      <div className="flex flex-col items-center mb-10">
        <div className="flex items-center gap-1.5 mb-2">
          <span className="text-3xl font-black tracking-tight text-
black">PSG</span>
          <span className="w-1.5 h-1.5 rounded-full bg-black inline-block mt-1" />
          <span className="text-base font-light tracking-widest text-gray-500
uppercase">SHOP</span>
        </div>
        <p className="text-sm text-gray-400 font-medium">Crea tu cuenta
ahora</p>

```

```

</div>

{ /* Alerts */
  {firebaseError && (
    <div className="mb-6 p-4 rounded-xl bg-yellow-50 border border-yellow-
100 text-sm text-yellow-700 font-medium leading-relaxed">
      <div className="flex gap-2 mb-1 uppercase text-[10px] font-bold
tracking-widest text-yellow-600">Configuración incompleta</div>
      {firebaseError}
    </div>
  )}
  {error && (
    <div className="mb-6 flex gap-3 items-start p-4 rounded-xl bg-red-50
border border-red-100 animate-in fade-in slide-in-from-top-2">
      <svg className="w-5 h-5 text-red-500 mt-0.5 flex-shrink-0"
fill="currentColor" viewBox="0 0 20 20"> <path fillRule="evenodd" d="M18 10a8 8
0 11-16 0 8 8 0 0116 0zm-7 4a1 1 0 11-2 0 1 1 0 012 0zm-1-9a1 1 0 00-1 1v4a1 1 0
102 0V6a1 1 0 00-1-1z" clipRule="evenodd" /> </svg>
      <p className="text-sm text-red-700 font-medium">{error}</p>
    </div>
  )}
  {message && (
    <div className="mb-6 flex gap-3 items-start p-4 rounded-xl bg-green-50
border border-green-100 animate-in fade-in">
      <svg className="w-5 h-5 text-green-500 mt-0.5 flex-shrink-0"
fill="currentColor" viewBox="0 0 20 20"> <path fillRule="evenodd" d="M10 18a8 8

```

```
0 100-16 8 8 0 000 16zm3.707-9.293a1 1 0 00-1.414-1.414L9 10.586 7.707 9.293a1
1 0 00-1.414 1.414l2 2a1 1 0 001.414 0l4-4z" clipRule="evenodd" /> </svg>
```

```
<p className="text-sm text-green-700 font-medium">{message}</p>

</div>

}}
```

```
{/* Google SignUp */}
```

```
<button

  onClick={handleGoogleSignUp}

  disabled={loading || !!firebaseError}

  className="flex items-center justify-center gap-3 w-full py-3 px-4 rounded-
xl border border-gray-200 bg-white text-sm font-semibold text-gray-700 hover:bg-
gray-50 hover:border-gray-300 transition-all duration-200 disabled:opacity-50 mb-
8 active:scale-[0.98]"
```

```
>

  <svg className="w-5 h-5" viewBox="0 0 24 24">

    <path d="M22.56 12.25c0-.78-.07-1.53-.2-2.25H12v4.26h5.92c-.26 1.37-
1.04 2.53-2.21 3.31v2.77h3.57c2.08-1.92 3.28-4.74 3.28-8.09z" fill="#4285F4" />

    <path d="M12 23c2.97 0 5.46-.98 7.28-2.66l-3.57-2.77c-.98.66-2.23 1.06-
3.71 1.06-2.86 0-5.29-1.93-6.16-4.53H2.18v2.84C3.99 20.53 7.7 23 12 23z"
fill="#34A853" />

    <path d="M5.84 14.09c-.22-.66-.35-1.36-.35-2.09s.13-1.43.35-
2.09V7.07H2.18C1.43 8.55 1 10.22 1 12s.43 3.45 1.18 4.93l2.85-2.22.81-.62z"
fill="#FBBC05" />

    <path d="M12 5.38c1.62 0 3.06.56 4.21 1.64l3.15-3.15C17.45 2.09 14.97 1
12 1 7.7 1 3.99 3.47 2.18 7.07l3.66 2.84c.87-2.6 3.3-4.53 6.16-4.53z" fill="#EA4335"
/>

  </svg>
```


Continuar con Google

```
</button>
```

```
{/* Divider */}
```

```
<div className="relative mb-8">
```

```
  <div className="absolute inset-0 flex items-center"> <div className="w-full border-t border-gray-100" /> </div>
```

```
  <div className="relative flex justify-center"> <span className="px-4 text-xs font-semibold text-gray-400 bg-white uppercase tracking-widest">o con tu email</span> </div>
```

```
</div>
```

```
{/* Form */}
```

```
<form onSubmit={handleSubmit} className="space-y-5">
```

```
  <div>
```

```
    <label htmlFor="email" className="block text-xs font-bold text-gray-500 uppercase tracking-widest mb-2">Correo Electrónico</label>
```

```
    <input id="email" name="email" type="email" autoComplete="email" required placeholder="tu@ejemplo.com" className={inputCls} value={email} onChange={(e) => setEmail(e.target.value)} disabled={!firebaseError || loading} />
```

```
  </div>
```

```
  <div>
```

```
    <label htmlFor="password" className="block text-xs font-bold text-gray-500 uppercase tracking-widest mb-2">Contraseña</label>
```

```
    <input id="password" name="password" type="password" autoComplete="new-password" required placeholder="Mínimo 6 caracteres"
```

```

    className={inputCls} value={password} onChange={(e) =>
    setPassword(e.target.value)} disabled={!firebaseError || loading} />

    </div>

    <div>

      <label htmlFor="confirm-password" className="block text-xs font-bold
      text-gray-500 uppercase tracking-widest mb-2">Confirmar contraseña</label>

      <input id="confirm-password" name="confirm-password"
      type="password" autoComplete="new-password" required placeholder="Repite tu
      contraseña" className={inputCls} value={confirmPassword} onChange={(e) =>
      setConfirmPassword(e.target.value)} disabled={!firebaseError || loading} />

    </div>

    <button

      type="submit"

      disabled={loading || !firebaseError}

      className="w-full py-3.5 px-4 bg-black text-white text-sm font-bold
      rounded-xl hover:bg-gray-800 active:bg-gray-900 active:scale-[0.99] transition-all
      duration-200 disabled:opacity-40 shadow-lg shadow-gray-200"

      >

      {loading ? 'Creando cuenta...' : 'Crear mi cuenta gratis'}

    </button>

  </form>

  <div className="mt-10 text-center">

    <p className="text-sm text-gray-500 font-medium">

      ¿Ya tienes cuenta?{' '}

```

```
      <Link to="/login" name="login-link" className="font-bold text-black
border-b-2 border-black/10 hover:border-black transition-all pb-0.5">Inicia
sesión</Link>
```

```
    </p>
```

```
  </div>
```

```
    <p className="mt-10 text-center text-[10px] font-bold text-gray-300
uppercase tracking-[0.2em]">
```

```
      © {new Date().getFullYear()} PSG SHOP · CALIDAD PREMIUM
```

```
    </p>
```

```
  </div>
```

```
</div>
```

```
);
```

```
};
```

```
export default Register;
```

```
📁 src/assets/components/ResetPassword.jsx
```

```
import React, { useState } from 'react';
```

```
import { auth } from '../firebase';
```

```
import { sendPasswordResetEmail } from 'firebase/auth';
```

```
import { useNavigate, Link } from 'react-router-dom';
```

```
const ResetPassword = () => {
```

```
  const [email, setEmail] = useState("");
```

```
  const [message, setMessage] = useState("");
```

```
  const [error, setError] = useState("");
```

```

const [loading, setLoading] = useState(false);

const navigate = useNavigate();

const handleSubmit = async (e) => {
  e.preventDefault();
  setError(''); setMessage(''); setLoading(true);
  try {
    await sendPasswordResetEmail(auth, email);

    setMessage('Se ha enviado un correo para restablecer tu contraseña. Revisa tu
bandeja de entrada.');
```

} catch (err) {
 switch (err.code) {
 case 'auth/user-not-found': setError('No se encontró una cuenta con ese
correo electrónico.');

break;

case 'auth/invalid-email': setError('El correo electrónico no es válido.');

break;

default: setError('Error al enviar el correo de restablecimiento. Por favor,
inténtalo de nuevo.');

}
 } finally {
 setLoading(false);
 }
 };

return (
 <div className="flex min-h-screen bg-gray-50 items-center justify-center p-6"
style={{ fontFamily: 'Inter, sans-serif' }}>

```
<div className="w-full max-w-md bg-white rounded-2xl shadow-xl shadow-
gray-200/50 p-8 md:p-12 border border-gray-100">
```

```
{/* Logo */}
```

```
<div className="flex flex-col items-center mb-10">
```

```
<div className="flex items-center gap-1.5 mb-2">
```

```
<span className="text-3xl font-black tracking-tight text-
black">PSG</span>
```

```
<span className="w-1.5 h-1.5 rounded-full bg-black inline-block mt-1" />
```

```
<span className="text-base font-light tracking-widest text-gray-500
uppercase">SHOP</span>
```

```
</div>
```

```
<p className="text-sm text-gray-400 font-medium text-center">Recupera
el acceso a tu cuenta</p>
```

```
</div>
```

```
{/* Icon */}
```

```
<div className="flex items-center justify-center w-16 h-16 rounded-2xl bg-
gray-50 mx-auto mb-8 border border-gray-100">
```

```
<svg className="w-8 h-8 text-gray-400" fill="none" viewBox="0 0 24 24"
stroke="currentColor">
```

```
<path strokeLinecap="round" strokeLinejoin="round" strokeWidth="1.5"
d="M15 7a2 2 0 0 1 2 2m4 0a6 6 0 0 1-7.743 5.743L11 17H9v2H7v2H4a1 1 0 0 1-1-
1v-2.586a1 1 0 0 1.293-.707l5.964-5.964A6 6 0 1 1 21 9z" />
```

```
</svg>
```

```
</div>
```

```
<div className="text-center mb-8">
```

```
<h1 className="text-xl font-bold text-gray-900 mb-2">¿Olvidaste tu
contraseña?</h1>
```

```
<p className="text-sm text-gray-500 font-medium">
```

```
  Ingresa tu email y te enviaremos las instrucciones de recuperación.
```

```
</p>
```

```
</div>
```

```
{/* Alerts */}
```

```
{message && (
```

```
  <div className="mb-8 flex gap-3 items-start p-4 rounded-xl bg-green-50
border border-green-100 animate-in fade-in">
```

```
    <svg className="w-5 h-5 text-green-500 mt-0.5 flex-shrink-0"
fill="currentColor" viewBox="0 0 20 20">
```

```
      <path fillRule="evenodd" d="M10 18a8 8 0 100-16 8 8 0 00 16zm3.707-
9.293a1 1 0 00-1.414-1.414L9 10.586 7.707 9.293a1 1 0 00-1.414 1.414l2 2a1 1 0
001.414 0l4-4z" clipRule="evenodd" />
```

```
    </svg>
```

```
    <p className="text-sm text-green-700 font-medium leading-
relaxed">{message}</p>
```

```
  </div>
```

```
)}
```

```
{error && (
```

```
  <div className="mb-8 flex gap-3 items-start p-4 rounded-xl bg-red-50
border border-red-100 animate-in fade-in slide-in-from-top-2">
```

```
    <svg className="w-5 h-5 text-red-500 mt-0.5 flex-shrink-0"
fill="currentColor" viewBox="0 0 20 20">
```

```
<path fillRule="evenodd" d="M18 10a8 8 0 11-16 0 8 8 0 0116 0zm-7 4a1
1 0 11-2 0 1 1 0 012 0zm-1-9a1 1 0 00-1 1v4a1 1 0 102 0V6a1 1 0 00-1-1z"
clipRule="evenodd" />
```

```
</svg>
```

```
<p className="text-sm text-red-700 font-medium leading-
relaxed">{error}</p>
```

```
</div>
```

```
)}
```

```
<form onSubmit={handleSubmit} className="space-y-6">
```

```
<div>
```

```
<label htmlFor="email-address" className="block text-xs font-bold text-
gray-500 uppercase tracking-widest mb-2 text-center">
```

```
Correo Electrónico
```

```
</label>
```

```
<input
```

```
id="email-address"
```

```
name="email"
```

```
type="email"
```

```
autoComplete="email"
```

```
required
```

```
placeholder="tu@email.com"
```

```
className="block w-full px-4 py-3 text-sm text-gray-900 bg-white border
border-gray-200 rounded-xl focus:outline-none focus:ring-2 focus:ring-black
focus:ring-offset-0 transition-all placeholder-gray-400 disabled:bg-gray-50 text-
center"
```

```
value={email}
```

```

        onChange={(e) => setEmail(e.target.value)}

        disabled={loading || !!message}

    />
</div>

<button

    type="submit"

    disabled={loading || !!message}

    className="w-full py-3.5 px-4 bg-black text-white text-sm font-bold
rounded-xl hover:bg-gray-800 active:bg-gray-900 active:scale-[0.99] transition-all
duration-200 disabled:opacity-40 shadow-lg shadow-gray-200"

    >

        {loading ? 'Enviando...' : 'Enviar instrucciones'}

    </button>

</form>

<div className="mt-10 text-center">

    <Link to="/login" name="login-link" className="inline-flex items-center
gap-2 text-sm font-bold text-gray-400 hover:text-black transition-all">

        <svg className="w-4 h-4" fill="none" viewBox="0 0 24 24"
stroke="currentColor">

            <path strokeLinecap="round" strokeLinejoin="round" strokeWidth="2.5"
d="M15 19l-7-7 7-7" />

        </svg>

        Volver al inicio

    </Link>

```



```
</div>
```

```
<p className="mt-10 text-center text-[10px] font-bold text-gray-300  
uppercase tracking-[0.2em]">
```

```
  © {new Date().getFullYear()} PSG SHOP · SEGURIDAD TOTAL
```

```
</p>
```

```
</div>
```

```
</div>
```

```
);
```

```
};
```

```
export default ResetPassword;
```

```
📁 src/assets/components/StartRating.jsx
```

```
import React from 'react';
```

```
const StarRating = ({ rating, size = 'md' }) => {
```

```
  // Ensure rating is between 0 and 5
```

```
  const validRating = Math.max(0, Math.min(5, rating || 0));
```

```
  // Calculate full stars and fractional part
```

```
  const fullStars = Math.floor(validRating);
```

```
  const fractionalPart = validRating - fullStars;
```

```
  // Define size classes
```

```
  const sizeClasses = {
```

```
    sm: 'w-4 h-4',
```

```

md: 'w-5 h-5',
lg: 'w-6 h-6'
};

```

```

const starSize = sizeClasses[size] || sizeClasses.md;

```

```

// Create array of star elements

```

```

const stars = [];

```

```

// Add full stars

```

```

for(let i = 0; i < fullStars; i++) {

```

```

  stars.push(

```

```

    <svg

```

```

      key={`full-${i}`}

```

```

      className={` ${starSize} text-yellow-400`}

```

```

      fill="currentColor"

```

```

      viewBox="0 0 20 20"

```

```

    >

```

```

      <path d="M9.049 2.927c.3-.921 1.603-.921 1.902 0l1.07 3.292a1 1 0
00.956 3.462c.969 0 1.371 1.245 1.811 2.8 2.034a1 1 0 00-.364 1.118l1.07
3.292c.3.921-.755 1.688-1.54 1.118l-2.8 2.034a1 1 0 00-1.175 0l-2.8
2.034c-.784 57-
1.838-.197-1.539-1.118l1.07-3.292a1 1 0 00-.364-1.118L2.98 8.72c-.783-.57-.38-
1.81 588-1.81h3.461a1 1 0 00.951-.69l1.07-3.292z" />

```

```

    </svg>

```

```

  );

```

```

}

```

```

// Add half star if there's a fractional part
if(fractionalPart > 0) {
  const percentage = fractionalPart * 100;
  stars.push(
    <div key="fractional" className="relative">
      <svg
        className={` ${starSize} text-gray-300`
        fill="currentColor"
        viewBox="0 0 20 20"
      >
        <path d="M9.049 2.927c.3-.921 1.603-.921 1.902 0l1.07 3.292a1 1 0
00.95.69h3.462c.969 0 1.371 1.24.588 1.81l-2.8 2.034a1 1 0 00-.364 1.118l1.07
3.292c.3.921-.755 1.688-1.54 1.118l-2.8-2.034a1 1 0 00-1.175 0l-2.8 2.034c-.784.57-
1.838-.197-1.539-1.118l1.07-3.292a1 1 0 00-.364-1.118L2.98 8.72c-.783-.57-.38-
1.81.588-1.81h3.461a1 1 0 00.951-.69l1.07-3.292z" />
      </svg>
      <div className="absolute top-0 left-0 overflow-hidden" style={{ width:
`${percentage}%` }}>
        <svg
          className={` ${starSize} text-yellow-400`
          fill="currentColor"
          viewBox="0 0 20 20"
        >
          <path d="M9.049 2.927c.3-.921 1.603-.921 1.902 0l1.07 3.292a1 1 0
00.95.69h3.462c.969 0 1.371 1.24.588 1.81l-2.8 2.034a1 1 0 00-.364 1.118l1.07
3.292c.3.921-.755 1.688-1.54 1.118l-2.8-2.034a1 1 0 00-1.175 0l-2.8 2.034c-.784.57-

```

```
1.838-.197-1.539-1.118l1.07-3.292a1 1 0 00-.364-1.118L2.98 8.72c-.783-.57-.38-
1.81.588-1.81h3.461a1 1 0 00.951-.69l1.07-3.292z" />
```

```
</svg>
```

```
</div>
```

```
</div>
```

```
);
```

```
}
```

```
// Add empty stars
```

```
const totalStars = stars.length;
```

```
for (let i = totalStars; i < 5; i++) {
```

```
  stars.push(
```

```
    <svg
```

```
      key={`empty-${i}`}
```

```
      className={` ${starSize} text-gray-300`}
```

```
      fill="currentColor"
```

```
      viewBox="0 0 20 20"
```

```
    >
```

```
      <path d="M9.049 2.927c.3-.921 1.603-.921 1.902 0l1.07 3.292a1 1 0
00.95.69h3.462c.969 0 1.371 1.24.588 1.81l-2.8 2.034a1 1 0 00-.364 1.118l1.07
3.292c.3.921-.755 1.688-1.54 1.118l-2.8-2.034a1 1 0 00-1.175 0l-2.8 2.034c-.784.57-
1.838-.197-1.539-1.118l1.07-3.292a1 1 0 00-.364-1.118L2.98 8.72c-.783-.57-.38-
1.81.588-1.81h3.461a1 1 0 00.951-.69l1.07-3.292z" />
```

```
    </svg>
```

```
  );
```

```
}
```

```
return (  
  <div className="flex items-center">  
    {stars}  
    <span className="ml-1 text-gray-600 text-sm">{validRating.toFixed(1)}</span>  
  </div>  
);  
};  
  
export default StarRating;
```

DESCRIPCIÓN DE LOS ACUERDOS DE NIVELES DE SERVICIOS

1. Nombre del Servicio

Soporte técnico integral y mantenimiento de la plataforma e-commerce **PSG-SHOP**

2. Información de autorización

2.1 Administrador del Nivel de Servicio: Desarrollador ADSO – Andres Rojas

2.2 Cliente: PSG-SHOP Accesorios Artesanales

3. Descripción/resultado deseado por el cliente

3.1 Proceso/ actividad de negocio del cliente que apoya el servicio: Digitalización, venta y distribución nacional de accesorios y moños artesanales.

3.2 Resultado deseado en términos de acceso al servicio:

TIPO DE SERVICIO	DETALLE	TIEMPO DE RESPUESTA	TIEMPO SOLUCIÓN
CRÍTICO	Caída total del sitio, imposibilidad de procesar pagos en Stripe o pérdida de acceso a Firebase.	1 hora	El 99.9% de las fallas serán reparadas en un máximo de 3 horas.
ALTO	Fallos en el módulo de carrito de compras o errores al cargar imágenes del catálogo de moños.	3 horas	El 80% de las incidencias serán resueltas en el transcurso de 4 horas laborales.
MEDIO	Errores en la aplicación de cupones de descuento o problemas visuales menores que no impiden la venta.	2 horas	Solución en un máximo de 24 horas (un día hábil).
BAJO	Consultas técnicas sobre el uso del Admin Panel o solicitud de cambios	6 horas	El 90% de las solicitudes serán

	menores en textos informativos.		atendidas en un plazo de 5 horas laborales.
--	---------------------------------	--	---

4. Criticidad del servicio y de los activos

4.1 Identificación de activos esenciales para el negocio conectados con el servicio:

4.1.1. Funciones vitales del Negocio: Integridad de la base de datos **NoSQL (Firestore)** para garantizar que los precios y el stock de moños sean correctos.

Protección de las llaves de API de **Stripe** para asegurar la recuperación de fondos y auditoría de transacciones.

5. Tiempo del servicio

La plataforma estará disponible **24/7** en la infraestructura de **Vercel**. El soporte técnico y acompañamiento se realizará mediante asistencia remota (Google Meet, AnyDesk o WhatsApp Web).

5.3. Tiempo de acompañamiento: Se realizará un mantenimiento preventivo **trimestral** con un máximo de 10 horas laborales para la actualización de dependencias (npm update), optimización de imágenes y revisión de reglas de seguridad de Firebase. El cliente será notificado con 10 días hábiles de anticipación.

ADMINISTRACIÓN DE USUARIOS - ROLES

Administrador (Dueño de Negocio): Se encarga de la gestión de inventario, precios y visualización de pedidos. Su principal necesidad es mantener la tienda operativa y rentable.

- **Gestor de Inventario (administrador):** Encargado de actualizar las fotos de las nuevas colecciones y controlar que los moños físicos coincidan con el stock del sistema.
- **Cliente (Usuario Final):** Persona interesada en la compra. Su necesidad es una interfaz rápida, segura para pagar y con información clara sobre sus moños adquiridos.